Advanced Machine Learning

The full teaching manual for the 13-session Advanced Machine Learning kit (Modules 7 to 13; one 3-hour session, two lessons each). Timing, live-demo scripts, the mistakes to pre-empt, lab facilitation, and checks for understanding, for every session.

13 sessions · 23 lessons | 13 sessions × 3 h · 39 h | Instructor copy · not for students

CONTENTS

S01 **Interpreting Models and SHAP**

7.1 Interpreting Models

7.2 SHAP Values

S02 **Local Explanations, Fairness and K-Means**

7.3 LIME and Fairness Checks

8.1 K-Means Clustering

S03 **Choosing k and Hierarchical Clustering**

8.2 Choosing k and Customer Segments

8.3 Hierarchical Clustering

S04 **Density-Based Clustering and Gaussian Mixtures**

8.4 Density-Based Clustering

8.5 Gaussian Mixture Models

S05 **Anomaly Detection and PCA**

8.6 Anomaly Detection

9.1 Principal Component Analysis

S06 **LDA, t-SNE and UMAP**

9.2 Linear Discriminant Analysis

9.3 t-SNE and UMAP

S07 **Neural Networks and PyTorch**

10.1 Neural Networks from Scratch

10.2 PyTorch Fundamentals

S08 **CNNs and Sequence Models**

10.3 Convolutional Neural Networks

10.4 Sequence Models

S09 **Transfer Learning and Text Classification**

10.5 Transfer Learning

11.1 Text Classification

S10 **Recommendations and Reproducibility**

11.2 Recommendation Systems

12.1 Reproducibility and Experiment Tracking

S11 **Serving, Monitoring and the Churn Case Study**

12.2 Serving and Monitoring Models

13.1 Case Study: Predicting Churn

S12 **Capstone Kickoff and Studio I**

13.2 Capstone Project

S13 **Capstone Studio II and Showcase**

13.2 Capstone Project

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 1: Interpreting Models and SHAP

How to teach the first session of Advanced Machine Learning. Welcome a room in which some learners finished Machine Learning Foundations and others bring equivalent experience; recap the conventions this kit builds on; prove that every machine has the full environment, PyTorch and the image data included; then open the churn model up: odds ratios, the impurity trap, permutation importance and partial dependence (7.1), and SHAP values that split one customer's prediction into exact shares (7.2).

2 lessons | 3 hrs session | Lessons 7.1–7.2 | Session 1 of 13

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame (the goal and one mental model) → live demo (you type, they type along) → guided lab (a kit lab, new practice on the lesson's skills) → start the lesson's own project. The lesson exercises, quiz and journal are homework.
- Type, do not paste. Every code block in a demo script below was run to produce the output printed under it (Python 3.12.14, pandas 3.0.6, scikit-learn 1.9.1, shap 0.53.0, PyTorch 2.14.1 on 4 CPU threads). Ask for a prediction before every run that prints a number, and read errors from the last line up, out loud.
- The lessons are the reference. Keep `model_interpretation.html` and `interpretability_shap.html` open in browser tabs for the pictures (the PDP and ICE panels, the waterfall, the beeswarm, the SHAP scatter plots). Labs live in `labs/module-01/`: `_starter.py` files are handouts, `_solution.py` files are yours.
- Session 1 of Advanced Machine Learning opens with 30 minutes of welcome, environment check and a Foundations refresher instead of the usual 10-minute warm-up. This kit is sold separately: some learners finished Machine Learning Foundations last week, others learned scikit-learn at work. The opening makes sure everyone shares the same conventions before Module 7 builds on them.
- Teal blocks are hands-on, amber blocks are watch-outs. Lesson codes (7.1, 7.2) match the course site.

> **THE #1 RISK OF SESSION 1: A ROOM WITH TWO DIFFERENT BACKGROUNDS**
>
> Learners who did Foundations know Pinecrest's churn table, the canonical split, `Pipeline([("prep", ...), ("model", ...)])` and the $15 / $60 campaign economics by heart. Learners who come "with equivalent experience" may know scikit-learn well and still have never seen this data, these names or these numbers. Module 7 quotes all of them without explanation. **Before:** send the setup guide and the one-page "What Kit B assumes" table (it is in the workbook's Start Here page) a week ahead, with `check_environment.py`. **In the room:** the refresher is six predict-the-output questions, one per convention; the tally tells you who needs a partner. **After:** `diagnostic_starter.py` is homework for anyone who missed two or more. The second risk is the image data: Fashion-MNIST, EuroSAT and two pretrained networks (about 175 MB) are needed from Session 5 on. The environment check reports them as "to download"; nobody downloads on venue Wi-Fi.

BEFORE THE SESSION

- ☐ **A week before:** email the setup guide (Python 3.12 or newer, a `pinecrest-ml` folder, `.venv`, on Linux the CPU-only PyTorch line FIRST, then `python -m pip install -r requirements.txt`, then `pinecrest-ml-data.zip` unzipped into the folder). Attach `labs/module-01/l7_0-orientation/check_environment.py`; ask learners to run it once plainly and once with `--download` at home (about 175 MB, once) and to reply with its last line. Learners who did Foundations already have the environment; they only need the download.
- ☐ **Files to hand out** (shared folder or USB stick): `requirements.txt`, `pinecrest-ml-data.zip` (1.4 MB), the whole `labs/` folder **without** the `*_solution.py` files (keep `labs_paths.py`: every lab imports it), and the workbook. Learners put `labs/` inside `pinecrest-ml`, next to `data/`.
- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-01` from this kit's folder. Every line should say PASS (10 files, about a minute and a half). Run `python labs/module-01/l7_0-orientation/check_environment.py` on the projector machine so you can show what READY looks like (up to a minute the first time).
- ☐ **Downloads and caches:** none needed today. Pre-load the image data on the projector machine anyway (`check_environment.py --download`), so you can show what the full READY line looks like, and so Session 5 does not start with a download.
- ☐ **Projector machine:** a terminal with `(.venv)` active in your own `pinecrest-ml`, JupyterLab (or VS Code) started from that folder, font size 18 pt or more, two empty notebooks: `lesson_7_1` and `lesson_7_2`. Browser tabs: the course home page, `model_interpretation.html`, `interpretability_shap.html`. Run `import shap` once before the session: the first import takes several seconds.
- ☐ **Board, before learners arrive:** the "What Kit B assumes" six rows (split, baseline, named pipeline, CV, metrics, $15 / $60); three boxes "recipe / taste test / dial" for 7.1; and "base value + shares = prediction (log-odds)" for 7.2.

- ☐ **Print or share:** the workbook (its Start Here page has the refresher), and sticky notes for help flags and the exit ticket.

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
`check_environment.py` ends with `READY for Sessions 1 to 4`	The image data and the pretrained weights are not downloaded yet	Nothing to do today. At home: `python labs/module-01/l7_0-orientation/check_environment.py --download` (about 175 MB, once), before Session 5.
`ModuleNotFoundError: No module named 'shap'` (or `lime`)	An older Foundations environment from before the full requirements, or the wrong kernel	With `(.venv)` active: `python -m pip install -r requirements.txt`. In a notebook, check the kernel is the `.venv` one.
`import shap` sits silent for 10 to 20 seconds, or prints a numba warning	shap compiles parts of itself the first time	Wait; the second import is fast. Warnings on the first import are harmless.
Their SHAP numbers differ in the third decimal from the page	Another shap or scikit-learn version	The check prints both versions (shap 0.53, scikit-learn 1.9). Reinstall from `requirements.txt`.
`ValueError: The column 0 contains integer data` from a partial dependence call	X was built without `.astype(float)`	Rebuild X with `.astype(float)` and refit: train and explain on the same X (demo Step 9).
On Linux, `pip install -r requirements.txt` starts downloading `nvidia-...` files	pip chose the GPU build of PyTorch	Ctrl+C. Install the CPU line from Lesson 0.1 first, then the requirements line again.
`FileNotFoundError: ... 'data/customers_churn.csv'`	Python is working in another folder	`print(Path.cwd())`. Start JupyterLab from `pinecrest-ml`.

SESSION 1 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	30	Welcome, environment check and Foundations refresher	Introductions, what Kit B assumes from Foundations, `check_environment.py` (now with PyTorch and the image data), a six-question refresher on pipelines, CV, metrics and the threshold (not graded).
0:30	10	**7.1** Frame and mental model	Three people, three questions; the recipe, the taste test and the dial.
0:40	25	**7.1** Live demo	Two models, coefficients as odds ratios, the noise trap, permutation importance, partial dependence, an interaction, two errors on purpose.
1:05	15	**7.1** Guided lab	`labs/module-01/l7_1-interpreting-models/fraud_drivers_starter.py`: what drives the fraud model?
1:20	10	**7.1** Start the lesson project	The churn-drivers briefing, Steps 1 and 2; Steps 3 to 5 go home.
1:30	10	Break	Fix NOT READY machines. Leave the recipe / taste test / dial on the board.
1:40	10	**7.2** Frame and mental model	Splitting a team bonus fairly; base value + shares = prediction.
1:50	25	**7.2** Live demo	Shapley by hand, TreeExplainer and additivity, log-odds, why this customer, the big picture, an interaction, an explainer for any model.
2:15	20	**7.2** Guided lab	`labs/module-01/l7_2-shap-values/shapley_by_hand_starter.py` in pairs, then `labs/module-01/l7_2-shap-values/fraud_reasons_starter.py`.
2:35	15	**7.2** Start the lesson project	Reasons for the call list, Steps 1 to 3; Steps 4 and 5 go home.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 2.
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 95 + 95 = 190 minutes for a solo learner; after the 30-minute opening, the break and the wrap-up the room has 130. So the room does, for each lesson: the mental model, the live demo, one

guided kit lab, and the first steps of the lesson project. The plots are shown from the lesson pages; the demo types the numbers behind them.

Goes home (2 to 3 hours). The churn-drivers briefing Steps 3 to 5 and the call-list reasons Steps 4 and 5; each lesson's Hands-on Exercises 1 to 3 (Exercise 4 is a stretch); both Quick Quizzes; both journal entries; the 7.2 "Before the Next Lesson" list. Anyone whose check said NOT READY fixes their machine; anyone at "READY for Sessions 1 to 4" downloads the image data before Session 5.

Protect, in this order: the environment check and the refresher tally (nobody leaves without knowing whether their machine works and which Foundations habits to revisit); 7.1 Steps 4 and 5 (the noise trap and permutation importance); 7.2 Steps 4 to 7 (additivity, log-odds, one customer explained). **Cut first:** 7.2 Step 9 (the beeswarm: show the lesson figure), then 7.1 Step 6 (side by side: say the 0.064 vs 0.003 sentence), then the 7.2 project start (send all five steps home).

OPENING SEGMENT

0:00 Welcome, Environment Check and Foundations Refresher — 30 min

TIMING

5m	Welcome and introductions
7m	How Kit B runs, and what it assumes from Foundations (the six-row table)
10m	Environment check: `check_environment.py` ends with READY on every machine (or the machine gets a helper)
8m	Refresher: six predict-the-output questions on the split, pipelines, CV, metrics and the threshold, not graded

SEQUENCE AND TALKING POINTS

- **Welcome (5m).** One minute on yourself, including a time an explanation of a model changed a decision. Then a fast round: name, where they use machine learning today, and whether they did Machine Learning Foundations. Note who did not: they get a Foundations graduate as a neighbor for the labs. Housekeeping in one breath: break at 1:30, where the lab files are, sticky note on the laptop lid when stuck.
- **How Kit B runs (3m).** "Thirteen sessions of three hours: explaining models and checking their fairness, then finding groups without labels, reducing dimensions, deep learning with PyTorch, text and recommendations, putting models into production, and a capstone you choose. Same company, Pinecrest Outfitters, same data. In the room we do what is hard alone: the mental model, a live demo, a guided lab and the start of each project. At home you finish the projects and do the exercises, quiz and journal: two to three hours a week."
- **What Kit B assumes (4m).** Walk the six rows on the board (the same table is on the workbook's Start Here page): the canonical split and its first test customer C102112; a baseline before any score; `Pipeline([("prep", preprocess), ("model", model)])`; cross-validation on training rows only; the metric that matches the decision; $15 per offer and $60 per churner reached, break-even 0.25. Say: "If any row is new to you, that is fine: the refresher in ten minutes shows which, and `diagnostic_starter.py` practices each one."
- **Environment check (10m).** On the projector, then theirs: **1** open a terminal; **2** `cd pinecrest-ml`; **3** activate and look for `(.venv)`; **4** `python labs/module-01/l7_0-orientation/check_environment.py`. Three possible last lines: `READY for Advanced Machine Learning`; `READY for Sessions 1 to 4` (images not downloaded yet: homework, with `--download`); `NOT READY` with a numbered list (activation and the data folder you fix now; a missing install you do not start on venue Wi-Fi: pair them).

- **Refresher (8m).** "Six quick questions, one per row of the table. Not a test, nobody collects it." Learners answer D1 to D6 on the Start Here page (about four minutes), then show fingers (1 to 4 for A to D) as you reveal each. Settle arguments by running the code. Tally wrong answers per question on a sticky note.

REFRESHER ANSWERS AND HOW TO READ THE TALLY

Answers: **D1** B) `4500 1500 C102112`; **D2** D) `['prep', 'model'] LogisticRegression`; **D3** A) `[900, 900, 900, 900, 900]`; **D4** C) `0.333 0.5`; **D5** B) `0.75`; **D6** D) `4 2 60`.

- **D1 (split) and D2 (named pipeline)** are the shape of every Module 7 code block: `logreg[-1].coef_`, `logreg[1].scale_`, the first test customer C102112. Two or more misses on D2: say "last step" every time you type `[-1]` today.
- **D3 (CV)** comes back in Lesson 7.1's FAQ ("validation set or test set?") and in Session 3's stability checks. Misses are not urgent today.
- **D4 (recall, precision) and D6 (economics)** are the vocabulary of Lesson 7.3's fairness audit in Session 2. A third of the room missing either: plan five extra minutes on them in Session 2's warm-up.
- **D5 (AUC)** is the number permutation importance measures drops in. Misses here: narrate demo Step 5 slowly ("0.14 of AUC is most of the way from 0.86 to a coin toss").
- **Three or more wrong:** pair them with a Foundations graduate today, and at the wrap-up hand them `labs/module-01/l7_0-orientation/diagnostic_starter.py` as extra homework: six small functions with a checker, one per question, ending with a real churn pipeline, a threshold chosen on out-of-fold probabilities and a profit.

WATCH-OUTS

- **Do not re-teach Foundations.** The refresher is diagnostic; the fixes are the homework file and a neighbor. Two minutes over is fine; ten is not.
- **"I already use SHAP at work."** Welcome them, and give them a job: watch for the units (log-odds) and the background, which experienced users get wrong most. Their stretch is Lab 7.2 A's interaction and Lesson 7.2's Exercise 3.
- **Do not debug one machine in front of the room.** Two minutes per machine, then pair and move on.

LESSON PLANS

7.1 Interpreting Models

60 min

OBJECTIVES (FROM THE LESSON PAGE)

- Read the coefficients of a scaled logistic regression as odds ratios, and say when they mislead
- Compute permutation importance on the test set with `permutation_importance` and explain why impurity-based importance can fool you
- Draw partial dependence and ICE plots with `PartialDependenceDisplay` and read thresholds, curves and interactions from them
- State what feature importance does and does not mean: it describes the model, not the causes of churn

TIMING IN THE ROOM

10m	Frame: three people, three questions (what drives churn, is it using anything it should not, why this person); the recipe, the taste test, the dial
25m	Live demo: two models, coefficients and odds ratios, natural units, the noise trap, permutation importance, PDP numbers, an interaction, two errors
15m	Guided lab: `fraud_drivers_starter.py`
10m	Start the lesson project: the churn-drivers briefing, Steps 1 and 2

MENTAL MODEL TO SAY

"Before the retention team acts on a churn model, three people ask questions. The marketing manager: what drives churn? The data scientist next to you: is it using anything it should not? An agent looking at one flagged customer: why this person? The first two are about the whole model, global explanations; the third is about one prediction, a local explanation. Today: all three, the first two this hour."

Then the picture, drawn as three boxes on the board: "Think of the model as a dish. **Coefficients are reading the recipe**: only possible when the recipe is short and written down, as in a linear model. **Permutation importance is a taste test**: swap one ingredient for a random one and see how much worse the dish gets; it works on any dish, even from a kitchen you cannot see into. **Partial dependence is a dial**: turn one ingredient up and down, keep everything else, and watch the taste change."

Then the one rule for the whole module, written under the boxes: **these tools describe the model, not the world.** "The model relies on recency" is a measurement. "Recency causes churn" needs an experiment.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook, `lesson_7_1`, in `pinecrest-ml`. The lesson's charts stay on the lesson page; you type the code that produces the numbers behind them, and ask for a prediction before every run.

1 **Two models to explain (3m).** "Before we explain anything, we check it is worth explaining. Same 11 numeric features for both, the canonical split, a baseline." Narrate `.astype(float)`: partial dependence refuses integer columns (Step 9 shows the error).

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.dummy import DummyClassifier
from sklearn.metrics import roc_auc_score

customers = pd.read_csv("data/customers_churn.csv")
features = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value", "da
            "returns_12m", "return_rate", "support_tickets_12m", "discount_sha
            "email_opens_90d"]

X = customers[features].astype(float)
y = customers["churned"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, stra

baseline = DummyClassifier().fit(X_train, y_train)
logreg = make_pipeline(SimpleImputer(strategy="median"), StandardScaler(), Log
logreg.fit(X_train, y_train)
hgb = HistGradientBoostingClassifier(random_state=42).fit(X_train, y_train)

for name, model in [("baseline", baseline), ("logistic regression", logreg), (
    auc = roc_auc_score(y_test, model.predict_proba(X_test)[:, 1])
    print(f"{name:20s} test ROC AUC {auc:.3f}")
```

```
baseline             test ROC AUC 0.500
logistic regression  test ROC AUC 0.828
gradient boosting    test ROC AUC 0.865
```

Write 0.500 / 0.828 / 0.865 on the board. Ask: "Where will the 0.04 gap between the two models come from?" Park the answers; Step 6 settles it.

2 **Read the recipe: coefficients (2m).** "A scaled logistic regression adds weighted features to get the log-odds. `np.exp` turns each weight into an odds ratio: above 1 raises the odds, below 1 lowers them."

```
coefs = pd.Series(logreg[-1].coef_[0], index=features)
table = pd.DataFrame({"coef": coefs, "odds_ratio": np.exp(coefs)}).sort_values
print(table.round(2))
```

```
                        coef  odds_ratio
orders_12m             -0.32        0.73
app_user               -0.29        0.75
email_opens_90d        -0.28        0.75
tenure_months          -0.13        0.88
spend_12m              -0.12        0.89
returns_12m            -0.03        0.97
avg_order_value        -0.01        0.99
return_rate             0.08        1.09
support_tickets_12m     0.40        1.50
discount_share          0.68        1.97
days_since_last_order   0.89        2.43
```

`logreg[-1]` is the last pipeline step, the fitted model: the Pipeline naming from Foundations. Read only the top and the bottom rows aloud.

3. **Say it in natural units (2m).** "Per standard deviation is precise and unfriendly. Divide by the scaler's standard deviation, multiply by a step that means something."

```
scale = pd.Series(logreg[1].scale_, index=features)
per_unit = coefs / scale

print(f"days_since_last_order: sd = {scale['days_since_last_order']:.1f} days"
print(f"odds ratio per 30 more days without an order: {np.exp(30 * per_unit['d
print(f"odds ratio for an app user vs a non-user:     {np.exp(per_unit['app_us
```

```
days_since_last_order: sd = 87.2 days
odds ratio per 30 more days without an order: 1.36
odds ratio for an app user vs a non-user:     0.55
```

The sentence for the briefing: every extra month without an order multiplies the odds by about 1.4; app users have about half the odds, other features held fixed. Then the three warnings on the slide: a line cannot bend, overlapping features share credit, "held fixed" may be impossible.

4. **The built-in importance trap (4m).** "Forests come with `feature_importances_`. Free, fast, tempting. Watch what it does with a column of pure noise."

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance

rng = np.random.default_rng(42)
X_train_noise = X_train.assign(random_noise=rng.normal(size=len(X_train)))
X_test_noise = X_test.assign(random_noise=rng.normal(size=len(X_test)))

forest = RandomForestClassifier(n_estimators=200, random_state=42, n_jobs=-1)
forest.fit(X_train_noise, y_train)

mdi = pd.Series(forest.feature_importances_, index=X_train_noise.columns)
perm = permutation_importance(forest, X_test_noise, y_test, scoring="roc_auc",
                              n_repeats=10, random_state=42, n_jobs=-1)
compare = pd.DataFrame({"impurity (train)": mdi,
                        "permutation (test)": perm.importances_mean}).sort_val
print(compare.round(3))
```

```
                       impurity (train)  permutation (test)
days_since_last_order             0.218               0.152
discount_share                    0.130               0.080
tenure_months                     0.125               0.057
spend_12m                         0.091               0.001
avg_order_value                   0.086               0.004
random_noise                      0.083               0.005
return_rate                       0.079               0.006
email_opens_90d                   0.072               0.010
orders_12m                        0.041               0.003
support_tickets_12m               0.041               0.012
app_user                          0.018               0.009
returns_12m                       0.016               0.001
```

Ask before you reveal the table: "Where will the noise land?" It lands sixth by impurity, above `app_user` and `support_tickets_12m`, and near 0.005 by permutation on the test set. Rule of thumb on the board: impurity for a quick look, permutation on held-out data for anything you tell someone.

5 **The taste test: permutation importance (3m).** "Shuffle one column of the test set so it no longer matches its customers. If the score drops, the model was relying on it. Nothing is retrained."

```
result = permutation_importance(hgb, X_test, y_test, scoring="roc_auc", n_repe
importance = pd.DataFrame({"mean": result.importances_mean, "std": result.impo
                           index=features).sort_values("mean", ascending=False)
print(importance.round(3))
```

```
                          mean    std
days_since_last_order    0.140  0.012
discount_share           0.088  0.006
tenure_months            0.064  0.009
app_user                 0.019  0.003
support_tickets_12m      0.015  0.003
email_opens_90d          0.009  0.004
orders_12m               0.006  0.005
return_rate              0.004  0.003
avg_order_value          0.003  0.001
returns_12m             -0.000  0.001
spend_12m               -0.004  0.002
```

A slightly negative value (spend_12m) reads as zero. Point at the std column: two features within one standard deviation of each other are not a ranking you defend.

6 **Same features, different model, different story (2m).** "Importance describes a model. Ask the logistic regression the same question."

```
lr_result = permutation_importance(logreg, X_test, y_test, scoring="roc_auc",
side_by_side = pd.DataFrame({"boosting": result.importances_mean,
                             "logistic": lr_result.importances_mean}, index=fe
print(side_by_side.sort_values("boosting", ascending=False).round(3).head(6))
```

```
                        boosting  logistic
days_since_last_order      0.140     0.150
discount_share             0.088     0.092
tenure_months              0.064     0.003
app_user                   0.019     0.010
support_tickets_12m        0.015     0.021
email_opens_90d            0.009     0.010
```

Tenure: 0.064 for boosting, 0.003 for the line. That is the coefficient warning, measured, and a large part of the AUC gap from Step 1.

7 **The dial: partial dependence in numbers (3m).** "Set everyone's days to 15, predict, average. Then 60, 120 ... That curve is partial dependence. `method="brute"` averages real probabilities."

```
from sklearn.inspection import partial_dependence

pd_days = partial_dependence(hgb, X_test, ["days_since_last_order"], method="b
                             custom_values={"days_since_last_order": [15, 60,
for days, p in zip(pd_days["grid_values"][0], pd_days["average"][0]):
    print(f"{days:5.0f} days -> average predicted churn {p:.3f}")
```

```
  15 days -> average predicted churn 0.077
  60 days -> average predicted churn 0.077
 120 days -> average predicted churn 0.095
 150 days -> average predicted churn 0.214
 180 days -> average predicted churn 0.390
 240 days -> average predicted churn 0.443
 300 days -> average predicted churn 0.490
```

Show the lesson page's three-panel PDP and ICE figure for the shapes (threshold, U, kink). The briefing sentence: risk roughly quadruples between 120 and 180 days.

8 **An interaction the average hides (3m).** "A PDP averages everybody. Split the customers by app use and compute it twice."

```
for flag, label in [(0, "non-app users"), (1, "app users")]:
    group = X_test[X_test["app_user"] == flag]
    pd_tickets = partial_dependence(hgb, group, ["support_tickets_12m"], metho
                                    custom_values={"support_tickets_12m": [0,
    print(f"{label:14s}", pd_tickets["average"][0].round(3))
```

```
non-app users  [0.137 0.2   0.234 0.288 0.296 0.438]
app users      [0.087 0.098 0.107 0.11  0.081 0.145]
```

For non-app users each ticket raises predicted churn; for app users tickets barely matter. "It depends" is what trees capture and a plain line does not.

9 **Error on purpose 1: integer columns (1m).** "Here is why we converted to float at the start."

```
from sklearn.inspection import PartialDependenceDisplay
raw = customers[features].iloc[:300]
PartialDependenceDisplay.from_estimator(hgb, raw, ["tenure_months"])
```

```
ValueError: The column 0 contains integer data. Partial dependence plots are r
```

Read only the first sentence aloud. Fix: `.astype(float)` when you build X, and train and explain on the same X.

10 **Error on purpose 2: a pipeline has no coefficients (1m).** "The pipeline is a container. The coefficients live on its last step."

```
logreg.coef_
```

```
AttributeError: 'Pipeline' object has no attribute 'coef_'
```

Fix: `logreg[-1].coef_` (or `logreg.named_steps["logisticregression"]`). Then show Common Mistake 3 from the page without running it: importance on the training rows looks several times bigger for the noisy columns (0.026 vs 0.009 for email opens). No error, and that is the problem.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Reading importance as a lever.** "App use is a top-5 driver, so the app campaign will cut churn." Coach with a question: "Would pushing a disengaged customer to install the app make them engaged? What would you need to know that?" The answer is a randomized test. Make them rephrase as "customers who use the app are predicted to churn less".
- **Importance on the training rows, or impurity importance as the answer.** No error, bigger and more confident numbers. Coach: "Which rows did you shuffle? Would the noise column survive this?" In the lab, ask them to add a random column if they doubt a ranking.
- **Partial dependence read off the edges.** The PDP sets every customer's spend to $2,500, including customers with one $60 order: a customer who cannot exist. Coach: "Check the curve against a groupby of the real rate" (Lab A's TODO 5 does exactly that for the hour).

GUIDED LAB

File: learners open `labs/module-01/l7_1-interpreting-models/fraud_drivers_starter.py`; your reference is `labs/module-01/l7_1-interpreting-models/fraud_drivers_solution.py`. The same tools on a new question: what drives the FRAUD model (the stratified 70/30 split from Foundations, a regularized booster)? Step 1 is done for them and prints test ROC AUC 0.863 and average precision 0.513.

- **TODO 1 and 2 together on the projector (5m).** Permutation importance by average precision, then by ROC AUC. Pause on the surprise before anyone explains it: `hour` costs 0.074 of AP and nothing (-0.005) of AUC.
- **TODO 3 to 5 in pairs (8m).** The hour dial (night 0.0174 vs day 0.0072), the same dial for known and new devices (2 a.m.: 0.006 vs 0.122), and the check against the data (night 6.40% real, 5.89% predicted; under 1% in every daytime band).
- **Debrief (2m):** "Which metric would the review team want, and why?" Average precision: they work from the top of the list, and the hour pushes a few night frauds there. Then: "Is the hour effect the same for every payment?" No: it is a night-and-new-device interaction. Lesson 7.2's lab finds it again in SHAP values.
- **Fast finishers:** `labs/module-01/l7_1-interpreting-models/pipeline_importance_starter.py`: the full Foundations pipeline with one-hot categories. Odds ratios with the pipeline's own column names; Asheville looks protective (0.66) while region Southeast looks risky (1.32), and loyalty tier gets 1.48 although Gold churns least; permutation

importance per business column puts all five descriptive columns at zero. Then `labs/module-01/l7_1-interpreting-models/debug_interpret.py`: two loud bugs and two silent ones.

START THE LESSON PROJECT

The lesson project is the **churn-drivers briefing**: a ranked table, grouped importance and turning points, saved as `churn_drivers.csv`. In the room, Steps 1 and 2 (10 minutes).

- Read the brief aloud ("Pinecrest's retention team meets on Monday and wants one page"). Step 1: train and confirm `test ROC AUC 0.865 (baseline 0.500)`. Step 2: permutation importance with mean and std; the top line is `days_since_last_order 0.140 0.012`.
- **Goes home:** Step 3 (`group_importance`: purchase volume 0.010, more than its single columns' 0.004), Step 4 (turning points: recency lowest 0.072 at 18 days, highest 0.490 at about 293), Step 5 (`churn_drivers.csv`, 11 rows, with the caveat line).

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- What does `np.exp` of a scaled logistic-regression coefficient give you? **Answer:** an odds ratio per standard deviation of that feature, other features held fixed.
- Why permutation importance on the TEST set? **Answer:** on training rows the model's memorized noise looks important; on new rows it does not help, so the test numbers are the honest ones.
- A feature has permutation importance 0. Is it unrelated to churn? **Answer:** not necessarily: the model may get the same information from overlapping columns (spend from orders and order value). Shuffle the group together.
- What does a fan of diverging ICE lines under a flat PDP tell you? **Answer:** an interaction: the feature's effect depends on another feature, and the average hides it.

Workbook self-check, items Q1 to Q6 (learners do them at the end of the block or at home).

Answers: **Q1** C) One more standard deviation of days without an order multiplies the odds of churn by about 2.4, other features held fixed; **Q2** `[1.49, 0.5]`; **Q3** B) Impurity importance cannot be trusted for the ranking: it is measured on training data and rewards columns the trees can memorize with; **Q4** D) Importance depends on the metric: the hour lifts a few night frauds to the very top of the list, which average precision rewards and the whole-ranking AUC hardly notices; **Q5** `[ 2. 12.]`; **Q6** A) Customers with long gaps since their last order are predicted to churn more; the risk jumps after about four months. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: a card with the three boxes (recipe, taste test, dial) and one line each on what it answers. In the lab, let a struggling pair copy the `permutation_importance` call from the demo notebook and change only the model, the data and `scoring`.

Stretch: Lab 7.1 B; then the lesson's Exercise 4 (permutation importance by hand) and the Bonus Challenge (add `cancellation_reason` and watch a routine importance check catch the leak).

7.2 SHAP Values

70 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain the Shapley idea (average contribution over every order in which features join) and compute it by hand for a tiny model
- Compute SHAP values for a gradient-boosting model with `shap.TreeExplainer` and check that they add up to the prediction
- Read waterfall, beeswarm and scatter plots, including the units (log-odds) they are drawn in
- Use a model-agnostic explainer on a pipeline, and turn SHAP values into plain-language reason codes

TIMING IN THE ROOM

10m	Frame: why this customer? Splitting a team bonus fairly; base value + shares = prediction
25m	Live demo: Shapley by hand, the Exact explainer, TreeExplainer and additivity, log-odds, one customer, mean |SHAP|, the interaction, a PermutationExplainer, two errors
20m	Guided lab: `shapley_by_hand_starter.py` in pairs (8m), then `fraud_reasons_starter.py` (12m)
15m	Start the lesson project: reasons for the call list, Steps 1 to 3

MENTAL MODEL TO SAY

"A team of three wins a bonus. How much did each person contribute? 'What would the team have earned without Ana?' is unfair when people work together, because the answer depends on who else is in the room. Lloyd Shapley's answer, in 1953: imagine them arriving one at a time, in every possible order; each time someone arrives, credit them with how much the result went up; average each person's credit over all the orders."

Then the translation, on the board: the players are the **features**; the bonus is **how far this customer's prediction is from the model's average prediction** (the base value). Under it: **base value + all shares = this prediction, exactly**. And the warning, boxed: for a boosting classifier, all of it is in **log-odds**; squash only the end points with `expit`.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook, `lesson_7_2`. The toy model first, so that every number can be checked by hand; then the churn model, so that the same bookkeeping holds for 1,500 real customers.

1. **Shapley values by hand (4m).** "A toy risk score with an interaction: a long silence is worse without the app. Let the features arrive in every order and credit each with what its arrival adds." Ask for a prediction first: how will the 0.10 interaction bonus be split?

```
import itertools

def toy_risk(days, disc, app):
    risk = 0.10
    if days > 150:
        risk += 0.30
    if disc > 0.35:
        risk += 0.15
    if days > 150 and app == 0:
        risk += 0.10
    return risk

reference = {"days": 40, "disc": 0.15, "app": 1}   # a typical customer
customer = {"days": 240, "disc": 0.45, "app": 0}   # the customer to explain
names = list(customer)

def value(present):
    """Risk when the features in `present` take the customer's values and the
    return toy_risk(**{n: customer[n] if n in present else reference[n] for n

shares = dict.fromkeys(names, 0.0)
orders = list(itertools.permutations(names))
for order in orders:
    present = set()
    for name in order:
        before = value(present)
        present.add(name)
        shares[name] += value(present) - before
shares = {n: round(s / len(orders), 3) for n, s in shares.items()}

print("orders tried:", len(orders))
print("reference risk:", value(set()), " customer risk:", value(set(names)))
print("Shapley values:", shares, " sum:", round(sum(shares.values()), 3))
```

```
orders tried: 6
reference risk: 0.1  customer risk: 0.65
Shapley values: {'days': 0.35, 'disc': 0.15, 'app': 0.05}  sum: 0.55
```

Days gets 0.30 + 0.05, app gets 0.05: whichever of the two arrives second collects the 0.10, and each arrives second in half the orders. The shares add up to the gap, 0.55, exactly.

2 **The library agrees (1m).** "shap's Exact explainer tries every subset, like our loop."

```
import numpy as np
import shap

toy_model = lambda rows: np.array([toy_risk(*row) for row in rows])
toy_explainer = shap.explainers.Exact(toy_model, np.array([[40, 0.15, 1]]))
print(toy_explainer(np.array([[240, 0.45, 0]])).values.round(3))
```

```
[[0.35 0.15 0.05]]
```

With 11 features there are 39,916,800 orders: that is why SHAP's fast tree algorithm matters.

3 **Back to the churn model (1m).** "Same 11 features, same split as 7.1."

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.metrics import roc_auc_score

customers = pd.read_csv("data/customers_churn.csv")
features = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value", "da
            "returns_12m", "return_rate", "support_tickets_12m", "discount_sha
            "email_opens_90d"]
X = customers[features].astype(float)
y = customers["churned"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, stra

hgb = HistGradientBoostingClassifier(random_state=42).fit(X_train, y_train)
print(f"test ROC AUC {roc_auc_score(y_test, hgb.predict_proba(X_test)[:, 1]):.
```

```
test ROC AUC 0.865
```

4 **TreeExplainer, and the bookkeeping (2m).** "One row of 11 shares per customer. Base value plus shares must equal the model's output: check it."

```
explainer = shap.TreeExplainer(hgb)
sv = explainer(X_test)

print(type(sv).__name__, sv.values.shape)
print(f"base value: {sv.base_values[0]:.3f}")

raw_score = hgb.decision_function(X_test)
gap = np.abs(sv.base_values + sv.values.sum(axis=1) - raw_score).max()
print(f"largest additivity gap over 1,500 customers: {gap:.1e}")
```

```
Explanation (1500, 11)
base value: -2.951
largest additivity gap over 1,500 customers: 1.5e-14
```

Stress which output: `decision_function`, the raw score, not `predict_proba`.

5 **The units are log-odds (2m).** "Boosting adds trees in log-odds and squashes at the very end. SHAP explains the sum before the squash."

```
from scipy.special import expit   # the logistic function: log-odds to probabi

print(f"base value as a probability: {expit(sv.base_values[0]):.3f}")
print(f"average predicted probability: {hgb.predict_proba(X_train)[:, 1].mean(
```

```
base value as a probability: 0.050
average predicted probability: 0.166
```

5% is not 16.6%: the squash of an average is not the average of the squashes. Compare sizes and signs of SHAP values freely; never add them to a probability.

6 **Why this customer? (3m).** "The riskiest test customer. What should the agent talk about?" Have the room guess the top three before you run it (this was 7.1's Before the Next Lesson task).

```
proba = hgb.predict_proba(X_test)[:, 1]
i = int(proba.argmax())
row_id = X_test.index[i]
print(customers.loc[row_id, "customer_id"], f"churn probability {proba[i]:.3f}

why = pd.DataFrame({"value": X_test.iloc[i], "shap": sv.values[i]}).sort_value
print(why.round(3))
```

```
C104051 churn probability 0.972
                         value   shap
days_since_last_order  364.000  3.095
discount_share           0.432  1.827
support_tickets_12m      3.000  0.767
app_user                 0.000  0.316
orders_12m               2.000  0.267
return_rate              0.051  0.157
spend_12m              223.780  0.110
email_opens_90d          6.000  0.084
avg_order_value        111.890  0.061
returns_12m              0.000 -0.032
tenure_months           65.000 -0.149
```

Silence first (+3.10 log-odds), discounts (+1.83), complaints (+0.77); a long tenure pulls slightly the other way. Draw the waterfall on the board as a staircase, or show it on the lesson page.

7. **Speak in probabilities at the end points only (2m).** "Squash the start and the end, not the bars."

```
start = sv.base_values[i]
end = start + sv.values[i].sum()
print(f"log-odds {start:.2f} -> {end:.2f}")
print(f"probability {expit(start):.3f} -> {expit(end):.3f}")
```

```
log-odds -2.95 -> 3.55
probability 0.050 -> 0.972
```

8. **From 1,500 explanations to the big picture (2m).** "Average the sizes and you get a global importance."

```
mean_abs = pd.Series(np.abs(sv.values).mean(axis=0), index=features).sort_valu
print(mean_abs.round(3))
```

```
days_since_last_order    0.903
tenure_months            0.802
discount_share           0.742
email_opens_90d          0.278
orders_12m               0.278
app_user                 0.273
support_tickets_12m      0.267
return_rate              0.193
spend_12m                0.171
avg_order_value          0.137
returns_12m              0.043
dtype: float64
```

Same top three as 7.1's permutation importance; tenure and discounts swap. Not a contradiction: mean |SHAP| measures how much a feature moves predictions, permutation how much the score depends on it.

9. **The beeswarm (optional, 2 minutes) (2m).** "Every customer is a dot: position is the share, color is the feature's value." Run it if your projector shows plots well; otherwise show the lesson page's figure.

```
import matplotlib.pyplot as plt

plt.figure()
shap.plots.beeswarm(sv, max_display=11, show=False)
plt.gcf().set_size_inches(9, 6)
plt.title("SHAP values for 1,500 test customers")
plt.tight_layout()
plt.show()
```

Read recency's long red tail: most customers ordered recently and get a small negative share; a few silent ones get a large positive one.

10 **The interaction, in SHAP numbers (2m).** "7.1's tickets-and-app interaction, now from 1,500 explanations."

```
tickets = pd.DataFrame({
    "tickets": X_test["support_tickets_12m"].clip(upper=4),
    "app_user": X_test["app_user"],
    "shap": sv[:, "support_tickets_12m"].values,
})
print(tickets.pivot_table(index="tickets", columns="app_user", values="shap",
```

```
app_user   0.0   1.0
tickets
0.0      -0.39 -0.23
1.0       0.09  0.07
2.0       0.35  0.23
3.0       0.78  0.46
4.0       0.92  0.40
```

Non-app users at 4+ tickets: +0.92 log-odds; app users +0.40.

11 **Any model: the PermutationExplainer (2m).** "TreeExplainer only reads trees. For a pipeline, pass a function and a background sample."

```
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

logreg = make_pipeline(SimpleImputer(strategy="median"), StandardScaler(), Log
logreg.fit(X_train, y_train)

def churn_probability(rows):
    return logreg.predict_proba(pd.DataFrame(rows, columns=features))[:, 1]

background = X_train.sample(100, random_state=42)
perm_explainer = shap.PermutationExplainer(churn_probability, background, seed
lr_sv = perm_explainer(X_test.iloc[[i]])

lr_why = pd.Series(lr_sv.values[0], index=features).sort_values(ascending=Fals
print(f"base {lr_sv.base_values[0]:.3f} + shares {lr_sv.values[0].sum():.3f} =
print(lr_why.round(3).head(5))
```

```
base 0.157 + shares 0.791 = 0.948
days_since_last_order    0.506
discount_share           0.113
support_tickets_12m      0.104
orders_12m               0.037
app_user                 0.028
dtype: float64
```

Probability units now, because the function returns probabilities. Two different models, the same story for this customer.

12 **Errors on purpose (1m).** "Two mistakes everybody makes once."

```
shap.TreeExplainer(logreg)

wrong = proba.mean() + sv.values[i].sum()
right = expit(sv.base_values[i] + sv.values[i].sum())
print(f"wrong: {wrong:.2f}   right: {right:.3f}")
```

```
InvalidModelError: Model type not yet supported by TreeExplainer: <class 'skl
wrong: 6.65   right: 0.972
```

First: TreeExplainer cannot read a Pipeline (use PermutationExplainer, or explain `pipe[-1]` on `pipe[:-1].transform(X)`). Second: adding log-odds to a probability gives 6.65 with no error. Add to the base value, then `expit`.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Adding log-odds to a probability.** "The base is 16% and discounts add 1.83, so ..." Coach: "What units is the base value in? What does `expit(-2.95)` give?" Then the rule: add in log-odds, squash at the end. Lab 7.2 A's base value (-5.84 log-odds, 0.0029) is the same trap on fraud.
- **Handing a Pipeline to TreeExplainer, or a forest's 3-D values to a plot.** Two errors with clear last lines. Coach: "Print `.values.shape` before you plot a new model's explanation." A forest gives (rows, features, 2): pick class 1.
- **Reading a top SHAP feature as the cause.** An agent offers fewer discounts because discount_share got +1.83. Coach: "flagged because", never "leaving because"; reasons start a conversation, they do not prescribe a treatment.

GUIDED LAB

- **Lab B first (8m, pairs):** `labs/module-01/l7_2-shap-values/shapley_by_hand_starter.py`; reference `labs/module-01/l7_2-shap-values/shapley_by_hand_solution.py`. A four-feature fraud rule with an interaction (a new device is only suspicious at night) and a feature that does nothing (`gift_card`). One types, one predicts: how is the 0.30 bonus split, and what does gift_card get? **Expected:** 24 orders; velocity 0.2, new_device 0.15, night 0.15, gift_card 0.0, sum 0.5; shap Exact agrees; against a typical NIGHT payment, night drops to 0.0 and new_device rises to 0.3 (the total stays 0.5).
- **Lab A (12m, everyone):** `labs/module-01/l7_2-shap-values/fraud_reasons_starter.py`; reference `labs/module-01/l7_2-shap-values/fraud_reasons_solution.py`. SHAP values for 15,000 test payments, the additivity check, mean |SHAP|, a review queue of the 25 riskiest payments with three reasons each, and the night x new-device interaction. **Expected:** base value -5.840 log-odds = 0.0029; mean |SHAP| led by device_new 0.247 and velocity_24h 0.235; the first queued payment (T031633, 0.99) has "5 payments on this card in 24 hours", "amount $836", "account only 10 days old"; top reasons velocity 19, new device 5, card attempts 1; 25 of 25 are real frauds; the hour adds +1.24 log-odds at night on a new device, +0.33 on a known one.
- **Circulate for:** queue rows looked up by label in `sv.values` (it is a NumPy array: positions, hence `test.index.get_loc`); `phrases[f].format(...)` on a NaN; reasons taken from negative shares.
- **Debrief (2m):** "7.1's lab found the hour interaction with partial dependence; this one finds it in SHAP values. Two tools, one story: that agreement is what makes you confident."
- **Fast finishers:** `labs/module-01/l7_2-shap-values/debug_shap.py`: TreeExplainer on a pipeline, a forest's class axis, and a silent probability of 6.652.

START THE LESSON PROJECT

The lesson project is **reasons for the call list**: the 20 riskiest test customers, three neutral reasons each, saved as `call_list_reasons.csv`. In the room, Steps 1 to 3 (15 minutes).

- Step 1: split the whole table so the IDs stay with their rows. **Done** = `4500 1500 C102112`. Step 2: explain only the top 20 (C104051 first: 3.10, 1.83, -0.15).
- Step 3, the phrases: let pairs write their own templates before showing the lesson's. Discuss: which phrases sound like an accusation? ("only 2 orders this year" vs "2 orders this year".) **Done** = `C104051 ['no order in 364 days', '43% of spend on discounted items', '3 support tickets this year']`.
- **Goes home:** Step 4 (the call list) and Step 5 (16 of 20 open with a long silence, 4 with discounts; 16 of 20 really churned; `call_list_reasons.csv`).

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- What do base value and the shares add up to for the boosting churn model? **Answer:** the model's raw score in log-odds (`decision_function`), exactly.
- Why is the base value 5% when 16.6% of customers churn? **Answer:** it is the average log-odds, squashed; the squash of an average is not the average of the squashed values.
- When do you need a PermutationExplainer instead of TreeExplainer? **Answer:** for any model TreeExplainer cannot read: a Pipeline, a linear model on its own data, a neural network, a model behind an API.
- Why only positive shares as reasons on a call list? **Answer:** the agent wants what raised the risk; negative shares are protective factors (useful for deciding whom NOT to call).

Workbook self-check, items Q7 to Q12 (learners do them at the end of the block or at home).

Answers: **Q7** B) 0, because adding it never changes the score in any arrival order; **Q8** `0.0 0.5`; **Q9** C) Mean |SHAP| measures how much a feature moves predictions; permutation importance measures how much the score depends on it, and the two need not agree; **Q10** D) `forest_sv[0, :, 1]`; **Q11** A) The base value and the shares change; the customer's predicted probability does not; **Q12** `43% of spend on discounted items | no order in 364 days`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: do Lab B's first order (velocity, new_device, night, gift_card) together on the board before the pairs start; a learner who sees one order worked out can do the other 23 with the loop. Give anyone lost in the units the two-line card: "add in log-odds; expit at the end".

Stretch: Lesson 7.2's Exercise 3 (interaction values: the strongest pair is days x tenure, 0.073) and Exercise 4 (does a forest give the same top reason? 20 of 20 for the call list; try customers ranked 200 to 300).

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Put the three questions on the slide. Learners answer in the workbook's Exit Ticket box or on a sticky note. Sort the notes before Session 2: the pile for question 2 tells you how much of Session 2's warm-up goes to units.

1. In one sentence each: what does permutation importance tell you about a model, what does partial dependence tell you, and what does neither tell you? **Answer:** how much the held-out score depends on a feature; how the average prediction moves as one feature changes; neither says what would happen if Pinecrest changed that feature (cause and effect).
2. A SHAP waterfall for the boosting churn model starts at -2.95 and ends at +0.50. What churn probability does the model give, and why can you not add the bars to 16.5%? **Answer:** `expit(0.50)` = 0.62. The bars are log-odds; 16.5% is a probability, and the base value is -2.95 log-odds (5%), not 16.5%.
3. Write the reason code an agent could read aloud for `days_since_last_order` = 240 with a SHAP value of +2.1, and one thing the agent must not say. **Answer:** "no order in 240 days" (neutral, factual, about the account). Not: "you are leaving because you stopped buying", or any promise based on the model ("order now and you will stay").

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** The churn-drivers briefing (Steps 3 to 5: grouped importance, turning points, `churn_drivers.csv`) and the call-list reasons (Steps 4 and 5: `call_list_reasons.csv`, 16 of 20 really churned).
- **Lesson exercises.** 7.1: Exercises 1 to 3 (another yardstick, remove a redundant feature, check a curve against reality); Exercise 4 is a stretch. 7.2: Exercises 1 to 3 (the safest customer, the tenure U in SHAP values, the strongest interaction); Exercise 4 is a stretch.
- **Both Quick Quizzes** on the lesson pages (five questions each; answers in the Answer Key).
- **Journal:** both entries. The 7.2 prompt asks for the three reasons for C104051 worded for an agent, and what the agent must not say.
- **Before the Next Lesson (from 7.2):** keep the project notebook (Lesson 7.3 explains the same customer with LIME); look at the call list and write down a guess: would it look the same for younger and older customers?
- **If the check said NOT READY:** follow its numbered list. **If it said READY for Sessions 1 to 4:** run it again with `--download` at home (about 175 MB) before Session 5.
- **Optional:** `labs/module-01/l7_0-orientation/diagnostic_starter.py` for anyone who missed two or more refresher questions; `labs/module-01/l7_1-interpreting-models/pipeline_importance_starter.py` for anyone who did not reach it.

BRING LIST FOR SESSION 2

- A laptop on which `check_environment.py` ends with READY (lime is used for the first time in Session 2; the check already covers it).
- The `pinecrest-ml` folder with `data/` and the `labs/` folder inside it. Session 2's starters are in `labs/module-02/`.
- The finished Lesson 7.2 project notebook and `call_list_reasons.csv`: Lesson 7.3 explains the same customer, C104051, with LIME.
- Your guess about the call list and age groups (7.2's Before the Next Lesson), the workbook and the journal. No downloads needed for Session 2.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** every machine's check result; 7.1 Steps 4 and 5; 7.2 Steps 4 to 7; Lab 7.2 B (Shapley by hand) in full.
- **Cut first (saves 4m):** 7.2 Step 9 (beeswarm) and Step 10 (tickets pivot): show the lesson figures and say the two sentences.
- **Cut second (saves 5m):** 7.1 Steps 6 and 8 become one sentence each ("the line gets 0.003 from tenure, boosting 0.064"; "tickets matter for non-app users only").
- **Cut third (saves 12m):** Lab 7.2 A moves to homework; do only its TODO 1 together (the additivity check and the base value in two units).

RUNNING SHORT: WHAT TO ADD

- Run Lab 7.1 B for everyone and debate the Asheville/Southeast and tier puzzles: why does a sensible model give "wrong-looking" coefficients?
- Lesson 7.1 Exercise 3 live: bin recency and compare the real churn rate with the PDP (0.133 → 0.416 between 91 to 150 and 151 to 250 days).
- Lesson 7.2 Exercise 1 live: the safest customer (C101889, 0.0002), and why protective reasons are not the riskiest customer's reasons reversed.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 2: Local Explanations, Fairness and K-Means

How to teach the second session of Advanced Machine Learning: explain one prediction with a flat local map (LIME), learn when not to trust it, and audit the churn campaign for fairness across age groups (7.3); then leave labels behind and find customer groups with K-Means, learning the algorithm, its pitfalls and why scaling is a choice (8.1).

2 lessons | 3 hrs session | Lessons 7.3-8.1 | Session 2 of 13

HOW TO USE THIS GUIDE

- Same rhythm as every session: warm-up → for each lesson: frame and mental model → live demo → guided lab → start the lesson project; exercises, quiz and journal go home.
- Every code block below was run to produce the output printed under it (Python 3.12, scikit-learn 1.9.1, shap 0.53, lime 0.2, 4 CPU threads). LIME is random: the outputs match because every explainer gets `random_state`.
- Keep `interpretability_lime.html` and `clustering_kmeans.html` open for the figures (LIME's bar chart, the four-panel Lloyd figure, the stuck run, raw vs scaled scatter, moons). Labs live in `labs/module-02/`.
- This session changes gear at the break: Module 7 ends with fairness, Module 8 starts unsupervised learning. Say so out loud; the second half has no target column at all.

> **THE #1 RISK OF SESSION 2: FAIRNESS BECOMES A DEBATE, OR A VERDICT**
>
> The age-group table invites strong reactions ("the model discriminates against young people!") and equally strong dismissals ("the model never sees age, so it is fair"). Both skip the lesson. Keep the room on the procedure: which harm, which metric, how many people per group, is the gap bigger than chance, what was not checked. The permutation test is the hinge: practice saying "no evidence of a difference, and not enough data to rule one out" before the session. Do not let the audit drift into whether Pinecrest should collect age at all; park it for the journal prompt.

BEFORE THE SESSION

- ☐ **Run the kit check:** `python3 labs/check_labs.py module-02` (8 files, about a minute). LIME and K-Means labs are all under 30 seconds.

- ☐ **Projector machine:** notebooks `lesson_7_3` and `lesson_8_1`; run `import lime, shap` once beforehand (first imports are slow). Browser tabs: the two lesson pages.
- ☐ **Downloads:** none. (Image data is needed from Session 5; remind anyone whose check said READY for Sessions 1 to 4.)
- ☐ **Board:** "fidelity · stability" (two checks before trusting LIME); the three fairness views (flag rate, recall, precision) with "which harm?" above them; for 8.1, a sketch of campfires: assign, move, repeat.
- ☐ **Sort the Session 1 exit tickets.** Question 2 (units) usually splits the room; W3 in the warm-up revisits it.

◆ KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
`ValueError: Domain error in arguments. The scale parameter must be positive ...`	Missing values in the data given to `LimeTabularExplainer`	Fill with training medians, in the explainer's data and in the row you explain.
`KeyError: 0` from `explain_instance`	A pandas row was passed	`.to_numpy()` on the row.
`NotImplementedError: LIME does not currently support classifier models without probability scores`	The function returns `predict` labels	Return `predict_proba(...)`, one column per class.
Learners' LIME weights differ from the slides in the second decimal	No `random_state`, or another lime version	`random_state=42` on the explainer; check `lime` 0.2.0.1 in `check_environment.py`.
A UserWarning about feature names from LIME runs	LIME passes NumPy arrays to a model fitted on a DataFrame	Harmless; the `predict_fn` that rebuilds a DataFrame avoids most of them.
`ValueError: could not convert string to float` or `Input X contains NaN` from KMeans	A text column or gaps in X	Cluster numeric behavior columns only (Common Mistakes 1 and 2).

SESSION 2 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Session 1 (odds ratios, permutation importance, SHAP units), homework check.
0:10	10	**7.3** Frame and mental model	The Earth looks flat from your street: a flat map of the model near one customer. Then a different question: fairness.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:20	25	**7.3** Live demo	LIME by hand, the lime library, fidelity and stability, a what-if, a fairness audit by age group, a permutation test, two errors.
0:45	20	**7.3** Guided lab	`labs/module-02/l7_3-lime-fairness/lime_fraud_starter.py`, then `labs/module-02/l7_3-lime-fairness/fairness_calibration_starter.py` in pairs.
1:05	15	**7.3** Start the lesson project	Auditing the retention campaign, Steps 1 and 2; Steps 3 to 5 go home.
1:20	10	Break	Leave "fidelity, stability" and "flag rate, recall, precision" on the board.
1:30	10	**8.1** Frame and mental model	No labels: campfires on a big campground. Assign, move, repeat.
1:40	30	**8.1** Live demo	A first K-Means, inertia by hand, Lloyd's loop, local minima and k-means++, scaling, new customers, three mistakes.
2:10	25	**8.1** Guided lab	`labs/module-02/l8_1-kmeans/scaling_choices_starter.py`, then `labs/module-02/l8_1-kmeans/payment_clusters_starter.py`.
2:35	15	**8.1** Start the lesson project	Pinecrest's first customer groups, Steps 1 to 3; Steps 4 to 6 go home.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 3.
Total	180		Ends at 3:00.

In the room. The lessons estimate 100 + 95 = 195 minutes alone; the room has 150 after the warm-up, break and wrap-up. Each lesson gets its mental model, demo, one guided lab and the start of its project.

Goes home (2 to 3 hours). The audit project Steps 3 to 5 and the customer-groups project Steps 4 to 6; Exercises 1 to 3 of each lesson; both quizzes; both journal entries; Lesson 8.1's "Before the Next Lesson" list.

Protect: 7.3 Steps 2, 6 and 7 (LIME by hand, the audit, the permutation test); 8.1 Steps 4, 5 and 8 (Lloyd's loop, local minima, raw vs scaled). **Cut first:** 7.3 Step 8 (errors: point at the trap table), then 8.1 Step 9 (new customers: Lesson 8.2 repeats it with a pipeline), then the fairness lab (B) moves to homework.

WARM-UP

0:00 Retrieval: Interpreting Models and SHAP

10 min

◆ QUESTIONS (ON THE SLIDE; LEARNERS ANSWER IN THE WORKBOOK)

W1

A scaled logistic regression gives `discount_share` a coefficient of 0.68. Say what that means in one sentence.

W2

Shuffling `days_since_last_order` in the TEST set lowers the boosting model's ROC AUC by 0.140. What does that measure, and why the test set?

W3

What does this print?

```
import numpy as np
from scipy.special import expit
base = -2.95                    # C104051's waterfall starts here (log-odds)
total_shares = 6.50             # and its 11 SHAP values add up to this
print(round(base + total_shares, 2), round(expit(base + total_shares), 3))
```

W4

Why should a briefing never rank features with a forest's `feature_importances_`?

W5

Which phrase can an agent read aloud: "you are leaving because you stopped buying" or "no order in 240 days"? Why?

ANSWERS

- **W1** One more standard deviation of discount share multiplies the odds of churn by e^0.68, about 2.0, other features held fixed.
- **W2** How much the model's held-out score depends on that feature (permutation importance); on training rows memorized noise would look important too.
- **W3** `3.55 0.972`: base + shares = 3.55 log-odds; `expit(3.55)` = 0.972, C104051's churn probability.
- **W4** It is measured on training data and favors many-valued columns: in Lesson 7.1 it ranked a column of pure noise sixth.
- **W5** "No order in 240 days": a neutral fact about the account. The other turns a model's association into a claim about the person's motives.

HOW TO RUN IT

Four minutes writing, four minutes cold-calling, two minutes on the homework: who finished both projects? Ask one learner to read their three reason codes for C104051 aloud, and the room to vote on whether an agent could say them. W3 is the units question from Session 1's exit ticket: if more than a third get it wrong, redo the end points on the board (add in log-odds, squash at the end).

LESSON PLANS

7.3 LIME and Fairness Checks

70 min

OBJECTIVES (FROM THE LESSON PAGE)

- Describe LIME's recipe (perturb, predict, weight by closeness, fit a simple model) and build a basic version by hand
- Explain a prediction with `lime.lime_tabular.LimeTabularExplainer` and judge it by its fidelity score and its stability across random seeds
- Say when LIME and SHAP answer different questions, and run simple what-if checks on one customer
- Run a fairness check: compare flag rate, recall and precision across age groups and test whether a gap is larger than chance

TIMING IN THE ROOM

10m	Frame: a flat map near one customer; fidelity and stability; then: does the model treat groups alike?
25m	Live demo: LIME by hand, the library, stability, a what-if, the age audit, a permutation test, two errors
20m	Guided lab: `lime_fraud_starter.py` (12m), then `fairness_calibration_starter.py` TODO 1 and 2 in pairs (8m)
15m	Start the lesson project: auditing the retention campaign, Steps 1 and 2

MENTAL MODEL TO SAY

"The Earth is round, but a map of your neighborhood can be flat and still be accurate, because you only use it nearby. LIME draws a flat map of the model around one customer: make thousands of nearby variants, ask the model about each, weight them by closeness, fit a straight line. Useful for 'what pushes this customer's risk around?'; misleading if you carry it across the country."

Then the second half, a different question entirely: "Explanations tell you how a model decides. A fairness check asks whether its decisions land differently on different groups of people. Not using a sensitive attribute guarantees nothing; you have to measure, and measure honestly when groups are small."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook, `lesson_7_3`.

1. **Same model, same customer (2m).** "Lesson 7.2's model and its riskiest customer. This time we split the whole table, because we will need the age groups later."

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import HistGradientBoostingClassifier

customers = pd.read_csv("data/customers_churn.csv")
features = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value", "da
            "returns_12m", "return_rate", "support_tickets_12m", "discount_sha
            "email_opens_90d"]
train, test = train_test_split(customers, test_size=0.25, stratify=customers["
X_train, X_test = train[features].astype(float), test[features].astype(float)

model = HistGradientBoostingClassifier(random_state=42).fit(X_train, train["ch
proba = model.predict_proba(X_test)[:, 1]
i = int(proba.argmax())
x0 = X_test.iloc[i]
print(test["customer_id"].iloc[i], f"churn probability {proba[i]:.3f}")
```

```
C104051 churn probability 0.972
```

2 **LIME by hand: the flat map (5m).** "Make 3,000 variants of this customer, ask the model about each, weight each by how close it is, fit a weighted straight line. The line's slopes are the explanation." Walk the four numbered comments.

```
from sklearn.linear_model import Ridge

rng = np.random.default_rng(42)
sd = X_train.std()

# 1. variants near the customer, kept inside the training range
noise = rng.normal(size=(3000, len(features)))
variants = pd.DataFrame(x0.to_numpy() + noise * sd.to_numpy(), columns=feature
variants = variants.clip(lower=X_train.min(), upper=X_train.max(), axis=1)
variants["app_user"] = (variants["app_user"] > 0.5).astype(float)

# 2. ask the model
scores = model.predict_proba(variants)[:, 1]

# 3. weight by closeness (distance in standard deviations)
steps = (variants - x0) / sd
distance = np.sqrt((steps ** 2).sum(axis=1))
width = 0.75 * np.sqrt(len(features))        # LIME's default kernel width
weights = np.exp(-distance ** 2 / width ** 2)

# 4. fit a simple weighted model: its slopes are the explanation
surrogate = Ridge(alpha=1.0).fit(steps, scores, sample_weight=weights)
local = pd.Series(surrogate.coef_, index=features).sort_values(key=abs, ascend
print(local.round(3).head(5))
print(f"fidelity (weighted R2): {surrogate.score(steps, scores, sample_weight=
print(f"surrogate at the customer: {surrogate.intercept_:.3f}   model: {proba[
```

```
discount_share           0.086
tenure_months            0.080
app_user                -0.059
days_since_last_order    0.050
orders_12m              -0.016
dtype: float64
fidelity (weighted R2): 0.496
surrogate at the customer: 0.897   model: 0.972
```

The surprise: recency is NOT the top slope, though SHAP gave it the biggest share. Ask why before you explain: at 364 days the risk curve is already flat, so nudging recency changes little; SHAP measures the gap from a typical customer, LIME the sensitivity nearby. Both right, different questions. Fidelity 0.50 means "roughly".

3 **The lime library (3m).** "The package wraps the same recipe. Two details: fill missing values first, and give it a function that returns probabilities."

```
from lime.lime_tabular import LimeTabularExplainer

fill = X_train.median()

def predict_fn(rows):
    return model.predict_proba(pd.DataFrame(rows, columns=features))

lime_explainer = LimeTabularExplainer(
    X_train.fillna(fill).to_numpy(), feature_names=features,
    class_names=["stayed", "churned"], mode="classification", random_state=42,
)
exp = lime_explainer.explain_instance(x0.fillna(fill).to_numpy(), predict_fn,

for rule, weight in exp.as_list():
    print(f"{rule:35s} {weight:+.3f}")
print(f"fidelity {exp.score:.3f}   surrogate prediction {exp.local_pred[0]:.3f
```

```
days_since_last_order > 110.00      +0.295
discount_share > 0.29               +0.238
39.00 < tenure_months <= 68.00      -0.069
support_tickets_12m > 1.00          +0.066
app_user <= 0.00                    +0.060
fidelity 0.512   surrogate prediction 0.580   model 0.972
```

Binned rules ("days > 110") turn the question back to "what about this customer is unusual and risky?", so the ranking resembles SHAP's. Point at the last line: the surrogate says 0.58 for a 0.97 customer.

4 **Is it stable? (3m).** "LIME is random. Rerun with three seeds, at 300 and at 5,000 variants."

```
for n_samples in [300, 5000]:
    print(f"num_samples={n_samples}")
    for seed in [1, 2, 3]:
        explainer_s = LimeTabularExplainer(X_train.fillna(fill).to_numpy(), fe
                                           class_names=["stayed", "churned"],
        e = explainer_s.explain_instance(x0.fillna(fill).to_numpy(), predict_f
                                         num_samples=n_samples)
        print(f"  seed {seed}:", " | ".join(f"{rule} {w:+.2f}" for rule, w in
```

```
num_samples=300
  seed 1: days_since_last_order > 110.00 +0.29 | discount_share > 0.29 +0.27
  seed 2: days_since_last_order > 110.00 +0.29 | discount_share > 0.29 +0.21
  seed 3: days_since_last_order > 110.00 +0.29 | discount_share > 0.29 +0.25
num_samples=5000
  seed 1: days_since_last_order > 110.00 +0.30 | discount_share > 0.29 +0.24
  seed 2: days_since_last_order > 110.00 +0.30 | discount_share > 0.29 +0.24
  seed 3: days_since_last_order > 110.00 +0.30 | discount_share > 0.29 +0.26
```

The top two hold everywhere; the third reason wanders at 300 variants and settles at 5,000. On the board: **fidelity** and **stability**, the two checks before trusting a LIME explanation.

5 **A what-if check (2m).** "The simplest local explanation: change one thing and ask again."

```
what_if = X_test.iloc[[i]].copy()
for days in [364, 200, 150, 120, 60]:
    what_if["days_since_last_order"] = days
    print(f"{days:3d} days -> predicted churn {model.predict_proba(what_if)[0,
```

```
364 days -> predicted churn 0.972
200 days -> predicted churn 0.899
150 days -> predicted churn 0.716
120 days -> predicted churn 0.619
 60 days -> predicted churn 0.645
```

Even at 60 days the model says 0.645: discounts and tickets still hold the risk up. And a what-if describes the model's answer, not what the customer would do.

6 **A fairness audit by age group (4m).** "The model never sees age. That settles nothing: other features can act as proxies. Age is used here only to audit, never as an input." Write the three definitions on the board: flag rate, recall, precision.

```
audit = test[["customer_id", "age_group", "churned"]].copy()
audit["age"] = audit["age_group"].fillna("unknown")
audit["p"] = proba
audit["flag"] = audit["p"] >= audit["p"].quantile(0.80)   # calls for the top

by_age = audit.groupby("age").agg(customers=("churned", "size"), churners=("ch
                                  churn_rate=("churned", "mean"), flag_rate=("
by_age["recall"] = audit[audit["churned"] == 1].groupby("age")["flag"].mean()
by_age["precision"] = audit[audit["flag"]].groupby("age")["churned"].mean()
print(by_age.round(3))
```

```
         customers  churners  churn_rate  flag_rate  recall  precision
age
18-24          162        28       0.173      0.173   0.536      0.536
25-34          309        55       0.178      0.184   0.564      0.544
35-44          295        51       0.173      0.173   0.569      0.569
45-54          255        31       0.122      0.212   0.710      0.407
55-64          204        34       0.167      0.255   0.765      0.500
65+            147        24       0.163      0.211   0.708      0.548
unknown        128        25       0.195      0.211   0.680      0.630
```

77% of churners aged 55 to 64 are reached, 54% of those aged 18 to 24: a 23-point gap. Before anyone reacts, point at the churners column: 28 people.

7 **Is the gap more than chance? (3m).** "Shuffle the age labels among the churners, so age has nothing to do with being flagged, and see how big the gap gets by chance. 2,000 times."

```
def recall_gap(frame):
    recall = frame.groupby("age")["flag"].mean()
    return recall.max() - recall.min()

churners = audit[audit["churned"] == 1]
observed = recall_gap(churners)

rng = np.random.default_rng(42)
shuffled_gaps = np.array([recall_gap(churners.assign(age=rng.permutation(churn
                         for _ in range(2000)])
print(f"observed recall gap: {observed:.3f}")
print(f"typical gap by chance (median): {np.median(shuffled_gaps):.3f}")
print(f"share of shuffles with a gap at least as large: {(shuffled_gaps >= obs
```

```
observed recall gap: 0.229
typical gap by chance (median): 0.223
share of shuffles with a gap at least as large: 0.473
```

Half the shuffles produce a gap this large. The honest sentence: no evidence that recall differs by age, and not enough data to rule out a moderate difference.

8 **Errors on purpose (3m).** "Two LIME errors whose messages do not say what is wrong."

```
broken = LimeTabularExplainer(X_train.to_numpy(), feature_names=features, rand
broken.explain_instance(x0.fillna(fill).to_numpy(), predict_fn)

lime_explainer.explain_instance(x0.fillna(fill).to_numpy(),
                                lambda rows: model.predict(pd.DataFrame(rows,
```

```
ValueError: Domain error in arguments. The `scale` parameter must be positive
NotImplementedError: LIME does not currently support classifier models without
```

The first is missing values in `email_opens_90d` (LIME's summary statistics become NaN); the second is `predict` instead of `predict_proba`. A third (a pandas row instead of `.to_numpy()`) raises `KeyError: 0`: it is in `debug_lime.py`.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Trusting a LIME explanation without its two checks.** Coach: "What is `exp.score`? Is `exp.local_pred` on the same side of your threshold as the model? Does the top reason survive another seed?" The fraud lab makes this vivid: fidelity 0.12, surrogate 0.077 for a 0.990 payment.
- **"We removed age, so the model is fair."** Coach with the audit table: the model never saw age, and recall still varies from 54% to 77%. Whether that is chance is a separate question; you only know because you measured.
- **A gap read as a verdict, or a non-significant test read as proof.** Coach the two-part sentence: "no evidence of a difference" AND "groups too small to rule one out". Ask for group sizes every time a rate is quoted.

GUIDED LAB

- **Lab A (12m, everyone):** `labs/module-02/l7_3-lime-fairness/lime_fraud_starter.py`; reference `labs/module-02/l7_3-lime-fairness/lime_fraud_solution.py`. LIME meets a rare class: three fraud payments at different risk levels. **Expected:** the riskiest payment (T031633, 0.990) gets velocity and country-mismatch rules with fidelity 0.12 and a surrogate of 0.077; at 500 variants the top rule agrees in 3, 2 and 1 of 3 seeds, at 5,000 in 2, 3 and 3, fidelities 0.05 to 0.35; SHAP names velocity, velocity, new device.
- **Debrief Lab A (2m):** "Why does the flat map fail here when it worked roughly on churn?" Fraud is 0.88% of the data: almost every random variant of a payment looks legitimate, so the neighborhood the line is fitted to is nearly flat at zero. For this model, report SHAP.
- **Lab B (8m, pairs, TODO 1 and 2 in class):** `labs/module-02/l7_3-lime-fairness/fairness_calibration_starter.py`; reference `labs/module-02/l7_3-lime-fairness/fairness_calibration_solution.py`. Two views the lesson did not take: calibration by group and campaign profit by group, at the $15 / $60 break-even (which flags exactly the same 300 customers as the lesson's top 20%). **Expected:** gaps from -0.042 (35 to 44) to +0.025 (45 to 54); 45 to 54 earns $2.00 per customer, unknown $4.80; the largest gap is matched by 64% of shuffles. TODO 3 and 4 go home.
- **Fast finishers:** `labs/module-02/l7_3-lime-fairness/debug_lime.py`: three errors whose messages hide the cause, and one silent problem (100 variants).

START THE LESSON PROJECT

The lesson project is **auditing the retention campaign**: LIME on three flagged customers with stability checks, flag rates by age with a permutation test, saved as `fairness_audit.csv`. In the room, Steps 1 and 2 (15 minutes).

- Step 1: train and flag the top 20%. **Done** = `flagged 300 of 1500; churners among them 52.3%`.
- Step 2: three customers at different risk levels. **Done** = C104051 fidelity 0.48, C103674 0.36 (its surrogate lands on the other side of 0.5), C104266 0.27. Ask: "Which of these three explanations would you give an agent?"
- **Goes home:** Step 3 (stability: 5 of 5 seeds for all three), Step 4 (flag rates by age, permutation p = 0.395), Step 5 (`fairness_audit.csv` and a note that lists what was NOT checked).

CHECKS FOR UNDERSTANDING

Ask these out loud:

- What are LIME's four steps? **Answer:** perturb, predict, weight by closeness, fit a simple weighted model.
- Why can LIME's top slope differ from SHAP's top share? **Answer:** the slope is sensitivity nearby; the share is the contribution to the gap from a typical customer. At 364 days the curve is flat, so the slope is small and the share large.
- Which fairness view matches "a churner who misses out on an offer"? **Answer:** recall by group (equal opportunity).
- What does a permutation test of a recall gap shuffle? **Answer:** the group labels among the churners, so group has nothing to do with being flagged.

Workbook self-check, items Q1 to Q6.

Answers: **Q1** A) Treat it as unreliable: the flat map does not imitate the model near this payment; report SHAP's reasons, or fewer reasons; **Q2** `[1. 0.368 0.018]`; **Q3** C) A single top reason hides the rest: discounts and tickets still hold the risk up, and the what-if describes the model, not the customer; **Q4** B) Recall among churners (equal opportunity); **Q5** `[0.5, 0.67]`; **Q6** D) No evidence of a real calibration difference at this sample size; the groups are small, so keep monitoring. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: a card with the two LIME checks and the three fairness definitions. In Lab A, let pairs skip the 500-variant loop and only compare seeds at 5,000.

Stretch: Lesson 7.3 Exercise 4 (a counterfactual: the smallest change that flips C104051 below 0.5) and the Bonus Challenge (the recall audit at 10%, 20% and 40% of customers called).

8.1 K-Means Clustering

80 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain what clustering is for and how it differs from the supervised models of Modules 1 to 7
- Describe Lloyd's algorithm (assign each point to the nearest center, move each center to the mean) and write it in NumPy
- Fit `KMeans` and read `labels_`, `cluster_centers_`, `inertia_` and `n_iter_`
- Explain why K-Means needs scaled features and several starts (`n_init`, k-means++), and show what goes wrong without them
- Profile clusters in the original units and recognize the cluster shapes K-Means cannot find

TIMING IN THE ROOM

10m	Frame: no answer column; campfires on a campground; inertia
30m	Live demo: blobs, K-Means, inertia by hand, Lloyd's loop, local minima, k-means++, raw vs scaled, new customers, mistakes
25m	Guided lab: `scaling_choices_starter.py` (12m), then `payment_clusters_starter.py` (13m)
15m	Start the lesson project: Pinecrest's first customer groups, Steps 1 to 3

MENTAL MODEL TO SAY

"Every model so far learned from examples with an answer. Marketing now asks what kinds of customers Pinecrest has, and nobody has labeled them. Picture a campground at night: hundreds of hikers and k campfires lit at random spots. Each hiker walks to the nearest fire. Each fire is moved to the middle of its crowd. Some hikers are now closer to another fire and walk over; the fires move again. After a few rounds nobody switches."

Then on the board: fires = **centroids**; walk to the nearest = **assign**; move to the middle = **update**; k = the number of fires, **you** choose it (Lesson 8.2 asks how). The one number each round lowers: **inertia**, the total squared distance to the own center. And the warning: "K-Means always answers, even in data with no groups. Checking the answer is your job."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook, `lesson_8_1`.

1. **Data where we know groups exist (2m).** "Before real customers, round clouds where we know the answer. The underscore throws the true labels away: a clustering algorithm never sees them."

```
import numpy as np
import pandas as pd
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

X_toy, _ = make_blobs(n_samples=300, centers=4, cluster_std=1.0, random_state=
print(X_toy.shape)
print(X_toy[:3].round(2))
```

```
(300, 2)
[[-9.3   6.47]
 [-9.7   6.94]
 [-1.69  7.79]]
```

2 **A first K-Means (3m).** "An estimator like any other, except that `fit` takes only X."

```
km = KMeans(n_clusters=4, n_init=10, random_state=42)
km.fit(X_toy)

print(km.labels_[:12])
print(km.cluster_centers_.round(2))
print(f"inertia: {km.inertia_:.1f}   iterations: {km.n_iter_}")
```

```
[3 3 0 1 3 1 2 1 0 2 0 2]
[[-2.71  8.97]
 [-6.83 -6.83]
 [ 4.72  2.04]
 [-8.87  7.17]]
inertia: 564.9   iterations: 2
```

Four learned attributes, trailing underscores as always. Labels are names, not ranks.

3 **Inertia is not magic (2m).** "Each point's own center, subtract, square, add up."

```
own_center = km.cluster_centers_[km.labels_]      # one row per point: the cen
inertia_by_hand = ((X_toy - own_center) ** 2).sum()
print(f"{inertia_by_hand:.1f}")
```

```
564.9
```

`cluster_centers_[labels_]` uses the 300 labels as row numbers into the 4-row table of centers.

4 **Lloyd's loop in fifteen lines (5m).** "Assign, move, repeat until nothing moves." Narrate the broadcasting line once: every point minus every center at once, shape (points, k, features).

```
def lloyd(X, centers, max_steps=20):
    """Plain K-Means: assign each point to its nearest center, move each cente
    history = [centers]
    for _ in range(max_steps):
        sq_dist = ((X[:, None, :] - centers[None, :, :]) ** 2).sum(axis=2)   #
        labels = sq_dist.argmin(axis=1)                                      #
        new_centers = np.array([X[labels == j].mean(axis=0) for j in range(len
        history.append(new_centers)
        if np.allclose(new_centers, centers):                                #
            break
        centers = new_centers
    return labels, centers, history

rng = np.random.default_rng(12)     # a seed whose random start is clearly poo
start = X_toy[rng.choice(len(X_toy), size=4, replace=False)]     # 4 random po
labels, centers, history = lloyd(X_toy, start)

my_inertia = ((X_toy - centers[labels]) ** 2).sum()
print(f"rounds: {len(history) - 1}   inertia: {my_inertia:.1f}")
```

```
rounds: 6   inertia: 564.9
```

Show the lesson page's four-panel figure of the centers drifting. Same inertia as scikit-learn.

5 **Local minima (3m).** "Downhill from where you start is not the lowest point there is. Five purely random starts:"

```
for seed in range(5):
    one_run = KMeans(n_clusters=4, init="random", n_init=1, random_state=seed)
    print(f"seed {seed}: inertia {one_run.inertia_:7.1f}   rounds {one_run.n_i
```

```
seed 0: inertia   564.9   rounds 3
seed 1: inertia  8422.6   rounds 4
seed 2: inertia  2062.1   rounds 4
seed 3: inertia  2059.1   rounds 2
seed 4: inertia   564.9   rounds 2
```

Same data, same k: 565 to 8,423. Show the "stuck run" figure: one center between two clouds, two splitting one.

6 **The two defenses (2m).** "k-means++ spreads the starting centers out; `n_init` runs several starts and keeps the best."

```
for seed in range(5):
    one_run = KMeans(n_clusters=4, init="k-means++", n_init=1, random_state=se
    print(f"seed {seed}: inertia {one_run.inertia_:7.1f}")

print("default n_init:", KMeans(n_clusters=4).n_init)
```

```
seed 0: inertia   564.9
seed 1: inertia   564.9
seed 2: inertia   564.9
seed 3: inertia   564.9
seed 4: inertia   564.9
default n_init: auto
```

`auto` means one run with k-means++. The course writes `n_init=10` whenever the result matters.

7 **Real customers, two columns (2m).** "Orders run from 1 to 21; order values from about $30 to $625."

```
customers = pd.read_csv("data/customers_churn.csv")
two = customers[["orders_12m", "avg_order_value"]]
print(two.describe().loc[["mean", "std", "min", "max"]].round(1))
```

```
      orders_12m  avg_order_value
mean         5.0            140.9
std          3.2            103.6
min          1.0             29.8
max         21.0            625.2
```

Ask: "K-Means measures straight-line distance. Which column will decide the clusters?"

8 **Raw versus scaled (5m).** "Cluster the raw numbers, then the standardized ones, and read both sets of centers in dollars and orders."

```
from sklearn.preprocessing import StandardScaler

raw_km = KMeans(n_clusters=4, n_init=10, random_state=42).fit(two)

scaler2 = StandardScaler().set_output(transform="pandas")
two_scaled = scaler2.fit_transform(two)
scaled_km = KMeans(n_clusters=4, n_init=10, random_state=42).fit(two_scaled)

print("raw centers:")
print(pd.DataFrame(raw_km.cluster_centers_, columns=two.columns).round(1).sort
print("scaled centers, back in original units:")
print(pd.DataFrame(scaler2.inverse_transform(scaled_km.cluster_centers_), colu
      .round(1).sort_values("avg_order_value"))
```

```
raw centers:
   orders_12m  avg_order_value
0         5.5             80.5
2         5.7            163.7
1         2.7            300.6
3         2.4            423.2
scaled centers, back in original units:
   orders_12m  avg_order_value
2         2.4             94.4
3        10.8             96.7
0         6.4             98.7
1         2.6            332.3
```

Raw: bands of dollars ($80, $160, $300, $420); orders had no say. Scaled: three groups with similar baskets separated by how often they buy, and one big-ticket group. Show the lesson's side-by-side scatter. Scaling is a choice about what "similar" means.

9 **New customers (3m).** "`predict` gives the nearest center; `transform` gives the distance to every center."

```
newcomers = pd.DataFrame({"orders_12m": [12, 2, 8.6], "avg_order_value": [80.0
newcomers_scaled = scaler2.transform(newcomers)

print(scaled_km.predict(newcomers_scaled))
print(scaled_km.transform(newcomers_scaled).round(2))
```

```
[3 1 3]
[[1.72 3.79 2.96 0.41]
 [3.66 1.15 3.44 4.35]
 [0.67 2.93 1.91 0.67]]
```

The third newcomer is 0.67 from two centers: K-Means gives a confident single label anyway. Gaussian mixtures (Lesson 8.5) soften that.

10 **Three mistakes (3m).** "Two errors, and one silent trap."

```
KMeans(n_clusters=3, n_init=10, random_state=42).fit(customers[["orders_12m",

KMeans(n_clusters=3, n_init=10, random_state=42).fit(customers[["orders_12m",

rerun = KMeans(n_clusters=4, n_init=10, random_state=7).fit(two_scaled)
print(pd.crosstab(scaled_km.labels_, rerun.labels_, rownames=["seed 42"], coln
```

```
ValueError: could not convert string to float: 'Bend'
ValueError: Input X contains NaN.
seed 7      0     1     2    3
seed 42
0        1978     0     0    0
1           0  1128     0    0
2           0     0  1993    0
3           0     0     0  901
```

Text and gaps raise errors. The crosstab is the trap: the same four groups with shuffled numbers. Code that says "cluster 2 is our premium segment" breaks when anything changes. Then show the lesson's moons / stretched / unequal-spread figure in one sentence: K-Means always answers, checking is your job.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Clustering unscaled columns.** No error; the biggest numbers decide. Coach: "Read the centers in real units. Does every column vary across clusters?" Lab 8.1 B shows it on recency, spend and tickets; Lab 8.1 A on payments.
- **Reading scaled centers as real values.** "0.44 orders of -0.41 dollars." Coach: `inverse_transform`, or simply a groupby of the original columns by label.
- **Treating cluster numbers as names.** Code that hard-codes "cluster 4" breaks when the seed or the data change (`debug_kmeans.py` has exactly this bug). Coach: name clusters from their profiles.

GUIDED LAB

- **Lab B first (12m, everyone):** `labs/module-02/l8_1-kmeans/scaling_choices_starter.py`; reference `labs/module-02/l8_1-kmeans/scaling_choices_solution.py`. Four groups from recency, spend and tickets, three ways. **Expected:** raw = spend bands ($236 to $1,692, churn 4% to 25%); standardized finds 780 silent customers (271 days, churn 51%) and 1,201 who complain (2.5 tickets, churn 22%); recency counted twice gives 539 at 307 days (churn 53%); the crosstab shows different groupings, not renamed ones.

- **Lab A (13m):** `labs/module-02/l8_1-kmeans/payment_clusters_starter.py`; reference `labs/module-02/l8_1-kmeans/payment_clusters_solution.py`. 50,000 payments. **Expected:** raw = bands of account age; after log + scale four named groups, fraud 0.27% (small everyday orders) to 2.13% (busy cards, 243 of the 438 frauds); inertia 97,067 for every seed.
- **Debrief (2m):** "Is 'Busy cards' a fraud detector?" No: 97.9% of its payments are legitimate. Differences in an outcome the clustering never saw show the groups are real; detecting rare events is Lesson 8.6's job.
- **Fast finishers:** `labs/module-02/l8_1-kmeans/debug_kmeans.py`: a text column, a gap, scaled centers shown as real ones, and a hard-coded cluster number.

START THE LESSON PROJECT

The lesson project is **Pinecrest's first customer groups**: five clusters on four behavior columns, profiled in plain units, with churn rates, saved as `customer_groups.csv`. In the room, Steps 1 to 3 (15 minutes).

- Step 1: `(6000, 4) missing: 0`. Step 2: `inertia 5713 after 5 rounds`. Step 3: the same 5713 for seeds 0, 1 and 2.
- Before anyone profiles, ask the room for the Before-the-Next-Lesson guess from 7.3: "which three or four types of customers do you picture?" Write them on the board; compare at the next session.
- **Goes home:** Step 4 (the profile: churn 3% to 32% across groups, though churn played no part), Step 5 (names: Discount Seekers 0.317 ... Loyal Regulars 0.034), Step 6 (`customer_groups.csv`, 6,000 rows).

CHECKS FOR UNDERSTANDING

Ask these out loud:

- What two steps does Lloyd's loop repeat, and what number can only go down? **Answer:** assign to the nearest center, move each center to its points' mean; inertia.
- Why scale before K-Means? **Answer:** distances depend on units; without scaling the column with the largest spread decides the clusters.
- What do k-means++ and `n_init` protect against? **Answer:** local minima from a poor start.
- Why is there no train/test split in this lesson? **Answer:** there are no answers to check predictions against; we describe the customers we have. (Stability checks come in 8.2.)

Workbook self-check, items Q7 to Q12.

Answers: **Q7** `10.0`; **Q8** D) Distances are dominated by the column with the largest spread: account age varies by about 700 days, items by about 1; **Q9** A) Runs the whole algorithm from ten starts and keeps the run with the lowest inertia; **Q10** B) Pass the fitted scaler's `transform` of their rows to the fitted model's `predict`; **Q11** `[[6.449]]`; **Q12** B) The groups differ in fraud, which shows they capture real behavior, but 97.9% of busy-card payments are legitimate: a cluster describes, it does not detect. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: a card with the campfire picture and the two steps. In Lab B, give struggling learners the `profile` function's output to read first, before they fit anything.

Stretch: Lesson 8.1 Exercise 4 (write k-means++ seeding) and the Bonus Challenge (color quantization of a picture drawn with NumPy).

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

1. Name the two checks before you trust a LIME explanation, and how you run each. **Answer:** fidelity (`exp.score`, and `exp.local_pred` on the same side of the threshold as the model) and stability (rerun with other seeds; the top reasons should hold).
2. The churn model never sees age. Why audit by age anyway, and which metric fits "a churner misses out on an offer"? **Answer:** other features can act as proxies, so outcomes can still differ; recall by group (equal opportunity), with group sizes and a permutation test.
3. Why scale features before K-Means, and what does `n_init=10` do? **Answer:** distances depend on units, so the largest-spread column would decide the clusters; ten starts, keeping the lowest-inertia run, to avoid a poor local minimum.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** The campaign audit (Steps 3 to 5: stability, flag rates with a permutation test, `fairness_audit.csv`) and the customer groups (Steps 4 to 6: profile, names, `customer_groups.csv`).
- **Lesson exercises.** 7.3: Exercises 1 to 3; 8.1: Exercises 1 to 3. Exercise 4 of each is a stretch.
- **Both Quick Quizzes** and **both journal entries** (the 7.3 prompt: report a 23-point gap and its permutation test to a manager in three sentences).
- **Lab 7.3 B** TODO 3 and 4 if not done in class.
- **Before the Next Lesson (from 8.1):** keep the notebook and `customer_groups.csv`; plot histograms of `avg_order_value` and `orders_12m` (symmetric or skewed?); rerun the project with 4 and 6 clusters and note which named groups merge or split.
- **Image data:** anyone whose check said READY for Sessions 1 to 4 runs it with `--download` before Session 5.

BRING LIST FOR SESSION 3

- A READY laptop; the `pinecrest-ml` folder with `data/` and `labs/` (Session 3's starters are in `labs/module-03/`).
- The Lesson 8.1 project notebook and `customer_groups.csv`: Lesson 8.2 asks whether five groups was the right number.

- Your histograms and your k = 4 / k = 6 notes from 8.1's Before the Next Lesson.
- The workbook and the journal. No downloads needed for Session 3.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** LIME by hand and its fidelity; the audit and the permutation test; Lloyd's loop; raw vs scaled.
- **Cut first (saves 3m):** 7.3 Step 8 (errors): point at the trap table.
- **Cut second (saves 3m):** 8.1 Step 9 (new customers): Lesson 8.2 repeats it with a pipeline.
- **Cut third (saves 8m):** Lab 7.3 B moves to homework in full.

RUNNING SHORT: WHAT TO ADD

- Lab 7.3 B's TODO 3 for everyone: a permutation test on a calibration gap, and the two-sentence report.
- Lesson 8.1 Exercise 2 live: borderline customers (114 within 0.1 of a second group).
- The workbook's worked example on Lloyd's loop from a poor start (the 30 that never gets its own cluster).

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 3: Choosing k and Hierarchical Clustering

How to teach the third session of Advanced Machine Learning: decide how many clusters the data supports from four kinds of evidence and turn them into segments with honest limits (8.2); then build a whole tree of groupings, choose a linkage and a distance that fit the question, and know when the cost rules it out (8.3).

2 lessons | 3 hrs session | Lessons 8.2-8.3 | Session 3 of 13

HOW TO USE THIS GUIDE

- Same rhythm: warm-up → for each lesson: frame → live demo → guided lab → start the lesson project; exercises, quiz and journal go home.
- Every code block below was run to produce its output (scikit-learn 1.9.1, SciPy 1.18, 4 threads). Plots stay on the lesson pages; the demo types the numbers behind them.
- Keep `clustering_kmeans_optimization.html` and `clustering_hierarchical.html` open for the elbow/silhouette figures, the silhouette plot, the dendrograms and the 2 x 4 linkage figure. Labs live in `labs/module-03/`.
- Lesson 8.2's MiniBatchKMeans section (300,000 rows, timings that differ by machine) is shown from the lesson page, not typed: it takes minutes under load and its point is one sentence.

> **THE #1 RISK OF SESSION 3: "SO WHAT IS THE RIGHT K?"**
> Learners want a number from a formula. The lesson's answer is a habit: collect several kinds of evidence (elbow, silhouette, stability, usefulness), look for agreement, and let the business choose within the range the data supports, in writing. Lab 8.2 A is built for that conversation: five and six are both defensible for the activity shares. Resist settling it for them; ask which groups a merchandising team would treat differently.

BEFORE THE SESSION

- ☐ **Run the kit check:** `python3 labs/check_labs.py module-03` (8 files, under a minute).
- ☐ **Projector machine:** notebooks `lesson_8_2` and `lesson_8_3`. The 8.2 stability loop takes a few seconds; the elbow loop computes eight silhouettes on 6,000 customers (also a few seconds). Browser tabs: the two lesson pages.
- ☐ **Downloads:** none. Remind anyone at READY for Sessions 1 to 4 that Session 5 is the last call for the image

download.

- ☐ **Board:** the four kinds of evidence (elbow · silhouette · stability · usefulness) with an empty "k =" next to each; for 8.3, a closet with nested boxes and the linkage table (single, complete, average, Ward).
- ☐ **Collect from Session 2:** learners' histograms of `avg_order_value` and `orders_12m` and their k = 4 / 6 notes from 8.1's Before the Next Lesson (both on Session 2's bring list). They come back in the 8.2 frame.

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
`TypeError: ... unexpected keyword argument 'affinity'`	An older tutorial	`metric=`.
`ValueError: ... Ward can only work with euclidean distances`	Ward with cosine, correlation or precomputed distances	Euclidean with Ward, or `linkage="average"`.
`ValueError: Exactly one of n_clusters and distance_threshold has to be set`	Both given (the default `n_clusters=2` counts)	`n_clusters=None` with `distance_threshold`.
`ValueError: Number of labels is 1`	A silhouette for k = 1	Start the k loop at 2.
The elbow loop seems to hang	Eight silhouettes on 6,000 rows, plus 80 K-Means fits	Wait a few seconds; on very slow laptops pass `sample_size=2000, random_state=42` to `silhouette_score`.
A dendrogram of 6,000 leaves is a black smear	Every leaf drawn	`truncate_mode="lastp", p=25` (the lesson does this).

SESSION 3 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Session 2 (LIME, fairness, K-Means), homework check.
0:10	10	**8.2** Frame and mental model	Sorting the returns pile: cohesion, separation, stability, usefulness.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:20	30	**8.2** Live demo	Skew and logs, the elbow, silhouettes by hand and per cluster, ARI, bootstrap stability, profiles and names, three mistakes.
0:50	20	**8.2** Guided lab	`labs/module-03/l8_2-choosing-k/choose_k_tastes_starter.py`: how many kinds of shoppers, by what they buy?
1:10	15	**8.2** Start the lesson project	The segment brief, Steps 1 to 3; Steps 4 to 6 go home.
1:25	10	Break	Leave the four kinds of evidence on the board.
1:35	10	**8.3** Frame and mental model	Tidying the gear closet: merge the two closest, record the height, repeat.
1:45	25	**8.3** Live demo	Six points and a linkage matrix, four linkages, cutting by height, Ward vs K-Means on customers, correlation distance, five mistakes.
2:10	25	**8.3** Guided lab	`labs/module-03/l8_3-hierarchical/seasonal_families_starter.py`, then `labs/module-03/l8_3-hierarchical/sample_then_assign_starter.py`.
2:35	15	**8.3** Start the lesson project	A product family tree, Steps 1 to 3; Steps 4 to 6 go home.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 4.
Total	180		Ends at 3:00.

In the room. The lessons estimate 100 + 95 = 195 minutes alone; the room has 150. Each lesson gets its mental model, demo, one guided lab and the start of its project.

Goes home (2 to 3 hours). The segment brief Steps 4 to 6 and the product family tree Steps 4 to 6; Exercises 1 to 3 of each lesson; both quizzes; both journal entries; Lesson 8.3's "Before the Next Lesson" list.

Protect: 8.2 Steps 3, 5 and 7 (elbow and silhouette, the per-cluster view, stability); 8.3 Steps 1, 3 and 7 (the linkage matrix, the four linkages, correlation distance). **Cut first:** 8.2 Step 6 (ARI examples: one sentence), then 8.3 Step 4 (cut by height: the customers' Ward tree shows the same idea), then Lab 8.3 B moves to homework.

WARM-UP

0:00 Retrieval: LIME, Fairness and K-Means

10 min

QUESTIONS (ON THE SLIDE; LEARNERS ANSWER IN THE WORKBOOK)

W1

Name LIME's two checks before you trust an explanation.

W2

Recall by age group ranges from 54% to 77% and a permutation test finds a gap this large in 47% of shuffles. Say the honest conclusion in one sentence.

W3

What does this print?

```
from sklearn.cluster import KMeans
km = KMeans(n_clusters=2, n_init=10, random_state=42).fit([[0], [4], [10], [14
print(km.inertia_)
```

W4

Why scale before K-Means?

W5

Yesterday cluster 3 was the discount segment; after a refit it is cluster 1. What happened?

ANSWERS

- **W1** Fidelity (`exp.score` , and the surrogate's prediction close to the model) and stability (the top reasons survive other seeds).
- **W2** No evidence that recall differs by age group, and the groups are too small to rule out a moderate difference.
- **W3** `16.0` : clusters {0, 4} and {10, 14}, centers 2 and 12; each point is 2 away: $4 \times 2^2 = 16$.
- **W4** Distances depend on units: the column with the largest spread would decide the clusters (dollars over orders, account age over items).
- **W5** Probably nothing: cluster numbers are arbitrary names. Name clusters from their profiles.

HOW TO RUN IT

Four minutes writing, four minutes cold-calling, two minutes on the homework: who finished `fairness_audit.csv` and `customer_groups.csv` ? Ask one learner to read the "Not checked" line of their audit note. W2 is the sentence most people get half right: insist on both halves.

8.2 Choosing k and Customer Segments

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Read an elbow plot and explain why inertia alone cannot choose k
- Compute and interpret silhouette scores, overall and per cluster, from the formula and with scikit-learn
- Test whether clusters are stable by refitting on resampled data and comparing labelings with the adjusted Rand index
- Build a reusable clustering pipeline (log transform, scaling, K-Means) and use `MiniBatchKMeans` when data gets large
- Profile, name and size segments, link them to an outcome such as churn, and write recommendations with honest limits

TIMING IN THE ROOM

10m	Frame: the returns pile; four kinds of evidence; the learners' k = 4 / 6 notes from 8.1
30m	Live demo: skew and logs, elbow, silhouettes, per-cluster view, ARI, stability, profiles and outcomes, mistakes
20m	Guided lab: `choose_k_tastes_starter.py`
15m	Start the lesson project: the segment brief, Steps 1 to 3

MENTAL MODEL TO SAY

"The back room of a Pinecrest store, piled with returned gear to sort into bins. Two bins: tents next to socks. Fifty bins: identical headlamps in different bins. A good number of bins has things in the same bin alike (**cohesion**), bins clearly different (**separation**), a colleague sorting the same pile would come up with much the same bins (**stability**), and the bins are **useful** to whoever unpacks them."

Then: "No single number captures all four, so we collect evidence and look for agreement. Elbow, silhouette, stability, usefulness." Ask for the 8.1 homework: which named groups merged at k = 4 and split at k = 6?

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook, `lesson_8_2`.

1. **Look at the shape first (2m).** "K-Means works with means and squared distances, so a long right tail pulls centers toward it."

```
import numpy as np
import pandas as pd
from sklearn.cluster import KMeans

pd.set_option("display.width", 120)
customers = pd.read_csv("data/customers_churn.csv")
behavior = ["orders_12m", "avg_order_value", "discount_share", "return_rate"]
X = customers[behavior]

print(X.skew().round(2))
```

```
orders_12m         0.92
avg_order_value    1.59
discount_share     1.18
return_rate        1.80
dtype: float64
```

Above 1 = a long right tail. Counts and money are the classic cases.

2 **Logs inside a pipeline (3m).** "A log turns 'twice as much' into 'one step more'. Inside a pipeline, so new customers get exactly the same treatment. Shares stay as they are: their high values are what we want noticed."

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import FunctionTransformer, StandardScaler

log_some = ColumnTransformer(
    [("log", FunctionTransformer(np.log, feature_names_out="one-to-one"), ["or
    remainder="passthrough", verbose_feature_names_out=False,
)
prep = make_pipeline(log_some, StandardScaler()).set_output(transform="pandas"
Z = prep.fit_transform(X)

print(Z.skew().round(2))
```

```
orders_12m        -0.19
avg_order_value    0.69
discount_share     1.18
return_rate        1.80
dtype: float64
```

`np.log` is safe because both columns are at least 1; use `np.log1p` when a column can be 0.

3 **The elbow, and the silhouette alongside (5m).** "Fit k = 2 to 9 and keep every model. Inertia always falls; read where the drops get small."

```
from sklearn.metrics import silhouette_score

fits, rows = {}, []
for k in range(2, 10):
    fits[k] = KMeans(n_clusters=k, n_init=10, random_state=42).fit(Z)
    rows.append({"k": k, "inertia": fits[k].inertia_, "silhouette": silhouette

scores = pd.DataFrame(rows).set_index("k")
scores["drop"] = -scores["inertia"].diff()
print(scores.round({"inertia": 0, "silhouette": 3, "drop": 0}))
```

```
   inertia  silhouette    drop
k
2  16849.0       0.302     NaN
3  12131.0       0.367  4718.0
4   8124.0       0.424  4008.0
5   5525.0       0.479  2599.0
6   5055.0       0.452   470.0
7   4717.0       0.395   338.0
8   4409.0       0.362   307.0
9   4178.0       0.365   232.0
```

4 → 5 removes 2,599, 5 → 6 only 470: an unusually clear elbow. The silhouette peaks at 5 (0.479). Show the lesson's two-panel figure.

4 **A silhouette by hand (3m).** "For each point: a = average distance to its own cluster, b = to the nearest other cluster, s = (b - a) / max(a, b)." Work the point 1 on the board: a = 1.5, b = 7.5, s = 0.8.

```
from sklearn.metrics import silhouette_samples

tiny = np.array([[1], [2], [3], [8], [9]])
tiny_labels = [0, 0, 0, 1, 1]
print(silhouette_samples(tiny, tiny_labels).round(3))
```

```
[0.8   0.846 0.727 0.833 0.857]
```

5 **Look inside the average (3m).** "An average can hide a bad cluster. Per-cluster means and minimums, the numbers behind the silhouette plot."

```
best = fits[5]
sil = silhouette_samples(Z, best.labels_)
print(pd.Series(sil).groupby(best.labels_).agg(["size", "mean", "min"]).round(
print(f"customers with a negative silhouette: {(sil < 0).sum()}")
```

```
   size   mean    min
0  1253  0.458 -0.029
1  1374  0.440 -0.022
2  1449  0.528  0.011
3   811  0.429 -0.058
4  1113  0.521  0.001
customers with a negative silhouette: 35
```

Every cluster has a solid body; only 35 customers are negative. Show the silhouette plot on the lesson page. Rough guide: 0.25 to 0.5 is weak but possibly real; customer data rarely forms islands.

6 **Comparing labelings: the ARI (2m).** "Pairs of points: do both labelings agree on 'together' or 'apart'? Corrected so random labels score about 0."

```
from sklearn.metrics import adjusted_rand_score

print(adjusted_rand_score([0, 0, 1, 1, 2, 2], [2, 2, 0, 0, 1, 1]))   # same gr
print(round(adjusted_rand_score([0, 0, 1, 1, 2, 2], [0, 1, 2, 0, 1, 2]), 3))
```

```
1.0
-0.25
```

7 **Stability: resample and refit (4m).** "Real groups should not depend on exactly which customers were in the table."

```
def stability(k, n_boot=8):
    """ARI between the full-data clustering and refits on bootstrap samples.""
    rng = np.random.default_rng(42)
    aris = []
    for b in range(n_boot):
        rows_b = rng.choice(len(Z), size=len(Z), replace=True)
        refit = KMeans(n_clusters=k, n_init=5, random_state=b).fit(Z.iloc[rows
        aris.append(adjusted_rand_score(fits[k].labels_, refit.predict(Z)))
    return np.mean(aris), np.min(aris)

for k in range(3, 8):
    mean_ari, worst_ari = stability(k)
    print(f"k={k}: mean ARI {mean_ari:.3f}   worst {worst_ari:.3f}")
```

```
k=3: mean ARI 0.843   worst 0.602
k=4: mean ARI 0.991   worst 0.984
k=5: mean ARI 0.994   worst 0.989
k=6: mean ARI 0.975   worst 0.848
k=7: mean ARI 0.984   worst 0.967
```

Three is unstable (worst 0.60), four and five rock-solid, six less consistent. Write the evidence on the board: elbow 5, silhouette 5, stability 4 or 5.

8 **Profile with an index (3m).** "Average each column per cluster, then divide by the overall average: 200 means twice the typical customer."

```python
customers["cluster"] = fits[5].labels_
profile = customers.groupby("cluster")[behavior].mean()
index = (profile / X.mean() * 100).round(0).astype(int)

print(profile.round(2))
print(index)
```

```
         orders_12m  avg_order_value  discount_share  return_rate
cluster
0              5.42            64.16            0.51         0.08
1              1.85            96.01            0.14         0.06
2              8.84            88.58            0.10         0.04
3              6.35           167.26            0.18         0.28
4              2.50           331.74            0.11         0.05
         orders_12m  avg_order_value  discount_share  return_rate
cluster
0               108               46             244           92
1                37               68              67           66
2               176               63              50           50
3               127              119              86          319
4                50              235              54           57
```

Each cluster stands out on one or two columns: naming is easy.

9 **Names, and outcomes the clustering never saw (3m).** "Usefulness: do segments built without churn differ in churn?"

```
names = {0: "Deal Hunters", 1: "Casual Campers", 2: "Trail Regulars", 3: "Gear
customers["segment"] = customers["cluster"].map(names)

outcomes = customers.groupby("segment").agg(
    customers=("customer_id", "size"), churn_rate=("churned", "mean"), app_use
outcomes["share"] = outcomes["customers"] / len(customers)
print(outcomes.sort_values("churn_rate").round(3))
```

```
                  customers  churn_rate  app_users  share
segment
Trail Regulars         1449       0.041      0.540  0.242
Gear Testers            811       0.101      0.501  0.135
Big-Trip Planners      1113       0.163      0.316  0.186
Casual Campers         1374       0.203      0.207  0.229
Deal Hunters           1253       0.312      0.467  0.209
```

4% to 31%: nearly eight-fold. Then the warning box: segments describe; they do not explain (Deal Hunters churn, which does not mean discounts cause it).

10 **Three mistakes (2m).** "One error and two silent traps."

```
print("k with the lowest inertia:", scores["inertia"].idxmin())

one_cluster = KMeans(n_clusters=1, n_init=10, random_state=42).fit(Z)
silhouette_score(Z, one_cluster.labels_)

new_batch = X.sample(5, random_state=42)
right = fits[5].predict(prep.transform(new_batch))
wrong = fits[5].predict(make_pipeline(log_some, StandardScaler()).set_output(t
print("right:", right)
print("wrong:", wrong)
```

```
k with the lowest inertia: 9
ValueError: Number of labels is 1. Valid values are 2 to n_samples - 1 (inclus
right: [0 1 1 2 3]
wrong: [0 4 1 2 3]
```

Lowest inertia always says 9. A silhouette needs two clusters. A scaler refitted on five newcomers moves the second one to another segment: fit once, then only transform (the project puts it all in one pipeline).

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **One score as the answer.** "The silhouette is highest at 5, done." Coach: "What do stability and the profiles say? Would marketing act differently on the groups?" Lab 8.2 A is the case where scores and

usefulness pull slightly apart.

- **Comparing silhouettes across feature sets.** "The log version scores 0.479, the raw 0.472, so logs are better." Coach: each score lives in its own space; choose features for business and statistical reasons, then compare k within one space.
- **Refitting preprocessing on new customers.** Silent. Coach: "Which customers did this scaler learn its mean from?" The pipeline (`segmenter.predict`) makes the mistake impossible.

GUIDED LAB

- **Lab A (20m, everyone, pairs):** `labs/module-03/l8_2-choosing-k/choose_k_tastes_starter.py`; reference `labs/module-03/l8_2-choosing-k/choose_k_tastes_solution.py`. The same four kinds of evidence for a different description of the customers: the share of their purchases in each activity. No scaling needed (every row adds up to 1).
- **Expected:** silhouette peaks at k = 5 (0.453; 6: 0.429); inertia drops 115 from 4 to 5 but 51 from 5 to 6; worst bootstrap ARI 0.641 (k = 4), 0.925 (5), 0.950 (6), 0.658 (7); five tastes (hiking 1,009, camping 738, climbing 424, paddling 423, running 406); at k = 6 a "general gear" group (484, general share 0.56) splits off the hiking group.
- **Debrief (3m):** "Five or six?" Take a vote, then ask for the reason. Both are defensible; the deciding question is whether "general gear" buyers would be treated differently. Note the contrast with Lesson 8.2: how people shop and what they buy each give five groups, which need not be the same five people (Lesson 8.3's Bonus Challenge checks that).
- **Fast finishers:** `labs/module-03/l8_2-choosing-k/segment_economics_starter.py`: usefulness in dollars with the $15 / $60 economics. Only Deal Hunters clear the 0.25 break-even for a blanket offer; on the test customers, offers to everyone lose $7,620 while the churn model's 300 offers earn $4,920. Then `labs/module-03/l8_2-choosing-k/debug_choose_k.py`.

START THE LESSON PROJECT

The lesson project is **the segment brief**: one pipeline (log, scale, K-Means) and a one-page table with sizes, profiles, churn, app use and actions, saved as `customer_segments.csv`. In the room, Steps 1 to 3 (15 minutes).

- Step 1: the `segmenter` pipeline. **Done** = sizes 1253, 1374, 1449, 811, 1113. Step 2: the evidence `{4: 0.424, 5: 0.479, 6: 0.452}`. Step 3: the index table and the five names.
- **Goes home:** Step 4 (the brief table: Deal Hunters churn 0.31 ... Trail Regulars 0.04), Step 5 (three newcomers: Trail Regulars, Big-Trip Planners, Deal Hunters), Step 6 (`customer_segments.csv` and the printed brief).

CHECKS FOR UNDERSTANDING

Ask these out loud:

- What are the four kinds of evidence for k? **Answer:** elbow, silhouette, stability under resampling, usefulness (sizes, outcomes, actions).
- A point's silhouette is negative. What does it mean? **Answer:** on average it is closer to another cluster than to its own: probably misassigned or on a border.
- Why does the ARI ignore cluster numbers? **Answer:** it compares pairs of points (together or apart), and the numbers are only names.
- Why put the log, the scaler and K-Means in one pipeline? **Answer:** new customers then go through exactly the fitted steps; nothing is refitted on them.

Workbook self-check, items Q1 to Q6.

Answers: **Q1** B) Inertia can only fall as k grows, so its minimum is always the largest k you tried; **Q2** `[0.8 0.75 0. ]`; **Q3** D) Three is not a natural number of groups here: K-Means must make an arbitrary choice about what to merge, and resamples disagree; **Q4** A) Segments describe who is together; choosing whom to call is the model's job; **Q5** `1.0 0.32`; **Q6** C) Compare the profiles for 5 and 6 and choose the one whose groups lead to distinct actions. Full set with explanations in the Answer Key.

◆ DIFFERENTIATION

Support: a card with the four kinds of evidence and the silhouette formula. In Lab A, give struggling pairs the stability function from the demo to adapt.

Stretch: Lesson 8.2 Exercise 3 (silhouettes of pure noise: 0.18 to 0.20 with no peak) and the Bonus Challenge (the gap statistic).

8.3 Hierarchical Clustering

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain how agglomerative clustering builds a hierarchy by repeatedly merging the two closest groups
- Read a SciPy linkage matrix and a dendrogram, and cut the tree by number of clusters or by height
- Compare single, complete, average and Ward linkage and predict how each behaves on chains, blobs and outliers
- Use `AgglomerativeClustering` with the right metric and linkage, including non-Euclidean distances
- Judge when hierarchical clustering beats K-Means, and when its cost rules it out

TIMING IN THE ROOM

10m	Frame: tidying the gear closet; n - 1 merges; the dendrogram and its cuts
25m	Live demo: six points, cutting, four linkages, cut by height, customers with Ward, Ward vs K-Means, correlation distance, mistakes
25m	Guided lab: `seasonal_families_starter.py` (13m), then `sample_then_assign_starter.py` (12m)
15m	Start the lesson project: a product family tree, Steps 1 to 3

MENTAL MODEL TO SAY

"Every item in the gear closet lies on its own. Find the two most similar and put them in a box. Treat the box as one item; repeat: the two closest things, items or boxes, go in a box. Keep going until everything is in one big box, writing down every merge and how far apart the two things were. That record is the hierarchy: you can open the boxes down to any level later."

On the board: n points → n − 1 merges; the **dendrogram** draws each merge at its height; a horizontal **cut** gives flat clusters; a long empty stretch in the tree is a natural answer. Then the one choice that changes everything: the **linkage**, the distance between two groups. And the price: every pairwise distance.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook, `lesson_8_3`.

1. **Six points, five merges (4m).** "Every point starts alone; merge the two closest; record the distance; repeat. n points, n - 1 merges." Read the linkage matrix row by row: the two merged clusters, the distance, the new size.

```
import numpy as np
import pandas as pd
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster

points = np.array([[1.0, 1.0], [1.4, 1.0], [5.0, 5.0], [5.6, 5.1], [5.3, 6.0],
names = ["A", "B", "C", "D", "E", "F"]

Z_tiny = linkage(points, method="single")
print(Z_tiny.round(3))
```

```
[[0.    1.    0.4   2.   ]
 [2.    3.    0.608 2.   ]
 [4.    7.    0.949 3.   ]
 [5.    8.    5.326 4.   ]
 [6.    9.    5.381 6.   ]]
```

Clusters 6, 7, 8 ... are the new boxes. Draw the dendrogram on the board as you read the rows.

2 **Cut the tree (2m).** "Keep every merge below 2.5: three groups."

```
print(fcluster(Z_tiny, t=2.5, criterion="distance"))
```

```
[1 1 2 2 2 3]
```

The long gap between about 1 and 5 is the tree saying "three".

3 **Four linkages, two datasets (5m).** "Distance between two GROUPS is a choice." The table on the board: single (closest pair), complete (farthest), average (all pairs), Ward (variance increase).

```
from sklearn.cluster import AgglomerativeClustering
from sklearn.datasets import make_moons, make_blobs

X_moons, _ = make_moons(n_samples=300, noise=0.05, random_state=42)
X_spread, _ = make_blobs(n_samples=300, centers=[[0, 0], [4, 0], [9, 0]],
                         cluster_std=[0.4, 0.4, 2.2], random_state=42)

for name, data, k in [("moons", X_moons, 2), ("blobs", X_spread, 3)]:
    for link in ["single", "complete", "average", "ward"]:
        labels_l = AgglomerativeClustering(n_clusters=k, linkage=link).fit_pre
        print(f"{name:6s} {link:9s} sizes {sorted(np.bincount(labels_l).tolist
```

```
moons  single    sizes [150, 150]
moons  complete  sizes [95, 205]
moons  average   sizes [84, 216]
moons  ward      sizes [108, 192]
blobs  single    sizes [1, 1, 298]
blobs  complete  sizes [38, 48, 214]
blobs  average   sizes [2, 73, 225]
blobs  ward      sizes [75, 100, 125]
```

Only single linkage traces the moons; the same chaining ruins it on the blobs (298 in one cluster). Only Ward comes close on the blobs. Show the lesson's 2 x 4 figure: no linkage wins everywhere.

4 **Cut by height (2m).** "Ask for a height and let the number follow."

```
by_height = AgglomerativeClustering(n_clusters=None, distance_threshold=30, li
print("clusters below height 30:", by_height.n_clusters_)

heights = np.sort(by_height.distances_)[::-1][:6]
print("six highest merges:", heights.round(1))
```

```
clusters below height 30: 3
six highest merges: [78.2 47.6 24.  15.7 12.4 11.8]
```

The top two merges far above the rest: the hierarchical elbow.

5 **Pinecrest customers, Ward linkage (3m).** "Same features as 8.2."

```
from sklearn.cluster import KMeans
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import FunctionTransformer, StandardScaler

customers = pd.read_csv("data/customers_churn.csv")
behavior = ["orders_12m", "avg_order_value", "discount_share", "return_rate"]
log_some = ColumnTransformer(
    [("log", FunctionTransformer(np.log, feature_names_out="one-to-one"), ["or
    remainder="passthrough", verbose_feature_names_out=False,
)
Z = make_pipeline(log_some, StandardScaler()).set_output(transform="pandas").f

Z_ward = linkage(Z, method="ward")
print("eight highest merges:", Z_ward[-8:, 2][::-1].round(1))
```

```
eight highest merges: [118.8  96.1  88.7  73.1  30.2  25.6  23.7  21.3]
```

Four merges far above the fifth: five clusters again. Show the truncated dendrogram.

6 **Ward versus K-Means (3m).** "Two different ideas: minimize inertia from random starts, or merge greedily with no randomness."

```
from sklearn.metrics import adjusted_rand_score, silhouette_score

ward5 = AgglomerativeClustering(n_clusters=5, linkage="ward").fit(Z)
km5 = KMeans(n_clusters=5, n_init=10, random_state=42).fit(Z)

print("Ward sizes   :", sorted(np.bincount(ward5.labels_).tolist()))
print("K-Means sizes:", sorted(np.bincount(km5.labels_).tolist()))
print(f"agreement (ARI): {adjusted_rand_score(km5.labels_, ward5.labels_):.3f}
print(f"silhouette  Ward {silhouette_score(Z, ward5.labels_):.3f}   K-Means {s
```

```
Ward sizes   : [810, 1107, 1286, 1326, 1471]
K-Means sizes: [811, 1113, 1253, 1374, 1449]
agreement (ARI): 0.907
silhouette  Ward 0.469   K-Means 0.479
```

ARI 0.907: two methods find nearly the same five groups. That is strong evidence the segments are real.

7 **Beyond straight-line distance (3m).** "Products described by who bought them: Euclidean mostly measures popularity. Correlation asks: when one is bought, is the other too?"

```
from scipy.spatial.distance import pdist

rows = np.array([[1, 1, 1, 1, 0, 0, 0, 0, 0, 0],    # popular tent: 4 buyers
                 [1, 1, 0, 0, 0, 0, 0, 0, 0, 0],    # niche tent, bought by th
                 [0, 0, 0, 0, 1, 1, 1, 1, 0, 0]])   # popular paddle: 4 differ

print("euclidean  :", pdist(rows, metric="euclidean").round(2))
print("correlation:", pdist(rows, metric="correlation").round(2))
```

```
euclidean  : [1.41 2.83 2.45]
correlation: [0.39 1.67 1.41]
```

Correlation distance runs from 0 (always together) through 1 (unrelated) to 2 (never together).

8 **Five mistakes, from older tutorials (3m).** "Most of these are renamed or removed parameters."

```
AgglomerativeClustering(n_clusters=5, affinity="cosine", linkage="average").fi

AgglomerativeClustering(n_clusters=5, metric="cosine", linkage="ward").fit(Z)

AgglomerativeClustering(n_clusters=5, distance_threshold=50).fit(Z)

ward5.predict(Z.head(3))

single5 = AgglomerativeClustering(n_clusters=5, linkage="single").fit(Z)
print("single linkage sizes:", sorted(np.bincount(single5.labels_).tolist()))
```

```
TypeError: AgglomerativeClustering.__init__() got an unexpected keyword argume
ValueError: cosine was provided as metric. Ward can only work with euclidean d
ValueError: Exactly one of n_clusters and distance_threshold has to be set, ar
AttributeError: 'AgglomerativeClustering' object has no attribute 'predict'
single linkage sizes: [1, 1, 1, 392, 5605]
```

`metric=` replaced `affinity=`; Ward needs Euclidean; one way to cut; no `predict` (assign by nearest mean, or train a classifier); and single linkage chains 5,605 customers into one cluster: always print the sizes.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Old tutorial code.** `affinity=`, both `n_clusters` and `distance_threshold`, Ward with cosine. Read the last line: each message says the fix.
- **Not printing cluster sizes.** Single linkage "finds five clusters": 5,605, 392 and three single customers. Coach: "Print `np.bincount(labels)` every time."
- **Euclidean distance for profiles.** Products, purchase vectors and seasonal curves differ mostly in volume; Euclidean groups by volume. Coach: "Is your question about how MUCH or about which TOGETHER / which SHAPE?"

GUIDED LAB

- **Lab A (13m):** `labs/module-03/l8_3-hierarchical/seasonal_families_starter.py`; reference `labs/module-03/l8_3-hierarchical/seasonal_families_solution.py`. The 24 products of `weekly_demand.csv` by WHEN they sell: average linkage, correlation distance on 12 monthly averages. **Expected:** highest merges [1.54, 0.292, 0.281, 0.215, 0.179]; any cut between 0.29 and 1.54 gives two families (6 winter products: 4 Apparel + 2 Snow; 18 others); at 3 families the two footwear products peak in May; Euclidean gives [22, 1, 1], isolating the two best sellers; cophenetic correlation 0.979.
- **Lab B (12m):** `labs/module-03/l8_3-hierarchical/sample_then_assign_starter.py`; reference `labs/module-03/l8_3-hierarchical/sample_then_assign_solution.py`. The big-data recipe from the lesson's warning box. **Expected:** 17,997,000 distances for everyone vs 124,750 for

500; ARI vs K-Means 0.971 (500) and 0.957 (1,500); K-Means started from the sample's means: inertia 5,525, ARI 1.000.

- **Debrief (2m):** "Which question did correlation distance answer in Lab A, and which did Euclidean answer?" Shape of the year vs volume. Then: "Would you run full agglomerative clustering on 100,000 customers?" No: sample, assign, polish.
- **Fast finishers:** `labs/module-03/l8_3-hierarchical/debug_hierarchical.py`: the lesson's product families rebuilt in scikit-learn from an older tutorial.

START THE LESSON PROJECT

The lesson project is **a product family tree**: 200 products clustered by who buys them (average linkage, correlation distance), checked against `primary_activity`, saved as `product_families.csv`. In the room, Steps 1 to 3 (15 minutes).

- Step 1: the purchase matrix. **Done** = `(200, 3000)`, 5.4% of cells filled. Step 2: the tree, 199 merges, top heights near 1. Step 3: the dendrogram cut at 0.97.
- **Goes home:** Step 4 (five families against the activities), Step 5 (single linkage chains: ARI 0.162 against 0.840), Step 6 (names and `product_families.csv`).

CHECKS FOR UNDERSTANDING

Ask these out loud:

- How many rows does a linkage matrix for 200 products have? **Answer:** 199, one per merge.
- Which linkage follows chains, and what is its danger? **Answer:** single; one bridge point glues groups together (chaining).
- Why can Ward not use cosine distance? **Answer:** it merges by increase in variance, which is defined only for Euclidean distance.
- How do you assign a new customer after agglomerative clustering? **Answer:** to the nearest cluster mean, or with a classifier trained on the labels; there is no `predict`.

Workbook self-check, items Q7 to Q12.

Answers: **Q7** `5 6.0`; **Q8** C) Euclidean distance mostly measures volume (how many units), correlation distance compares the shape of the year; **Q9** A) `AgglomerativeClustering(n_clusters=None, distance_threshold=50, linkage="ward")`; **Q10** D) Two: the long empty stretch between 0.29 and 1.54 means any cut there gives the same two groups; **Q11** `[1.89 0. 1.89]`; **Q12** B) Agglomerative clustering needs every pairwise distance (18 million for 6,000, billions for 100,000) and has no predict, so: tree on a sample, nearest mean for the rest. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: a card with the four linkages in one line each. In Lab A, let pairs skip TODO 5 (Euclidean) and do it as homework.

Stretch: Lesson 8.3 Exercise 4 (cophenetic correlation by linkage) and the Bonus Challenge (customers by what they buy, cross-tabulated with the 8.2 segments).

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

1. Name the four kinds of evidence for choosing k, and one thing each can tell you. **Answer:** elbow (where extra clusters stop paying off), silhouette (cohesion vs separation, per cluster too), stability (groups survive resampling, ARI), usefulness (sizes, outcomes like churn, distinct actions).
2. Why did correlation distance find product families (by buyers, or by season) when Euclidean distance would not? **Answer:** Euclidean mostly measures volume or popularity; correlation compares the pattern (who buys it, when it sells) regardless of level.
3. When would you choose K-Means over agglomerative clustering? **Answer:** many rows (hundreds of thousands), a need to assign new rows with `predict`, round groups on Euclidean features; agglomerative when you want the whole hierarchy, a non-Euclidean distance, or no random start.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** The segment brief (Steps 4 to 6, `customer_segments.csv`) and the product family tree (Steps 4 to 6, `product_families.csv`).
- **Lesson exercises.** 8.2: Exercises 1 to 3; 8.3: Exercises 1 to 3. Exercise 4 of each is a stretch.
- **Both Quick Quizzes** and **both journal entries** (the 8.2 prompt: the paragraph that explains "why five" with three kinds of evidence).
- **Lab 8.2 B** (segment economics) if not reached in class.
- **Before the Next Lesson (from 8.3):** keep `product_families.csv` (it returns in Lesson 11.2); rerun the linkage comparison with `make_moons(noise=0.12)`; one sentence each on when you would pick K-Means, Ward and average linkage with a custom distance.
- **Image data:** last call. Anyone whose check said READY for Sessions 1 to 4 runs it with `--download` before Session 5.

BRING LIST FOR SESSION 4

- A READY laptop; the `pinecrest-ml` folder with `data/` and `labs/` (Session 4's starters are in `labs/module-04/`).
- The Lesson 8.2 notebook with the log + scale features and `customer_segments.csv`: Lessons 8.4 and 8.5 cluster the same customers by density and with mixtures, and compare.
- Your moons experiment and the three one-sentence answers from 8.3's Before the Next Lesson.

- The workbook and the journal.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** the elbow + silhouette loop, stability, the linkage matrix, the four linkages, correlation distance.
- **Cut first (saves 2m):** 8.2 Step 6 (ARI examples).
- **Cut second (saves 2m):** 8.3 Step 4 (cut by height).
- **Cut third (saves 12m):** Lab 8.3 B goes home.

RUNNING SHORT: WHAT TO ADD

- Lab 8.2 B for everyone: the money test of usefulness.
- Lesson 8.2 Exercise 3 live: silhouettes of pure noise, flat at 0.18 to 0.20.
- Lesson 8.3 Exercise 2 live: the product tree cut at 0.90, 0.95, 0.97 and 1.0.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 4: Density-Based Clustering and Gaussian Mixtures

How to teach the fourth session of Advanced Machine Learning. Learners leave Session 3 able to give every customer one segment; today adds two honest answers: "fits nowhere" (noise, from DBSCAN, OPTICS and HDBSCAN) and "partly this, partly that" (probabilities, from Gaussian mixtures). They find dense payment patterns whose misfits hold most of the frauds, and soft segments whose confident members churn at very different rates.

2 lessons | 3 hrs session | Lessons 8.4-8.5 | Session 4 of 13

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame (one mental model) → live demo (you type, they type along) → guided lab (a kit lab, new practice on the lesson's skills) → start the lesson project. Exercises, quiz and journal are homework.
- Every code block below was run to produce the output printed under it (Python 3.12, pandas 3.0, scikit-learn 1.9.1, 4 CPU threads). Type it live, ask for a prediction first, read errors from the last line up.
- Keep `clustering_dbscan.html` and `clustering_gaussian_mixture.html` open for the pictures (moons, k-distance plot, reachability plot, method table, weighted bell curves, stretched blobs). Labs live in `labs/module-04/`.
- This session is the hinge of Module 8: Lessons 8.1 to 8.3 gave every customer one hard label; today adds two honest answers, "fits nowhere" (noise) and "partly this, partly that" (probabilities). Lesson 8.6 next session turns both into anomaly scores.
- Teal blocks are hands-on, amber blocks are watch-outs. Lesson codes (8.4, 8.5) match the course site.

> **THE #1 RISK OF SESSION 4: BELIEVING A CLUSTER BECAUSE AN ALGORITHM PRODUCED IT**
>
> Both lessons, and both labs, contain a cluster that is not a segment: customers or payments stacked on a whole-number value (one order, exactly two items) look like the densest crowd in the data. A learner who reports "HDBSCAN found 8 kinds of payment" without reading the profile table has learned the API and missed the lesson. Ask "what do the members of this cluster have in common, besides being in it?" at every profile table today. The second risk is quieter: treating `predict` as the whole answer of a mixture. Every mixture output today goes with its confidence.

BEFORE THE SESSION

- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-04` from this kit's folder. Every line should say PASS (8 files, about 30 seconds).
- ☐ **Downloads:** none. Everything today uses the course CSV files and data generated in code.
- ☐ **Projector machine:** `(.venv)` active in `pinecrest-ml`, JupyterLab or VS Code started there, two empty notebooks `lesson_8_4` and `lesson_8_5`, font 18 pt or more. Browser tabs: `clustering_dbscan.html`, `clustering_gaussian_mixture.html`, and the Session 3 customer segments notebook if you kept one (8.2's five named segments come back in both projects).
- ☐ **Board, before learners arrive:** the core/border/noise decision diagram (three boxes), an empty method table with the columns of Lesson 8.4's "Which Clustering Method When" (You choose · Cluster shape · Noise label · Labels new data?), and three headings for the second block: means · covariances · weights.
- ☐ **Print or share:** the workbook pages for Session 4 and sticky notes for the exit ticket.
- ☐ **Timing note:** Lesson 8.5's project Step 2 fits 24 mixtures (8 values of k × 3 starts) and takes about 10 to 20 seconds on a laptop; start it running, then talk.

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
`FutureWarning` about HDBSCAN's `copy` parameter	scikit-learn 1.9 is changing the default	Pass `copy=True` (as the lesson does). The warning is harmless but noisy.
`ValueError: Expected 2D array, got 1D array instead`	A single column passed as a Series or a 1-D array to `GaussianMixture` or `DBSCAN`	Double brackets (`customers[["discount_share"]]`) or `.reshape(-1, 1)`.
OPTICS takes many seconds on the customers	OPTICS is slow on thousands of rows	Use it on the generated data only, as the lesson does; HDBSCAN on the customers.
Different cluster numbers from the lesson page	Component and cluster numbers are arbitrary	Compare sizes and profiles, not numbers. Exact numbers match with the same seeds, 4 threads and scikit-learn 1.9.
`ConvergenceWarning` in the project	Unscaled features or too few iterations	Check that `Z` is the scaled table; raise `max_iter`; check `gm.converged_`.

SESSION 4 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Lessons 8.2 and 8.3 (choosing k, ARI, linkage), homework check.
0:10	10	**8.4** Frame and mental model	A city at night from a plane: crowded places, dark fields, the lone farmhouse.
0:20	25	**8.4** Live demo	Moons, core/border/noise, eps sweep, k-distances, varying density, OPTICS, HDBSCAN, three mistakes.
0:45	25	**8.4** Guided lab	`labs/module-04/l8_4-density-clustering/payment_density_starter.py`: dense payment patterns and the misfits.
1:10	15	**8.4** Start the lesson project	Core Segments and Misfits, Steps 1 and 2; Steps 3 to 6 go home.
1:25	10	Break	Leave the core/border/noise diagram and the method table on the board.
1:35	10	**8.5** Frame and mental model	Paint mixed from a few base colors: components, weights, soft membership.
1:45	25	**8.5** Live demo	One-dimensional mixture, probabilities, ellipses, covariance types and BIC, score_samples, label switching.
2:10	25	**8.5** Guided lab	`labs/module-04/l8_5-gaussian-mixtures/discount_mixture_starter.py`, then `labs/module-04/l8_5-gaussian-mixtures/lifecycle_mixture_starter.py` in pairs.
2:35	15	**8.5** Start the lesson project	Soft Customer Segments, Steps 1 and 2 (the BIC sweep); Steps 3 to 6 go home.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 5.
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 100 + 95 = 195 minutes for a solo learner; the room has 150 after the warm-up, the break and the wrap-up. So the room does, for each lesson: the mental model, the live demo, one guided kit lab, and the first steps of the lesson project.

Goes home (2 to 3 hours). Core Segments and Misfits Steps 3 to 6 and Soft Customer Segments Steps 3 to 6; each lesson's Exercises 1 to 3 (Exercise 4 is a stretch); both Quick Quizzes; both journal entries; `lifecycle_mixture_starter.py` for anyone who did not reach it.

Protect, in this order: the varying-density demo through HDBSCAN (8.4 Steps 5 to 7: the reason HDBSCAN exists); the payment lab through TODO 5 (the noise holds most of the frauds: the bridge to 8.6); 8.5 Steps 2 and 4 (soft membership and BIC). **Cut first:** 8.4 demo Step 2 (core/border counts: say one sentence), then 8.5 Steps 8 and 9 (two mistakes: name them), then the 8.4 project start (send all six steps home with the two check numbers).

OPENING SEGMENT

0:00 Warm-Up: Choosing k and Hierarchical Clustering — 10 min

HOW TO RUN IT

- Put W1 to W5 on the slides (or read them out). Learners answer on the workbook's warm-up page, two minutes, no notes; reveal with fingers 1 to 4 for A to D. Settle W1 and W3 by running the code.
- Homework check (2 minutes): who finished the 8.2 segment brief and the 8.3 product family tree? Ask one learner to read the five segment names from 8.2. They are back in both projects today.

WARM-UP ANSWERS

Answers: **W1** C) `1.0`; **W2** D) The k where inertia reaches its minimum; **W3** B) `[1. 1. 9.]`; **W4** A) Single; **W5** B) The whole fitted pipeline (log, scaler, K-Means): call predict on the new rows.

- **W1 wrong (0.0 or an error):** they think cluster numbers must match. Today's quiz and Lesson 8.5's label-switching mistake depend on this; spend one extra minute.
- **W2 wrong (D):** they still read inertia's minimum. Say: "It always falls; we read where it stops falling fast." The same reading comes back for BIC today.
- **W3 or W4 wrong:** single linkage and chaining. Not urgent, but the trail of in-between points is exactly what density methods call noise.

LESSON PLANS

8.4 Density-Based Clustering

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain core, border and noise points, and what eps and min_samples control in DBSCAN
- Choose a starting eps from a k-distance plot and read the effect of changing it
- Read an OPTICS reachability plot and explain why one eps fails when clusters have different densities
- Cluster with HDBSCAN, count clusters and noise correctly, and profile the result
- Judge whether a density-based clustering found real structure or an artifact of the features

TIMING IN THE ROOM

10m	Frame: the city at night; why every-point-in-a-cluster is a quiet assumption
25m	Live demo: moons, roles, eps sweep, k-distances, varying density, OPTICS, HDBSCAN, three mistakes
25m	Guided lab: `payment_density_starter.py` (fast finishers: `rings_starter.py`, `debug_density.py`)
15m	Start the lesson project: Core Segments and Misfits, Steps 1 and 2

MENTAL MODEL TO SAY

"Look down at a region at night from a plane. Towns are where the lights are packed close together, and their shapes follow rivers and valleys, not circles. Between them are dark fields with a lone farmhouse here and there. You did not need to be told how many towns there are: you counted the bright patches. And you would not call a farmhouse a town."

Then the contrast with Session 3: "K-Means and hierarchical clustering put EVERY customer in some cluster and assume roughly round blobs. Density methods give two new answers: clusters of any shape, and noise, label -1, for points that belong nowhere. The number of clusters comes out of the data."

Name the three algorithms in one breath: DBSCAN (one radius, easy to reason about), OPTICS (every radius at once, a picture), HDBSCAN (keeps the clusters that are stable across densities: the one to reach for).

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook `lesson_8_4`. Steps 1 to 7 are the lesson's sections in order; 8 to 10 are its Common Mistakes. Ask for a prediction before every sweep.

1. **K-Means cannot follow a moon (3m).** "Two interleaving half-moons. K-Means draws straight boundaries between centers; DBSCAN follows the crowd. Scale first: eps is a distance." Show the

lesson's side-by-side figure after the run.

```
import numpy as np
import pandas as pd
from sklearn.datasets import make_moons
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, DBSCAN
from sklearn.metrics import adjusted_rand_score

def label_counts(labels):
    """Cluster sizes as a dict; the key -1 is noise."""
    return pd.Series(labels).value_counts().sort_index().to_dict()

X_moons, y_moons = make_moons(n_samples=400, noise=0.08, random_state=42)
Z_moons = StandardScaler().fit_transform(X_moons)

km_labels = KMeans(n_clusters=2, n_init=10, random_state=42).fit_predict(Z_moo
db = DBSCAN(eps=0.3, min_samples=5).fit(Z_moons)

print("K-Means:", label_counts(km_labels), " ARI", round(adjusted_rand_score(y
print("DBSCAN :", label_counts(db.labels_), " ARI", round(adjusted_rand_score(
```

```
K-Means: {0: 200, 1: 200}  ARI 0.475
DBSCAN : {-1: 1, 0: 200, 1: 199}  ARI 0.995
```

The adjusted Rand index is back from Lesson 8.2 (today's warm-up): 1 means the true moons. Ask: "Did I tell DBSCAN there were two clusters?" (No: the count came out of the data.)

2 **Core, border, noise (2m).** "Every point gets a role. Core: at least min_samples within eps, itself included. Border: not crowded itself, but next to a core point. Noise: neither, label -1." Draw the decision diagram from the lesson.

```
is_core = np.zeros(len(Z_moons), dtype=bool)
is_core[db.core_sample_indices_] = True
is_noise = db.labels_ == -1
print("core  :", is_core.sum())
print("border:", (~is_core & ~is_noise).sum())
print("noise :", is_noise.sum())
```

```
core  : 398
border: 1
noise : 1
```

3 **eps is unforgiving (3m).** "Too small: crumbs and noise. Too large: one blob." Ask for a prediction for eps 0.1 and eps 1.0 before running.

```
for eps in [0.1, 0.2, 0.3, 0.5, 1.0]:
    labels = DBSCAN(eps=eps, min_samples=5).fit_predict(Z_moons)
    n_clusters = len(set(labels) - {-1})
    print(f"eps={eps:<4} clusters={n_clusters:<3} noise={(labels == -1).sum():
```

```
eps=0.1  clusters=29  noise=157  ARI=0.031
eps=0.2  clusters=5   noise=5    ARI=0.886
eps=0.3  clusters=2   noise=1    ARI=0.995
eps=0.5  clusters=2   noise=0    ARI=1.000
eps=1.0  clusters=1   noise=0    ARI=0.000
```

The useful range is 0.3 to 0.5. On real data there is no ARI to check against, which is why the next step exists.

4 **A starting eps from the data (3m).** "For each point: the distance to its min_samples-th neighbor, itself included. That is the smallest eps that makes it core. Sort, plot, look for the knee." Show the lesson's k-distance plot.

```
from sklearn.neighbors import NearestNeighbors
min_samples = 5
nn = NearestNeighbors(n_neighbors=min_samples).fit(Z_moons)
distances, _ = nn.kneighbors(Z_moons)        # column 0 is each point itself,
k_dist = np.sort(distances[:, -1])
print("k-distance percentiles (50, 90, 99):", np.percentile(k_dist, [50, 90, 9
print("share that would be core at eps=0.3:", (k_dist <= 0.3).mean())
```

```
k-distance percentiles (50, 90, 99): [0.112 0.186 0.278]
share that would be core at eps=0.3: 0.995
```

Say the warning card: on real data the knee is a gentle curve. Treat it as the center of a short sweep.

5 **When density varies (3m).** "Two tight groups close together, one loose group further away, a sprinkle of noise. True sizes 200, 200, 300. Find an eps that gets all three."

```
from sklearn.datasets import make_blobs
rng = np.random.default_rng(42)
X_blobs, y_blobs = make_blobs(n_samples=[200, 200, 300], centers=[[0, 0], [2,
                              cluster_std=[0.3, 0.3, 1.3], random_state=42)
X_var = np.vstack([X_blobs, rng.uniform(-3, 10, size=(30, 2))])
y_var = np.concatenate([y_blobs, np.full(30, -1)])     # -1 marks the sprinkle
for eps in [0.2, 0.4, 0.6, 1.0]:
    labels = DBSCAN(eps=eps, min_samples=10).fit_predict(X_var)
    print(f"eps={eps:<4} {label_counts(labels)}")
```

```
eps=0.2  {-1: 361, 0: 189, 1: 180}
eps=0.4  {-1: 180, 0: 199, 1: 202, 2: 149}
eps=0.6  {-1: 60, 0: 403, 1: 267}
eps=1.0  {-1: 20, 0: 403, 1: 307}
```

No eps works: small radii lose the loose group, large radii merge the tight ones. The gap between the tight groups is narrower than the spacing inside the loose one.

6 **OPTICS: every eps at once (3m).** "OPTICS records, for every point, roughly the eps at which it would be reached. Plotted in visiting order, clusters are valleys." Show the reachability plot; point at the higher valley (the loose group).

```
from sklearn.cluster import OPTICS
optics = OPTICS(min_samples=10, xi=0.1, min_cluster_size=0.05).fit(X_var)
print("OPTICS:", label_counts(optics.labels_), " ARI", round(adjusted_rand_sco
```

```
OPTICS: {-1: 18, 0: 200, 1: 203, 2: 309}  ARI 0.959
```

Running DBSCAN = drawing one horizontal line across this plot. One line cannot cut a high valley and two low ones well.

7 **HDBSCAN: the practical default (3m).** "Build the whole hierarchy over every eps, keep the clusters that live longest. The setting you can reason about: the smallest group you care about." Mention why `copy=True` (scikit-learn 1.9 FutureWarning).

```
from sklearn.cluster import HDBSCAN
hdb = HDBSCAN(min_cluster_size=20, copy=True).fit(X_var)
print("HDBSCAN:", label_counts(hdb.labels_), " ARI", round(adjusted_rand_score
print("membership strength of the first 5 points:", hdb.probabilities_[:5].rou
```

```
HDBSCAN: {-1: 30, 0: 302, 1: 197, 2: 201}  ARI 0.95
membership strength of the first 5 points: [1.   0.55 0.36 0.93 0.65]
```

`probabilities_`: 1 is the dense core, near 0 the fringe, noise 0. Put the lesson's method table on the board now; it is the summary of the block.

8 **Mistake: counting noise as a cluster (1m).** "No error, wrong number."

```
labels_04 = DBSCAN(eps=0.4, min_samples=10).fit_predict(X_var)
print("len(set(labels)):", len(set(labels_04)))
print("real clusters    :", len(set(labels_04) - {-1}))
```

```
len(set(labels)): 4
real clusters    : 3
```

9 **Mistake: asking DBSCAN to predict (1m).** "These models describe the data they saw."

```
db.predict(Z_moons[:3])
```

```
AttributeError: 'DBSCAN' object has no attribute 'predict'
```

Fix: refit, or a nearest-neighbor assignment (Exercise 4). Gaussian mixtures, after the break, can predict.

10 **Mistake: forgetting to scale, on real customers (3m).** "Dollars against fractions: a radius of 0.5 means 'within 50 cents of order value'."

```
customers_raw = pd.read_csv("data/customers_churn.csv")
raw = customers_raw[["orders_12m", "avg_order_value", "discount_share", "retur
raw_labels = DBSCAN(eps=0.5, min_samples=10).fit_predict(raw)
print("clusters:", len(set(raw_labels) - {-1}), " noise share:", round((raw_la
```

```
clusters: 57  noise share: 0.776
```

57 clusters and 78% noise, no error. Transition: "The project fixes this with logs and a scaler, then finds a third of Pinecrest's customers fit no segment."

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Counting -1 as a cluster.** Symptom: "HDBSCAN found 8 clusters" when it found 7 and noise. Coach: "Print `set(labels)`. Which of those is not a cluster?" Make them report noise as its own number every time.
- **Sweeping eps forever on real data.** Symptom: a learner in the lab tries eps 0.55, 0.6, 0.65 hoping for a clean answer. Coach: "Do the k-distances show a knee? Do the clusters have different densities?" When no eps works, that is the finding, and the cue for HDBSCAN.

- **Trusting a cluster without its profile.** Symptom: naming lab clusters "one-item shoppers" and "two-item shoppers" as segments. Coach: "What else differs between those clusters?" (Little.) Whole-number columns make shelves.

GUIDED LAB

File: `labs/module-04/l8_4-density-clustering/payment_density_starter.py`; reference `labs/module-04/l8_4-density-clustering/payment_density_solution.py`. The lesson's moves on a new question: 5,000 card payments, five features (log amount, log account age, hour, velocity, items), fraud never used to fit.

- **TODO 1 and 2 together (7m):** k-distances and the eps sweep. No knee, and no eps works: 69.7% noise at 0.3, 5.6% at 1.0 with clusters that split on whole numbers. Say: "Same story as the varying-density demo."
- **TODO 3 to 5 in pairs (13m):** HDBSCAN (8 clusters, 8.8% noise), the profile table, noise against the clusters.
- **Expected:** most clusters sit on one exact items count (1 to 4) and velocity value (0, 1 or 2); the noise holds 24 of the sample's 38 frauds, a fraud rate of 5.4% against 0.31% in the clusters.
- **Debrief (3m):** "Which result would you take to the payments team?" Not the eight clusters: the noise. "Unusual payments are where the fraud is" is exactly the idea of Lesson 8.6. TODO 6 (stability) is the habit: 50 and 100 agree; 25 and 200 still put 3.9% to 4.5% fraud in the noise.
- **Fast finishers:** `labs/module-04/l8_4-density-clustering/rings_starter.py` (a ring inside a ring; the silhouette votes 0.328 for the wrong K-Means answer and 0.158 for the right DBSCAN one), then `labs/module-04/l8_4-density-clustering/debug_density.py`.

START THE LESSON PROJECT

Core Segments and Misfits (lesson page, Steps 1 to 6, about 45 minutes alone). In the room: Steps 1 and 2 (15 minutes).

- Read the brief aloud: marketing wants the tight segments AND the customers who fit none. Step 1: log the two skewed columns, scale; **done** = `(6000, 4)` with means 0 and standard deviations 1.
- Step 2: the k-distances `[0.28 0.41 0.6 ]` and the sweep from 19 clusters with 31.3% noise (eps 0.3) to 2 clusters holding 5,528 customers (eps 0.6). Ask: "Would you present any of these?"
- **Goes home:** Steps 3 to 6: HDBSCAN finds 7 clusters and 32.3% noise; two pairs differ only by a whole-number order count and are merged; misfits churn at 0.203; `customer_density_segments.csv` (6,000 rows).

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- Why do we scale before DBSCAN? **Answer:** eps is a distance; columns in large units would decide it alone.
- What does a point with label -1 mean? **Answer:** noise: not in any crowd. It is a result to report, not a cluster.
- Why can no single eps find the tight and the loose groups? **Answer:** the gap between the tight groups is narrower than the spacing inside the loose one; any radius that connects the loose group bridges the gap.
- How do you label a new customer with a fitted HDBSCAN? **Answer:** you cannot predict: refit, or assign to the nearest clustered neighbor's segment and call it noise when it is too far.

Workbook self-check, items Q1 to Q6:

Answers: **Q1** C) `[ 0 0 0 -1 -1 -1]`; **Q2** B) Choosing a starting eps for DBSCAN with min_samples=10: just above the knee; **Q3** `4 3`; **Q4** C) `min_cluster_size`, from the smallest group you would act on, then check that nearby values agree; **Q5** D) items only takes whole values, so payments stack on exact values and look dense: profile further and treat the split as an artifact unless the business cares about it; **Q6** B) `True False`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: give strugglers the three-role card (core, border, noise) and have them label five points of the moons figure by hand before the lab. In the lab, pair them on TODO 3 to 5 only; TODO 1 and 2 are done on the projector.

Stretch: `rings_starter.py` and `debug_density.py`; then the lesson's Bonus Challenge (replace `log_orders` with `log(spend_12m)` to remove the integer artifact).

8.5 Gaussian Mixture Models

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain a Gaussian mixture as a weighted sum of components with means, covariances and weights
- Fit a mixture with EM and read predict_proba as soft cluster membership
- Choose the covariance type and the number of components with BIC and AIC
- Compare a mixture with K-Means on elongated or overlapping clusters
- Use a fitted mixture to score how typical a point is and to generate new samples

TIMING IN THE ROOM

10m	Frame: paint mixed from a few base colors; hard labels to probabilities
25m	Live demo: 1-D mixture, probabilities, ellipses, covariance types and BIC, choosing k, score_samples, three mistakes
25m	Guided lab: `discount_mixture_starter.py` (everyone), then `lifecycle_mixture_starter.py` in pairs
15m	Start the lesson project: Soft Customer Segments, Steps 1 and 2

MENTAL MODEL TO SAY

"Every customer's behavior is a dab of paint, and all the paint on the palette was mixed from a few base colors. A Gaussian mixture works out the base colors (the components), how much of the palette each supplied (the weights), and, for any dab, how likely it is to have come from each color. A dab between two colors gets a split answer, and that split is information, not indecision."

Write under the three headings on the board: **means_** (where), **covariances_** (spread and tilt: an ellipse), **weights_** (share). Then: "K-Means is close to a special case: tiny, equal, round components and all-or-nothing answers. A mixture can tilt, stretch, share, and predict for new customers, which the density methods before the break could not."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook `lesson_8_5`. Steps 1 to 6 follow the lesson's sections; 7 to 9 are its mistakes.

1. **One feature, two hidden groups (3m).** "Six hundred values from one bell curve, four hundred from another. The model never sees which is which." Remind them: scikit-learn wants 2-D X, hence the reshape.

```
import numpy as np
import pandas as pd
from sklearn.mixture import GaussianMixture
rng = np.random.default_rng(42)
group_a = rng.normal(loc=0.0, scale=1.0, size=600)
group_b = rng.normal(loc=4.0, scale=1.5, size=400)
x = np.concatenate([group_a, group_b]).reshape(-1, 1)   # scikit-learn wants a
gm_1d = GaussianMixture(n_components=2, random_state=42).fit(x)
print("means  :", gm_1d.means_.ravel().round(2))
print("stds   :", np.sqrt(gm_1d.covariances_.ravel()).round(2))
print("weights:", gm_1d.weights_.round(3))
```

```
means  : [0.03 4.16]
stds   : [1.01 1.35]
weights: [0.629 0.371]
```

True values: means 0 and 4, spreads 1 and 1.5, weights 60/40. Recovered from the mixed-up numbers alone.

2. **Soft membership (3m).** "predict_proba: one row per value, one column per component, each row adds up to 1." Ask the room to predict the row for 2.0 before running.

```
probe = np.array([[0.0], [1.5], [2.0], [2.5], [4.0]])
print(pd.DataFrame(gm_1d.predict_proba(probe).round(3), index=probe.ravel(),
                   columns=[f"P(component {k})" for k in range(2)]))
print("predict:", gm_1d.predict(probe))
```

```
     P(component 0)  P(component 1)
0.0           0.996           0.004
1.5           0.846           0.154
2.0           0.550           0.450
2.5           0.197           0.803
4.0           0.001           0.999
predict: [0 0 0 1 1]
```

`predict` picks the larger probability and hides the 0.55 / 0.45 split: the information that makes a mixture useful. Show the lesson's figure of two weighted bell curves.

3. **Ellipses, not circles (3m).** "Three blobs stretched the same way. K-Means slices across them; a full-covariance component bends to each shape."

```
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import adjusted_rand_score
X_round, y_true = make_blobs(n_samples=600, centers=3, random_state=170)
X_stretched = X_round @ np.array([[0.6, -0.6], [-0.4, 0.8]])
km_labels = KMeans(n_clusters=3, n_init=10, random_state=42).fit_predict(X_str
gm_full = GaussianMixture(n_components=3, covariance_type="full", random_state
gm_labels = gm_full.predict(X_stretched)
print("K-Means ARI:", round(adjusted_rand_score(y_true, km_labels), 3))
print("GMM ARI    :", round(adjusted_rand_score(y_true, gm_labels), 3))
```

```
K-Means ARI: 0.577
GMM ARI    : 1.0
```

Draw the EM loop from the lesson while it runs: E-step shares out each point, M-step refits each component; repeat until the log-likelihood stops improving.

4 **Covariance types, judged by BIC (4m).** "BIC rewards fit and charges for every parameter. Lower is better. It never sees the true labels."

```
for cov in ["spherical", "diag", "tied", "full"]:
    gm = GaussianMixture(n_components=3, covariance_type=cov, random_state=42)
    ari = adjusted_rand_score(y_true, gm.predict(X_stretched))
    print(f"{cov:10s} BIC {gm.bic(X_stretched):8.1f}   ARI {ari:.3f}")
```

```
spherical  BIC   4019.6   ARI 0.568
diag       BIC   3988.1   ARI 0.516
tied       BIC   3032.8   ARI 1.000
full       BIC   3061.9   ARI 1.000
```

tied wins because the groups really share one shape; full also gets them right but pays for two covariances it does not need. The axis-aligned types cannot tilt.

5 **How many components (3m).** "Same tool, different question."

```
for k in range(1, 7):
    gm = GaussianMixture(n_components=k, covariance_type="tied", random_state=
    print(f"k={k}  BIC {gm.bic(X_stretched):8.1f}   AIC {gm.aic(X_stretched):8
```

```
k=1  BIC   4272.7   AIC   4250.7
k=2  BIC   3825.1   AIC   3789.9
k=3  BIC   3032.8   AIC   2984.5
k=4  BIC   3053.1   AIC   2991.6
k=5  BIC   3072.4   AIC   2997.6
k=6  BIC   3087.2   AIC   2999.2
```

A clean minimum at 3 on toy data. Warn: on the customers (the project) BIC keeps creeping down; read where the big drops stop.

6 **A mixture is a density model (3m).** "score_samples is the log of the model's density. High in the middle of a component, very low far from everything."

```
gm_tied = GaussianMixture(n_components=3, covariance_type="tied", random_state
typical_point = gm_tied.means_[:1]                # the center of one compon
odd_point = np.array([[0.0, -20.0]])              # far from every group
print("log-density, center of a component:", gm_tied.score_samples(typical_poi
print("log-density, far-away point       :", gm_tied.score_samples(odd_point).
print("lowest log-density among the data :", gm_tied.score_samples(X_stretched
```

```
log-density, center of a component: [-1.47]
log-density, far-away point       : [-1397.33]
lowest log-density among the data : -7.94
```

Compare log-densities within one model only. This is the anomaly score of Lesson 8.6.

7 **Mistake: component numbers mean nothing (3m).** "Two seeds, the same clustering, different numbers."

```
run_a = GaussianMixture(n_components=3, covariance_type="tied", random_state=4
run_b = GaussianMixture(n_components=3, covariance_type="tied", random_state=7
print("first labels, run A:", run_a.predict(X_stretched)[:8])
print("first labels, run B:", run_b.predict(X_stretched)[:8])
print("ARI between runs   :", adjusted_rand_score(run_a.predict(X_stretched),
```

```
first labels, run A: [1 1 2 1 1 1 2 1]
first labels, run B: [0 0 2 0 0 0 2 0]
ARI between runs   : 1.0
```

A report that says 'segment 2 churns most' breaks on the next refit. Name segments by their profile.

8 **Mistake: more components than rows (1m).** "A filtered subset is how this happens."

```
GaussianMixture(n_components=5, random_state=42).fit(X_stretched[:3])
```

```
ValueError: Expected n_samples >= n_components but got n_components = 5, n_sar
```

9 **Mistake: ignoring a ConvergenceWarning (2m).** "With max_iter=2 EM stops half-way and only warns."

```
import warnings
with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always")
    GaussianMixture(n_components=3, max_iter=2, random_state=42).fit(X_stretch
print(caught[0].category.__name__)
```

```
ConvergenceWarning
```

Fix: raise max_iter, scale features, check `converged_`; try `n_init=3` or a larger `reg_covar` if a component collapses.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Stopping at predict.** Symptom: a lab answer that only lists component labels. Coach: "How sure is the model about this customer?" Every mixture result today comes with its top probability.
- **Assuming component numbers are stable.** Symptom: `deal = 1` hard-coded in the discount lab. It happens to be right with seed 42 and wrong with another seed. Coach: "Which component has the larger mean?" (`gm.means_.ravel().argmax()`).
- **Reading BIC's minimum on real data.** Symptom: "BIC is lowest at 8, so 8 segments" (project Step 2, or the lifecycle lab). Coach: "How big is each drop?" Same reading as the elbow in the warm-up.

GUIDED LAB

- **Lab A (15m, everyone):** `labs/module-04/l8_5-gaussian-mixtures/discount_mixture_starter.py`; reference `labs/module-04/l8_5-gaussian-mixtures/discount_mixture_solution.py`. The lesson's first section on a real column: is discount use one population or two?
- **Expected, Lab A:** BIC drops 3,596 from one to two components, then only 330; means 0.11 and 0.44 with weights 0.705 and 0.295; the vote switches between a discount share of 0.25 (P = 0.471) and 0.30 (0.911); churn 0.119 for customers with P(deal-driven) up to 0.1 and 0.305 above 0.9; 430 borderline customers (P between 0.2 and 0.8) with shares 0.219 to 0.281.
- **Circulate for:** a 1-D Series passed to `fit` (the 2-D error); `deal` hard-coded; `pd.cut` without `include_lowest=True` (customers at exactly 0 vanish). **Debrief:** "The mixture found the Deal Hunters from one column and never saw churn, yet its confident members churn at 30%."

- **Lab B (10m, pairs, finish at home):** `labs/module-04/l8_5-gaussian-mixtures/lifecycle_mixture_starter.py`; reference `labs/module-04/l8_5-gaussian-mixtures/lifecycle_mixture_solution.py`. Lifecycle stages from tenure, recency and spend.
- **Expected, Lab B:** BIC drops 3,408, 493, 385, 243, 40 (no minimum, a judgment call); four stages: long-tenure regulars (2,169, churn 0.109), established (1,272, 0.071), fading (1,293, median 101 days since the last order, 0.205), newcomers (1,266, 3 months, 0.317); seeds 42 and 7 give ARI 0.981 with the components renumbered; 1,340 customers below 0.8 confidence; a new two-month customer is 0.977 newcomer.

START THE LESSON PROJECT

Soft Customer Segments (lesson page, Steps 1 to 6, about 40 minutes alone). In the room: Steps 1 and 2 (15 minutes).

- Read the warning card "Why Not the Logs from Lesson 8.2?" aloud: logs of whole numbers create stripes a full-covariance component collapses onto. Step 1 scales the raw four features; **done** = `(6000, 4)`.
- Step 2 fits 1 to 8 components, three starts each (start it, then talk). **Done** = the BIC table: drops of thousands up to 5 (2,216 at k=5), then 185 and 187. Ask for a choice and a reason.
- **Goes home:** Steps 3 to 6: five named segments, 305 customers below 0.8 confidence, what a sixth component would add (it splits the Deal Hunters), two new customers scored, `customer_soft_segments.csv`.

CHECKS FOR UNDERSTANDING

Ask these out loud:

- What do the three learned attributes of a mixture describe? **Answer:** `means_` where each component sits, `covariances_` its spread and tilt, `weights_` its share of the data.
- Why can a mixture separate stretched groups that K-Means cuts across? **Answer:** a full or tied component is an ellipse with its own tilt; K-Means assumes round clusters around centers.
- A customer gets probabilities 0.51 and 0.49. What do you do with that? **Answer:** treat them as borderline; keep the probability next to the label; leave them out of narrowly targeted decisions.
- What tells you a customer is unlike every segment? **Answer:** a low `score_samples` (log-density), not a low probability.

Workbook self-check, items Q7 to Q12:

Answers: **Q7** D) `1.0`; **Q8** A) About 2 or 3: the big drops stop there; then check whether the third component means something; **Q9** C) `"tied"`; **Q10** `1.0`: the two runs found the same groups under different numbers; **Q11** A) `score_samples`: a very low log-density; **Q12** `ValueError: Expected n_samples >= n_components but got n_components = 4, n_samples = 3`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: draw two overlapping bell curves and have learners shade where they would say "not sure"; that region is what `predict_proba` reports. In Lab A, give TODO 1 to them pre-typed if the loop slows them down.

Stretch: Lab B in full; then the lesson's Bonus Challenge (merge the soft segments with 8.4's misfits: are misfits borderline or atypical?).

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Learners answer in the workbook's Exit Ticket box or on a sticky note. Sort into "got it" and "not yet" before Session 5: question 3 decides how long you spend on `score_samples` in the 8.6 framing.

1. HDBSCAN returns the labels `-1, 0, 1, 2` and 30% of points are -1. How many clusters did it find, and what do you tell the marketing team about the 30%? **Answer:** three clusters. The 30% are customers in no dense crowd: report them as a group of their own (they are often mixed or unusual behavior), not as a failure.
2. Name one thing a Gaussian mixture can do that HDBSCAN cannot, and one thing HDBSCAN can do that a mixture cannot. **Answer:** a mixture can `predict` for new customers and gives probabilities (and a density); HDBSCAN finds clusters of any shape and labels noise.
3. A customer far from every segment gets `predict_proba` 0.99 for one component. Is the model sure this is a typical member? What would you check? **Answer:** no: probabilities only compare components. Check `score_samples`: a very low log-density means atypical.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** Core Segments and Misfits Steps 3 to 6 (7 clusters, 32.3% noise, merged pairs, misfits churn 0.203, `customer_density_segments.csv`) and Soft Customer Segments Steps 3 to 6 (305 below 0.8, the sixth component, `customer_soft_segments.csv`).
- **Lesson exercises.** 8.4: Exercises 1 to 3 (min_samples, OPTICS cuts, segment size); Exercise 4 is a stretch. 8.5: Exercises 1 to 3 (covariance types on customers, uncertainty by segment, synthetic customers); Exercise 4 is a stretch and previews 8.6.
- **Both Quick Quizzes** on the lesson pages (answers are in the Answer Key).
- **Journal:** both entries. The 8.5 prompt asks for one borderline and one atypical customer from their project.
- **Before the Next Lesson (from 8.5):** keep `customer_soft_segments.csv`; look at `data/transactions.csv` (how many frauds, what does a normal payment look like?); think of one situation where "unusual" is good news and one where it is bad news.
- **Optional:** `lifecycle_mixture_starter.py` if not finished in class; `rings_starter.py` and `debug_density.py`.

BRING LIST FOR SESSION 5

- A laptop with the course environment (`(.venv)` active, `check_environment.py` READY). Nothing new to install, no downloads.
- The `pinecrest-ml` folder with `data/` and `labs/`; Session 5's starters are in `labs/module-05/`.
- Both project notebooks from today and their files `customer_density_segments.csv` and `customer_soft_segments.csv` (8.6 compares detectors with the density ideas from today).
- Your answers to the 8.5 "Before the Next Lesson" questions about `transactions.csv`.
- The workbook and the journal.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** 8.4 demo Steps 5 to 7 (varying density, OPTICS, HDBSCAN); the payment lab through TODO 5; 8.5 demo Steps 2 and 4.
- **Cut first (saves 4m):** 8.4 demo Steps 2 and 8 (roles and counting noise): one sentence each.
- **Cut second (saves 5m):** 8.5 demo Steps 8 and 9: name the two mistakes and show the messages on the slide.
- **Cut third (saves 15m):** the 8.4 project start: read the brief, give the check numbers (k-distances `[0.28 0.41 0.6 ]`, no usable eps), send all six steps home.

RUNNING SHORT: WHAT TO ADD

- Run `rings_starter.py` for everyone and debrief the silhouette's vote: why does a score favor the wrong answer?
- Lesson 8.4 Exercise 2 live: one OPTICS fit, DBSCAN cuts at 0.4 and 0.6 without refitting.
- Lesson 8.5 Exercise 3 live: synthetic customers from the mixture, and the 13% that are impossible.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 5: Anomaly Detection and PCA

How to teach the fifth session of Advanced Machine Learning. Lesson 8.6 turns "how typical is this point?" into a fraud review queue with Isolation Forest, LOF and honest conventions; Lesson 9.1 opens Module 9 with PCA: scale, choose, name, test inside a pipeline, and rebuild. Learners leave knowing that a detector's ranking can hold for months while a fixed threshold drifts, and that the direction of most spread is not always the useful one.

`2 lessons` `3 hrs session` `Lessons 8.6-9.1` `Session 5 of 13`

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame → live demo → guided lab (new practice) → start the lesson project. Exercises, quiz and journal go home.
- Every code block below was run to produce the output under it (Python 3.12, pandas 3.0, scikit-learn 1.9.1, 4 CPU threads). Ask for a prediction before each run.
- Keep `applications_anomaly_detection.html` and `dimensionality_pca.html` open for the pictures (IF vs LOF panels, LOF diagram, review loop, PCA arrows, scree, churn map). Labs live in `labs/module-05/`.
- This session closes Module 8 and opens Module 9. The thread through both halves: 8.6 learns what normal looks like and scores distance from it; 9.1 learns the main directions of the data, and its reconstruction error is another distance from normal.
- Teal blocks are hands-on, amber blocks are watch-outs. Lesson codes (8.6, 9.1) match the course site.

> **THE #1 RISK OF SESSION 5: SIGNS AND CONVENTIONS**
>
> Outlier detectors return -1 for outliers, `score_samples` is higher for NORMAL points, and `contamination` changes nothing about the ranking. Every one of these produces a plausible-looking wrong number with no error. Put the convention table on the board in the 8.6 demo and leave it up through the lab; when a learner reports an AUC below 0.5, ask "which way does your score point?" before anything else. In 9.1 the equivalent trap is scale: unscaled PCA quietly becomes "the column with the biggest units".

BEFORE THE SESSION

- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-05` (8 files, about 30 seconds, every line PASS).

- ☐ **Downloads:** none for today. **Session 6 needs Fashion-MNIST** (about 30 MB, by torchvision): tell learners now and put the download command in the bring list (it is in the wrap-up). If your room has poor Wi-Fi, have the `FashionMNIST/` folder on a USB stick for Session 6.
- ☐ **Projector machine:** `(.venv)` active, two notebooks `lesson_8_6` and `lesson_9_1`, font 18 pt. Tabs: both lesson pages.
- ☐ **Board:** the night guard (normal → score → review the top N → labels), an empty three-row convention table (score_samples · decision_function · predict), and for the second block a flashlight drawing: object, two shadows.
- ☐ **Timing note:** the 8.6 project's LOF step (home) takes several seconds; the lab and demo do not.

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
`AttributeError: ... 'LocalOutlierFactor' has no attribute 'predict'` (or `score_samples`)	Default `novelty=False`	`LocalOutlierFactor(novelty=True)` to score new rows.
An AUC like 0.15 for a detector	`score_samples` used without a minus sign	`anomaly_score = -model.score_samples(X)`.
`ValueError: Input X contains NaN` in PCA	`email_opens_90d` has gaps	Leave it out, or `SimpleImputer` first in the pipeline.
Loadings with the opposite sign from the page	PCA signs are arbitrary	Nothing to fix; compare absolute values or which columns move together.
`ValueError: n_components=... must be between 0 and min(n_samples, n_features)`	More components than columns	A smaller integer or a fraction like 0.90.

SESSION 5 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Lessons 8.4 and 8.5 (noise, predict, probabilities, BIC, atypical), homework check.
0:10	10	**8.6** Frame and mental model	The night guard: learn normal, score how far from it, review what you can afford.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:20	25	**8.6** Live demo	Rules, isolation by hand, Isolation Forest vs LOF, contamination, three sign and label mistakes.
0:45	25	**8.6** Guided lab	`labs/module-05/l8_6-anomaly-detection/monthly_queue_starter.py`, then `labs/module-05/l8_6-anomaly-detection/unusual_customers_starter.py` in pairs.
1:10	15	**8.6** Start the lesson project	A Fraud Review Queue, Steps 1 and 2; Steps 3 to 6 go home.
1:25	10	Break	Leave the score/decision_function/predict convention table on the board.
1:35	10	**9.1** Frame and mental model	The flashlight and the shadow: the angle with the widest shadow.
1:45	25	**9.1** Live demo	Toy components, eigenvectors, unscaled vs scaled churn, scree, loadings, the map, PCA before a model, reconstruction.
2:10	25	**9.1** Guided lab	`labs/module-05/l9_1-pca/payment_pca_starter.py`, then `labs/module-05/l9_1-pca/daily_sales_pca_starter.py` in pairs.
2:35	15	**9.1** Start the lesson project	Seasonal Fingerprints, Steps 1 to 3; Steps 4 to 6 go home.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 6 (Fashion-MNIST download).
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 110 + 100 = 210 minutes alone; the room has 150. Each lesson gets its mental model, demo, guided lab and project start.

Goes home (2 to 3 hours). A Fraud Review Queue Steps 3 to 6 and Seasonal Fingerprints Steps 4 to 6; Exercises 1 to 3 of each lesson (Exercise 4 is a stretch; 9.1's downloads Fashion-MNIST, which Session 6 needs anyway); both Quick Quizzes; both journal entries; the second lab of each block if not reached.

Protect: the contamination and sign steps of the 8.6 demo (Steps 5 to 7); the monthly queue lab through TODO 3; 9.1 Steps 3, 4 and 8 (unscaled, scaled, PCA before a model). **Cut first:** 8.6 Step 3 (isolation by hand: describe it), then 9.1 Steps 2 and 5, then the 9.1 project start (send Steps 1 to 6 home with the check numbers).

OPENING SEGMENT

0:00 Warm-Up: Density Methods and Mixtures — 10 min

HOW TO RUN IT

- W1 to W5 on the slides, two minutes, no notes, fingers 1 to 4. Run W1 and W3 to settle them.
- Homework check (2 minutes): who has `customer_density_segments.csv` and `customer_soft_segments.csv`? Ask for one borderline and one atypical customer from a journal: the distinction is today's bridge.

WARM-UP ANSWERS

Answers: **W1** B) `[ 0 0 0 1 1 1 -1] 2`; **W2** B) It has fit_predict but no predict: it describes the data it saw; **W3** C) `1.0`; **W4** B) The big drop is from 1 to 2; after that gains shrink: choose 2 to 4 and defend it with profiles; **W5** B) Atypical.

- **W1 wrong (3 clusters):** the noise label again. Today's predict(-1/1) convention will confuse the same learners: pair them in the lab.
- **W3 wrong (a very small number):** they expect probabilities to fall far from the data. That is exactly what score_samples measures, the 8.6 anomaly score.
- **W5 wrong:** spend one extra minute; borderline (low top probability) and atypical (low density) are the two ideas 8.6 builds on.

8.6 Anomaly Detection

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Tell point, contextual and collective anomalies apart and choose between supervised and unsupervised detection
- Apply simple statistical rules (z-score, interquartile range) as a baseline
- Score anomalies with Isolation Forest and Local Outlier Factor, and read the -1/1 and score conventions correctly
- Choose a threshold from a review budget instead of tuning contamination
- Evaluate a detector with ROC AUC, average precision and catches within a budget on a time-based split

TIMING IN THE ROOM

10m	Frame: the night guard; point, contextual and collective anomalies; supervised vs unsupervised vs novelty
25m	Live demo: rules, isolation by hand, IF vs LOF, contamination, three convention mistakes, accuracy
25m	Guided lab: `monthly_queue_starter.py`, then `unusual_customers_starter.py` in pairs
15m	Start the lesson project: A Fraud Review Queue, Steps 1 and 2

MENTAL MODEL TO SAY

"A night guard in a big building does not carry a list of every possible burglary. After a few weeks they know what normal sounds like: the server room hum, the cleaner at 2 a.m., the heating at five. Anything that does not fit gets a closer look. Most closer looks are nothing, and that is fine: the point is to look in the right places."

Then the three kinds (point, contextual, collective) with one Pinecrest example each, and the setup table: supervised (labels, Lesson 2.5), unsupervised (no labels, training data may contain anomalies), novelty (trained on clean normal data). "Labels for fraud arrive weeks late and only for the fraud someone noticed. A detector works on day one."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook `lesson_8_6`. Steps follow the lesson's sections, then its mistakes.

1. **How rare is fraud? (2m).** "Under 1% of payments. Remember the number every detector must beat: doing nothing is 99.1% accurate."

```
import numpy as np
import pandas as pd
pd.set_option("display.width", 150)
tx = pd.read_csv("data/transactions.csv", parse_dates=["timestamp"])
print(tx.shape)
print(f"fraud: {tx['is_fraud'].sum()} of {len(tx):,} ({tx['is_fraud'].mean():.
```

```
(50000, 13)
fraud: 438 of 50,000 (0.876%)
```

2 **Two classic rules (3m).** "One column and a threshold. The z-score uses the mean and standard deviation, which the extremes drag around; the IQR fence uses quartiles."

```
amount = tx["amount"]
z = (amount - amount.mean()) / amount.std()
q1, q3 = amount.quantile([0.25, 0.75])
upper_fence = q3 + 1.5 * (q3 - q1)
rules = {"z-score above 3": z > 3, f"IQR: amount above ${upper_fence:.0f}": am
for name, flagged in rules.items():
    print(f"{name:26s} flags {flagged.sum():5d}, fraud among flagged {tx.loc[f
          f"frauds caught {tx.loc[flagged, 'is_fraud'].sum()} of {tx['is_fraud
```

```
z-score above 3            flags   909, fraud among flagged 4.7%, frauds caugh
IQR: amount above $482     flags  2693, fraud among flagged 4.3%, frauds caugh
```

Both raise the fraud rate among flagged payments far above 0.9%, and both miss most fraud and flag hundreds of honest tents. Rules are the baseline, and they stay in real systems because they are explainable.

3 **Isolation by hand (3m).** "Pick a random cut between the min and max; keep the side with your point; repeat. A point in a crowd needs many cuts; an odd one, very few." Draw a number line and do two cuts on the board.

```
rng = np.random.default_rng(42)
amounts = np.append(rng.normal(150, 40, size=30).round(2), 900.0)

def isolation_depth(values, target, rng):
    """Random cuts needed until target is the only value left."""
    depth = 0
    while len(values) > 1:
        cut = rng.uniform(values.min(), values.max())
        values = values[values < cut] if target < cut else values[values >= cu
        depth += 1
    return depth

for target in [np.median(amounts), 900.0]:
    depths = [isolation_depth(amounts, target, rng) for _ in range(1000)]
    print(f"amount {target:7.2f}: average cuts to isolate {np.mean(depths):.2f
```

```
amount  155.11: average cuts to isolate 7.55
amount  900.00: average cuts to isolate 1.15
```

That is the whole idea; a forest does it with many random trees in all features at once.

4 **Isolation Forest vs Local Outlier Factor (4m).** "A tight cluster, a loose cluster, five planted oddballs. One sits just outside the tight cluster." Show the lesson's two-panel figure after the run.

```
from sklearn.datasets import make_blobs
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
X_toy, _ = make_blobs(n_samples=[300, 200], centers=[[0, 0], [6, 6]], cluster_
planted = np.array([[1.8, 1.8], [-1.6, 1.4], [10, -2], [-3, 9], [12, 12]])
X_toy = np.vstack([X_toy, planted])
y_toy = np.r_[np.zeros(500, dtype=int), np.ones(5, dtype=int)]
iso_toy = IsolationForest(n_estimators=200, random_state=42).fit(X_toy)
lof_toy = LocalOutlierFactor(n_neighbors=20).fit(X_toy)
scores = pd.DataFrame({"isolation_forest": -iso_toy.score_samples(X_toy), "lof
ranks = scores.rank(ascending=False).astype(int)
print(pd.concat([pd.DataFrame(planted, columns=["x", "y"]), ranks.tail(5).rese
```

```
      x     y  isolation_forest  lof
0   1.8   1.8                 8    1
1  -1.6   1.4                 4    4
2  10.0  -2.0                 2    3
3  -3.0   9.0                 3    2
4  12.0  12.0                 1    5
```

They disagree on (1.8, 1.8): globally it is near things (IF ranks it 8th), locally it is much sparser than its packed neighbors (LOF ranks it 1st). Draw the LOF diagram from the lesson.

5 **contamination only moves the line (3m).** "Same forest, three contamination values. Predict: do the scores change?" Then write the convention table: score_samples (higher = normal), decision_function (negative = outlier), predict (-1 / 1).

```
for contamination in ["auto", 0.01, 0.05]:
    model = IsolationForest(n_estimators=200, contamination=contamination, ran
    flagged = (model.predict(X_toy) == -1).sum()
    same_scores = np.allclose(model.score_samples(X_toy), iso_toy.score_sample
    print(f"contamination={str(contamination):5s} offset {model.offset_:.3f}
```

```
contamination=auto  offset -0.500  flagged 103  same scores: True
contamination=0.01  offset -0.645  flagged   6  same scores: True
contamination=0.05  offset -0.591  flagged  26  same scores: True
```

Never tune contamination to make a detector better: it cannot reorder anything. Choose the threshold from the review budget.

6 **Mistake: reading predict as 0 and 1 (2m).** "Code written for classifiers counts the wrong group, silently."

```
toy_pred = iso_toy.predict(X_toy)
print("labels returned:", np.unique(toy_pred))
print("wrong 'outliers found':", (toy_pred == 1).sum())
print("real outliers flagged :", (toy_pred == -1).sum())
```

```
labels returned: [-1  1]
wrong 'outliers found': 402
real outliers flagged : 103
```

7 **Mistake: the sign (2m).** "score_samples is a normality score. Fed into a metric that wants 'higher = more anomalous'..."

```
from sklearn.metrics import roc_auc_score
print("AUC of score_samples         :", round(roc_auc_score(y_toy, iso_toy.sco
print("AUC of -score_samples        :", round(roc_auc_score(y_toy, -iso_toy.sc
```

```
AUC of score_samples         : 0.001
AUC of -score_samples        : 0.999
```

An AUC far below 0.5 is almost always a flipped sign. Negate once, name it `anomaly_score`.

8 **Mistake: default LOF on new data (1m).** "novelty=False describes the data it saw."

```
lof_toy.predict(X_toy[:3])
```

```
AttributeError: This 'LocalOutlierFactor' has no attribute 'predict'
```

Fix: `LocalOutlierFactor(novelty=True)` for fit-on-one-month, score-the-next.

9 **Mistake: accuracy (1m).** "Last one, and it closes the loop with the first step."

```
all_normal_accuracy = 1 - tx["is_fraud"].mean()
print(f"accuracy of flagging nothing at all: {all_normal_accuracy:.4f}")
```

```
accuracy of flagging nothing at all: 0.9912
```

Judge with ROC AUC, average precision and catches within the budget, next to a rule baseline. Draw the review loop from the lesson: reviewed cases become labels.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **A flipped sign.** Symptom: an AUC of 0.153 in TODO 2 of the lab, or a queue that catches nothing. Coach: "Which way does score_samples point?"
- **Counting predict == 1 as outliers.** Symptom: "the detector flags 98% of payments". Coach: "What does an inlier get?"
- **Tuning contamination to catch more.** Symptom: a learner raising contamination and expecting a better AUC. Coach: "Did the scores change?" Then: "How many can the team review?"

GUIDED LAB

- **Lab A (15m, everyone):** `labs/module-05/l8_6-anomaly-detection/monthly_queue_starter.py`; reference `labs/module-05/l8_6-anomaly-detection/monthly_queue_solution.py`. Fit once on January to March, score April, May and June month by month.
- **Expected, Lab A:** April AUC 0.153 for `score_samples` and 0.847 negated; caught 33 of 72 (April), 35 of 65 (May), 33 of 64 (June) in 2% queues, against about 1.3 for a random queue; a fixed `contamination=0.02` threshold flags 1.88% to 2.16% in the fit months and 1.72% in June.
- **Debrief Lab A (2m):** "The ranking held up for three months; the fixed threshold did not. That is why the team reviews the top N."

- **Lab B (10m, pairs):** `labs/module-05/l8_6-anomaly-detection/unusual_customers_starter.py`; reference `labs/module-05/l8_6-anomaly-detection/unusual_customers_solution.py`. The 60 most unusual customers by Isolation Forest and by LOF.
- **Expected, Lab B:** rank correlation 0.515; only 9 customers in both top-60 lists; the forest's top 60 are heavy buyers (9.67 orders, $1,810 spend, 4.78 returns) who churn at 0.067; LOF's top 60 are closer to average but churn at 0.333 (overall 0.165). Debrief: unusual is not bad, and two detectors find two kinds of unusual.
- **Fast finishers:** `labs/module-05/l8_6-anomaly-detection/debug_anomaly.py` (a random split for time data, `pred == 1`, the sign, a default LOF).

START THE LESSON PROJECT

A Fraud Review Queue (lesson page, about 55 minutes alone). In the room: Steps 1 and 2.

- Step 1: the time split and the drift table. **Done** = `train 41,738 payments, test (June) 8,262 payments with 64 frauds; budget 165`.
- Step 2: `review_report` and two rules. **Done** = amount rule AUC 0.726, caught 8 of 64; velocity rule AUC 0.830, caught 26.
- **Goes home:** Steps 3 to 6: Isolation Forest 32, LOF 16, mixture 24, supervised boosting 41; review budgets with $10 per review and $375 per missed fraud; the false-alarm check for new accounts.

CHECKS FOR UNDERSTANDING

Ask out loud:

- Why a time split for payments? **Answer:** you deploy on the future; fraud patterns drift (1.0% in January to 0.77% in June).
- What does contamination change? **Answer:** only the threshold (offset_); never the scores or the ranking.
- Why does LOF catch the point next to the tight cluster? **Answer:** it compares each point's density with its neighbors'.
- Which number does the payments manager care about? **Answer:** frauds caught within the review budget, against a rule baseline.

Workbook self-check, items Q1 to Q6:

Answers: **Q1** B) `(array([-1, 1]), array([ 1, 49]))`; **Q2** D) score_samples is a normality score (higher = more normal); negate it and the AUC is 0.88; **Q3** `0.01 2 / 0.05 10`; **Q4** B) June's payments are distributed differently from the fit months, so a fixed threshold drifts: rank each month and review the top N the team can handle; **Q5** C) An anomaly score finds the unusual, not the bad: these are heavy buyers; whether unusual means a problem is a separate question; **Q6** C) `LocalOutlierFactor(novelty=True)` fitted on May, then `-score_samples(june)`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: a printed card with the convention table; in Lab A give the loop skeleton of TODO 3 on the projector.

Stretch: `debug_anomaly.py`; the lesson's Exercise 1 (a contextual amount rule) and the three-tier Bonus Challenge.

9.1 Principal Component Analysis

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain what a principal component is and why scaling matters
- Choose the number of components from explained variance
- Interpret loadings to name what each component captures
- Test whether PCA before a model helps, inside a pipeline
- Use reconstruction for compression, denoising and anomaly scores

TIMING IN THE ROOM

10m	Frame: the flashlight and the shadow; unsupervised, a mix of all columns, linear
25m	Live demo: toy components, eigenvectors, unscaled vs scaled, scree, loadings, the map, PCA before a model, reconstruction
25m	Guided lab: `payment_pca_starter.py`, then `daily_sales_pca_starter.py` in pairs
15m	Start the lesson project: Seasonal Fingerprints, Steps 1 to 3

MENTAL MODEL TO SAY

"Hold a hiking boot under a flashlight. From the side its shadow is clearly a boot; from straight above the toe it is a blob. Both are flat pictures of the same object, but one keeps far more of its shape. PCA chooses the angle for you: the one where the shadow is most spread out, because spread is where the differences between rows live."

Three facts on the board: **unsupervised** (it never sees churn), **each component is a mix of all columns** (not a selection), **linear** (straight directions; curves are Lesson 9.3).

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook `lesson_9_1`.

1. **The direction of most spread (3m).** "Two columns that move together. PCA turns the cloud until its shadow is widest." Show the arrow figure.

```
import numpy as np
import pandas as pd
from sklearn.decomposition import PCA
rng = np.random.default_rng(42)
a = rng.normal(0, 1, 300)
toy = pd.DataFrame({"feature_a": a, "feature_b": 0.6 * a + rng.normal(0, 0.35,
toy_pca = PCA(n_components=2).fit(toy)
print("components (one row per component):")
print(toy_pca.components_.round(3))
print("share of variance:", toy_pca.explained_variance_ratio_.round(3))
```

```
components (one row per component):
[[ 0.826  0.564]
 [-0.564  0.826]]
share of variance: [0.932 0.068]
```

Each row of `components_` is a unit-length direction; the first explains 93% of the variance.

2 **It is an eigenvector (2m).** "For the curious: the top eigenvector of the covariance matrix."

```
cov = np.cov(toy.to_numpy(), rowvar=False)
eigenvalues, eigenvectors = np.linalg.eigh(cov)
print("eigenvalues, largest first:", eigenvalues[::-1].round(3))
print("top eigenvector:", eigenvectors[:, -1].round(3))
```

```
eigenvalues, largest first: [1.227 0.089]
top eigenvector: [-0.826 -0.564]
```

Same numbers, opposite sign: an arrow and its opposite describe the same line. Signs are arbitrary.

3 **Unscaled churn columns (3m).** "The ten churn columns, the usual split. Predict: what does PC1 look like?"

```
from sklearn.model_selection import train_test_split
pd.set_option("display.width", 150)
customers = pd.read_csv("data/customers_churn.csv")
features = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value", "da
            "returns_12m", "return_rate", "support_tickets_12m", "discount_sha
X = customers[features]
y = customers["churned"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, stra
raw_pca = PCA().fit(X_train)
print("share of variance, first three:", raw_pca.explained_variance_ratio_[:3]
print(pd.Series(raw_pca.components_[0], index=features).round(3).iloc[[1, 2, 3
```

```
share of variance, first three: [0.916 0.052 0.027]
orders_12m               0.004
spend_12m                0.991
avg_order_value          0.125
days_since_last_order   -0.052
dtype: float64
```

PC1 IS spend_12m (weight 0.991): dollars dwarf fractions. PCA on unscaled data tells you which column has the largest units.

4 **Scale first (2m).** "Every column starts with variance 1."

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler().set_output(transform="pandas")
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)
pca = PCA().fit(X_train_s)
print("share of variance:", pca.explained_variance_ratio_.round(3))
print("cumulative:       ", pca.explained_variance_ratio_.cumsum().round(3))
```

```
share of variance: [0.253 0.175 0.131 0.099 0.09  0.085 0.072 0.061 0.029 0.0(
cumulative:        [0.253 0.427 0.558 0.657 0.748 0.833 0.905 0.966 0.994 1.
```

No elbow: the variance is spread over many components. Honest: these columns are only partly redundant.

5 **A variance target (1m).** "A fraction as n_components keeps enough components to reach it."

```
pca90 = PCA(n_components=0.90).fit(X_train_s)
print("components kept:", pca90.n_components_)
print("variance kept:  ", pca90.explained_variance_ratio_.sum().round(3))
```

```
components kept: 7
variance kept:   0.905
```

6 **Reading the loadings (3m).** "Large weights, either sign."

```
loadings = pd.DataFrame(pca.components_[:3].T, index=features, columns=["PC1",
print(loadings.round(2))
```

```
                          PC1   PC2   PC3
tenure_months            0.08 -0.01 -0.24
orders_12m               0.45 -0.17 -0.41
spend_12m                0.46  0.41 -0.22
avg_order_value         -0.01  0.65  0.03
days_since_last_order -0.31  0.18  0.35
returns_12m              0.47  0.04  0.42
return_rate              0.40  0.07  0.56
support_tickets_12m      0.27 -0.23  0.13
discount_share           0.02 -0.52  0.22
app_user                 0.18 -0.12 -0.21
```

PC1 how active, PC2 big tickets versus deals, PC3 returns relative to orders. Names are your interpretation.

7 **A two-dimensional map (2m).** "43% of the variance in one picture. Where are the churners?" Show the lesson's scatter.

```
pca2 = PCA(n_components=2).fit(X_train_s)
map_train = pd.DataFrame(pca2.transform(X_train_s), columns=["PC1", "PC2"], in
map_train["churned"] = y_train
print(map_train.groupby("churned")[["PC1", "PC2"]].mean().round(2))
```

```
           PC1   PC2
churned
0         0.14  0.06
1        -0.72 -0.28
```

A lean, heavy overlap: the biggest sources of variation are not the churn signal.

8 **PCA before a model: test it (4m).** "Scaler, PCA and logistic regression in one pipeline, so each fold fits its own. Compare with no PCA."

```
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score, StratifiedKFold
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
for k in [2, 4, 6, 8]:
    pipe = make_pipeline(StandardScaler(), PCA(n_components=k), LogisticRegres
    auc = cross_val_score(pipe, X_train, y_train, cv=cv, scoring="roc_auc")
    print(f"PCA {k:2d} components   AUC {auc.mean():.3f}")
auc = cross_val_score(make_pipeline(StandardScaler(), LogisticRegression()), X
print(f"no PCA (10 columns)  AUC {auc.mean():.3f}")
```

```
PCA  2 components   AUC 0.687
PCA  4 components   AUC 0.786
PCA  6 components   AUC 0.812
PCA  8 components   AUC 0.829
no PCA (10 columns)  AUC 0.829
```

Eight components to match the plain model. PCA ranks directions by spread, and it never saw churn (Lesson 9.2 fixes exactly this).

9 **Running PCA backward (3m).** "inverse_transform rebuilds rows from the kept components; the gap is what they could not express."

```python
for k in [2, 5, 7, 9, 10]:
    model_k = PCA(n_components=k).fit(X_train_s)
    rebuilt = model_k.inverse_transform(model_k.transform(X_test_s))
    rmse = np.sqrt(((X_test_s.to_numpy() - rebuilt) ** 2).mean())
    print(f"{k:2d} components   reconstruction RMSE {rmse:.3f}")
```

```
 2 components   reconstruction RMSE 0.765
 5 components   reconstruction RMSE 0.509
 7 components   reconstruction RMSE 0.313
 9 components   reconstruction RMSE 0.081
10 components   reconstruction RMSE 0.000
```

This is compression, denoising and an anomaly score. The lesson's fraud version: AUC 0.819 but 19 caught against the forest's 32. Say why (extreme columns vs broken correlations).

10 **Mistake: more components than columns (1m).** "Ten columns, at most ten directions."

```python
PCA(n_components=12).fit(X_train_s)
```

```
ValueError: n_components=12 must be between 0 and min(n_samples, n_features)=1
```

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Forgetting to scale.** Symptom: one column with weight near 1.0 in PC1 (account age in the lab). Coach: "What units is that column in?"
- **Reading signs.** Symptom: "PC1 is negative for loyal customers, so loyalty is bad". Coach: multiply the arrow by -1; does anything change?
- **Most variance = most useful.** Symptom: expecting PC1 to separate fraud in the lab. Coach: "Which component does?" (PC2.)

GUIDED LAB

- **Lab A (15m, everyone):** `labs/module-05/l9_1-pca/payment_pca_starter.py`; reference `labs/module-05/l9_1-pca/payment_pca_solution.py`. PCA on the nine payment features.
- **Expected, Lab A:** unscaled PC1 explains 0.95 and is `account_age_days` alone; scaled shares from 0.159 down to 0.064, eight components for 90%; PC1 = basket size (amount and items 0.70 each), PC2 = risk signals (card attempts and velocity 0.52, country mismatch 0.39, new device 0.35); frauds average PC2 3.10 against -0.03; fraud rate 0.084 in the top 2% of PC1 and 0.206 in the top 2% of PC2 (base 0.009).
- **Debrief:** "Here a direction of spread DID carry the fraud signal, but the second one, not the first. PCA never knew; you found out by checking."
- **Lab B (10m, pairs, finish at home):** `labs/module-05/l9_1-pca/daily_sales_pca_starter.py`; reference `labs/module-05/l9_1-pca/daily_sales_pca_solution.py`. Days × four stores, one unit: PCA's home ground.
- **Expected, Lab B:** 1,090 open days; shares 0.841, 0.067, 0.051, 0.042; PC1 weights about 0.5 for every store (the chain's day: 1.16 to 1.23 on the three sale events, -0.05 on ordinary days); PC2 Asheville 0.70 vs Flagstaff -0.66, falling from 0.117 in 2023 to -0.130 in 2025 as Flagstaff grows from $4,997 to $6,597 a day; PC1 alone leaves a gap of 0.105 log units.

START THE LESSON PROJECT

Seasonal Fingerprints (lesson page, about 45 minutes alone). In the room: Steps 1 to 3.

- Step 1: monthly averages pivoted. **Done** = `(96, 24)`.
- Step 2: log1p and remove each row's mean; ask why the columns are NOT standardized (one unit, swings are information).
- Step 3: **done** = first six shares `[0.799 0.019 0.018 0.018 0.015 0.014]`: one pattern plus noise.
- **Goes home:** Steps 4 to 6: the PC1 wave, seasonal scores per product (Snow -6.58, Tents 2.08), one history rebuilt from PC1 (gap 0.189), `seasonal_scores.csv`.

CHECKS FOR UNDERSTANDING

Ask out loud:

- What does explained_variance_ratio_ tell you? **Answer:** each component's share of the total variance.
- When should you NOT standardize before PCA? **Answer:** when all columns share one meaningful unit (monthly sales, pixels).
- Why put PCA inside the pipeline for cross-validation? **Answer:** each fold fits its own scaler and PCA: no leakage.
- What is reconstruction error? **Answer:** the gap between a row and its rebuild from the kept components: what they cannot express.

Workbook self-check, items Q7 to Q12:

Answers: **Q7** D) `[0.732 0.268 0. ]`; **Q8** B) Account age is measured in days, up to about 7,000, so its variance dwarfs the 0/1 flags and small counts; **Q9** `(50, 3) (3, 6)`; **Q10** D) 784 highly correlated pixel columns feeding a slow k-nearest-neighbors model; **Q11** A) Much of this fraud is extreme on a few columns rather than breaking the correlations most payments share; **Q12** `7`: hardly a reduction. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: draw the flashlight again with two shadows and ask which component each shadow is. Let pairs skip TODO 1 of Lab A if time is short.

Stretch: Lab B in full; the lesson's Exercise 1 (PCA by hand with SVD) and the Bonus Challenge (reconstruction error vs Isolation Forest on customers).

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Answers on sticky notes. Question 3 tells you how much of the Session 6 warm-up goes to scaling and to supervised vs unsupervised projections.

1. An Isolation Forest's `predict` returns -1 and 1, and `score_samples` gives an AUC of 0.2. Write the two lines you would use instead to build a review queue of 165. **Answer:** `anomaly_score = -model.score_samples(X)` and `queue = np.argsort(-anomaly_score)[:165]` (then count frauds in it).
2. Your manager asks you to raise contamination from 0.01 to 0.05 "so the detector catches more fraud". What do you explain? **Answer:** contamination only moves the threshold; the ranking (and the AUC) is unchanged. Choose how many to review from the team's budget and the costs.
3. You run PCA on ten customer columns without scaling and PC1 explains 92%. What happened, and when would skipping the scaler be right? **Answer:** the column with the largest units (spend) took over. Skip scaling only when all columns share one meaningful unit, like monthly sales or pixels.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** A Fraud Review Queue Steps 3 to 6 (IF 32, LOF 16, mixture 24, boosting 41; budgets and costs; false alarms by account age) and Seasonal Fingerprints Steps 4 to 6 (`seasonal_scores.csv`).
- **Lesson exercises.** 8.6: Exercises 1 to 3 (contextual rule, a drifting threshold, clean training data); Exercise 4 is a stretch. 9.1: Exercises 1 to 3 (SVD by hand, 95%, a segment map); Exercise 4 (eigen-clothes) is a stretch and downloads Fashion-MNIST.
- **Both Quick Quizzes** and **both journal entries**.

- **Before the Next Lesson (from 9.1):** rerun the project with weekly averages; predict which column a label-aware direction would lean on for churn.
- **Optional:** `unusual_customers_starter.py`, `daily_sales_pca_starter.py`, `debug_anomaly.py` if not finished in class.

BRING LIST FOR SESSION 6

- A laptop with the course environment; `import torch`, `import torchvision` and `import umap` must work (they are in `requirements.txt`; `check_environment.py` tests them).
- **Fashion-MNIST downloaded before the session** (about 30 MB). If `check_environment.py` already ends with READY for Advanced Machine Learning (the `--download` run from Session 1), you have it. Otherwise, from the `pinecrest-ml` folder, run `python labs/module-01/l7_0-orientation/check_environment.py --download` (all the course's image data and weights, about 175 MB, once), or for Fashion-MNIST alone `python -c "from torchvision import datasets; datasets.FashionMNIST('data', train=True, download=True); datasets.FashionMNIST('data', train=False, download=True)"`, or do Lesson 9.1's Exercise 4. Labs find it in `data/FashionMNIST/`.
- The `pinecrest-ml` folder with `data/` and `labs/`; Session 6's starters are in `labs/module-06/`.
- Your project notebooks from Sessions 4 and 5 (Session 6 holds **Checkpoint 1 on Modules 7 to 9**, open notebook: your Module 7 to 9 notebooks are allowed).
- The workbook and the journal.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** 8.6 Steps 5 to 7, Lab A through TODO 3, 9.1 Steps 3, 4 and 8.
- **Cut first (saves 3m):** 8.6 Step 3 (isolation by hand): describe it with the number line.
- **Cut second (saves 3m):** 9.1 Steps 2 and 5.
- **Cut third (saves 15m):** the 9.1 project start: give the check numbers and send it home.

RUNNING SHORT: WHAT TO ADD

- Run `debug_anomaly.py` together and fix it in four steps.
- Lesson 8.6 Exercise 2 live (the drifting threshold), then compare with Lab A's months.
- Lesson 9.1 Exercise 1 live: PCA as an SVD in three lines.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 6: LDA, t-SNE and UMAP

How to teach the sixth session of Advanced Machine Learning: LDA, the supervised counterpart of PCA (and its leak), then t-SNE and UMAP, the most persuasive and most over-read pictures in machine learning, checked with numbers. The session ends with Checkpoint 1, a scored 30-minute open-notebook task on Modules 7 to 9.

`2 lessons` `3 hrs session` `Lessons 9.2–9.3` `Checkpoint 1` `Session 6 of 13`

HOW TO USE THIS GUIDE

- Two lessons in shorter blocks (60 minutes each) because the last 30 minutes before the wrap-up are **Checkpoint 1**, a scored open-notebook task on Modules 7 to 9.
- Every code block below was run to produce the output under it (Python 3.12, scikit-learn 1.9.1, umap-learn, 4 CPU threads). t-SNE and UMAP outputs can differ in the last digits on another machine: say "yours may vary slightly" once.
- Keep `dimensionality_lda.html` and `dimensionality_tsne.html` open for the figures (PCA vs LDA panels, loyalty-tier maps, the garment maps, the three-blob and noise panels). Labs live in `labs/module-06/`.
- Teal blocks are hands-on, amber blocks are watch-outs.

> **THE #1 RISK OF SESSION 6: RUNNING OUT OF TIME FOR THE CHECKPOINT**
>
> t-SNE and UMAP take seconds per run, and a room of laptops will be slower than yours. Protect the 2:20 start of Checkpoint 1: if the 9.3 block runs over, cut the lab to TODO 1 and 2 and send the rest home. The second risk is technical: Fashion-MNIST not downloaded. Check it in the warm-up (one command, below) and have the `FashionMNIST/` folder on a USB stick.

BEFORE THE SESSION

- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-06` (10 files, about 90 seconds; the Fashion-MNIST lab about 30 s).
- ☐ **Downloads and caches:** Fashion-MNIST (about 30 MB download, about 86 MB on disk) must be in each learner's `pinecrest-ml/data/FashionMNIST/` (Session 5 bring list). Bring a USB copy of that folder. No pretrained weights today.

- ☐ **Projector machine:** run `import umap` once before the session (it compiles its routines on first import, several seconds). Two notebooks `lesson_9_2` and `lesson_9_3`.
- ☐ **Checkpoint:** print or share `labs/module-06/checkpoint-1/checkpoint_starter.py` and the workbook's Checkpoint 1 page; keep `checkpoint_solution.py` and the rubric (answer key) to yourself.
- ☐ **Board:** the team photo (two clubs, two camera angles), and the five reading rules for maps.

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
`RuntimeError: Dataset not found. You can use download=True to download it`	Fashion-MNIST not downloaded and offline	Copy `FashionMNIST/` from the USB stick into `data/`, or download over the network (about 30 MB).
`ModuleNotFoundError: No module named 'umap'`	umap-learn missing (it is in `requirements.txt`)	`python -m pip install umap-learn`; it imports as `umap`.
`UserWarning: n_jobs value 1 overridden to 1 by setting random_state`	A seeded UMAP runs on one core	Expected (Lesson 9.3, Common Mistake 3); the labs silence it.
t-SNE takes over a minute	Too many rows, or no PCA first	Sample (1,000 to 2,000 rows) and reduce to 50 components first.
A different-looking map from the page	Another seed, library or machine	Compare neighbor scores, not pictures.

SESSION 6 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Lessons 8.6 and 9.1, homework check, Fashion-MNIST check.
0:10	10	**9.2** Frame and mental model	The team photo: the angle where the clubs separate.
0:20	20	**9.2** Live demo	PCA vs LDA on one picture, the churn axis, LDA as a classifier, more classes, shrinkage, the noise leak.
0:40	20	**9.2** Guided lab	`labs/module-06/l9_2-lda/payment_category_lda_starter.py`, then `labs/module-06/l9_2-lda/noise_leak_starter.py` in pairs.
1:00	10	**9.2** Start the lesson project	The Loyalty-Tier Map, Steps 1 and 2; Steps 3 to 6 go home.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
1:10	10	Break	Start the UMAP import on the projector machine (it compiles on first use).
1:20	10	**9.3** Frame and mental model	The seating chart: friends together; table sizes and distances mean nothing.
1:30	20	**9.3** Live demo	A t-SNE map of 1,000 garments, perplexity, what maps distort, neighbor checks, UMAP and transform.
1:50	20	**9.3** Guided lab	`labs/module-06/l9_3-tsne-umap/fashion_map_starter.py`, then `labs/module-06/l9_3-tsne-umap/fraud_map_starter.py` in pairs.
2:10	10	**9.3** Start the lesson project	The Customer Map, Steps 1 and 2; Steps 3 to 6 go home.
2:20	30	Checkpoint 1 (Modules 7 to 9)	`labs/module-06/checkpoint-1/checkpoint_starter.py`: open notebook, 30 minutes, scored with the 5-row rubric.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 7 (PyTorch).
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 90 + 100 minutes alone; the room has 120 after the warm-up, break, checkpoint and wrap-up, so each lesson gets a 10-minute frame, a 20-minute demo, a 20-minute lab and a 10-minute project start.

Goes home (2 to 3 hours). The Loyalty-Tier Map Steps 3 to 6 and The Customer Map Steps 3 to 6; Exercises 1 to 3 of each lesson (Exercise 4 is a stretch); both Quick Quizzes; both journal entries; the second lab of each block.

Protect: the noise-leak step of the 9.2 demo, the distortion and noise steps of 9.3, and all 30 minutes of the checkpoint. **Cut first:** 9.2 Steps 5 and 6 (more classes, shrinkage), then 9.3 Step 3 (perplexity), then both project starts (give the check numbers).

OPENING SEGMENT

0:00 Warm-Up: Anomaly Detection and PCA

10 min

HOW TO RUN IT

- W1 to W5, two minutes, no notes; run W1 and W3.
- Fashion-MNIST check (1 minute): everyone runs `python -c "from torchvision import datasets; print(len(datasets.FashionMNIST('data', train=True)))"` from `pinecrest-ml`. 60000 = ready. An error = USB stick now.
- Announce the checkpoint: at 2:20, 30 minutes, open notebook, scored with a rubric. It is a check, not an exam.

WARM-UP ANSWERS

Answers: **W1** B) `2 0.67`; **W2** C) Only the threshold: more rows get -1, in the same order; **W3** C) `[1. 0.]`; **W4** B) PCA picks directions of spread, not of churn; the recency signal is spread over several components; **W5** C) When all columns share one meaningful unit, like monthly sales or pixels.

- **W4 wrong:** the bridge to 9.2. Ask them to hold the AUC 0.687 in mind; LD1 alone reaches 0.824 today.
- **W5 wrong:** pixels share one unit: today's image maps are not standardized.

LESSON PLANS

9.2 Linear Discriminant Analysis

60 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain how LDA differs from PCA: it uses the labels to find directions that separate classes
- Use LDA to reduce dimensions for a plot or as a step before another model
- Use LDA as a classifier and compare it with logistic regression
- Know the limit on components (classes minus one) and when shrinkage helps
- Keep LDA inside the pipeline so cross-validation stays honest

TIMING IN THE ROOM

10m	Frame: the team photo; Fisher's criterion in words; at most classes minus one axes
20m	Live demo: PCA vs LDA, the churn axis, LDA as a classifier, more classes, shrinkage, mistakes, the noise leak
20m	Guided lab: `payment_category_lda_starter.py`, then `noise_leak_starter.py` in pairs
10m	Start the lesson project: The Loyalty-Tier Map, Steps 1 and 2

MENTAL MODEL TO SAY

"Two hiking clubs pose on a field and you photograph them from the side, so the picture flattens everyone onto one line. PCA picks the angle that makes the crowd look widest, even if the clubs end up mixed. LDA walks around the field until the two clubs appear as two tight, separate groups: each bunched up, the two far apart."

Then the consequence to memorize: the class means of C classes span at most C - 1 directions, so two classes give one axis and three tiers give two, however many features.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

1. **Two methods, one picture (3m).** "Two long, thin, parallel classes. Predict: where does PCA point, where does LDA point?" Show the lesson's three-panel figure.

```
import numpy as np
import pandas as pd
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
rng = np.random.default_rng(42)
n = 200
along = rng.normal(0, 3, 2 * n)
across = rng.normal(0, 0.6, 2 * n) + np.repeat([-1.0, 1.0], n)
angle = np.deg2rad(30)
rotation = np.array([[np.cos(angle), -np.sin(angle)], [np.sin(angle), np.cos(a
toy_X = np.column_stack([along, across]) @ rotation.T
toy_y = np.repeat([0, 1], n)
toy_pca = PCA(n_components=1).fit(toy_X)
toy_lda = LinearDiscriminantAnalysis(n_components=1).fit(toy_X, toy_y)
lda_dir = toy_lda.scalings_[:, 0] / np.linalg.norm(toy_lda.scalings_[:, 0])
print("PCA direction:", toy_pca.components_[0].round(3))
print("LDA direction:", lda_dir.round(3))
```

```
PCA direction: [0.866 0.5  ]
LDA direction: [-0.489  0.872]
```

Almost at right angles: PCA follows the long axis (the spread), LDA points across the classes.

2. **What each one keeps (2m).** "Distance between the class means, in within-class standard deviations."

```
def separation(z, labels):
    """Distance between the two class means, in units of the average within-cl
    a, b = z[labels == 0], z[labels == 1]
    return abs(a.mean() - b.mean()) / np.sqrt((a.var() + b.var()) / 2)
print(f"separation on PC1: {separation(toy_pca.transform(toy_X)[:, 0], toy_y):
print(f"separation on LD1: {separation(toy_lda.transform(toy_X)[:, 0], toy_y):
```

```
separation on PC1: 0.05
separation on LD1: 3.32
```

3. **One axis for churn (4m).** "Two classes, at most one axis. Its weights read like a churn story."

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
customers = pd.read_csv("data/customers_churn.csv")
features = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value", "da
            "returns_12m", "return_rate", "support_tickets_12m", "discount_sha
X = customers[features]
y = customers["churned"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, stra
scaler = StandardScaler().set_output(transform="pandas")
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)
lda = LinearDiscriminantAnalysis()
lda.fit(X_train_s, y_train)
churn_axis = lda.transform(X_test_s)
print("transformed shape:", churn_axis.shape)
print(pd.Series(lda.scalings_[:, 0], index=features).round(2).sort_values().il
```

```
transformed shape: (1500, 1)
app_user                -0.21
tenure_months           -0.09
support_tickets_12m      0.30
discount_share           0.50
days_since_last_order    0.92
dtype: float64
```

Days since the last order dominates, then discounts and tickets; app users on the other side. Lesson 9.1's PCA buried recency; LDA puts it first because the labels told it to.

4 **LDA as a classifier (3m).** "The same object predicts: Gaussian classes with one shared covariance, so a straight boundary."

```
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
from sklearn.metrics import accuracy_score, roc_auc_score
models = {"baseline": DummyClassifier(strategy="prior"), "LDA": LinearDiscrimi
          "logistic regression": LogisticRegression(), "QDA": QuadraticDiscrim
for name, model in models.items():
    model.fit(X_train_s, y_train)
    proba = model.predict_proba(X_test_s)[:, 1]
    acc = accuracy_score(y_test, model.predict(X_test_s))
    print(f"{name:20s} accuracy {acc:.3f}   ROC AUC {roc_auc_score(y_test, pro
```

```
baseline             accuracy 0.835   ROC AUC 0.500
LDA                  accuracy 0.853   ROC AUC 0.824
logistic regression  accuracy 0.848   ROC AUC 0.823
QDA                  accuracy 0.813   ROC AUC 0.803
```

Neck and neck with logistic regression. QDA (one covariance per class) is worse: flexibility is not free.

5 **More classes, more axes (2m).** "Four classes in ten dimensions."

```
from sklearn.datasets import make_blobs
blobs_X, blobs_y = make_blobs(n_samples=400, centers=4, n_features=10, cluster
blob_lda = LinearDiscriminantAnalysis().fit(blobs_X, blobs_y)
print("transform shape:", blob_lda.transform(blobs_X).shape)
print("separation carried by each axis:", blob_lda.explained_variance_ratio_.r
```

```
transform shape: (400, 3)
separation carried by each axis: [0.781 0.136 0.083]
```

The ratio is the share of class SEPARATION, not of total variance.

6 **Many features, few rows: shrinkage (2m).** "100 rows, 200 features."

```
from sklearn.datasets import make_classification
from sklearn.model_selection import cross_val_score, StratifiedKFold
wide_X, wide_y = make_classification(n_samples=100, n_features=200, n_informat
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
for name, model in [("plain LDA (svd)", LinearDiscriminantAnalysis()),
                    ("LDA with shrinkage", LinearDiscriminantAnalysis(solver="
    print(f"{name:20s} CV accuracy {cross_val_score(model, wide_X, wide_y, cv=
```

```
plain LDA (svd)      CV accuracy 0.540
LDA with shrinkage   CV accuracy 0.680
```

7 **Two loud mistakes (1m).** "Too many axes; shrinkage with the default solver."

```
LinearDiscriminantAnalysis(n_components=2).fit(X_train_s, y_train)
LinearDiscriminantAnalysis(shrinkage="auto").fit(X_train_s, y_train)
```

```
ValueError: n_components cannot be larger than min(n_features, n_classes - 1).
NotImplementedError: shrinkage not supported with 'svd' solver.
```

8 **The quiet mistake: LDA before cross-validation (3m).** "Pure noise, random labels. No honest model beats one in three."

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
noise_X = rng.normal(size=(100, 500))
noise_y = rng.integers(0, 3, size=100)
reduced = LinearDiscriminantAnalysis(n_components=2).fit_transform(noise_X, no
leaky = cross_val_score(KNeighborsClassifier(n_neighbors=5), reduced, noise_y,
honest_pipe = make_pipeline(LinearDiscriminantAnalysis(n_components=2), KNeigh
honest = cross_val_score(honest_pipe, noise_X, noise_y, cv=cv).mean()
print(f"LDA fitted on all rows, then CV: {leaky:.2f}")
print(f"LDA inside the pipeline:         {honest:.2f}")
```

```
LDA fitted on all rows, then CV: 0.62
LDA inside the pipeline:         0.34
```

Write both numbers on the board. Every supervised step (LDA, target encoding, selection that looks at y) belongs inside the pipeline. The lab repeats this on real customers.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Asking for too many axes.** `n_components=2` for churn. Coach: "How many classes? Minus one."
- **Reading LDA's explained_variance_ratio_ as PCA's.** Coach: "Share of what? Of the separation between classes."
- **Reducing before cross-validating.** Coach with the board: 0.62 vs 0.34, and in the lab 0.998 vs 0.481.

GUIDED LAB

- **Lab A (12m, everyone):** `labs/module-06/l9_2-lda/payment_category_lda_starter.py`; reference `labs/module-06/l9_2-lda/payment_category_lda_solution.py`. Ten product categories from nine payment columns.
- **Expected, Lab A:** 9 axes; LD1 carries 0.995 of the separation and is amount (1.42) against items (-0.59): price per item; median LD1 from Accessories (-0.93, $65) to Tents (1.41, $350); accuracy 0.290 against a 0.219 baseline; six categories never predicted, Apparel recall 0.86.
- **Debrief:** "A weak result, honestly reported, and LDA told you WHY in one axis: categories differ mainly by price per item."
- **Lab B (8m, pairs):** `labs/module-06/l9_2-lda/noise_leak_starter.py`; reference `labs/module-06/l9_2-lda/noise_leak_solution.py`. 400 customers, 10 real + 290 noise columns.

- **Expected, Lab B:** shuffled labels: leaky AUC 0.998, honest 0.481; true labels: leaky 1.000, honest 0.541; the 10 real columns alone: 0.776. Second lesson: noise columns drown real signal for a reducer that has to estimate a covariance.

START THE LESSON PROJECT

The Loyalty-Tier Map (about 45 minutes alone). In the room: Steps 1 and 2.

- Step 1: stratify on `loyalty_tier`; **done** = Basic 2129, Silver 2015, Gold 356.
- Step 2: baseline 0.473, LDA 0.909, logistic regression 0.921.
- **Goes home:** Steps 3 to 6: LD1 carries 0.948 and is spend plus tenure; PCA vs LDA maps; errors only between neighboring tiers; `tier_map.csv`.

CHECKS FOR UNDERSTANDING

Ask out loud:

- How many axes for three tiers? **Answer:** two.
- What do priors change? **Answer:** the threshold, not the axis or the ranking.
- When do you need shrinkage? **Answer:** many features, few rows; eigen solver if you also need transform.

Workbook self-check, items Q1 to Q6:

Answers: **Q1** B) `(200, 2)`; **Q2** A) Class means far apart relative to the spread inside each class; PCA chooses the largest total spread and never sees the labels; **Q3** `[0.5 0.5]`; **Q4** C) The payment details separate categories along essentially one direction, price per item, and only weakly (accuracy 0.290 against 0.219); **Q5** D) LDA used every row's label, including the test folds', to build its axis; with 300 columns it can separate any labels; **Q6** B) `LinearDiscriminantAnalysis(solver="eigen", shrinkage="auto")`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: the team-photo drawing with both camera angles. **Stretch:** Lab B; the lesson's Exercise 1 (kNN on two LDA axes, 0.916 vs 0.864).

9.3 t-SNE and UMAP

60 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain what t-SNE and UMAP preserve and what they distort
- Tune perplexity (t-SNE) and n_neighbors and min_dist (UMAP), and see how the picture changes
- Read an embedding without over-interpreting cluster sizes or distances
- Check a map with numbers (neighbors kept, trustworthiness) instead of by eye
- Choose between PCA, t-SNE and UMAP for a task, and place new rows with UMAP

TIMING IN THE ROOM

10m	Frame: the seating chart; what is kept, what is lost
20m	Live demo: the garment map, perplexity, distortion, noise, neighbor checks, UMAP, transform
20m	Guided lab: `fashion_map_starter.py` (start TODO 1, talk while it runs), then `fraud_map_starter.py`
10m	Start the lesson project: The Customer Map, Steps 1 and 2

MENTAL MODEL TO SAY

"You seat 1,000 wedding guests in one hall; the only rule is: sit near the people you know best. The tables of friends mean something. How far one table is from another, and how big a table is, mean nothing: the only goal was friends together." Then the five reading rules on the board: neighbors meaningful; cluster sizes not; distances between clusters not; axes meaningless; validate clusters in the original data.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

1. **1,000 garments (2m).** "Each image is a row of 784 pixels. The labels only color the picture."

```
import numpy as np
import pandas as pd
from torchvision import datasets
fashion = datasets.FashionMNIST("data", train=False, download=True)
images = fashion.data.numpy().reshape(len(fashion), -1) / 255
labels = fashion.targets.numpy()
class_names = fashion.classes
rng = np.random.default_rng(42)
pick = rng.choice(len(images), size=1000, replace=False)
img, lab = images[pick], labels[pick]
print(img.shape, img.min(), img.max())
print(np.bincount(lab))
```

```
(1000, 784) 0.0 1.0
[102  90  88 101  99 103 106 100 102 109]
```

2 **PCA to 50, then t-SNE (3m).** "The standard recipe: PCA first removes pixel noise and is much faster." Show the lesson's PCA vs t-SNE figure.

```
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
pca50 = PCA(n_components=50, random_state=42).fit(img)
img50 = pca50.transform(img)
pca2 = img50[:, :2]
tsne = TSNE(n_components=2, perplexity=30, random_state=42)
emb_tsne = tsne.fit_transform(img50)
print(emb_tsne.shape, "KL divergence:", round(tsne.kl_divergence_, 3))
```

```
(1000, 2) KL divergence: 0.676
```

Footwear, trousers and bags get islands; shirts, pullovers and coats stay mixed because they really look alike. Output label: yours may vary slightly (another machine's math libraries).

3 **Perplexity is a lens (3m).** "Roughly how many neighbors each point listens to."

```
small50, small_lab = img50[:500], lab[:500]
for perp in [5, 30, 100]:
    emb = TSNE(perplexity=perp, random_state=42).fit_transform(small50)
    print(f"perplexity {perp:3d}: map spans {np.ptp(emb[:, 0]):.0f} by {np.ptp
```

```
perplexity   5: map spans 132 by 115 units
perplexity  30: map spans 47 by 57 units
perplexity 100: map spans 19 by 19 units
```

The scale of the map changes with its settings: axis numbers have no units. Look at more than one perplexity.

4 **What a map distorts (3m).** "Three groups whose truth we know: tight, wide, far away."

```
from sklearn.datasets import make_blobs
blob_X, blob_y = make_blobs(n_samples=[100, 100, 100], centers=[[0, 0], [6, 0]
                            cluster_std=[0.3, 2.0, 1.0], random_state=42)
blob_emb = TSNE(perplexity=30, random_state=42).fit_transform(blob_X)
def spread(Z, g):
    return Z[blob_y == g].std(axis=0).mean()
def gap(Z, g1, g2):
    return np.linalg.norm(Z[blob_y == g1].mean(axis=0) - Z[blob_y == g2].mean(
print("spread original:", "  ".join(f"{spread(blob_X, g):.2f}" for g in range(
      "   map:", "  ".join(f"{spread(blob_emb, g):.2f}" for g in range(3)))
print(f"gap A-B vs B-C original: {gap(blob_X, 0, 1):.1f} vs {gap(blob_X, 1, 2)
      f"   map: {gap(blob_emb, 0, 1):.1f} vs {gap(blob_emb, 1, 2):.1f}")
```

```
spread original: 0.28  1.96  0.99    map: 3.04  4.48  3.08
gap A-B vs B-C original: 6.3 vs 23.7   map: 31.9 vs 41.1
```

Sizes nearly equalized, gaps compressed. Then the pure-noise map (next step) invents clumps. Put the red reading-rules card on the board.

5 **Clumps from nothing (1m).** "Random noise, perplexity 5." Show the lesson's right-hand panel.

```
noise = rng.normal(size=(200, 10))
noise_emb = TSNE(perplexity=5, random_state=42).fit_transform(noise)
print("rows of pure noise mapped:", len(noise_emb))
```

```
rows of pure noise mapped: 200
```

6 **Check a map with numbers (3m).** "Of each image's 10 true neighbors, how many are map neighbors?"

```
from sklearn.neighbors import NearestNeighbors
from sklearn.manifold import trustworthiness
def neighbors_kept(original, embedded, k=10):
    """Average share of each point's k nearest original neighbors that are als
    nn_orig = NearestNeighbors(n_neighbors=k + 1).fit(original).kneighbors(ori
    nn_emb = NearestNeighbors(n_neighbors=k + 1).fit(embedded).kneighbors(embe
    return np.mean([len(set(a) & set(b)) / k for a, b in zip(nn_orig, nn_emb)]
for name, emb in [("PCA (2 components)", pca2), ("t-SNE", emb_tsne)]:
    print(f"{name:20s} neighbors kept {neighbors_kept(img50, emb):.3f}"
          f"   trustworthiness {trustworthiness(img50, emb, n_neighbors=10):.3
```

```
PCA (2 components)   neighbors kept 0.235   trustworthiness 0.923
t-SNE                neighbors kept 0.599   trustworthiness 0.988
```

7 **UMAP (2m).** "Same spirit, a graph laid out flat, faster, and it can place new rows." The import compiles on first use.

```
import warnings
import umap
warnings.filterwarnings("ignore", message="n_jobs value")
reducer = umap.UMAP(n_neighbors=15, min_dist=0.1, random_state=42)
emb_umap = reducer.fit_transform(img50)
print(f"neighbors kept {neighbors_kept(img50, emb_umap):.3f}"
      f"   trustworthiness {trustworthiness(img50, emb_umap, n_neighbors=10):.
```

```
neighbors kept 0.519   trustworthiness 0.981
```

8 **Placing new rows (2m).** "Through the same fitted PCA, then the fitted UMAP."

```
from sklearn.neighbors import KNeighborsClassifier
rest = np.setdiff1d(np.arange(len(images)), pick)
new_pick = rng.choice(rest, size=500, replace=False)
new_umap = reducer.transform(pca50.transform(images[new_pick]))
knn_on_map = KNeighborsClassifier(n_neighbors=10).fit(emb_umap, lab)
print("new rows placed:", new_umap.shape)
print(f"accuracy on 500 new images: {knn_on_map.score(new_umap, labels[new_pic
```

```
new rows placed: (500, 2)
accuracy on 500 new images: 0.690
```

In real work: `make_pipeline(PCA(50), umap.UMAP(...))` so the order cannot be forgotten.

9 **Two loud mistakes (1m).** "t-SNE has no transform; perplexity must be below the number of rows."

```
tsne.transform(img50[:5])
TSNE(perplexity=30, random_state=42).fit_transform(img50[:20])
```

```
AttributeError: 'TSNE' object has no attribute 'transform'
ValueError: perplexity (30) must be less than n_samples (20)
```

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Reading sizes and distances.** Coach: "Which of the five rules does that sentence break?"
- **Clustering on the map.** Coach: "Cluster in the original space; the map only illustrates."
- **Unscaled business columns.** Coach with Lesson 9.3's Mistake 4: neighbors kept 0.201 raw vs 0.739 scaled.

GUIDED LAB

- **Lab A (12m, everyone):** `labs/module-06/l9_3-tsne-umap/fashion_map_starter.py`; reference `labs/module-06/l9_3-tsne-umap/fashion_map_solution.py`. 1,500 TRAINING images; t-SNE and UMAP; neighbor checks; 500 new images placed. TODO 1 runs about 20 seconds: start it, then explain TODO 2.
- **Expected, Lab A:** 50 components keep 0.869 of the pixel variance; neighbors kept 0.195 (PCA view), 0.573 (t-SNE), 0.495 (UMAP); trustworthiness 0.922 / 0.991 / 0.986; hardest classes Shirt 0.32, Coat 0.51, Pullover 0.62, easiest Trouser 0.90; 0.730 of the new images land among their own class. Yours may vary slightly.
- **Lab B (8m, pairs):** `labs/module-06/l9_3-tsne-umap/fraud_map_starter.py`; reference `labs/module-06/l9_3-tsne-umap/fraud_map_solution.py`. Is fraud its own island?
- **Expected, Lab B:** 0.614 of a fraud's neighbors are frauds in the original space; 0.630 and 0.631 on the maps (perplexity 5 and 30, neighbors kept 0.633 and 0.696); K-Means in the original space: one cluster of 239 frauds and 20 honest payments, the other 199 frauds among 1,042 honest ones. The colleague is partly right, and the evidence is the original space.

START THE LESSON PROJECT

The Customer Map (about 45 minutes alone). In the room: Steps 1 and 2.

- Step 1: four behavior columns, two logged, scaled: `(6000, 4)`.
- Step 2: K-Means in the 4-D space: silhouette 0.479; profile and name the five segments from the table, never from a map.
- **Goes home:** Steps 3 to 6: t-SNE and UMAP maps of 2,000; silhouette 0.477 in 4-D vs 0.572 / 0.596 on the maps (maps flatter); 98.5% of 4,000 new customers land in their region; `customer_map.csv`.

CHECKS FOR UNDERSTANDING

Ask out loud:

- What does a t-SNE map keep? **Answer:** neighborhoods.
- Which method can place new rows? **Answer:** UMAP (and PCA), not t-SNE.
- How do you check a map? **Answer:** neighbors kept, trustworthiness, and clusters validated in the original space.

Workbook self-check, items Q7 to Q12:

Answers: **Q7** C) Little about size or distance: check spread and separation in the original data; **Q8** A) The map reproduces a neighborhood structure that exists in the data; now confirm any group in the original space (a clustering found 239 frauds together, 199 elsewhere); **Q9** `ValueError: perplexity (30.0) must be less than n_samples (12)`; **Q10** D) UMAP: fit once, then transform new rows; it can sit in a pipeline; **Q11** B) Speed, and the last few hundred components of image data are mostly noise; **Q12** D) Try several perplexities and confirm with a clustering and its profiles in the original space. Full set with explanations in the Answer Key.

◆ DIFFERENTIATION

Support: the five rules as a card. **Stretch:** Lab B and the lesson's stability Bonus Challenge.

CHECKPOINT 1

2:20 Checkpoint 1: Modules 7 to 9

30 min

TIMING

2m	Hand out `checkpoint_starter.py`; read the four questions aloud; open notebook, no partners
25m	Work: Part A (6m), Part B (8m), Part C (8m), Part D sentences (5m); call the time every 8 minutes
3m	Collect files (or screenshots of Part D); tell them the debrief comes at the start of Session 7

WHAT IT CHECKS

- **Module 7:** permutation importance on held-out rows, and the sentence that importance describes the model, not causes.
- **Module 8:** a Gaussian mixture on behavior only, profiles with churn, named by profile, borderline customers counted.
- **Module 9:** a variance count with PCA, a supervised axis with LDA fitted on training rows, the two compared with one number each.
- Scoring: five rubric rows, 0 to 2 each (the answer key holds the rubric and the model answer). 8 to 10 = on track; 5 to 7 = review the weakest module before Session 8; under 5 = a one-to-one before Module 10.

WATCH-OUTS WHILE THEY WORK

- Learners who fit the scaler on all rows in Part C: note it for row 1 of the rubric; do not interrupt.
- Part B takes about 5 seconds to fit (three starts); learners who see nothing should wait, not rerun.
- If the room is slower than planned, drop Part D to one sentence and say so; score row 5 on that sentence.

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 3 MINUTES)

The checkpoint ran long on attention; keep this brief.

1. LDA on churn gives one axis, PCA gives ten components. Why the difference? **Answer:** LDA's axes are at most classes minus one (two classes, one axis); PCA has one component per column.
2. Name two things you must NOT read off a t-SNE map. **Answer:** cluster sizes, distances between clusters (also: the axes).

3. You need tomorrow's customers on today's map. Which method, and how? **Answer:** UMAP: fit once, `transform` the new rows through the same fitted steps (scaler or PCA first).

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both projects:** The Loyalty-Tier Map Steps 3 to 6 (`tier_map.csv`) and The Customer Map Steps 3 to 6 (`customer_map.csv`).
- **Lesson exercises:** 9.2 Exercises 1 to 3 (kNN on LDA axes, priors, Fisher's ratio); 9.3 Exercises 1 to 3 (seeds, perplexity, min_dist). Exercise 4 of each is a stretch.
- **Both Quick Quizzes** and **both journal entries**.
- **Before the Next Lesson (from 9.3):** make sure `import torch` works; keep `customer_map.csv` for the capstone.
- **Optional:** `noise_leak_starter.py`, `fraud_map_starter.py`.

BRING LIST FOR SESSION 7

- A laptop on which `import torch` works (it did in `check_environment.py` in Session 1). Session 7 builds a neural network in NumPy, then in PyTorch, on the CPU.
- The `pinecrest-ml` folder with `data/` and `labs/`; Session 7's starters are in `labs/module-07/`.
- Fashion-MNIST stays in `data/`: Module 10 uses it again for a CNN.
- Your checkpoint file if you want to finish Part D at home (the debrief opens Session 7).
- The workbook and the journal.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** the checkpoint's 30 minutes; the noise leak (9.2); distortion and noise (9.3).
- **Cut first (saves 4m):** 9.2 Steps 5 and 6.
- **Cut second (saves 3m):** 9.3 Step 3.
- **Cut third (saves 20m):** both project starts: give the check numbers.

RUNNING SHORT: WHAT TO ADD

- Debrief the checkpoint immediately with the model answer (otherwise it opens Session 7).
- Lesson 9.3 Exercise 4 live: UMAP features inside a pipeline do not beat PCA here (0.725 vs 0.735).

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 7: Neural Networks and PyTorch

How to teach Session 7 of Advanced Machine Learning. Build a neural network by hand in NumPy (forward pass, loss, backpropagation checked numerically, the four-line training loop), let PyTorch's autograd prove it right, then rebuild everything the professional way: tensors, nn.Module, a loss and an optimizer, a DataLoader, early stopping and a saved state_dict. Both projects end with the same honest finding: on Pinecrest's churn table a network beats logistic regression and, at best, ties gradient boosting.

`2 lessons` `3 hrs session` `Lessons 10.1–10.2` `Session 7 of 13`

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame (the goal and one mental model) → live demo (you type, they type along) → guided lab (a kit lab, new practice on the lesson's skills) → start the lesson's own project. Exercises, quiz and journal are homework.
- Type, do not paste. Every code block in the demo scripts was run to produce the output printed under it (Python 3.12, scikit-learn 1.9.1, PyTorch 2.14.1 on 4 CPU threads). Ask for a prediction before every run that prints a number, and read errors from the last line up, out loud.
- Outputs marked "yours may vary slightly" are PyTorch results. On one machine with a fixed seed and thread count they repeat exactly; another processor or PyTorch build can change the last digits. Say this once at the first such output, then compare trends, not third decimals.
- The lessons are the reference. Keep `deep_learning_basics.html` and `deep_learning_pytorch.html` open for the pictures (the panel diagram, the loss curve and decision boundary, the two-curve chart). Labs live in `labs/module-07/`: `_starter.py` files are handouts, `_solution.py` files are yours.
- Teal blocks are hands-on, amber blocks are watch-outs. Lesson codes (10.1, 10.2) match the course site.

THE #1 RISK OF SESSION 7: THE MATH SWALLOWS THE SESSION

Lesson 10.1 contains a forward pass, a loss, gradient descent, backpropagation, a numeric gradient check and autograd. Each is one demo step of two or three minutes, not a lecture. Derive nothing on the board beyond the shape table and p - y. The learners who want the calculus have it on the lesson page; the ones who do not need to leave knowing the four-line loop, the coin-flip loss 0.693, and that autograd computes the same gradients they could check by nudging. The second risk is the opposite: PyTorch syntax without the honest comparison. Both projects end with the same verdict (a network beats logistic regression on churn and ties gradient boosting at best). Protect the moment the room sees 0.846 against 0.850: it is the course's habit applied to a shiny new tool.

BEFORE THE SESSION

- ☐ **Environment:** nothing new to install; Session 6's bring list asked for a laptop on which `import torch` works and for Fashion-MNIST to stay in `data/`. If anyone skipped Session 1's environment check, have them run `python labs/module-01/l7_0-orientation/check_environment.py` before you start; it confirms torch and torchvision import.
- ☐ **Checkpoint 1:** score the files from Session 6 on its five-row rubric (Session 6 Answer Key) and bring them back for the opening debrief.
- ☐ **Files to hand out:** `labs/module-07/` without the `*_solution.py` files (keep `labs_paths.py` in the `labs/` folder: every lab imports it). No data beyond the Pinecrest CSVs: Session 7 uses `customers_churn.csv` and `transactions.csv` only.
- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-07` from this kit's folder. Every line should say PASS (8 files, about 70 seconds).
- ☐ **Rehearse both demos once on the projector machine** in two notebooks, `lesson_10_1` and `lesson_10_2`. Keep the rehearsal's `churn_net.pt` in the project folder: if anything goes wrong in demo Step 10 of 10.2, load it instead of retraining.
- ☐ **Downloads for next week (announce today):** Session 8 needs Fashion-MNIST (about 30 MB) in each learner's `data/` folder. Everyone should have it since Session 6 (Lesson 9.3 and its labs used it). Test it on the projector machine now: `datasets.FashionMNIST("data", download=True)` must find the files without downloading.
- ☐ **Board, before learners arrive:** the four-line loop (forward → loss → backward → update) with the PyTorch line next to each, left empty for now; the number 0.693 in a box ("the coin-flip loss"); and the mapping table from Lesson 10.2 with its left column filled (NumPy arrays, dictionary of weights, bce_loss, backward, update loop, full batch, np.savez).
- ☐ **Print or share:** the workbook pages for Session 7 and sticky notes for the exit ticket.

CPU-TIME PLAN (MEASURED ON A 4-THREAD LAPTOP CPU)

WHAT	TIME	PLAN
First `import torch` / `import sklearn` in a fresh notebook	about 3 s each (10 to 30 s on a cold laptop)	Run the first import cell of both demo notebooks during the warm-up, so nobody waits in front of a silent cell.
10.1 NumPy training, 2,000 full-batch epochs on 750 rows	under 1 s	Live.
10.1 autograd training, 2,000 epochs	about 2 s	Live.
10.1 project network, 1,000 epochs on 4,500 customers	under 1 s	Live, in the project start.
10.2 thirty epochs (demo Step 9) and early stopping (Step 10)	about 2 s each	Live. Keep the rehearsal's `churn_net.pt` as the fallback checkpoint.
10.2 project training with early stopping (Step 4)	a few seconds	Live in the room.
Labs: neuron 3 s, circles 5 s, fraud 20 s, batch-size sweep 22 s		Everything trains live; no fixtures needed this session.

GPU note. Nothing in this session needs a GPU, and the numbers in this kit come from the CPU code. A learner with an NVIDIA GPU can try `.to("cuda")` at home (Lesson 10.2's devices card); in the room, keep everyone on the CPU so outputs match the pages and their neighbors. GPU arithmetic is not bit-identical to CPU arithmetic.

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
`ModuleNotFoundError: No module named 'torch'` in the notebook only	The notebook kernel is not the course `.venv`	Pick the `.venv` kernel (VS Code: Select Kernel), or start JupyterLab from the activated terminal.
`import torch` prints `UserWarning: Failed to initialize NumPy`	A NumPy installed after PyTorch, outside the requirements file	`python -m pip install -r requirements.txt` again in the activated environment.
Training is several times slower than the guide says	Battery saver, many other programs, or a laptop limiting threads	Plug in; close the browser tabs; the timings here are for 4 threads. Nothing in Session 7 takes more than a few seconds even so.
Numbers differ from the page in the third decimal	A different CPU or PyTorch build (or a missing	Check the seed lines first. If they are there, it is the expected "yours may vary slightly"

SYMPTOM	CAUSE	FIX
	`torch.manual_seed(42)`)	drift: compare trends.
`RuntimeError: Can't call numpy() on Tensor that requires grad`	A tracked tensor sent to NumPy	`.detach().numpy()`, or predict inside `with torch.no_grad():`.
`RuntimeError: mat1 and mat2 must have the same dtype, but got Double and Float`	float64 data from NumPy into a float32 layer	Create tensors with `dtype=torch.float32`.

SESSION 7 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	15	Warm-up and Checkpoint 1 debrief	Checkpoint 1 handed back with the five rubric rows (5 min); five retrieval questions on 9.2 and 9.3 while notebooks import torch (10 min).
0:15	10	**10.1** Frame and mental model	The panel of junior analysts; forward, loss, backward, update on the board.
0:25	25	**10.1** Live demo	One neuron, the moons, a layer as a matrix product, the loss, walking downhill, backprop and its numeric check, training, autograd.
0:50	20	**10.1** Guided lab	`labs/module-07/l10_1-neural-networks/circles_network_starter.py` (15 min), then `labs/module-07/l10_1-neural-networks/churn_neuron_starter.py` in pairs (5 min, finish at home).
1:10	10	**10.1** Start the lesson project	A churn network from scratch, Steps 1 to 3; Steps 4 to 6 go home.
1:20	10	Break	Leave the four-line loop and the mapping table on the board.
1:30	10	**10.2** Frame and mental model	The workshop with labeled drawers: what each 10.1 part becomes in PyTorch.
1:40	30	**10.2** Live demo	Tensors and the dtype trap, churn tensors, nn.Linear and nn.Module, loss and optimizer, DataLoader, the loop, early stopping, saving, dropout and eval().
2:10	25	**10.2** Guided lab	`labs/module-07/l10_2-pytorch/fraud_mlp_starter.py`: the whole workflow on card fraud.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
2:35	15	**10.2** Start the lesson project	A PyTorch churn model, Steps 1 to 4 (training takes seconds); Steps 5 and 6 go home.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 8.
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 105 + 110 = 215 minutes for a solo learner; after the warm-up with the Checkpoint 1 debrief, the break and the wrap-up the room has 145. So the room does, for each lesson: the mental model, the live demo, one guided kit lab, and the first steps of the lesson project. The lessons' Common Mistakes are folded into the demos (Steps 11 of 10.1 and 12 of 10.2) and the debug handouts.

Goes home (2 to 3 hours). 10.1 project Steps 4 to 6 (evaluate, the boosting comparison, save `churn_net.npz`); 10.2 project Steps 5 and 6 (the honest comparison, saving model, settings and preprocessing); each lesson's Exercises 1 to 3 (Exercise 4 is a stretch); both Quick Quizzes; both journal entries; the second half of the churn-neuron lab; 10.2's "Before the Next Lesson" (save `training.py`, check torchvision and Fashion-MNIST).

Protect, in this order: the four-line loop and autograd agreeing with the hand-written gradients (10.1 Steps 8 to 11); `train_one_epoch` / `evaluate` and early stopping (10.2 Steps 8 to 10); the honest comparison in the fraud lab (Step 5). **Cut first:** 10.1 demo Step 7 (the numeric check: say one sentence), then 10.2 Step 1 to 2 (tensor basics: show the outputs on the slide), then the 10.1 project start (send all six steps home with the check numbers).

WARM-UP

0:00 Warm-up: Checkpoint 1 Debrief and Retrieval on 9.2 and 9.3 — 15 min

TIMING

5m	Checkpoint 1 debrief: hand back the scored files; the five rubric rows and the model answer's numbers on the slide
2m	Start the import cells of both demo notebooks; learners do the same
5m	Five retrieval questions; learners answer in the workbook's Warm-up box
3m	Reveal, one sentence each; homework check by show of hands

CHECKPOINT 1 DEBRIEF (5 MINUTES; THE RUBRIC AND MODEL ANSWER ARE IN SESSION 6'S ANSWER KEY)

- **Before the session:** score every file on the five rows (0 to 2 each): leak-free setup, interpretation (Module 7), soft segments (Module 8), projections (Module 9), honest written conclusions. 8 to 10: on track; 5 to 7: name the weakest module to review before Session 8; under 5: book a one-to-one before Module 10 gets heavy.
- **On the slide, the model answer's numbers:** shuffling `days_since_last_order` costs 0.154 of test AUC, `discount_share` 0.099 (the model, not causes); the deal-driven component churns at about 0.30, with 305 customers below 0.8 confidence; 7 components for 90% of the variance; LD1 (led by recency, 0.92) ranks churn with AUC 0.824 against PC1's 0.631.
- **Name the two most common slips, not the people:** a scaler or importance computed on the wrong rows (the leak row), and PC1's AUC reported as 0.369 (a flipped sign: PCA signs are arbitrary).
- **Bridge:** "Every method in that checkpoint needed a person to decide the features. From today the model learns its own." Learners who want to finish Part D hand it in at the break.

QUESTIONS AND ANSWERS

1 **W1.** What does this print? (Lesson 9.2)

```
from sklearn.datasets import make_classification
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
X, y = make_classification(n_samples=300, n_features=6, n_informative=4, n_cla
print(LinearDiscriminantAnalysis().fit(X, y).transform(X).shape)
```

Answer: `(300, 2)`: LDA gives at most (classes - 1) axes, so 3 classes give 2 columns, whatever the number of features.

2. **W2.** PCA and LDA both flatten data onto a few directions. What is the one big difference in how they choose them? (9.2) **Answer:** PCA looks only at X and keeps the directions of largest total spread; LDA uses the labels y and keeps the directions where the class means are far apart relative to the spread inside each class.

3. **W3.** A colleague fits LDA on all 200 rows of 500 random features, then cross-validates a classifier on the LDA output and gets 0.99. What happened? (9.2) **Answer:** Leakage: LDA saw the labels of the rows later used as test folds. With 500 features it can separate any labels. Inside a pipeline, refitted per fold, the score falls to chance.

4. **W4.** On a t-SNE map of customers, one island is twice the size of the others and far away from them. What may you conclude from the size and the gap? (9.3) **Answer:** Very little: t-SNE keeps neighborhoods, not sizes or distances between groups. Check spread and separation in the original data (silhouette, profiles).

5. **W5.** Next month 200 new customers arrive and you want to place them on this month's 2-D map. t-SNE or UMAP, and why? (9.3) **Answer:** UMAP: a fitted UMAP has transform for new rows. t-SNE only has fit_transform; it positions the rows it was given and learns no function.

◆ BRIDGE TO TODAY

Say: "In Lesson 9.3 every Fashion-MNIST image was a row of 784 pixel numbers, and t-SNE drew a map of them. Every method so far, from logistic regression to LDA, needed someone to decide what the features are. Today you build the model that learns its own features from raw inputs, by hand first and then in PyTorch. Next week it learns them from those same pixels."

LESSON PLANS

10.1 Neural Networks from Scratch

65 min

OBJECTIVES (FROM THE LESSON PAGE)

- Describe a neuron as a weighted sum plus a bias passed through an activation, and compute one by hand
- Write the forward pass of a one-hidden-layer network with matrix shapes that line up
- Explain cross-entropy loss, gradient descent and the learning rate, and implement backpropagation for a small network
- Check hand-written gradients numerically, and reproduce them with PyTorch's autograd
- Judge a network honestly against a baseline, logistic regression and gradient boosting on tabular data

TIMING IN THE ROOM

10m	Frame: the panel of junior analysts; why a network can bend a boundary
25m	Live demo: neuron, moons, layer shapes, loss, walking downhill, backprop, numeric check, training, autograd, the gradient trap
20m	Guided lab: `circles_network_starter.py`, then `churn_neuron_starter.py` in pairs
10m	Start the lesson project: Steps 1 to 3

MENTAL MODEL TO SAY

Say: "Pinecrest's churn question goes to a panel. Each junior analyst looks at the raw customer numbers and writes down one score of their own; nobody tells them what to look for. A senior analyst reads only the juniors' scores and makes the final call. After every round, each analyst hears how much their score contributed to the final mistake, and adjusts. The juniors are the hidden layer, the senior is the output layer, and the feedback is backpropagation."

Then the reason, pointing at a drawn straight line: "Logistic regression draws one straight boundary. Each junior can draw their own line, and the senior combines them, so the panel can bend its boundary around curves and interactions, like the churn threshold near 150 days." Then the four steps on the board, which never change for the rest of Module 10: **forward → loss → backward → update**. "Today you write all four by hand once, so that when training goes wrong later you know which part to suspect. Then PyTorch takes the hardest one off your hands."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook `lesson_10_1` in the project folder. The demo follows the lesson page; every output below is real. Keep the shape table and p - y on the board the whole time. Total training time across the demo is

under 5 seconds.

1. **One neuron (2m).** "You already know this unit: Lesson 3.1's logistic regression. Weighted sum, plus a bias, through an activation." Do the arithmetic on the board before running it.

```
import numpy as np

x = np.array([0.5, -1.2, 2.0])      # one customer: three scaled features
w = np.array([0.8, 0.1, -0.4])      # one weight per feature
b = 0.2                             # the bias

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

z = x @ w + b
print(f"weighted sum z    = {z:.3f}")
print(f"sigmoid(z)        = {sigmoid(z):.3f}")
print(f"ReLU(z) = max(0,z) = {max(0.0, z):.3f}")
```

```
weighted sum z    = -0.320
sigmoid(z)        = 0.421
ReLU(z) = max(0,z) = 0.000
```

With a sigmoid, this neuron IS logistic regression: probability 0.42. ReLU of a negative number is 0. The activation is the only thing that changes between neuron types.

2. **A curve a line cannot follow (2m).** "Two interleaving moons. A straight-line model gets about 90%: the tips of each moon reach past any line."

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

X, y = make_moons(n_samples=1000, noise=0.25, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, stra
print(X_train.shape, X_test.shape, y_train[:10])

linear = LogisticRegression().fit(X_train, y_train)
print(f"logistic regression test accuracy: {linear.score(X_test, y_test):.3f}"
```

```
(750, 2) (250, 2) [1 0 1 0 0 0 0 0 1 0]
logistic regression test accuracy: 0.896
```

Write 0.896 on the board: the network's target is the remaining 10%.

3 **A layer is a matrix multiplication (3m).** "Inner dimensions must match; the result takes the outer ones." Write the shape table on the board as you type: (n, 2) @ (2, 16) → (n, 16), then (n, 16) @ (16, 1) → (n, 1).

```
def init_params(n_in, n_hidden, rng):
    return {
        "W1": rng.normal(0, np.sqrt(2 / n_in), size=(n_in, n_hidden)),
        "b1": np.zeros(n_hidden),
        "W2": rng.normal(0, np.sqrt(1 / n_hidden), size=(n_hidden, 1)),
        "b2": np.zeros(1),
    }

def forward(params, X):
    z1 = X @ params["W1"] + params["b1"]       # (n, hidden)
    a1 = np.maximum(0, z1)                     # ReLU
    z2 = a1 @ params["W2"] + params["b2"]      # (n, 1)
    p = sigmoid(z2).ravel()                    # (n,) probabilities
    return p, (X, z1, a1)                      # keep the pieces for the backw

params = init_params(2, 16, np.random.default_rng(42))
for name, value in params.items():
    print(f"{name}: shape {value.shape}")

p, cache = forward(params, X_train[:5])
print("first five probabilities:", p.round(3))
```

```
W1: shape (2, 16)
b1: shape (16,)
W2: shape (16, 1)
b2: shape (1,)
first five probabilities: [0.705 0.587 0.59  0.596 0.413]
```

Random weights, random opinions. Point at the two deliberate details: random (not zero) starts break the tie between units; the spread sqrt(2 / n_in) is He initialization for ReLU.

4 **How wrong is it? The loss (2m).** "Cross-entropy punishes confident mistakes hardest. Remember 0.693: a binary classifier whose loss sits there has learned nothing."

```
def bce_loss(p, y, eps=1e-12):
    p = np.clip(p, eps, 1 - eps)               # avoid log(0)
    return -np.mean(y * np.log(p) + (1 - y) * np.log(1 - p))

p, cache = forward(params, X_train)
print(f"untrained network loss: {bce_loss(p, y_train):.4f}")
print(f"always guessing 0.5:    {bce_loss(np.full(len(y_train), 0.5), y_train)
```

```
untrained network loss: 0.6714
always guessing 0.5:    0.6931
```

5 **Walking downhill (2m).** "One parameter, a valley at 3, and one rule: step against the slope." Ask the room to predict w after step 1 (0.6).

```python
w = 0.0
learning_rate = 0.1
for step in range(1, 26):
    grad = 2 * (w - 3)                # slope of (w - 3)**2 at the current w
    w = w - learning_rate * grad      # step downhill
    if step in (1, 2, 3, 5, 10, 25):
        print(f"step {step:2d}  w = {w:.4f}  loss = {(w - 3) ** 2:.6f}")
```

```
step  1  w = 0.6000  loss = 5.760000
step  2  w = 1.0800  loss = 3.686400
step  3  w = 1.4640  loss = 2.359296
step  5  w = 2.0170  loss = 0.966368
step 10  w = 2.6779  loss = 0.103763
step 25  w = 2.9887  loss = 0.000128
```

Big steps where it is steep, small ones near the bottom. Nobody told the loop where the valley is.

6 **Backpropagation: shapes first (3m).** "The output error is simply p minus y. Then it is matrix multiplication with the shapes reversed, and ReLU lets the error through only where it was active." Read the comments aloud; do not derive calculus.

```python
def backward(params, cache, p, y):
    X, z1, a1 = cache
    n = len(y)
    dz2 = ((p - y) / n).reshape(-1, 1)        # (n, 1): slope of the loss w.r.
    grads = {"W2": a1.T @ dz2,                 # (16, n) @ (n, 1) -> (16, 1)
             "b2": dz2.sum(axis=0)}
    da1 = dz2 @ params["W2"].T                 # (n, 1) @ (1, 16) -> (n, 16):
    dz1 = da1 * (z1 > 0)                       # ReLU passes the error only wh
    grads["W1"] = X.T @ dz1                    # (2, n) @ (n, 16) -> (2, 16)
    grads["b1"] = dz1.sum(axis=0)
    return grads

grads = backward(params, cache, p, y_train)
for name, g in grads.items():
    print(f"{name}: gradient shape {g.shape}  (parameter shape {params[name].s
```

```
W2: gradient shape (16, 1)  (parameter shape (16, 1))
b2: gradient shape (1,)  (parameter shape (1,))
W1: gradient shape (2, 16)  (parameter shape (2, 16))
b1: gradient shape (16,)  (parameter shape (16,))
```

First sanity check for any backward pass: every gradient has its parameter's shape.

7 **Check the calculus with a nudge (2m).** "Hand-derived gradients are easy to get subtly wrong, and a wrong gradient does not crash. So nudge one weight and measure."

```
eps = 1e-6
i, j = 0, 3                                    # check one weight: W1[0, 3]
original = params["W1"][i, j]

params["W1"][i, j] = original + eps
loss_plus = bce_loss(forward(params, X_train)[0], y_train)
params["W1"][i, j] = original - eps
loss_minus = bce_loss(forward(params, X_train)[0], y_train)
params["W1"][i, j] = original                  # put it back

numeric = (loss_plus - loss_minus) / (2 * eps)
print(f"backprop gradient: {grads['W1'][i, j]:.8f}")
print(f"numeric gradient:  {numeric:.8f}")
```

```
backprop gradient: -0.02341771
numeric gradient:  -0.02341771
```

Ask: "Why not always do this?" (two forward passes per parameter; backprop gets all of them for about one).

8 **Train: forward, loss, backward, update (3m).** "These four lines are the engine of every network in Module 10." Point at each numbered comment as it runs (it takes under a second).

```
def train(params, X, y, learning_rate=0.5, epochs=2000, report_every=400):
    history = []
    for epoch in range(1, epochs + 1):
        p, cache = forward(params, X)                  # 1. forward
        history.append(bce_loss(p, y))                 # 2. loss
        grads = backward(params, cache, p, y)          # 3. backward
        for name in params:                            # 4. update every param
            params[name] -= learning_rate * grads[name]
        if report_every and epoch % report_every == 0:
            print(f"epoch {epoch:4d}  loss {history[-1]:.4f}")
    return history

params = init_params(2, 16, np.random.default_rng(42))
history = train(params, X_train, y_train)

p_test, _ = forward(params, X_test)
print(f"network test accuracy: {((p_test > 0.5) == y_test).mean():.3f}")
```

```
epoch  400  loss 0.2924
epoch  800  loss 0.1811
epoch 1200  loss 0.1547
epoch 1600  loss 0.1467
epoch 2000  loss 0.1432
network test accuracy: 0.972
```

0.972 against logistic regression's 0.896. Show the lesson page's boundary chart: the network bends around both moons. The plateau near 0.30 is the network sitting on a straight-line solution until its units find the bend.

9 **Autograd in one line (1m).** "Now the payoff: PyTorch records the operations and replays them backwards for you."

```
import torch

w = torch.tensor(0.0, requires_grad=True)   # "track every operation on w"
loss = (w - 3) ** 2
loss.backward()                              # chain rule, automatically
print(w.grad)
```

```
tensor(-6.)
```

-6 is exactly the slope the walking-downhill loop used at w = 0.

10 **Same network, same gradients (2m).** "If my hand-written backward is right, autograd must agree."

```
import torch.nn.functional as F

params = init_params(2, 16, np.random.default_rng(42))           # a fresh cop
t_params = {name: torch.tensor(value, requires_grad=True) for name, value in p
X_train_t = torch.tensor(X_train)                                  # float64,
y_train_t = torch.tensor(y_train, dtype=torch.float64)

def forward_torch(tp, X):
    a1 = torch.relu(X @ tp["W1"] + tp["b1"])
    return torch.sigmoid(a1 @ tp["W2"] + tp["b2"]).squeeze(1)

loss_t = F.binary_cross_entropy(forward_torch(t_params, X_train_t), y_train_t)
loss_t.backward()

p, cache = forward(params, X_train)
grads = backward(params, cache, p, y_train)
print(f"loss: NumPy {bce_loss(p, y_train):.6f}   PyTorch {loss_t.item():.6f}")
for name in params:
    gap = np.abs(t_params[name].grad.numpy() - grads[name]).max()
    print(f"{name}: largest gradient difference {gap:.1e}")
```

Output (yours may vary slightly)

```
loss: NumPy 0.671389   PyTorch 0.671389
W1: largest gradient difference 6.9e-18
b1: largest gradient difference 4.2e-17
W2: largest gradient difference 5.6e-17
b2: largest gradient difference 0.0e+00
```

Differences around 1e-17 are floating-point rounding. Say once why this kit labels such outputs "yours may vary slightly": the last digits depend on the processor and the library build, the agreement does not.

11 **The loop with autograd, and the trap (3m).** "Only step 3 changes. Two new lines: the update runs under no_grad, and the gradient is reset." Then run the trap: no reset.

```
t_params = {name: torch.tensor(value, requires_grad=True)
            for name, value in init_params(2, 16, np.random.default_rng(42)).i

for epoch in range(1, 2001):
    loss = F.binary_cross_entropy(forward_torch(t_params, X_train_t), y_train_
    loss.backward()
    with torch.no_grad():
        for tensor in t_params.values():
            tensor -= 0.5 * tensor.grad
            tensor.grad.zero_()
    if epoch % 400 == 0:
        print(f"epoch {epoch:4d}  loss {loss.item():.4f}")

with torch.no_grad():
    p_test_t = forward_torch(t_params, torch.tensor(X_test)).numpy()
print(f"autograd network test accuracy: {((p_test_t > 0.5) == y_test).mean():.

w = torch.tensor(0.0, requires_grad=True)
for step in range(3):
    loss = (w - 3) ** 2
    loss.backward()
    print(w.grad)          # no w.grad.zero_() in this loop
```

Output (yours may vary slightly)

```
epoch  400  loss 0.2924
epoch  800  loss 0.1811
epoch 1200  loss 0.1547
epoch 1600  loss 0.1467
epoch 2000  loss 0.1432
autograd network test accuracy: 0.972
tensor(-6.)
tensor(-12.)
tensor(-18.)
```

Identical losses to the NumPy run. Then -6, -12, -18: `.grad` ADDS. This is why every PyTorch loop has `zero_grad()`.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Shapes the wrong way round.** `ValueError: matmul: Input operand 1 has a mismatch in its core dimension 0, with gufunc signature (n?,k),(k,m?)->(n?,m?)...` when someone writes `W1` as (hidden, inputs). Coach: "Write the shape next to the line. Which two numbers must match?" The course convention in 10.1 is (inputs, outputs); `nn.Linear` in 10.2 stores the transpose, which is why shapes are always read, never assumed.

- **A network that trains but learns nothing.** All-zero weights: the loss sits at 0.6931 and accuracy at 0.500 (Mistake 3: `test accuracy: 0.500`). A far too large learning rate: the loss jumps around or climbs (`debug_backprop.py` shows a sign error that does the same: 0.6714 → 13.8155). Coach: "What does the loss curve do? Does it sit at 0.693, bounce, or fall?" Each shape names a different bug.
- **Tensors that still remember their history.** `RuntimeError: Can't call numpy() on Tensor that requires grad. Use tensor.detach().numpy() instead.` Coach: predictions go inside `with torch.no_grad():` (as the lesson does) or through `.detach()`. And in loops, the gradient trap of demo Step 11: `.grad` adds up unless it is reset.

GUIDED LAB

- **Lab A (15m, everyone):** `labs/module-07/l10_1-neural-networks/circles_network_starter.py`; reference `labs/module-07/l10_1-neural-networks/circles_network_solution.py`. One class sits inside a ring of the other, so logistic regression scores a coin flip (0.488). Learners write `forward` and `bce_loss` themselves, check the gradient of an OUTPUT weight numerically (the lesson checked a hidden one), write the training loop, then answer the width question with a twist: what is the smallest number of ReLU units that can enclose a ring?
- **Expected, Lab A:** untrained loss 0.6927 against 0.6931; `W2[5, 0]` backprop and numeric both 0.03266046; 8 units: loss 0.6927 → 0.0156, test accuracy 0.984; width sweep 1 / 2 / 3 / 4 / 8 units: 0.648 / 0.752 / **0.992** / 0.984 / 0.984; learning rate 0.05 still at 0.487 after 500 epochs, 0.5 at 0.045, 50 bouncing near 4.
- **Circulate in Lab A for:** `p` left as shape (n, 1) (the loss then broadcasts (n, 1) against (n,) into an (n, n) matrix and is silently wrong: ask "what shape is p?"); forgetting to put the nudged weight back (every later number drifts); calling `train` on the same `params` dictionary twice in the sweep (the second run starts already trained). **Debrief (2m):** "Each ReLU unit adds one straight edge. One edge: a half-plane. Two: a wedge. Three: a triangle, the first shape that closes around the ring." That is the junior analysts, drawn.
- **Lab B (5m in pairs, finish at home):** `labs/module-07/l10_1-neural-networks/churn_neuron_starter.py`; reference `labs/module-07/l10_1-neural-networks/churn_neuron_solution.py`. One sigmoid neuron trained by hand on churn reaches test AUC 0.8235, exactly `LogisticRegression`'s 0.8235, with weights equal to two decimals (largest difference 0.0022). It also explains why zeros are fine for ONE neuron (its gradient p - y is not zero) and fatal for a hidden layer. Start it in the room so pairs see the loss fall from 0.6931 to 0.3689 in ten epochs.
- **Fast finishers:** `labs/module-07/l10_1-neural-networks/debug_backprop.py`: a transposed `W2` (a `ValueError`), a backward pass without the ReLU gate (the check prints backprop 0.03319750 against numeric 0.00780703), and an update with `+=` (loss 0.6714 → 13.8155, accuracy 0.500). Fixed, it prints the lesson's 0.1432 and 0.972.

START THE LESSON PROJECT

The lesson project is **a churn network from scratch**: the NumPy network on the ten numeric churn features, judged against a baseline, logistic regression and gradient boosting. In the room, Steps 1 to 3 (10 minutes); the functions from the demo notebook are reused, so learners continue in `lesson_10_1`.

- Step 1: load, split, scale (fit on the 4,500 training rows only). **Done** = `(4500, 10) (1500, 10) churn rate 0.165`.
- Step 2: the yardsticks. **Done** = baseline accuracy 0.835 / AUC 0.500, logistic regression 0.848 / 0.823.
- Step 3: train 10 → 16 → 1 (193 parameters) for 1,000 epochs: loss 0.3204 at epoch 200, 0.2994 at 1,000.
- **Goes home:** Step 4 (network test AUC 0.856, beating logistic regression's 0.823), Step 5 (gradient boosting 0.858 with zero tuning), Step 6 (save `churn_net.npz` and the summary). Tell them both numbers so they can check themselves, and ask them to write, in the journal, why they would still ship boosting.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- A batch has shape (32, 10) and the layer's weights (10, 16). Shape of the product? **Answer:** (32, 16): inner dimensions match and disappear.
- Remove every ReLU from a five-layer network. What boundary can it draw? **Answer:** only a straight line: matrix products in a row collapse into one.
- The loss reads 0.69, 0.52, 3.10, 0.95, 4.80, nan. Diagnosis? **Answer:** learning rate too large; divide it by 10.
- Why does the update run under `torch.no_grad()`? **Answer:** the update itself must not be recorded as part of the next forward pass.

Workbook self-check, items Q1 to Q6 (learners do them at the end of the block or at home).

Answers: **Q1** B) 176; **Q2** `-0.5 0.0 0.378`; **Q3** C) Every weight was initialized to zero; **Q4** `0.75`; **Q5** A) Its slope is 1 wherever it is active, so gradients pass back through many layers undiminished; **Q6** D) `RuntimeError`: Can't call numpy() on Tensor that requires grad. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: give anyone lost in the matrices a card with the shape table and have them say the shape before each run. In Lab A, a struggling pair may copy `forward` from the demo notebook and spend their time on the width sweep and its debrief, which is the new idea.

Stretch: the lesson's Exercise 4 (a second hidden layer with autograd only) and the Bonus Challenge (mini-batch gradient descent in NumPy: count the update steps; Lab 10.2 B measures the same thing in PyTorch).

10.2 PyTorch Fundamentals

80 min

OBJECTIVES (FROM THE LESSON PAGE)

- Create and reshape tensors, convert between NumPy and PyTorch, and get the float32 dtype right
- Define a network with `nn.Linear`, `nn.Sequential` and your own `nn.Module` subclass
- Choose the loss function that matches the task, and train with an optimizer, a Dataset and a DataLoader
- Write the standard training loop with `model.train()`, `model.eval()`, `torch.no_grad()`, a validation set and early stopping
- Save and reload a trained model with `state_dict`, and judge it honestly against gradient boosting

TIMING IN THE ROOM

10m	Frame: the workshop with labeled drawers; fill in the mapping table's right column
30m	Live demo: tensors and dtypes, churn tensors, nn.Linear and nn.Module, loss and optimizer, DataLoader, the loop, early stopping, saving, two silent traps
25m	Guided lab: `fraud_mlp_starter.py`
15m	Start the lesson project: Steps 1 to 4

MENTAL MODEL TO SAY

Say: "In 10.1 you carved every tool yourself before building a chair. PyTorch is a workshop where each tool sits in a labeled drawer: tensors, layers, losses, optimizers, data loaders. You still decide what to build and you still hold the tools: PyTorch does not hide the training loop from you. You just never carve a chisel again."

Then fill the right column of the mapping table on the board while they watch: NumPy arrays → `torch.Tensor`; dictionary of weights + forward → `nn.Module` with `nn.Linear`; `bce_loss` → `nn.BCEWithLogitsLoss`; `backward()` → `loss.backward()`; update loop and `grad.zero_()` → `optimizer.step()` / `optimizer.zero_grad()`; full batch → `DataLoader`; `np.savez` → `torch.save(model.state_dict())`. And the honest word from the lesson: "We use the churn table because you know its answers. It is not where networks shine; Session 8 is."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook `lesson_10_2`. Steps 1 to 7 are fast and build the pieces; Steps 8 to 10 are the loop that Sessions 8 and 9 reuse, so slow down there. Every training step finishes in a few seconds.

1. **Tensors (2m).** "A tensor is NumPy's array plus two powers: it can record operations for autograd, and it can live on a GPU."

```
import numpy as np
import pandas as pd
import torch

print(torch.__version__)

t = torch.tensor([[1.0, 2.0, 3.0],
                  [4.0, 5.0, 6.0]])
print(t)
print(t.shape, t.dtype, t.device)
```

```
2.14.1+cpu
tensor([[1., 2., 3.],
        [4., 5., 6.]])
torch.Size([2, 3]) torch.float32 cpu
```

Three attributes describe every tensor: shape, dtype, device. `+cpu` is a CPU-only build: all this course needs.

2 **NumPy in, NumPy out: the dtype trap (2m).** "Two lessons in one block." Ask them to predict both prints before running.

```
arr = np.array([0.5, 1.5, 2.5])
shared = torch.from_numpy(arr)       # shares memory with arr
copied = torch.tensor(arr, dtype=torch.float32)

arr[0] = 99.0
print(shared, shared.dtype)
print(copied, copied.dtype)
print(copied.numpy())
```

```
tensor([99.0000,  1.5000,  2.5000], dtype=torch.float64) torch.float64
tensor([0.5000, 1.5000, 2.5000]) torch.float32
[0.5 1.5 2.5]
```

`from_numpy` shares memory; `torch.tensor` copies. And float64 from NumPy vs float32 in PyTorch layers: convert as you create (Mistake 1).

3 **The churn data as tensors (3m).** "The standard test split, then a validation split carved out of training: 3,600 to fit, 900 to decide when to stop, 1,500 locked away."

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

customers = pd.read_csv("data/customers_churn.csv")
features = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value", "da
            "returns_12m", "return_rate", "support_tickets_12m", "discount_sha

X_train_df, X_test_df, y_train_s, y_test_s = train_test_split(
    customers[features], customers["churned"], test_size=0.25, stratify=custom
)
X_fit_df, X_val_df, y_fit_s, y_val_s = train_test_split(
    X_train_df, y_train_s, test_size=0.2, stratify=y_train_s, random_state=42
)
scaler = StandardScaler().fit(X_fit_df)          # fit on the fitting rows onl

def to_tensors(X_df, y_s):
    X_t = torch.tensor(scaler.transform(X_df), dtype=torch.float32)
    y_t = torch.tensor(y_s.to_numpy(), dtype=torch.float32).unsqueeze(1)   # s
    return X_t, y_t

X_fit, y_fit = to_tensors(X_fit_df, y_fit_s)
X_val, y_val = to_tensors(X_val_df, y_val_s)
X_test, y_test = to_tensors(X_test_df, y_test_s)
print(X_fit.shape, y_fit.shape, X_val.shape, X_test.shape)
print(X_fit.dtype, y_fit[:5].squeeze(1))
```

```
torch.Size([3600, 10]) torch.Size([3600, 1]) torch.Size([900, 10]) torch.Size(
torch.float32 tensor([0., 0., 0., 0., 0.])
```

Targets are float32 and shaped (n, 1) to match one output column: `unsqueeze(1)`.

4 **nn.Linear (2m).** "`X @ W + b` from 10.1, with the weights created and initialized for you."

```
from torch import nn

torch.manual_seed(42)
layer = nn.Linear(10, 16)
print(layer)
print(layer.weight.shape, layer.bias.shape)
print(layer(X_fit[:3]).shape)
```

```
Linear(in_features=10, out_features=16, bias=True)
torch.Size([16, 10]) torch.Size([16])
torch.Size([3, 16])
```

Surprise: the weight is (16, 10), outputs by inputs, the transpose of 10.1. `nn.Linear(10, 16)` still means 10 in, 16 out.

5 **Your own nn.Module (3m).** "Layers in `__init__`, wiring in `forward`. Call `model(x)`, never `model.forward(x)`. And no sigmoid at the end: the model returns logits."

```
class ChurnNet(nn.Module):
    def __init__(self, n_features, n_hidden=16):
        super().__init__()                          # always call this first
        self.hidden = nn.Linear(n_features, n_hidden)
        self.output = nn.Linear(n_hidden, 1)

    def forward(self, x):
        return self.output(torch.relu(self.hidden(x)))   # raw scores (logits)

torch.manual_seed(42)
model = ChurnNet(n_features=10)
for name, param in model.named_parameters():
    print(f"{name:14s} {tuple(param.shape)}")

logits = model(X_fit[:4])
print(logits.squeeze(1))
print(torch.sigmoid(logits).squeeze(1))
```

```
hidden.weight  (16, 10)
hidden.bias    (16,)
output.weight  (1, 16)
output.bias    (1,)
tensor([-0.0142,  0.3132,  0.1372,  0.0113], grad_fn=<SqueezeBackward1>)
tensor([0.4965, 0.5777, 0.5343, 0.5028], grad_fn=<SqueezeBackward1>)
```

`grad_fn=...`: autograd is already recording.

6 **A loss and an optimizer: one step (2m).** "The optimizer owns the update: `zero_grad`, `backward`, `step`."

```
loss_fn = nn.BCEWithLogitsLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

before = loss_fn(model(X_fit), y_fit)
optimizer.zero_grad()      # clear old gradients
before.backward()          # compute new ones
optimizer.step()           # update every parameter
after = loss_fn(model(X_fit), y_fit)
print(f"loss before one step {before.item():.4f}, after {after.item():.4f}")
```

```
loss before one step 0.7570, after 0.7288
```

Write on the board: logits out of the model, the sigmoid lives in the loss (`BCEWithLogitsLoss`); for k classes, `CrossEntropyLoss` with integer targets.

7 **Dataset and DataLoader (2m).** "Shuffle, cut into batches of 64, one optimizer step per batch." Ask: "How many batches per epoch?" (57; the last has 16 rows).

```
from torch.utils.data import TensorDataset, DataLoader

fit_set = TensorDataset(X_fit, y_fit)
first_x, first_y = fit_set[0]
print(len(fit_set), first_x.shape, first_y)

torch.manual_seed(42)
fit_loader = DataLoader(fit_set, batch_size=64, shuffle=True)
xb, yb = next(iter(fit_loader))
print(xb.shape, yb.shape, "batches per epoch:", len(fit_loader))
```

```
3600 torch.Size([10]) tensor([0.])
torch.Size([64, 10]) torch.Size([64, 1]) batches per epoch: 57
```

8 **The two functions you will reuse (2m).** Type `train_one_epoch` and `evaluate` and narrate the four numbered lines, `model.train()` / `model.eval()` and `torch.no_grad()`. Nothing prints.

```
from sklearn.metrics import roc_auc_score

def train_one_epoch(model, loader, loss_fn, optimizer):
    model.train()                                   # training mode (matters fo
    total_loss, n = 0.0, 0
    for xb, yb in loader:
        optimizer.zero_grad()                       # 1. reset gradients
        loss = loss_fn(model(xb), yb)               # 2. forward pass and loss
        loss.backward()                             # 3. backward pass
        optimizer.step()                            # 4. update
        total_loss += loss.item() * len(yb)
        n += len(yb)
    return total_loss / n                           # average loss over the epo

def evaluate(model, X, y, loss_fn):
    model.eval()                                    # prediction mode
    with torch.no_grad():                           # no gradient bookkeeping n
        logits = model(X)
        loss = loss_fn(logits, y).item()
        auc = roc_auc_score(y.squeeze(1).numpy(), logits.squeeze(1).numpy())
    return loss, auc
```

Tell them to save these in `training.py` tonight: Sessions 8 and 9 reuse the shape.

9 **Thirty epochs, two curves (3m).** "Watch both losses. Which one keeps falling?" (Under 3 seconds.)

```
torch.manual_seed(42)
model = ChurnNet(n_features=10, n_hidden=64)
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
fit_loader = DataLoader(fit_set, batch_size=64, shuffle=True)

history = []
for epoch in range(1, 31):
    train_loss = train_one_epoch(model, fit_loader, loss_fn, optimizer)
    val_loss, val_auc = evaluate(model, X_val, y_val, loss_fn)
    history.append({"epoch": epoch, "train": train_loss, "validation": val_los
    if epoch % 5 == 0:
        print(f"epoch {epoch:2d}  train loss {train_loss:.3f}  val loss {val_l
```

Output (yours may vary slightly)

```
epoch  5  train loss 0.323  val loss 0.311  val AUC 0.862
epoch 10  train loss 0.313  val loss 0.315  val AUC 0.869
epoch 15  train loss 0.309  val loss 0.326  val AUC 0.848
epoch 20  train loss 0.301  val loss 0.304  val AUC 0.870
epoch 25  train loss 0.296  val loss 0.311  val AUC 0.863
epoch 30  train loss 0.294  val loss 0.341  val AUC 0.841
```

Training loss keeps falling; validation loss zigzags and drifts up by epoch 30. Show the lesson page's two-curve chart.

10 **Early stopping (3m).** "Keep the best weights; stop after five epochs without improvement; roll back."

```
import copy

def fit(model, train_loader, X_val, y_val, loss_fn, optimizer, max_epochs=50,
    best_loss, best_state, best_epoch = float("inf"), None, 0
    for epoch in range(1, max_epochs + 1):
        train_one_epoch(model, train_loader, loss_fn, optimizer)
        val_loss, val_auc = evaluate(model, X_val, y_val, loss_fn)
        if val_loss < best_loss:
            best_loss, best_epoch = val_loss, epoch
            best_state = copy.deepcopy(model.state_dict())
        elif epoch - best_epoch >= patience:
            break
    model.load_state_dict(best_state)              # roll back to the best epo
    print(f"stopped after epoch {epoch}; best epoch {best_epoch}, validation l
    return model

torch.manual_seed(42)
model = ChurnNet(n_features=10, n_hidden=64)
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
fit_loader = DataLoader(fit_set, batch_size=64, shuffle=True)
model = fit(model, fit_loader, X_val, y_val, loss_fn, optimizer)

print(f"validation AUC of the restored model: {evaluate(model, X_val, y_val, l
```

Output (yours may vary slightly)

```
stopped after epoch 23; best epoch 18, validation loss 0.2937
validation AUC of the restored model: 0.881
```

Then the warning card: the validation set chose the epoch, so its AUC is slightly optimistic. The test set is scored once, at the end.

11 **Save and reload the right way (2m).** "Save the state_dict, not the object. Load with `weights_only=True`."

```
torch.save(model.state_dict(), "churn_net.pt")

state = torch.load("churn_net.pt", weights_only=True)
print(list(state.keys()))

reloaded = ChurnNet(n_features=10, n_hidden=64)    # same architecture as the
reloaded.load_state_dict(state)
reloaded.eval()

model.eval()
with torch.no_grad():
    print("same test predictions:", torch.equal(model(X_test), reloaded(X_test
```

```
['hidden.weight', 'hidden.bias', 'output.weight', 'output.bias']
same test predictions: True
```

The class definition and the fitted scaler are part of the model too: keep them next to the weights.

12 **Two silent traps (3m).** "Neither raises an error, which is why they cost the most time." First dropout in training mode, then a sigmoid applied twice.

```
torch.manual_seed(42)
dropout_net = nn.Sequential(nn.Linear(10, 32), nn.ReLU(), nn.Dropout(0.5), nn.
one_customer = X_test[:1]

dropout_net.train()
with torch.no_grad():
    print("train mode:", [round(dropout_net(one_customer).item(), 3) for _ in
dropout_net.eval()
with torch.no_grad():
    print("eval mode: ", [round(dropout_net(one_customer).item(), 3) for _ in

sure_churner = torch.tensor([[1.0]])
for name, output in [("logit 8.0 (correct)", torch.tensor([[8.0]])),
                     ("sigmoid(8.0) passed as a logit", torch.sigmoid(torch.te
    print(f"{name:32s} loss {loss_fn(output, sure_churner).item():.4f}")
```

```
train mode: [0.367, 0.958, -0.361]
eval mode:  [0.348, 0.348, 0.348]
logit 8.0 (correct)              loss 0.0003
sigmoid(8.0) passed as a logit   loss 0.3134
```

`no_grad()` does not switch modes; `eval()` does. And with a sigmoid inside the model plus `BCEWithLogitsLoss`, the most confident possible output still costs 0.3134.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **dtype and shape errors.** `RuntimeError: mat1 and mat2 must have the same dtype, but got Double and Float` (float64 from NumPy), `ValueError: Target size (torch.Size([900])) must be the same as input size (torch.Size([900, 1]))` (targets not unsqueezed), `RuntimeError: mat1 and mat2 shapes cannot be multiplied (900x9 and 10x64)` (a column short). Coach: "Print `X.dtype`, `X.shape` and `y.shape` before the first forward pass." Every one of these messages names the two things that disagree.
- **Modes confused with gradients.** Evaluating without `model.eval()` (dropout makes the same customer score differently every call) or believing `no_grad()` is enough. Coach with the demo Step 12 output: three different numbers in train mode, the same number three times in eval mode.
- **The validation set treated as the test set.** Reporting the early-stopping validation AUC (0.881) as the model's quality. Coach: "Which rows chose the epoch? Then which rows are left that never influenced anything?" The test set is opened once, at the end, in the project's Step 5 and in the fraud lab's Step 5.

GUIDED LAB

- **Lab A (25m):** `labs/module-07/l10_2-pytorch/fraud_mlp_starter.py`; reference `labs/module-07/l10_2-pytorch/fraud_mlp_solution.py`. The whole workflow on a dataset the lesson did not use: 50,000 card payments, 0.88% fraud. A three-way split and a `ColumnTransformer` are given; learners write the float32 tensors, a `FraudNet` module, the loop with early stopping, the one-time test, and the comparison. It runs in about 20 seconds.
- **Expected, Lab A:** 30,000 / 7,500 / 12,500 rows with 263 / 66 / 109 frauds; 673 parameters; stopped after epoch 24, best epoch 19, validation loss 0.0245; test ROC AUC network 0.855 (average precision 0.483), logistic regression 0.844 (0.456), default gradient boosting 0.825 (0.399); 50 frauds among the network's 100 riskiest payments against 0.9 for a random 100.
- **Circulate for:** targets without `unsqueeze(1)` (the `ValueError` names both shapes); a `nn.Sigmoid()` at the end of `FraudNet` (no error, worse scores); the validation loss computed in training mode; `torch.manual_seed(42)` missing before the loader (numbers differ from the neighbor's).
- **Debrief (3m):** "Here the network wins, and the comparison is honest: same rows, same features. But look at the default boosting score: 263 training frauds is a small class, and boosting with defaults overfits it. Lesson 5.3's tuned boosting does better on fraud. So the verdict is not 'networks win on fraud'; it is 'measure, and tune the challenger as carefully as the favorite'." Also: the validation set holds only 66 frauds, so the early-stopping decision is noisy.
- **Fast finishers:** `labs/module-07/l10_2-pytorch/debug_training_loop.py`: two loud bugs (float64 tensors, flat targets) and three silent ones (a sigmoid before `BCEWithLogitsLoss`, no `zero_grad()`, no `model.eval()`). Fixed, the validation AUC is 0.839 three times in a row. Then `labs/module-07/l10_2-pytorch/batch_size_starter.py` as homework (batch size against number of steps).

START THE LESSON PROJECT

The lesson project is **a PyTorch churn model on every usable feature**: 11 numeric and 5 categorical columns through a `ColumnTransformer`, dropout, early stopping, one test, an honest comparison, and saving everything needed to reuse it. New notebook `lesson_10_2_project`; learners copy `train_one_epoch`, `evaluate` and `fit` from the demo. In the room: Steps 1 to 4 (15 minutes; Steps 1 to 3 are copy-and-check, Step 4 trains in seconds).

- Step 1: leave out `customer_id`, `signup_date` and `cancellation_reason` (the leak from 1.2). **Done** = `3600 900 1500`.
- Step 2: preprocessing fitted on the fitting rows only. **Done** = `torch.Size([3600, 40]) torch.Size([900, 40]) torch.Size([1500, 40])` (16 columns became 40).
- Step 3: `ChurnMLP` with dropout: parameters: 1345.
- Step 4: Adam 0.003, batch 64, up to 30 epochs, patience 5: `stopped after epoch 22; best epoch 17, validation loss 0.3138` (yours may vary slightly).
- **Goes home:** Step 5 (test AUC: logistic regression 0.826, network 0.846, gradient boosting 0.850: a tie within the wobble of one test set, and boosting needed none of the choices) and Step 6 (save `churn_mlp.pt` with its settings and `churn_preprocess.joblib`, rebuild from disk).

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- What dtype does `torch.from_numpy(np.array([1.0, 2.0]))` have? **Answer:** float64; layers want float32.
- Put the four loop lines in order. **Answer:** `optimizer.zero_grad()`, forward and loss, `loss.backward()`, `optimizer.step()`.
- Your model ends with `nn.Linear(32, 1)` and trains with `BCEWithLogitsLoss`. How do you get probabilities for a report? **Answer:** `torch.sigmoid(model(X))` in eval mode under `no_grad()`.
- Which AUC do you report as the expected performance on new customers? **Answer:** the test AUC, computed once after training; the validation AUC chose the epoch.

Workbook self-check, items Q7 to Q12 (learners do them at the end of the block or at home).

Answers: **Q7** `torch.Size([3, 4]) 15`; **Q8** A) `ValueError`: Target size (torch.Size([8])) must be the same as input size (torch.Size([8, 1])); **Q9** C) `model.eval()` changes how layers such as dropout behave; `torch.no_grad()` stops gradient tracking; prediction code uses both; **Q10** `16 40`; **Q11** B) `state_dict()` returns references to the live tensors, which later epochs would keep changing; **Q12** D) integer class indices 0, 1, 2 (`torch.long`), shape (n,). Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: a card with the mapping table and the four loop lines. In the fraud lab, let a struggling pair copy the loop from the demo notebook and spend their effort on Step 5's comparison and what it means. Pair learners new to classes with someone who has written one: `__init__` and `forward` are the only two methods they need.

Stretch: the lesson's Exercise 4 (a regression network for weekly demand that narrowly beats default boosting: 8.09 against 8.36 units) and the Bonus Challenge (`pos_weight`): ask them to predict, before running, whether the AUC or the share of customers above 0.5 moves more.

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Put the three questions on the slide. Learners answer in the workbook's Exit Ticket box or on a sticky note. Sort the notes before Session 8: the pile for question 2 (logits and probabilities) tells you how much of the Session 8 warm-up goes to the loss/last-layer pairing, which 10.3 relies on with ten classes.

1. Write the four steps of the training loop and the PyTorch line for each. **Answer:** reset gradients `optimizer.zero_grad()`; forward and loss `loss = loss_fn(model(xb), yb)`; backward `loss.backward()`; update `optimizer.step()`.
2. Your churn model returns logits. How do you get a churn probability for a report, and why is there no sigmoid inside the model? **Answer:** `torch.sigmoid(model(X))` in eval mode under `no_grad()`; `BCEWithLogitsLoss` applies the sigmoid itself (in a numerically safe way), and a second sigmoid would squash the outputs and stall training.
3. On churn the network's test AUC is 0.846 and gradient boosting's 0.850. Which do you recommend, and why? **Answer:** gradient boosting: the gap is within the wobble of one test set, and boosting needed no scaling, architecture, learning rate, dropout or early stopping. Networks earn their keep on images and sequences (Session 8).

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** 10.1 Steps 4 to 6 (network AUC 0.856 vs logistic regression 0.823 vs boosting 0.858; `churn_net.npz` reloads to identical predictions). 10.2 Steps 5 and 6 (test AUCs 0.826 / 0.846 / 0.850; `churn_mlp.pt` and `churn_preprocess.joblib` rebuild a working model).
- **Lesson exercises.** 10.1: Exercises 1 to 3 (a neuron by hand, hidden units, a learning-rate sweep); Exercise 4 is a stretch. 10.2: Exercises 1 to 3 (tensor warm-up, SGD vs Adam, three classes with `CrossEntropyLoss`); Exercise 4 is a stretch.
- **Both Quick Quizzes** on the lesson pages (answers are in the Answer Key).
- **Journal:** both entries. The 10.2 prompt asks which line of the PyTorch loop replaces which piece of their 10.1 code, and for the recommendation with the two test AUCs.

- **Labs:** finish `churn_neuron_starter.py`; `batch_size_starter.py` (about 20 minutes: why 10 epochs at batch 3,600 learn almost nothing).
- **Before the Next Lesson (from 10.2):** save `train_one_epoch`, `evaluate` and `fit` in `training.py`; run `from torchvision import datasets` and make sure Fashion-MNIST is in `data/` (see the bring list); try `n_hidden` 8 and 128 on the validation set only.

BRING LIST FOR SESSION 8

- A laptop with the course environment (nothing new to install; torchvision came with PyTorch in Lesson 0.1). Plug it in: training on battery saver can be several times slower.
- **Fashion-MNIST in data/FashionMNIST/.** It has been there since Session 6 (Lesson 9.3). Anyone without it, at home before Session 8: run `python labs/module-01/l7_0-orientation/check_environment.py --download`, or `from torchvision import datasets; datasets.FashionMNIST("data", train=True, download=True); datasets.FashionMNIST("data", train=False, download=True)` from the project folder (about 30 MB, once). Venue Wi-Fi is not the place for it.
- The `pinecrest-ml` folder with `data/` (including `daily_store_sales.csv`, used by Lesson 10.4) and the `labs/` folder; Session 8's starters are in `labs/module-08/`, and its `_fixtures/` folder holds two small saved models the labs load.
- `training.py` with `train_one_epoch`, `evaluate` and `fit` from Lesson 10.2, and the 10.2 project notebook.
- The workbook and the journal.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** 10.1 demo Steps 8 to 11 (training, autograd agreeing, the gradient trap); 10.2 demo Steps 8 to 10 (the loop, the two curves, early stopping); fraud lab Step 5 and its debrief.
- **Cut first (saves 4m):** 10.1 demo Steps 6 and 7 (backprop shapes and the numeric check): say "every gradient has its parameter's shape, and a nudge confirms them" and show the slide.
- **Cut second (saves 5m):** 10.2 demo Steps 1 and 2 (tensor basics and the dtype trap): show the outputs on the slide, keep the float32 rule on the board.
- **Cut third (saves 10m):** the 10.1 project start: read the brief, give the check numbers (0.835 / 0.823 / 0.856 / 0.858), send all six steps home.

RUNNING SHORT: WHAT TO ADD

- Run `batch_size_starter.py` in pairs and debrief Step 4: given the same number of steps, the full batch catches up (val AUC 0.873); small batches simply give many more steps per pass.

- The lesson's 10.2 Exercise 2 live (SGD vs Adam): SGD's validation loss after 10 epochs (0.392) is still worse than Adam's after one (0.360).
- Start the 10.2 Bonus Challenge (`pos_weight=5.0`) together and ask for predictions first.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 8: CNNs and Sequence Models

How to teach Session 8 of Advanced Machine Learning. Convolutions and pooling, built up from a hand-applied filter to a CNN that beats a dense network on Fashion-MNIST with the same loop; then sequences: naive forecasts, windows split by time, an RNN opened up, an LSTM forecaster that beats "same day last week" by a third, and attention built by hand. Includes a CPU-time plan (what to start early, what trains through the break, a checkpoint fallback) for the first session where training takes real time.

`2 lessons` `3 hrs session` `Lessons 10.3–10.4` `Session 8 of 13`

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame → live demo → guided lab → start the lesson's project. Exercises, quiz and journal are homework.
- Type, do not paste, except the training helpers (they come from `training.py`, made as Session 7 homework). Every output below was produced by the code above it (Python 3.12, PyTorch 2.14.1, torchvision, scikit-learn 1.9.1 on 4 CPU threads).
- Outputs marked "yours may vary slightly" are PyTorch results: repeatable on one machine, possibly different in the last digits elsewhere. Compare trends and rankings.
- The lessons are the reference: keep `deep_learning_cnn.html` and `deep_learning_rnn.html` open for the pictures (filter maps, the CNN diagram, the confusion matrix, the 28-day chart, the holiday-season chart). Labs live in `labs/module-08/`.
- This is the session where training takes real time on a CPU. The CPU-time plan below says what to start early, what runs during the break, and what to load from a checkpoint.

THE #1 RISK OF SESSION 8: WAITING FOR TRAINING

Ten epochs of CNN training in the demo, four more in the project, two GRUs in a lab: on a slow laptop the room spends its time watching progress lines. The plan: start every training cell, then TALK over it (each demo step says what to discuss while it runs); load the kit's checkpoint the moment a laptop falls behind; and launch the project's training right before the break so it finishes while people get coffee. The second risk is the opposite of Session 7's: a room that now believes networks always win. Protect the end of 10.4's project (on learners' own time, so make the point in the demo): with the same information, default gradient boosting on a few lag columns ($616) sits between the weekday-only LSTM ($702) and the calendar-aware one ($515). The information fed in mattered more than the architecture.

BEFORE THE SESSION

- ☐ **Fashion-MNIST on every laptop** (about 30 MB in `data/FashionMNIST`). It was on Session 7's bring list. On the projector machine, check that `datasets.FashionMNIST("data", download=True)` finds the files without downloading. Bring a USB stick with a copy of the `FashionMNIST` folder for anyone who did not download it.
- ☐ **Fixtures:** `labs/module-08/_fixtures/` must hold `fashion_dense.pt` and `fashion_small_cnn.pt` (about 0.4 MB each; the lesson's two models). Hand the folder out with the starters. To rebuild: `python labs/module-08/l10_3-cnn/prepare_fashion_checkpoints.py` (prints 0.840 and 0.864).
- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-08` (8 files, about 2 minutes).
- ☐ **Pre-run both lesson notebooks** on the projector machine the day before, all the way through the projects. Keep the outputs: on the slide or a second screen they are your answer when a live run is slow. Keep `fashion_cnn.pt` and `sales_lstm.pt` from the rehearsal.
- ☐ **Downloads for next week (announce today):** Session 9 (Lesson 10.5) needs EuroSAT in `data/eurosat/` (94 MB, about 186 MB unpacked) and the pretrained ResNet-18 (45 MB) and MobileNetV3-small (10 MB) weights in PyTorch's cache. Too big for venue Wi-Fi: the bring list gives the one-line command for home. Prepare a USB stick with `eurosat/` and the two `.pth` files as the fallback Session 9 offers.
- ☐ **Board:** the shape formula out = (n + 2p - k) / s + 1 and the parameter formula; the CNN pipeline (conv → ReLU → pool, twice, flatten, dense); for 10.4, a row of boxes for a 28-day window with an arrow to day 29, and "$1,072" in a box.

CPU-TIME PLAN (4 THREADS, MEASURED ON THE BUILD MACHINE WITH OTHER JOBS RUNNING; A QUIET MODERN LAPTOP IS SIMILAR, AN OLDER ONE UP TO TWICE AS SLOW)

WHAT	MEASURED	PLAN
Loading Fashion-MNIST and `import torch` (demo Step 1)	about 5 s	Learners run it during the warm-up.

WHAT	MEASURED	PLAN
10.3 dense baseline, 5 epochs on 20,000 images (Step 9)	about 9 s	Live; talk over it.
10.3 SmallCNN, 5 epochs (Step 10)	about 22 s	Start it, then discuss where the parameters live. Laptops still running after a minute: load `_fixtures/fashion_small_cnn.pt` (Step 11's fallback) and continue.
10.3 project FashionCNN, 4 epochs (Step 3)	about 45 s (up to 2 min on older laptops)	Launch right before the break; it finishes during it. Anyone whose run fails: use the rehearsal's `fashion_cnn.pt` from the shared folder for Steps 4 to 6 at home.
10.4 LSTM, 15 epochs on 2,812 windows (Step 9)	about 9 s	Live.
10.4 project calendar LSTM (Step 3) and boosting (Step 5)	about 11 s and 6 s	Step 3 in the room; Step 5 at home.
Labs: edge race 30 s, shift test 20 s (loads the fixtures, no training), transactions GRU 30 s, attention 3 s		All live; under 90 s each.

GPU note. Nobody needs a GPU today, and every number in this kit comes from the CPU code. A learner with a CUDA laptop may try the lesson's GPU card at home (move the model once, every batch in the loops); in the room, stay on the CPU so results match the pages and the neighbors. GPU arithmetic is not bit-identical to CPU arithmetic, so even a correct GPU run differs in the last digits.

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
The first cell starts downloading Fashion-MNIST	The data is not in this project's `data/` (often: JupyterLab started in another folder)	`print(Path.cwd())`. Start from `pinecrest-ml`; copy the `FashionMNIST` folder from the USB stick into `data/`.
`RuntimeError: Given groups=1, weight of size [16, 1, 3, 3], expected input[1, 64, 28, 28] to have 1 channels`	A batch built by hand without the channel axis	`.unsqueeze(1)`: (64, 1, 28, 28).
`RuntimeError: mat1 and mat2 shapes cannot be multiplied (64x1568 and`	First Linear sized for 28 × 28 maps	Trace a dummy batch; use the flattened size you see.

SYMPTOM	CAUSE	FIX
`25088x10)`		
Training takes many minutes	Battery saver, the full 60,000 images, or training inside an evaluation loop	Plug in; check the `Subset(..., range(20_000))`; load the checkpoint and move on.
`FileNotFoundError` for `_fixtures/fashion_small_cnn.pt`	The `_fixtures` folder was not copied with the starters	Copy `labs/module-08/_fixtures/` from the shared folder.
`ValueError: RNN input dtype (torch.float64) does not match weight dtype (torch.float32)`	Windows built from NumPy without a dtype	`torch.tensor(np.array(X), dtype=torch.float32)`.

SESSION 8 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on 10.1 and 10.2 while everyone's first cell loads Fashion-MNIST; homework check.
0:10	10	**10.3** Frame and mental model	A stencil and a flashlight; why a dense network wastes an image's structure.
0:20	30	**10.3** Live demo	Fashion-MNIST, a convolution by hand, edge filters on a boot, Conv2d shapes and parameters, tracing SmallCNN, dense baseline vs CNN (trained live, checkpoint as fallback), per-class accuracy.
0:50	20	**10.3** Guided lab	`labs/module-08/l10_3-cnn/edge_features_starter.py` (12 min), then `labs/module-08/l10_3-cnn/shift_test_starter.py` in pairs (8 min).
1:10	10	**10.3** Start the lesson project	A better clothing classifier, Steps 1 to 3; Step 3's training runs through the break.
1:20	10	Break	The FashionCNN trains on every laptop (about a minute, longer on older machines).
1:30	10	**10.4** Frame and mental model	The store manager's notebook and the binder; check the project training finished.
1:40	30	**10.4** Live demo	Daily sales, naive forecasts, windows, an RNN cell by hand, LSTM and GRU, scaling, the LSTM forecaster, attention by hand, the silent mistakes.
2:10	25	**10.4** Guided lab	`labs/module-08/l10_4-sequences/transactions_gru_starter.py` (15 min), then `labs/module-08/l10_4-`

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
			`sequences/attention_lookup_starter.py` (10 min).
2:35	15	**10.4** Start the lesson project	A calendar-aware forecaster, Steps 1 to 3; Steps 4 to 7 go home.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 9 (downloads at home).
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 110 + 115 = 225 minutes for a solo learner; the room has 150. So the room does the mental model, the live demo (with training started early or loaded from a checkpoint), one or two guided kit labs and the first project steps. The lessons' Common Mistakes are folded into the demos and the two debug handouts.

Goes home (2 to 3 hours). 10.3 project Steps 4 to 6 (test once, class by class; save and reload; predictions and summary); 10.4 project Steps 4 to 7 (per store and special days, the boosting challenger, the holiday chart, save and summarize); Exercises 1 to 3 of each lesson (Exercise 4 is a stretch); both Quick Quizzes; both journals; the second halves of the shift and attention labs; the Session 9 downloads.

Protect, in this order: the dense-vs-CNN comparison with the same seed and loop (10.3 Steps 9 to 12); naive forecasts and the time split (10.4 Steps 4 and 5); the LSTM against $1,072 (Step 9); the three silent mistakes (Step 12). **Cut first:** 10.3 Step 5 (edge filters on a boot: show the page's picture), then 10.4 Step 6's manual loop (say "same numbers, step by step"), then 10.4 Step 11 (PyTorch's attention function).

WARM-UP

0:00 Warm-up: Retrieval on 10.1 and 10.2

10 min

TIMING

2m	Everyone opens `lesson_10_3` and runs the Fashion-MNIST cell (demo Step 1's code is on the slide); it loads while they think
5m	Five questions; learners answer in the workbook's Warm-up box
3m	Reveal; homework check: who has `training.py`? Who has Fashion-MNIST in `data/`?

QUESTIONS AND ANSWERS

1. **W1.** What does this print? (Lesson 10.2)

```
import numpy as np
import torch
print(torch.tensor(np.zeros(3)).dtype, torch.tensor(np.zeros(3), dtype=torch.f
```

Answer: `torch.float64 torch.float32`: torch.tensor keeps NumPy's float64; layers want float32, so convert as you create.

2. **W2.** Put the four lines of the batch loop in order: loss.backward(), optimizer.step(), optimizer.zero_grad(), loss = loss_fn(model(xb), yb). (10.2) **Answer:** optimizer.zero_grad(), loss = loss_fn(model(xb), yb), loss.backward(), optimizer.step().

3. **W3.** Your binary model ends with nn.Linear(32, 1) and trains with BCEWithLogitsLoss. Where does the sigmoid go? (10.2) **Answer:** Inside the loss during training; apply torch.sigmoid yourself only to report probabilities. Today's ten-class models follow the same rule with softmax and CrossEntropyLoss.

4. **W4.** A model with dropout gives a different score for the same customer on every call, inside with torch.no_grad(). Why? (10.2) **Answer:** It is still in training mode: no_grad() stops gradient tracking but does not switch dropout off. Call model.eval().

5. **W5.** On the churn table the PyTorch network scored test AUC 0.846 and gradient boosting 0.850. What was the verdict, and what kind of data should change it? (10.1, 10.2) **Answer:** Recommend boosting: a tie within the wobble, with far fewer choices. Images, sequences and text are where networks earn their keep: today's session.

BRIDGE TO TODAY

Say: "Last week a network tied gradient boosting on a table and we recommended boosting. Today the data has structure a network can exploit: pixels that belong together, and days that come in order. Same training

loop as 10.2; what changes is the architecture, and that is exactly the point."

LESSON PLANS

10.3 Convolutional Neural Networks

70 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain what a convolution filter and a pooling layer compute, and why sharing one small filter across the whole image suits pictures
- Predict the output shape and the parameter count of `nn.Conv2d` and `nn.MaxPool2d` layers
- Build a CNN as an `nn.Module` and train it on Fashion-MNIST with a validation set and a fixed seed
- Compare it with a dense-network baseline using test accuracy, per-class accuracy and a confusion matrix
- Save trained weights with `state_dict` and reload them safely with `weights_only=True`

TIMING IN THE ROOM

10m	Frame: a stencil and a flashlight; local connectivity and weight sharing
30m	Live demo: data, a convolution by hand, Conv2d shapes and parameters, tracing SmallCNN, dense vs CNN, per-class accuracy, two silent traps
20m	Guided lab: `edge_features_starter.py`, then `shift_test_starter.py` in pairs
10m	Start the project: Steps 1 to 3 (Step 3 trains through the break)

MENTAL MODEL TO SAY

Say: "Picture a 3 × 3 stencil cut to match one pattern: dark on the left, bright on the right. Slide it across the photo one step at a time and write down how well each patch matches. You get a new image, bright wherever that pattern appears: a feature map. The stencil is a filter, the sliding is a convolution. A layer holds many stencils, and training learns their patterns: edges first, then corners and stripes, then collars and soles."

Then the two ideas on the board: **local connectivity** (each output looks at a small window) and **weight sharing** (the same stencil at every position, so a pattern learned once is found anywhere, for 9 weights plus a bias). Contrast: "A dense network flattens the image into 784 unrelated numbers; if a sneaker moves three pixels, every input changes." Today's shift lab tests how far that promise really goes.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Notebook `lesson_10_3`. Steps 1 to 8 are instant; Steps 9 and 10 train (start them and talk); Step 11 has the checkpoint fallback for slow laptops.

1 **Load Fashion-MNIST (2m).** "Seventy thousand small grayscale photos of clothing in ten classes. It is already in your `data/` folder, so `download=True` just finds it."

```
import torch
from torch import nn
from torch.utils.data import DataLoader, Subset
from torchvision import datasets, transforms

train_full = datasets.FashionMNIST("data", train=True, download=True, transfor
test_set = datasets.FashionMNIST("data", train=False, download=True, transform

print(len(train_full), len(test_set))
print(train_full.classes)
```

```
60000 10000
['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sandal', 'Shirt', 'Sn
```

Anyone whose cell starts downloading: let it run (about 30 MB) or pair them up; this is why it was homework.

2 **One image is (channels, height, width) (2m).** "Channel first, always. A batch adds a dimension in front: (64, 1, 28, 28). Most CNN errors are shape errors."

```
image, label = train_full[0]
print(image.shape, image.dtype)
print(f"pixel values {image.min():.1f} to {image.max():.1f}; label {label} = {
```

```
torch.Size([1, 28, 28]) torch.float32
pixel values 0.0 to 1.0; label 9 = Ankle boot
```

3 **Three sets, and a fair headline metric (2m).** "20,000 to train, 5,000 other training images to watch progress, the official 10,000 locked away."

```
train_set = Subset(train_full, range(20_000))
val_set = Subset(train_full, range(55_000, 60_000))

print(len(train_set), len(val_set), len(test_set))
print(torch.bincount(train_full.targets[:20_000]))
```

```
20000 5000 10000
tensor([1935, 2025, 1982, 2011, 1967, 2010, 2068, 2003, 1971, 2028])
```

About 2,000 per class: chance is 10%, and accuracy is a fair headline here, unlike churn.

4 **A convolution by hand (3m).** "A 3 × 3 stencil for 'dark on the left, bright on the right'. Slide it, multiply, add." Ask for the value where the window straddles the edge before running (27).

```
import torch.nn.functional as F

x = torch.tensor([[0, 0, 0, 9, 9]] * 5, dtype=torch.float32)
vertical_edge = torch.tensor([[-1, 0, 1]] * 3, dtype=torch.float32)

out = F.conv2d(x.reshape(1, 1, 5, 5), vertical_edge.reshape(1, 1, 3, 3))
print(out.squeeze())
```

```
tensor([[ 0., 27., 27.],
        [ 0., 27., 27.],
        [ 0., 27., 27.]])
```

The output is 3 × 3: a 3-wide window fits three positions across five pixels.

5 **Two edge filters on a real boot (2m).** "`padding=1` keeps 28 × 28. Two filters, two output channels." Show the lesson page's three-panel picture.

```
first_of_each = [int((train_full.targets == c).nonzero()[0]) for c in range(10
boot = train_full[first_of_each[9]][0].unsqueeze(0)    # shape (1, 1, 28, 28)
edge_filters = torch.stack([vertical_edge, vertical_edge.T]).unsqueeze(1)
print(boot.shape, edge_filters.shape)

edges = F.conv2d(boot, edge_filters, padding=1)
print(edges.shape)
```

```
torch.Size([1, 1, 28, 28]) torch.Size([2, 1, 3, 3])
torch.Size([1, 2, 28, 28])
```

6 **Conv2d: shapes and the parameter bargain (3m).** "Sixteen learnable 3 × 3 filters: 160 parameters. A dense layer with the same outputs: almost ten million."

```
conv1 = nn.Conv2d(in_channels=1, out_channels=16, kernel_size=3, padding=1)
batch = torch.zeros(64, 1, 28, 28)

print("weight", tuple(conv1.weight.shape), " bias", tuple(conv1.bias.shape))
print("after conv:", tuple(conv1(batch).shape))
print("after pool:", tuple(nn.MaxPool2d(2)(conv1(batch)).shape))
print("conv parameters:", sum(p.numel() for p in conv1.parameters()))
print(f"dense layer with the same output size: {784 * 16 * 28 * 28 + 16 * 28 *
```

```
weight (16, 1, 3, 3)  bias (16,)
after conv: (64, 16, 28, 28)
after pool: (64, 16, 14, 14)
conv parameters: 160
dense layer with the same output size: 9,847,040
```

Write the two formulas on the board: out = (n + 2p - k) / s + 1, and out_channels × (in_channels × k × k) + out_channels.

7 **Trace a dummy batch through SmallCNN (3m).** "The most useful CNN debugging habit: push zeros through and print every shape."

```
class SmallCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2
            nn.Conv2d(16, 32, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool
        )
        self.classifier = nn.Sequential(
            nn.Flatten(), nn.Linear(32 * 7 * 7, 64), nn.ReLU(), nn.Linear(64,
        )

    def forward(self, x):
        return self.classifier(self.features(x))

torch.manual_seed(42)
cnn = SmallCNN()

x = torch.zeros(64, 1, 28, 28)
for layer in list(cnn.features) + list(cnn.classifier):
    x = layer(x)
    print(f"{layer.__class__.__name__:10s} {tuple(x.shape)}")
```

```
Conv2d     (64, 16, 28, 28)
ReLU       (64, 16, 28, 28)
MaxPool2d  (64, 16, 14, 14)
Conv2d     (64, 32, 14, 14)
ReLU       (64, 32, 14, 14)
MaxPool2d  (64, 32, 7, 7)
Flatten    (64, 1568)
Linear     (64, 64)
ReLU       (64, 64)
Linear     (64, 10)
```

The trace gives the first Linear its $32 \times 7 \times 7 = 1{,}568$ inputs. No softmax at the end: CrossEntropyLoss applies it.

8 **The same helpers as 10.2 (1m).** Type `train_one_epoch`, `accuracy` and `fit` (or paste from `training.py`, made as 10.2 homework). Nothing prints.

```
val_loader = DataLoader(val_set, batch_size=1000)
test_loader = DataLoader(test_set, batch_size=1000)

def train_one_epoch(model, loader, optimizer, loss_fn):
    model.train()
    total_loss, n = 0.0, 0
    for xb, yb in loader:
        optimizer.zero_grad()
        loss = loss_fn(model(xb), yb)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * len(yb)
        n += len(yb)
    return total_loss / n

def accuracy(model, loader):
    model.eval()
    correct, n = 0, 0
    with torch.no_grad():
        for xb, yb in loader:
            correct += (model(xb).argmax(dim=1) == yb).sum().item()
            n += len(yb)
    return correct / n

def fit(model, epochs=5, lr=1e-3):
    torch.manual_seed(42)
    train_loader = DataLoader(train_set, batch_size=64, shuffle=True)
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    loss_fn = nn.CrossEntropyLoss()
    for epoch in range(1, epochs + 1):
        loss = train_one_epoch(model, train_loader, optimizer, loss_fn)
        print(f"epoch {epoch}: train loss {loss:.3f}  validation accuracy {acc
```

9 **The dense baseline (start it, then talk) (3m).** Start the cell, then talk over it: "Same loop, five epochs, a network that flattens every image to 784 numbers." It takes 10 to 20 seconds on a laptop.

```
torch.manual_seed(42)
mlp = nn.Sequential(nn.Flatten(), nn.Linear(28 * 28, 128), nn.ReLU(), nn.Linea
print(f"dense network parameters: {sum(p.numel() for p in mlp.parameters()):,}

fit(mlp)
```

Output (yours may vary slightly)

```
dense network parameters: 101,770
epoch 1: train loss 0.697  validation accuracy 0.818
epoch 2: train loss 0.474  validation accuracy 0.811
epoch 3: train loss 0.429  validation accuracy 0.849
epoch 4: train loss 0.394  validation accuracy 0.860
epoch 5: train loss 0.369  validation accuracy 0.854
```

Training output is "yours may vary slightly": same seed and thread count repeat exactly; another processor can change the last digits.

10 **The CNN, same everything except the model (4m).** "Only the architecture changes." Start it, and while it trains (20 to 40 seconds) ask where its 105,866 parameters live (almost all in the dense head; the convolutions use 4,800).

```
torch.manual_seed(42)
cnn = SmallCNN()
print(f"CNN parameters: {sum(p.numel() for p in cnn.parameters()):,}")

fit(cnn)
```

Output (yours may vary slightly)

```
CNN parameters: 105,866
epoch 1: train loss 0.759  validation accuracy 0.803
epoch 2: train loss 0.463  validation accuracy 0.830
epoch 3: train loss 0.397  validation accuracy 0.859
epoch 4: train loss 0.357  validation accuracy 0.878
epoch 5: train loss 0.329  validation accuracy 0.873
```

Slow laptops: do not wait. Load the kit's checkpoint (next step) and move on; the numbers are the same.

11 **Open the test set once (2m).** "Validation says the CNN is ahead. Now, once, the test set."

```
for name, model in [("dense network", mlp), ("CNN", cnn)]:
    print(f"{name:14s} test accuracy {accuracy(model, test_loader):.3f}")
```

```
dense network  test accuracy 0.840
CNN            test accuracy 0.864
```

Fallback for any laptop that could not train in time (the kit's `_fixtures/` hold exactly these two models): <pre class="code">path = "labs/module-08/_fixtures/" mlp.load_state_dict(torch.load(path + "fashion_dense.pt", weights_only= True)) cnn.load_state_dict(torch.load(path + <span

class="s">"fashion_small_cnn.pt", weights_only=True)) for name, model in [("dense network" , mlp), ("CNN", cnn)]: acc = accuracy(model, test_loader) print(f"{name:14s} test accuracy {acc:.3f} (from the checkpoint)")</pre> <pre class="out">dense network test accuracy 0.840 (from the checkpoint) CNN test accuracy 0.864 (from the checkpoint)</pre>

12 **Which classes, not just how many (3m).** "An average hides where the model fails." Show the confusion matrix on the lesson page.

```
cnn.eval()
with torch.no_grad():
    test_pred = torch.cat([cnn(xb).argmax(dim=1) for xb, _ in test_loader])
y_test = test_set.targets

for c, name in enumerate(test_set.classes):
    mask = y_test == c
    print(f"{name:12s} {(test_pred[mask] == c).float().mean():.3f}")
```

```
T-shirt/top  0.706
Trouser      0.978
Pullover     0.747
Dress        0.863
Coat         0.846
Sandal       0.878
Shirt        0.720
Sneaker      0.969
Bag          0.971
Ankle boot   0.958
```

Trousers, bags, sneakers, boots are nearly solved; T-shirts, pullovers, coats and shirts are confused with each other: a collar a few pixels wide. Report the weakest classes.

13 **Two silent traps (2m).** Softmax before CrossEntropyLoss, then dropout in training mode.

```
loss_fn = nn.CrossEntropyLoss()
confident_logits = torch.tensor([[12.0] + [-12.0] * 9])    # very sure it is c
target = torch.tensor([0])

print(f"logits into the loss : {loss_fn(confident_logits, target):.3f}")
print(f"softmax first        : {loss_fn(torch.softmax(confident_logits, dim=1)

torch.manual_seed(42)
dropout = nn.Dropout(p=0.5)
ones = torch.ones(8)

print(dropout(ones))
print(dropout(ones))
dropout.eval()
print(dropout(ones))
```

```
logits into the loss : 0.000
softmax first        : 1.461
tensor([2., 2., 2., 2., 0., 2., 0., 0.])
tensor([2., 2., 2., 2., 0., 0., 2., 0.])
tensor([1., 1., 1., 1., 1., 1., 1., 1.])
```

A perfect answer keeps a loss of 1.46 when softmax is applied twice; dropout changes the output on every call until `eval()`.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **The channel axis.** `RuntimeError: Given groups=1, weight of size [16, 1, 3, 3], expected input[1, 64, 28, 28] to have 1 channels, but got 64 channels instead` Coach: "How many dimensions does a batch of images need, and in what order?" (batch, channels, height, width); `.unsqueeze(1)`.
- **The first Linear sized on paper.** `RuntimeError: mat1 and mat2 shapes cannot be multiplied (64x1568 and 25088x10)` Coach: "Trace a dummy batch through `features` and read the size." The message always shows the actual size first.
- **Probabilities into CrossEntropyLoss, or predictions in training mode.** Both silent (demo Step 13): a softmax layer before the loss keeps even a perfect answer at a loss of 1.461; dropout and batch norm in training mode make predictions noisy and worse. Coach: "What does your last layer output: logits or probabilities? Which mode is the model in right now?"

GUIDED LAB

- **Lab A (12m, everyone):** `labs/module-08/l10_3-cnn/edge_features_starter.py`; reference `labs/module-08/l10_3-cnn/edge_features_solution.py`. Learners build a classic computer-vision pipeline from the lesson's two edge filters (edge strength and brightness pooled over a 4 × 4 grid:

48 numbers per image, then logistic regression) and race it against SmallCNN at 1,000, 5,000 and 20,000 training images. The CNN code is given.

- **Expected, Lab A:** hand-made + logistic regression 0.795 / 0.819 / 0.827; SmallCNN (3 epochs) 0.676 / 0.782 / 0.863. **Debrief (2m):** "With little data the person's two filters win: they carry knowledge the CNN must learn from examples. With 20,000 images the learned filters overtake them. That is the whole case for deep learning on images, and also the case for transfer learning next week: what if someone else already learned the filters from a million images?"
- **Lab B (8m, pairs; finish at home):** `labs/module-08/l10_3-cnn/shift_test_starter.py`; reference `labs/module-08/l10_3-cnn/shift_test_solution.py`. Load the lesson's two models from `_fixtures/` (no training), shift the test images 1 to 4 pixels right and down, and watch both accuracies.
- **Expected, Lab B:** shifted right by 1 / 2 / 3 / 4 pixels: dense 0.756 / 0.552 / 0.355 / 0.252, CNN 0.838 / 0.743 / 0.471 / 0.301 (down: dense 0.778 / 0.708 / 0.661 / 0.553, CNN 0.821 / 0.770 / 0.726 / 0.592). Worst CNN classes at 2 pixels: Dress, T-shirt/top, Coat. **Debrief:** the CNN is more tolerant at every shift but not shift-proof: the convolutions find a pattern anywhere, the Flatten + Linear head still reads where. Augmentation (the lesson's Exercise 4) is how you teach it more.
- **Circulate for:** `torch.roll` instead of a zero-filled shift (pixels wrap around from the other side: ask what appears at the left edge); forgetting `eval()` after loading (harmless for these two models, which have no dropout, but the habit matters); in Lab A, `.abs()` forgotten (edge responses cancel out when averaged).
- **Fast finishers:** `labs/module-08/l10_3-cnn/debug_cnn.py`: two shape errors and two silent bugs; fixed it prints 0.747 twice.

START THE LESSON PROJECT

The lesson project is **a better clothing classifier**: batch normalization and dropout, the best epoch kept on validation, a per-class report, saved and reloaded weights. New notebook `lesson_10_3_project`. In the room: Steps 1 to 3, timed so Step 3's training runs during the break.

- Step 1: the three sets. **Done** = `20000 5000 10000`.
- Step 2: `FashionCNN`, traced before training. **Done** = `parameters: 421,834 · features out: (2, 64, 7, 7) · logits out : (2, 10)`.
- Step 3 (launch right before the break): four epochs, keeping the best. **Done** = `kept epoch 4 (validation accuracy 0.903)` (yours may vary slightly).
- **Goes home:** Step 4 (test once: 0.892; hardest class Shirt 0.632), Step 5 (save, reload with `weights_only=True`, 18 tensors including batch-norm statistics), Step 6 (predictions and summary). Anyone whose training did not finish can load the rehearsal's `fashion_cnn.pt`.

CHECKS FOR UNDERSTANDING

Ask these out loud:

- Shape of `nn.Conv2d(1, 8, kernel_size=5)` on (4, 1, 28, 28)? **Answer:** (4, 8, 24, 24): 28 - 5 + 1.
- Parameters of `nn.Conv2d(16, 32, kernel_size=3)`? **Answer:** 32 × (16 × 9) + 32 = 4,640.
- Why does the CNN beat the dense network with about the same number of parameters? **Answer:** small filters reused at every position: patterns learned once, found anywhere.
- Why call `model.eval()` before testing a model with batch norm and dropout? **Answer:** dropout off, batch norm uses its stored training statistics.

Workbook self-check, items Q1 to Q6.

Answers: **Q1** `(8, 16, 16, 16)`; **Q2** A) 224; **Q3** C) It keeps the height and width unchanged: (n + 2 - 3) / 1 + 1 = n; **Q4** `tensor([[ 5., 7.], [13., 15.]])`; **Q5** D) The `Flatten` + `Linear` head reads WHERE each feature map lit up, so positions still matter; **Q6** B) 3136 (64 × 7 × 7). Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: hand out the CNN pipeline card (conv → ReLU → pool, twice, flatten, dense, logits) with the shape after each layer, and ask struggling learners to say the next shape before every line of the trace. In Lab A they may skip TODO 1 by copying `hand_features` from the solution and spend the time on the race and its explanation.

Stretch: the lesson's Exercise 4 (augmentation: in a short run it HURTS, 0.874 vs 0.892: an honest result to explain) and Bonus Challenge (global average pooling); or retrain SmallCNN on shifted copies and repeat the shift test.

10.4 Sequence Models

80 min

OBJECTIVES (FROM THE LESSON PAGE)

- Measure naive and seasonal-naive forecasts and a gradient-boosting model on lag features, and use them as the yardsticks any sequence model must beat
- Turn a time series into sliding windows with a time-based train/test split, and explain why a random split leaks
- Explain how a recurrent cell carries a hidden state through a sequence, and why LSTMs and GRUs add gates
- Build, train and evaluate an LSTM forecaster in PyTorch, reporting the error in dollars
- Describe attention as a soft lookup, compute it by hand, and say when a transformer is (and is not) worth it

TIMING IN THE ROOM

10m	Frame: the notebook (hidden state) and the binder (attention); check the project training finished
30m	Live demo: data, naive forecasts and the time split, windows, the RNN loop, gates, scaling, the LSTM, attention by hand, three silent mistakes
25m	Guided lab: `transactions_gru_starter.py`, then `attention_lookup_starter.py`
15m	Start the project: Steps 1 to 3

MENTAL MODEL TO SAY

Say: "A store manager reads the till report every evening. They do not memorize every number; they keep a short notebook of impressions, update it each evening with the same habits, and guess tomorrow from the notebook alone. That notebook is a recurrent network's hidden state. An LSTM is a manager with a better notebook: it decides what to erase, what to write and what to read out, so a note from four weeks ago can survive. Attention is a different manager: every report goes in a binder, and each evening they flip to the pages that look most relevant to tomorrow."

Then the discipline, which matters more than the architecture: **split by time**, **beat the naive forecasts**, **report in dollars**. "A deep model that cannot beat 'same as last week' is not worth deploying, and in forecasting that happens more often than tutorials admit."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Notebook `lesson_10_4`. Training happens only in Step 9 (about 10 seconds). The LSTM numbers are the course's reference: the project and the capstone come back to them.

1 **Three years of daily sales (2m).** "One row per store per day; the managers want tomorrow's number."

```
import numpy as np
import pandas as pd
import torch
from torch import nn

print(torch.__version__)

sales = pd.read_csv("data/daily_store_sales.csv", parse_dates=["date"])
print(sales.shape)
print(sales.head(4))
```

```
2.14.1+cpu
(4384, 6)
        date      store    sales  transactions  promo event
0 2023-01-01  Asheville  4838.69            69      0   NaN
1 2023-01-01       Bend  4533.55            65      0   NaN
2 2023-01-01      Boise  6026.92            80      0   NaN
3 2023-01-01  Flagstaff  4217.59            59      0   NaN
```

First line may differ: any recent 2.x PyTorch works (yours may vary slightly).

2 **Wide table, trends, the weekly cycle (2m).** "Boise and Flagstaff grow, Bend is flat, Asheville declines; Saturday sells almost twice Tuesday."

```
wide = sales.pivot(index="date", columns="store", values="sales")
print(wide.shape)
print(wide.groupby(wide.index.year).mean().round(0))

weekday_order = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Satu
by_weekday = sales.groupby(sales["date"].dt.day_name())["sales"].mean().reinde
print(by_weekday.round(0))
```

```
(1096, 4)
store  Asheville    Bend   Boise  Flagstaff
date
2023      4596.0  5158.0  6831.0     4970.0
2024      4649.0  5235.0  7821.0     5744.0
2025      4380.0  5527.0  8342.0     6561.0
date
Monday       4807.0
Tuesday      4437.0
Wednesday    4684.0
Thursday     4894.0
Friday       6226.0
Saturday     8481.0
Sunday       7210.0
Name: sales, dtype: float64
```

3 **How much does today tell you? (1m).** Ask the room to guess which lag correlates most before running.

```
for lag in [1, 2, 7, 14]:
    print(f"Boise: correlation with {lag:2d} days earlier: {wide['Boise'].auto
```

```
Boise: correlation with  1 days earlier: 0.673
Boise: correlation with  2 days earlier: 0.238
Boise: correlation with  7 days earlier: 0.748
Boise: correlation with 14 days earlier: 0.732
```

Lag 7 is strongest (same weekday), but lag 1 is high too: recent days carry information beyond the calendar.

4 **Naive forecasts first, split by time (3m).** "Train 2023 and 2024, test every day of 2025. Never shuffle a time series into a random split."

```
from sklearn.metrics import mean_absolute_error

is_test_day = wide.index >= "2025-01-01"
actual_2025 = wide[is_test_day].to_numpy().ravel()

baselines = {}
for name, lag in [("naive (yesterday)", 1), ("seasonal naive (same day last we
    forecast = wide.shift(lag)[is_test_day].to_numpy().ravel()
    baselines[name] = mean_absolute_error(actual_2025, forecast)
    print(f"{name:36s} MAE ${baselines[name]:,.0f}")
```

```
naive (yesterday)                    MAE $1,350
seasonal naive (same day last week)  MAE $1,072
```

Write $1,072 on the board: the number every model must beat, in dollars.

5 **From a series to windows (2m).** "An example is a window of recent days and the day after it." Point at the overlap: that is why a random split leaks (Step 12).

```python
def make_windows(series, window):
    X = np.stack([series[i:i + window] for i in range(len(series) - window)])
    y = series[window:]
    return X, y

toy_X, toy_y = make_windows(np.arange(10), window=4)
print(toy_X)
print(toy_y)
```

```
[[0 1 2 3]
 [1 2 3 4]
 [2 3 4 5]
 [3 4 5 6]
 [4 5 6 7]
 [5 6 7 8]]
[4 5 6 7 8 9]
```

6 **A recurrent cell, and the loop inside it (3m).** "The same small set of weights updates the notebook every day." Run nn.RNN, then the plain loop.

```python
torch.manual_seed(42)
rnn = nn.RNN(input_size=1, hidden_size=3, batch_first=True)

x = torch.tensor([[[0.5], [-1.0], [2.0], [0.0]]])     # batch 1, 4 time steps,
out, h_last = rnn(x)
print("input", tuple(x.shape), " out", tuple(out.shape), " h_last", tuple(h_la
print(out[0].detach().round(decimals=3))

h = torch.zeros(3)
with torch.no_grad():
    for t in range(4):
        h = torch.tanh(rnn.weight_ih_l0 @ x[0, t] + rnn.bias_ih_l0 + rnn.weigh
        print(t, h.round(decimals=3))

print("matches nn.RNN:", torch.allclose(h, h_last[0, 0]))
```

Output (yours may vary slightly)

```
input (1, 4, 1)  out (1, 4, 3)  h_last (1, 1, 3)
tensor([[0.5120, 0.6430, 0.2880],
        [0.1270, 0.2580, 0.5620],
        [0.8690, 0.9490, 0.2260],
        [0.6120, 0.6140, 0.4590]])
0 tensor([0.5120, 0.6430, 0.2880])
1 tensor([0.1270, 0.2580, 0.5620])
2 tensor([0.8690, 0.9490, 0.2260])
3 tensor([0.6120, 0.6140, 0.4590])
matches nn.RNN: True
```

Same numbers step by step: a recurrent layer really is a loop.

7 **Gates cost parameters (2m).** "LSTM: forget, input and output gates around a cell state that is updated by adding and scaling, so gradients survive many steps."

```
for name, layer in [("RNN", nn.RNN(8, 32, batch_first=True)),
                    ("GRU", nn.GRU(8, 32, batch_first=True)),
                    ("LSTM", nn.LSTM(8, 32, batch_first=True))]:
    n_params = sum(p.numel() for p in layer.parameters())
    print(f"{name:5s} {n_params:5,d} parameters")

lstm = nn.LSTM(input_size=8, hidden_size=32, batch_first=True)
out, (h_n, c_n) = lstm(torch.zeros(64, 28, 8))
print("out", tuple(out.shape), " h_n", tuple(h_n.shape), " c_n", tuple(c_n.sha
```

```
RNN   1,344 parameters
GRU   4,032 parameters
LSTM  5,376 parameters
out (64, 28, 32)  h_n (1, 64, 32)  c_n (1, 64, 32)
```

GRU 3×, LSTM 4× the plain RNN's parameters: one set per gate plus the candidate.

8 **Scale the target, then windows for four stores (3m).** "log1p so a 20% swing is the same distance in every store; standardize with the training years only."

```
log_sales = np.log1p(wide)
mu = log_sales[~is_test_day].to_numpy().mean()
sd = log_sales[~is_test_day].to_numpy().std()
z = ((log_sales - mu) / sd).to_numpy()            # days x stores, about mean

weekday = np.eye(7)[wide.index.dayofweek]         # one-hot day of week, one r
print(round(mu, 3), round(sd, 3), z.shape, weekday.shape)

def to_dollars(z_values):
    return np.expm1(np.asarray(z_values) * sd + mu)

WINDOW = 28

def build_windows(z, calendar, window=WINDOW):
    """Windows of `window` days for every store: (sales, next day's calendar)
    X, y, day, store = [], [], [], []
    for s in range(z.shape[1]):
        for t in range(window, len(z)):
            X.append(np.column_stack([z[t - window:t, s], calendar[t - window
            y.append(z[t, s])
            day.append(t)
            store.append(s)
    return (torch.tensor(np.array(X), dtype=torch.float32), torch.tensor(np.ar
            np.array(day), np.array(store))

X, y, day, store = build_windows(z, weekday)
test_mask = is_test_day[day]
print("X", tuple(X.shape), " y", tuple(y.shape))
print("training windows:", (~test_mask).sum(), " test windows:", test_mask.sum
```

```
8.524 0.73 (1096, 4) (1096, 7)
X (4272, 28, 8)  y (4272,)
training windows: 2812  test windows: 1460
```

Each step sees that day's scaled sales and the NEXT day's weekday: the calendar is known in advance, not a leak.

9 **The model and the training run (4m).** Type `SalesRNN`, then `train_forecaster` / `predict_dollars` and run (about 10 seconds).

```
class SalesRNN(nn.Module):
    def __init__(self, n_features, hidden=32, cell=nn.LSTM):
        super().__init__()
        self.rnn = cell(n_features, hidden, batch_first=True)
        self.head = nn.Linear(hidden, 1)

    def forward(self, x):                       # x: (batch, 28, n_features)
        out, _ = self.rnn(x)                    # out: (batch, 28, hidden)
        return self.head(out[:, -1]).squeeze(-1)   # last step's hidden state

torch.manual_seed(42)
model = SalesRNN(n_features=X.shape[2])
print(model)
print(sum(p.numel() for p in model.parameters()), "parameters")

from torch.utils.data import TensorDataset, DataLoader

def train_forecaster(model, X, y, train_mask, epochs=15, lr=3e-3):
    torch.manual_seed(42)                       # same batch order every run
    loader = DataLoader(TensorDataset(X[train_mask], y[train_mask]), batch_siz
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    loss_fn = nn.MSELoss()
    for epoch in range(1, epochs + 1):
        model.train()
        total = 0.0
        for xb, yb in loader:
            optimizer.zero_grad()
            loss = loss_fn(model(xb), yb)
            loss.backward()
            optimizer.step()
            total += loss.item() * len(xb)
        if epoch % 5 == 0:
            print(f"epoch {epoch:2d}  train MSE {total / len(loader.dataset):.
    return model

def predict_dollars(model, X):
    model.eval()
    with torch.inference_mode():
        return to_dollars(model(X).numpy())

model = train_forecaster(model, X, y, ~test_mask)
lstm_pred = predict_dollars(model, X[test_mask])
lstm_actual = to_dollars(y[test_mask].numpy())
lstm_mae = mean_absolute_error(lstm_actual, lstm_pred)

for name, mae in baselines.items():
```

```
    print(f"{name:36s} MAE ${mae:,.0f}")
print(f"{'LSTM (28-day window + weekday)':36s} MAE ${lstm_mae:,.0f}")
```

Output (yours may vary slightly)

```
SalesRNN(
  (rnn): LSTM(8, 32, batch_first=True)
  (head): Linear(in_features=32, out_features=1, bias=True)
)
5409 parameters
epoch  5  train MSE 0.778
epoch 10  train MSE 0.762
epoch 15  train MSE 0.743
naive (yesterday)                      MAE $1,350
seasonal naive (same day last week)  MAE $1,072
LSTM (28-day window + weekday)         MAE $702
```

$702 against $1,072. The flat training MSE is the closed days: a $0 day is 11.7 standard deviations below normal and the model cannot know it is coming. The project gives it that information.

10 **Attention by hand: the binder (3m).** "Keys: the weekday of each past day. Query: tomorrow's weekday. A softmax turns matches into weights."

```
def attend(query, keys, values, sharpness=5.0):
    scores = sharpness * (keys @ query)
    weights = np.exp(scores - scores.max())
    weights = weights / weights.sum()
    return weights @ values, weights

t = int(np.flatnonzero(wide.index == "2025-03-15")[0])        # a Saturday in
boise = wide["Boise"].to_numpy()
forecast, weights = attend(weekday[t], weekday[t - 28:t], boise[t - 28:t])
print(wide.index[t].day_name(), "forecast", round(forecast), " actual", round(
print("weights on the 28 days:", weights.round(3))

attn_pred, attn_actual = [], []
for s, store_name in enumerate(wide.columns):
    values = wide[store_name].to_numpy()
    for t in np.flatnonzero(is_test_day):
        forecast, _ = attend(weekday[t], weekday[t - 28:t], values[t - 28:t])
        attn_pred.append(forecast)
        attn_actual.append(values[t])

print(f"{'hand-built attention (4 same weekdays)':40s} MAE ${mean_absolute_err
print(f"{'seasonal naive (1 same weekday)':40s} MAE ${baselines['seasonal naiv
```

```
Saturday forecast 8497  actual 11099
weights on the 28 days: [0.24  0.002 0.002 0.002 0.002 0.002 0.002 0.24  0.00
 0.002 0.002 0.24  0.002 0.002 0.002 0.002 0.002 0.002 0.24  0.002 0.002
 0.002 0.002 0.002 0.002]
hand-built attention (4 same weekdays)   MAE $979
seasonal naive (1 same weekday)          MAE $1,072
```

Four Saturdays get almost all the weight. Averaging four same weekdays ($979) beats copying one ($1,072) but loses to the LSTM, which also tracks the recent level.

11 **PyTorch's attention equals the formula (1m).** "Transformers stack learned versions of this."

```python
import torch.nn.functional as F

torch.manual_seed(42)
q = torch.randn(2, 5, 16)     # batch 2, 5 steps, 16 numbers per query
k = torch.randn(2, 5, 16)
v = torch.randn(2, 5, 16)

by_hand = torch.softmax(q @ k.transpose(1, 2) / 16 ** 0.5, dim=-1) @ v
built_in = F.scaled_dot_product_attention(q, k, v)
print(tuple(built_in.shape), "same result:", torch.allclose(by_hand, built_in,
```

```
(2, 5, 16) same result: True
```

12 **Three silent mistakes (4m).** No `batch_first`, a random split of windows, an error in scaled units. None raises.

```
time_first = nn.LSTM(input_size=8, hidden_size=32)           # batch_first def
out_wrong, _ = time_first(X[:64])
print(tuple(out_wrong.shape), "-> out[:, -1] has shape", tuple(out_wrong[:, -1

from sklearn.model_selection import train_test_split

toy_X, toy_y = make_windows(np.arange(40), window=4)
rows_train, rows_test = train_test_split(np.arange(len(toy_y)), test_size=0.25
seen = set(toy_X[rows_train].ravel()) | set(toy_y[rows_train])
leaked = sum(set(toy_X[r]) | {toy_y[r]} <= seen for r in rows_test)
print(f"test windows whose inputs and target all appear in training: {leaked}

model.eval()
with torch.inference_mode():
    scaled_pred = model(X[test_mask]).numpy()
print(f"MAE in scaled log units: {mean_absolute_error(y[test_mask].numpy(), sc
print(f"MAE in dollars         : {mean_absolute_error(to_dollars(y[test_mask].
```

```
(64, 28, 32) -> out[:, -1] has shape (64, 32)
test windows whose inputs and target all appear in training: 8 of 9
MAE in scaled log units: 0.222
MAE in dollars         : 702
```

8 of 9 random test windows were fully seen in training; 0.222 looks wonderful and means nothing to a manager. Split by time and report dollars.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Shapes and dtypes of windows.** A 2-D input (`RuntimeError: input.size(-1) must be equal to input_size. Expected 8, got 28`) or float64 windows (`ValueError: RNN input dtype (torch.float64) does not match weight dtype (torch.float32)...`). Coach: "Say the three dimensions: batch, time steps, features."
- **Random splits of windows.** The most dangerous one: no error, a too-good score (demo Step 12: 8 of 9 test windows fully seen). In `debug_sequence.py` the random split flatters Boise's MAE from $1,039 to $826. Coach: "Could you have made this forecast on the day before the target, knowing only the past?"
- **Errors in the wrong units.** 0.222 in scaled log units vs $702. Coach: "Would the store manager understand this number? Can you put it next to the seasonal naive forecast?"

GUIDED LAB

- **Lab A (15m):** `labs/module-08/l10_4-sequences/transactions_gru_starter.py`; reference `labs/module-08/l10_4-sequences/transactions_gru_solution.py`. A new target from the same file: tomorrow's number of transactions (staffing the tills), with a GRU. Learners write the naive

yardsticks and `build_windows`; the model and loop are given. Two trainings of about 15 seconds each.

- **Expected, Lab A:** naive 19.09, seasonal naive 15.50, GRU weekday only 9.15, GRU with promotion and closed flags 7.56 transactions per store-day (2025 means: 61 to 116 per store). **Debrief:** "The calendar known in advance halves the seasonal naive error. Same lesson as the sales project: what you feed the model matters most."
- **Lab B (10m; finish at home):** `labs/module-08/l10_4-sequences/attention_lookup_starter.py`; reference `labs/module-08/l10_4-sequences/attention_lookup_solution.py`. The lesson's attention, with richer keys (weekday + promotion + closed) and a longer binder (364 days), scored separately on ordinary and special days. No training.
- **Expected, Lab B (MAE all / ordinary / special):** lesson attention $979 / $856 / $3,097; calendar keys over 364 days $1,039 / $1,030 / $1,198; routed (28 days for ordinary, 364 for special) $875 / $856 / $1,198. For Black Friday in Boise the query puts 0.226 weight on each of four past sale Fridays. **Debrief:** the keys decide what is found; the memory trades recency for coverage. A trained attention layer learns both.
- **Fast finishers:** `labs/module-08/l10_4-sequences/debug_sequence.py`: one loud bug and three silent ones; fixed it prints $1,039.2 against seasonal naive $1,355.6.

START THE LESSON PROJECT

The lesson project is **a calendar-aware forecaster**: tomorrow's promotion and closed flags added to every step, the model checked per store and on special days, held up against a gradient-boosting model on lag features, and saved. Learners continue in `lesson_10_4` (it reuses the demo's functions). In the room: Steps 1 to 3 (15 minutes).

- Step 1: the calendar. **Done** = `(1096, 9) promo days: 54 closed days: 6`.
- Step 2: windows with 10 features per step. **Done** = `X (4272, 28, 10) test windows: 1460`.
- Step 3: train. **Done** = `calendar-aware LSTM MAE $515` (yours may vary slightly).
- **Goes home:** Step 4 (per store and special days: $838 on special days vs $3,194 for seasonal naive), Step 5 (default boosting on lags: $616 with the same information, $569 with the event type), Step 6 (the holiday chart), Step 7 (save `sales_lstm.pt` with mu and sd as Python floats, reload, summary).

CHECKS FOR UNDERSTANDING

Ask these out loud:

- Output shape of `nn.LSTM(input_size=3, hidden_size=16, batch_first=True)` on (10, 7, 3)? **Answer:** (10, 7, 16).
- Why compare every model with the seasonal naive forecast? **Answer:** it is free and already captures the weekly cycle; a model must beat it to be worth anything.

- What do the LSTM's gates and cell state fix? **Answer:** vanishing gradients over many steps.
- A colleague's random split gives an MAE far below seasonal naive. Explanation? **Answer:** overlapping windows leaked the test days into training.

Workbook self-check, items Q7 to Q12.

Answers: **Q7** `(4, 10, 8) (1, 4, 8)`; **Q8** C) 93; **Q9** A) Tomorrow = the same weekday last week; **Q10** `[0.498 0.003 0.498] 20.1`; **Q11** D) Undo the standardization, then the log (expm1), and compare in dollars; **Q12** B) It runs and trains on nonsense: PyTorch reads (batch, time) as (time, batch). Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: a card with the three shapes (series → windows (n, 28, features) → targets (n,)) and the undo chain (z → × sd + mu → expm1 → dollars). In Lab A, a struggling pair may copy `build_windows` from the demo; the new skills are the yardsticks and reading the result.

Stretch: the lesson's Exercise 4 (a tiny transformer: $514 vs the LSTM's $515 with twice the parameters) and Bonus Challenge (make the boosting comparison fair in both directions with a validation period).

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Sort the notes before Session 9: question 1 tells you whether shapes need another pass before transfer learning, where every layer of ResNet-18 is a shape question.

1. A batch (64, 1, 28, 28) goes through `nn.Conv2d(1, 16, kernel_size=3, padding=1)` and then `nn.MaxPool2d(2)`. Shape after each? **Answer:** (64, 16, 28, 28), then (64, 16, 14, 14).
2. Why must a time series be split by time and not at random? **Answer:** the model must be graded on days after everything it learned from, as in real use; overlapping windows in a random split put each test day's inputs and target into training, so the score is too good to be true.
3. With the same information, the calendar-aware LSTM forecast daily sales with MAE $515 and default gradient boosting on lag features $616 (and $569 given the event type). What do you tell the store managers? **Answer:** both are far better than "same day last week" ($1,072) and in the same league; the LSTM leads here, but boosting was simpler, faster and untuned, and the information fed in mattered more than the architecture. Start with the simpler one, keep testing both.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both projects.** 10.3 Steps 4 to 6 (test 0.892, Shirt 0.632, `fashion_cnn.pt` reloads identically). 10.4 Steps 4 to 7 (per store and special days; boosting $616 / $569 vs LSTM $515; the holiday chart; `sales_lstm.pt`).

- **Lesson exercises.** 10.3: Exercises 1 to 3 (shape detective, where the parameters live, the most confident mistakes); Exercise 4 is a stretch. 10.4: Exercises 1 to 3 (hardest weekday, a GRU, how much history); Exercise 4 is a stretch.
- **Both Quick Quizzes** and **both journal entries** (10.4's asks for all six MAE numbers and one sentence for a store manager).
- **Labs:** finish `shift_test_starter.py` and `attention_lookup_starter.py`.
- **Before the Next Lesson (from 10.4):** keep the notebook and `sales_lstm.pt`; retrain with `torch.manual_seed(7)` and compare the wobble with the gap to seasonal naive; make sure the pretrained weights can download (the Session 9 downloads below).

BRING LIST FOR SESSION 9

- A plugged-in laptop with the course environment (nothing new to install).
- **EuroSAT downloaded into data/eurosat/** (94 MB download, about 186 MB on disk) and **the ResNet-18 (45 MB) and MobileNetV3-small (10 MB) pretrained weights cached** in PyTorch's cache (`.cache/torch/hub/checkpoints` in your home folder). If `check_environment.py` already ends with READY for Advanced Machine Learning (the `--download` run from Session 1), you have all of it. Otherwise do it AT HOME, once, from the `pinecrest-ml` folder with the environment active: `python labs/module-01/l7_0-orientation/check_environment.py --download`, or `python -c "from torchvision import datasets, models; datasets.EuroSAT('data', download=True); models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1); models.mobilenet_v3_small(weights=models.MobileNet_V3_Small_Weights.IMAGENET1K_V1)"`. Venue Wi-Fi will not cope with a room doing this; anyone who cannot download gets the files from the instructor's USB stick at the start of Session 9.
- **sales_lstm.pt kept from the 10.4 project** (with the project notebook), and `fashion_cnn.pt` from 10.3: Lesson 10.5 contrasts training from scratch with transfer learning.
- The `pinecrest-ml` folder with `data/` (Lesson 11.1 uses `reviews.csv`) and the `labs/` folder; Session 9's starters are in `labs/module-09/`.
- `training.py`, the workbook and the journal.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** dense vs CNN on the same loop and seed (10.3 Steps 9 to 12, with the checkpoint fallback); naive forecasts, time split, the LSTM against $1,072 and the silent mistakes (10.4 Steps 4, 5, 9 and 12).
- **Cut first (saves 4m):** 10.3 Step 5 (edge filters on a boot): show the page's three panels.
- **Cut second (saves 3m):** 10.4 Step 6's plain loop (keep `nn.RNN`, say "the same numbers come out of a five-line loop").

- **Cut third (saves 8m):** Lab B of 10.3 (shift test) and Lab B of 10.4 (attention) move to homework; their expected outputs are in the README and the workbook.

RUNNING SHORT: WHAT TO ADD

- 10.3 Exercise 3 live (the most confident mistakes): how often is a 99% confident network wrong, and would a person hesitate on those images?
- 10.4 Exercise 2 (swap in a GRU: $509 vs $515, a tie with a quarter fewer parameters).
- Run the routed attention from Lab B on the projector and ask how a trained attention layer could learn such a routing by itself.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 9: Transfer Learning and Text Classification

How to teach Session 9 of Advanced Machine Learning: borrow a network trained on a million photos and turn it into a satellite land-cover classifier with 2,000 labels on a CPU, then teach a model to read Pinecrest's product reviews. Learners leave knowing that good features come first: pretrained features beat a from-scratch CNN with a twentieth of the labels (0.913 vs 0.719), and a TF-IDF pipeline beats the baseline by 13 points, as long as nobody throws away the word not.

`2 lessons` `3 hrs session` `Lessons 10.5-11.1` `Session 9 of 13`

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame (one mental model) → live demo (you type, they follow) → guided lab (a kit lab, new practice) → start the lesson project. Exercises, quizzes and journals go home.
- Every code block below was run to produce the output printed under it (Python 3.12, scikit-learn 1.9.1, PyTorch 2.14 / torchvision 0.29 on 4 CPU threads). PyTorch training output is labeled "yours may vary slightly" on the lesson pages: on another machine or thread count the last digits can differ; with 4 threads on one machine runs are bit-identical. Timings always differ.
- Keep `deep_learning_transfer.html` and `nlp_text_preprocessing.html` open for the figures (one tile per class, the label curve, the confusion matrix, the 20-word coefficient chart). Labs are in `labs/module-09/`.
- This is the last deep-learning session: 10.5 is the most CPU-hungry lesson of the course. The CPU-time plan below is not optional.

CPU-TIME PLAN FOR 10.5 (READ BEFORE THE SESSION)

- **Measured on the kit machine (4 threads, other jobs running):** scratch CNN 37 s, features 26 s; expect 15 to 40 seconds for each on a laptop. The lesson project's stage 1 (two epochs through the frozen ResNet-18 at 128 px, plus validation) takes about a minute; stage 2 (layer4 unfrozen) costs 15 to 30 seconds per epoch on a laptop. Feature extraction for the lab is cached, so the lab itself runs in about 5 seconds.
- **Before the warm-up (0:00):** open the demo notebook `lesson_10_5` with demo Steps 1 to 5 already typed, and Run All while you start the warm-up. The from-scratch CNN (Step 5) finishes during the warm-up; in the demo you scroll to its output instead of waiting for it.
- **Live:** feature extraction (Step 7, about 15 seconds) stays live on purpose: the room should feel that it is the expensive step and that everything after it is fast.
- **Learners' machines:** nobody trains the from-scratch CNN in the room. The lab loads cached features from `labs/module-09/_fixtures/eurosat_resnet18_feats.npz`; if a learner's copy is missing it, `prepare_eurosat_features.py` rebuilds it in about 30 seconds.
- **The project in the room:** Steps 1 to 3 only (stage 1). Stage 2 fine-tuning goes home.
- **GPU note:** nothing in the course needs a GPU. A learner with CUDA can follow the lesson's "Scaling Up on a GPU" box at home (all 27,000 tiles, 224 pixels, augmentation, layer3 unfrozen); the numbers on the page and in this kit come from the CPU code.

BEFORE THE SESSION

- ☐ **Caches on every machine (the Session 8 bring list asked for this):** EuroSAT in `data/eurosat/` (94 MB download, 186 MB unpacked) and the ResNet-18 (45 MB) and MobileNetV3-small (10 MB) weights in PyTorch's cache. Anyone who has not: run Lesson 10.5's first two code blocks at home BEFORE the session. On venue Wi-Fi, have the files on a USB stick: `EuroSAT.zip` goes into `data/eurosat/`, the two `.pth` files into `~/.cache/torch/hub/checkpoints/` (Windows: `%USERPROFILE%\.cache\torch\hub\checkpoints`).
- ☐ **Kit check:** `python3 labs/check_labs.py module-09` from the kit folder: 8 PASS lines, about 35 seconds. It uses the fixture, the cached EuroSAT and weights, and never downloads.
- ☐ **Fixture:** `labs/module-09/_fixtures/eurosat_resnet18_feats.npz` (2.4 MB) ships with the kit; hand out the `_fixtures/` folder with the starters. If it is ever lost: `python labs/module-09/l10_5-transfer-learning/prepare_eurosat_features.py` (about 30 seconds).
- ☐ **Projector:** two notebooks, `lesson_10_5` (demo Steps 1 to 5 typed in advance, see the CPU-time plan) and `lesson_11_1`. Browser tabs: both lesson pages.
- ☐ **Board before learners arrive:** a two-column box "from scratch | frozen + head | fine-tuned" for the three accuracies, and the TF-IDF formula `idf = ln((1 + n) / (1 + df)) + 1`.

- ☐ **Hand out:** the starters, `_fixtures/`, `debug_transfer.py`, `debug_text_pipeline.py` (no solutions).

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
A download starts in the room (`Downloading https://zenodo.org/...EuroSAT.zip` or `resnet18-f37072fd.pth`)	EuroSAT or the weights are not cached on that machine	Stop it on venue Wi-Fi; copy from the USB stick (see above). Pair the learner meanwhile.
`RuntimeError: ... Missing key(s) in state_dict` when reloading the project model	The rebuilt model's head differs (still 1,000 classes)	Rebuild exactly: `resnet18(weights=None)`, then `fc = nn.Linear(512, 10)`, then `load_state_dict`.
`UserWarning: The parameter 'pretrained' is deprecated`	Old tutorial code	`weights=models.ResNet18_Weights.DEFAULT`.
Feature extraction takes minutes	No `Resize(128)` limit (224 px recipe) or other programs on the CPU	Use the lesson's 128-pixel transform; close other notebooks. Four times the pixels is four times the time.
`ValueError: Found input variables with inconsistent numbers of samples: [1, ...]`	A DataFrame (`df[["text"]]`) passed to a vectorizer	Single brackets: `df["text"]`.
`ValueError: np.nan is an invalid document`	Missing text	Decide what it means; `fillna("")` or drop the rows.

SESSION 9 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Lessons 10.3 and 10.4 (CNN shapes and parameters, eval(), sequence baselines and leakage); homework check. The demo notebook's first five cells run meanwhile (CPU-time plan).
0:10	10	**10.5** Frame and mental model	The experienced photo editor; backbone and head; the four strategies table.
0:20	30	**10.5** Live demo	Weights and their recipe, EuroSAT split and baseline, the from-scratch CNN (pre-run), frozen features and a linear head, the label curve, freezing

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
			and parameter groups, two silent and loud mistakes.
0:50	20	**10.5** Guided lab	`labs/module-09/l10_5-transfer-learning/feature_table_starter.py`; pairs who finish take `labs/module-09/l10_5-transfer-learning/freeze_audit_starter.py`.
1:10	15	**10.5** Start the lesson project	The land-cover classifier, Steps 1 to 3 (stage 1 trains about a minute); Steps 4 to 6 go home.
1:25	10	Break	Leave the stage-1 cell running if it has not finished. Board: backbone / head / frozen / fine-tune.
1:35	10	**11.1** Frame and mental model	The clerk with the tally sheet; text to a fixed-length row; the pipeline picture.
1:45	30	**11.1** Live demo	Read the reviews, frame and baseline, what the vectorizer does, a toy bag of words, the sparse matrix, TF-IDF by hand, the classifier and its words, the stop-word trap, bigrams, two loud mistakes.
2:15	20	**11.1** Guided lab	`labs/module-09/l11_1-text-classification/recommend_flag_starter.py`; fast pairs take `labs/module-09/l11_1-text-classification/category_router_starter.py`.
2:35	15	**11.1** Start the lesson project	Review triage, Steps 1 to 3 (threshold from cross-validated probabilities); Steps 4 to 6 go home.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 10.
Total	180		Ends at 3:00.

In the room. The lessons estimate 110 + 95 = 205 minutes for a solo learner; the room has 150 after the warm-up, the break and the wrap-up. So for each lesson the room does the mental model, the demo, one guided lab and the first steps of the project.

Goes home (2 to 3 hours). Project 10.5 Steps 4 to 6 (stage 2 fine-tuning, test once, save `landcover_resnet18.pt`: about 45 seconds to a few minutes of CPU) and Project 11.1 Steps 4 to 6 (`review_follow_up.csv`); Exercises 1 to 3 of each lesson; both Quick Quizzes; both journal entries; the unfinished lab.

Protect, in this order: 10.5 demo Steps 6 to 9 (cut the head, features once, linear head, label curve: the "0.719 vs 0.913" moment); 11.1 demo Steps 8 to 12 (the classifier, its words, the stop-word trap and bigrams); each lab

through its TODO 3. **Cut first:** 10.5 demo Step 13 (loud mistakes: say them), then 11.1 demo Steps 4 and 13, then the 11.1 project start (send Steps 1 to 3 home with the check numbers).

WARM-UP

0:00 Warm-up: CNNs and Sequences — 10 min

HOW TO RUN IT

- Start the demo notebook's Run All first (CPU-time plan), then put the five questions on the slide. Two minutes alone, then fingers up for the code ones and quick answers for the rest.
- Homework check in one minute: hands up for anyone who saved `sales_lstm.pt` and anyone whose EuroSAT and weights are cached. The second group must be everyone; fix the rest at the break from the USB stick.

QUESTIONS AND ANSWERS

W1

What shape does this print?

```
import torch
from torch import nn
print(nn.Conv2d(1, 8, kernel_size=3)(torch.zeros(4, 1, 28, 28)).shape)
```

`torch.Size([4, 8, 26, 26])`

> **Why:** A 3 × 3 kernel with no padding loses one pixel on each side: (28 + 0 - 3) / 1 + 1 = 26. Batch 4, 8 output channels.

W2

How many parameters? (This is ResNet-18's first layer; Lesson 10.3's Before the Next Lesson asked the same.)

```
from torch import nn
print(sum(p.numel() for p in nn.Conv2d(3, 64, kernel_size=7, bias=False).param
```

`9408`

> **Why:** 64 filters × 3 channels × 7 × 7 = 9,408, and no bias. Today we borrow those 9,408 numbers, and 11 million more, ready-trained.

W3

What does this print?

```
import torch
from torch import nn
lstm = nn.LSTM(input_size=2, hidden_size=8, batch_first=True)
out, (h, c) = lstm(torch.zeros(5, 14, 2))
print(tuple(out.shape), tuple(h.shape))
```

`(5, 14, 8) (1, 5, 8)`

> **Why:** batch_first=True: out is (batch, time steps, hidden) = (5, 14, 8); h is (layers, batch, hidden) = (1, 5, 8), the last step only.

W4

Why must `model.eval()` come before measuring test accuracy of a model with dropout and batch norm?

So dropout stops zeroing activations and batch norm uses its stored training statistics (not the test batch's).

> **Why:** This comes straight back in 10.5: features extracted in training mode are silently wrong.

W5

A colleague splits overlapping sales windows with `train_test_split(..., random_state=42)` and beats the seasonal naive forecast by a mile. Your first suspicion?

Leakage: neighboring windows share days, so test days were in training. Split by time.

> **Why:** Same habit as today's text pipeline: anything learned from data (windows, vocabulary, IDF) must come from the training part only.

LESSON PLANS

10.5 Transfer Learning

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain why features learned on ImageNet transfer to a new image task, and when they will not
- Load a pretrained torchvision model with the `weights=` API and prepare images the way it expects
- Use a frozen backbone as a feature extractor and train a small head on its features
- Fine-tune the last block with a lower learning rate, and compare the result with a CNN trained from scratch
- Measure how many labeled images a model needs before you pay for labeling more

TIMING IN THE ROOM

10m	Frame: the experienced photo editor; backbone and head; the four strategies
30m	Live demo: recipe, data, from scratch (pre-run), frozen features, linear head, label curve, freezing, parameter groups, mistakes
20m	Guided lab: `feature_table_starter.py`, then `freeze_audit_starter.py`
15m	Start the project: the land-cover classifier, Steps 1 to 3

MENTAL MODEL TO SAY

"Imagine hiring someone to sort aerial photos into ten land types. A newcomer needs thousands of examples just to learn what a texture or a straight edge is. An experienced photo editor already sees textures, edges and color gradients without thinking; shown a few dozen examples per land type, they learn which combinations mean forest. The pretrained network is that editor. You only teach it the last step."

Draw it: tile → **backbone** (all the convolutions, ImageNet-trained) → 512 features → **head** (new, your 10 classes). Then the four strategies from the lesson's table: feature extraction (frozen backbone, train only the head), fine-tuning (head plus the top block, small learning rate), full fine-tuning (everything, needs data and a GPU), from scratch (huge data, or inputs nothing like photos). "Today: the first two, on a CPU, honestly compared with the fourth."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Notebook `lesson_10_5`. Steps 1 to 5 were typed before the session and ran during the warm-up; narrate them quickly. Type from Step 6 on.

1. **The weights know their recipe (2m).** "Start from the weights object, not the model: it knows how its images must be prepared." Say why `weights=` and not the old `pretrained=True` (deprecated

since torchvision 0.13).

```
import time
import torch
from torch import nn
from torch.utils.data import DataLoader, Subset, TensorDataset
from torchvision import datasets, models, transforms

weights = models.ResNet18_Weights.DEFAULT
prep = weights.transforms()
print(weights)
print(prep.mean, prep.std)
```

```
ResNet18_Weights.IMAGENET1K_V1
[0.485, 0.456, 0.406] [0.229, 0.224, 0.225]
```

The two lists are the ImageNet channel means and standard deviations. Every image we feed this network must be shifted and scaled by exactly these numbers.

2 **Look at the data (2m).** "27,000 satellite tiles, 64 × 64 pixels, ten land types." Show the lesson page's one-tile-per-class figure instead of plotting.

```
raw = datasets.EuroSAT("data", download=True)
print(len(raw), raw[0][0].size)
print(raw.classes)
```

```
27000 (64, 64)
['AnnualCrop', 'Forest', 'HerbaceousVegetation', 'Highway', 'Industrial', 'Pas
```

3 **A seeded split and the do-nothing score (2m).** "No official split, so we make a seeded one: 2,000 labeled tiles to learn from, 1,000 to grade. That is a realistic 'we labeled a couple of thousand images'."

```
order = torch.randperm(len(raw), generator=torch.Generator().manual_seed(42))
train_idx, test_idx = order[:2000].tolist(), order[2000:3000].tolist()
targets = torch.tensor(raw.targets)
y_train, y_test = targets[train_idx], targets[test_idx]
print(torch.bincount(y_train))
print(f"majority-class baseline accuracy: {(y_test == y_train.mode().values).f
```

```
tensor([208, 226, 233, 189, 188, 152, 183, 212, 183, 226])
majority-class baseline accuracy: 0.106
```

Roughly balanced classes, baseline 0.106: accuracy is a fair headline metric here (unlike fraud).

4 **Prepare tiles the way the network expects (2m).** "Two rules: normalize with the network's own statistics, and keep objects at a sensible size. 128 pixels is our CPU compromise."

```
eurosat_tf = transforms.Compose([
    transforms.Resize(128),
    transforms.ToTensor(),
    transforms.Normalize(mean=prep.mean, std=prep.std),
])
eurosat = datasets.EuroSAT("data", download=True, transform=eurosat_tf)
train_set, test_set = Subset(eurosat, train_idx), Subset(eurosat, test_idx)
x0, label0 = train_set[0]
print(tuple(x0.shape), eurosat.classes[label0])
print(f"channel means after normalizing: {x0.mean(dim=(1, 2)).numpy().round(2)
```

```
(3, 128, 128) Forest
channel means after normalizing: [-1.01 -0.64  0.11]
```

The means are not zero: satellite tiles are greener and darker than ImageNet photos. That is fine: the statistics must be the network's, not the dataset's (lesson quiz Q5).

5 **The from-scratch CNN (already run) (2m).** Do not type this one: it ran during the warm-up (CPU-time plan). Scroll to its output and say: "This is Lesson 10.3's recipe with one more block, trained 10 epochs on the same 2,000 tiles."

```
start = time.perf_counter()
plain_tf = transforms.Compose([transforms.ToTensor(), transforms.Normalize(mea
eurosat64 = datasets.EuroSAT("data", download=True, transform=plain_tf)
X64_train = torch.stack([eurosat64[i][0] for i in train_idx])
X64_test = torch.stack([eurosat64[i][0] for i in test_idx])

torch.manual_seed(42)
scratch = nn.Sequential(
    nn.Conv2d(3, 16, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
    nn.Conv2d(16, 32, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
    nn.Conv2d(32, 64, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
    nn.Flatten(), nn.Linear(64 * 8 * 8, 10),
)
torch.manual_seed(42)
loader = DataLoader(TensorDataset(X64_train, y_train), batch_size=64, shuffle=
optimizer = torch.optim.Adam(scratch.parameters(), lr=1e-3)
for epoch in range(1, 11):
    scratch.train()
    for xb, yb in loader:
        optimizer.zero_grad()
        nn.functional.cross_entropy(scratch(xb), yb).backward()
        optimizer.step()
scratch.eval()
with torch.no_grad():
    scratch_acc = (scratch(X64_test).argmax(dim=1) == y_test).float().mean().i
print(f"from-scratch CNN test accuracy: {scratch_acc:.3f}")
SCRATCH_SECONDS = time.perf_counter() - start
```

```
from-scratch CNN test accuracy: 0.719
```

Write 0.719 on the board under "from scratch". The training loss was still falling after 10 epochs (lesson page): more epochs would help a little.

6 **Cut off the head (3m).** "fc maps 512 features to 1,000 ImageNet classes. Replace it with a layer that does nothing and the network outputs its features." Insist on `eval()` out loud: "even though nothing trains".

```
backbone = models.resnet18(weights=weights)
print(backbone.fc)
print(f"parameters: {sum(p.numel() for p in backbone.parameters()):,}")
backbone.fc = nn.Identity()
backbone.eval()
print(tuple(backbone(torch.zeros(2, 3, 128, 128)).shape))
```

```
Linear(in_features=512, out_features=1000, bias=True)
parameters: 11,689,512
(2, 512)
```

7 **Features once, under inference_mode (3m).** "The backbone is frozen, so a tile's features never change: compute them once. This is the expensive cell: watch the clock." (About 15 seconds here.)

```
def extract(model, dataset, batch_size=100):
    feats = []
    with torch.inference_mode():
        for xb, _ in DataLoader(dataset, batch_size=batch_size):
            feats.append(model(xb))
    return torch.cat(feats)

start = time.perf_counter()
F_train = extract(backbone, train_set)
F_test = extract(backbone, test_set)
EXTRACT_SECONDS = time.perf_counter() - start
print(tuple(F_train.shape), tuple(F_test.shape))
```

```
(2000, 512) (1000, 512)
```

From now on 3,000 tiles are a table of 512 numbers each. Everything from Modules 1 to 9 applies.

8 **A linear head on the table (3m).** "One nn.Linear(512, 10): 5,130 numbers to learn, a second or two of training."

```
torch.manual_seed(42)
head = nn.Linear(512, 10)
optimizer = torch.optim.Adam(head.parameters(), lr=1e-3)
loader = DataLoader(TensorDataset(F_train, y_train), batch_size=64, shuffle=Tr
for epoch in range(1, 31):
    for fb, yb in loader:
        optimizer.zero_grad()
        nn.functional.cross_entropy(head(fb), yb).backward()
        optimizer.step()
with torch.no_grad():
    head_acc = (head(F_test).argmax(dim=1) == y_test).float().mean().item()
print(f"frozen ResNet-18 + linear head, test accuracy: {head_acc:.3f}")
```

```
frozen ResNet-18 + linear head, test accuracy: 0.913
```

Board: 0.719 from scratch, 0.913 frozen + linear head. Ask: "Nothing in ResNet-18 ever saw a satellite. Why does this work?" (textures, edges, colors, regular patterns are general).

9 **How many labels do you need? (4m).** "Labeling costs someone's time. With cached features each point of this curve takes a fraction of a second." Ask for predictions for 100 tiles before running.

```
from sklearn.linear_model import LogisticRegression

for n in [50, 100, 250, 500, 1000, 2000]:
    clf = LogisticRegression(max_iter=2000).fit(F_train[:n].numpy(), y_train[:
    print(f"{n:5d} labeled tiles: test accuracy {clf.score(F_test.numpy(), y_t
```

```
   50 labeled tiles: test accuracy 0.632
  100 labeled tiles: test accuracy 0.755
  250 labeled tiles: test accuracy 0.839
  500 labeled tiles: test accuracy 0.885
 1000 labeled tiles: test accuracy 0.895
 2000 labeled tiles: test accuracy 0.902
```

Frozen features with 100 labels beat the CNN with 2,000. After about 500 the gains shrink; a 1,000-tile test set has a margin of about ±2 points, so read the top of the curve loosely.

10 **Freeze, new head, unfreeze layer4 (3m).** "Freezing is one attribute per tensor. Order matters: freeze first, THEN attach the new head, because new layers are trainable by default."

```
def count_trainable(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

torch.manual_seed(42)
ft_model = models.resnet18(weights=weights)
for p in ft_model.parameters():
    p.requires_grad = False
ft_model.fc = nn.Linear(ft_model.fc.in_features, 10)      # new layers are tra
print(f"head only      : {count_trainable(ft_model):,} trainable")
for p in ft_model.layer4.parameters():
    p.requires_grad = True
print(f"head + layer4  : {count_trainable(ft_model):,} trainable")
```

```
head only      : 5,130 trainable
head + layer4  : 8,398,858 trainable
```

11 **Parameter groups (2m).** "Pretrained layers get a ten times smaller learning rate: they only need nudging."

```
optimizer = torch.optim.Adam([
    {"params": ft_model.layer4.parameters(), "lr": 1e-4},
    {"params": ft_model.fc.parameters(), "lr": 1e-3},
])
for group in optimizer.param_groups:
    print(group["lr"], sum(p.numel() for p in group["params"]))
```

```
0.0001 8393728
0.001 5130
```

Stage 2 of the project uses exactly this optimizer. One epoch through layer4 costs 15 to 30 seconds on a laptop CPU: that is why it goes home.

12 **A silent mistake: features in training mode (2m).** "No error. Watch."

```
probe = models.resnet18(weights=weights)
probe.fc = nn.Identity()
tiles = torch.stack([train_set[i][0] for i in range(8)])
before = probe.bn1.running_mean.clone()
probe.train()
with torch.no_grad():
    in_batch_of_8 = probe(tiles)[0]
    in_batch_of_2 = probe(tiles[:2])[0]
print("same tile, same features?", torch.allclose(in_batch_of_8, in_batch_of_2
print("stored ImageNet statistics changed?", not torch.equal(before, probe.bn1
```

```
same tile, same features? False
stored ImageNet statistics changed? True
```

Same tile, different features depending on its batch, and the stored statistics overwritten even under no_grad. Fix: `model.eval()`.

13 **Two loud mistakes (2m).** "Freeze after attaching the head, and a grayscale image. Read the last line."

```
frozen_all = models.resnet18(weights=weights)
frozen_all.fc = nn.Linear(512, 10)
for p in frozen_all.parameters():          # runs AFTER the new head was attac
    p.requires_grad = False
loss = nn.functional.cross_entropy(frozen_all(torch.zeros(2, 3, 128, 128)), to
loss.backward()

backbone(torch.zeros(1, 1, 128, 128))
```

```
RuntimeError: element 0 of tensors does not require grad and does not have a g
RuntimeError: Given groups=1, weight of size [64, 3, 7, 7], expected input[1,
```

Fixes: freeze first, check `count_trainable` before training; repeat the gray channel three times.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Normalizing with the wrong statistics, or not at all.** No error: the lesson measured 0.840 instead of 0.894 for unnormalized features (Mistake 4). Coach: "Where did your mean and std come from? Print one batch's channel means." The `debug_transfer.py` handout makes them find it.
- **Freezing in the wrong order, or an optimizer that never got the head.** Loud (`element 0 of tensors does not require grad`) or silent (a loss that never moves). Coach: "Print `count_trainable(model)` before you train. Is the head in the optimizer?" Lab B builds exactly this audit.
- **Forgetting `eval()` for features or prediction.** Silent: features depend on the batch, stored statistics drift. Coach: "Which mode is your model in right now?" (`model.training`).

GUIDED LAB

- **Lab A (everyone, 12m):** `labs/module-09/l10_5-transfer-learning/feature_table_starter.py`; reference `labs/module-09/l10_5-transfer-learning/feature_table_solution.py`. Frozen ResNet-18 features of 3,000 NEW tiles (positions 3,500 to 6,499 of the seeded order, cached in `_fixtures/`) treated as an ordinary table: baseline, a cross-validated head, k-nearest neighbors, the weakest classes, a "send to a human" queue.
- **Expected, Lab A:** baseline (SeaLake) 0.126; CV accuracy 0.899 ± 0.011 plain, 0.896 ± 0.009 with a scaler (no gain); test 0.903; kNN 0.850 / 0.866 / 0.861 (k = 1 / 5 / 15); weakest River 0.759, Highway 0.835, AnnualCrop 0.86; most common mix-up River → Highway (11); below 0.6 confidence 3.8% of tiles (accuracy 0.474 inside the queue, 0.920 outside); below 0.8, 10.5% (0.571 vs 0.942).
- **Debrief (2m):** "Deep learning made the features; classic machine learning took it from there." The human queue is the business answer: send a tenth of the tiles to a person and the rest are 94% right. River ↔ Highway is the same confusion the lesson's project found (long thin lines).
- **Lab B (pairs who finish, 8m):** `labs/module-09/l10_5-transfer-learning/freeze_audit_starter.py`; reference `labs/module-09/l10_5-transfer-learning/freeze_audit_solution.py`. Audit a fine-tuning setup on ONE batch of 16 tiles: trainable counts for three strategies, an optimizer that forgot the head, and which tensors change after one step in eval() and train() mode. About 8 seconds.
- **Expected, Lab B:** 5,130 trainable, 0 when the head is attached before freezing; head + layer4 8,398,858 (75.11%), + layer3 10,498,570 (93.89%); the layer4-only optimizer misses `fc.weight` and `fc.bias`; after one step in eval() mode layer4 and fc changed, conv1 and `bn1.running_mean` did not; in train() mode `bn1.running_mean` changed too (frozen weights, moving statistics).

- **Fast finishers:** `labs/module-09/l10_5-transfer-learning/debug_transfer.py`: two loud bugs (`classifier` instead of `fc` → a 1,000-wide feature; the head frozen too) and two silent ones (no Normalize, no `eval()`). Fixed: 0.895 on 200 test tiles.

START THE LESSON PROJECT

The **land-cover classifier**: pretrained ResNet-18, new head, trained in two stages, tested once, saved. In the room: Steps 1 to 3 (15 minutes, about a minute of it CPU).

- Step 1 together: the same seeded order; 2,000 / 500 / 1,000. **Check:** `2000 500 1000`.
- Step 2: freeze, new head. **Check:** `trainable parameters: 5,130`, validation accuracy before training 0.164 (near chance, as it should be).
- Step 3: stage 1, two epochs with the whole model in eval() mode. **Check (yours may vary slightly):** validation 0.824 then 0.876. Start it before the break if the clock is tight; it can finish during the break.
- **Goes home:** Step 4 (unfreeze layer4, parameter groups, keep stage 2 only if validation improves: 0.924), Step 5 (test once: 0.928, weakest Highway 0.808), Step 6 (save and reload `landcover_resnet18.pt` with `weights_only=True`). Keep the file: Lesson 12.1 is about saving models like it.

CHECKS FOR UNDERSTANDING

- Why can we compute the features once? **Answer:** the backbone is frozen and in eval() mode, so each tile's features never change.
- What does `backbone.fc = nn.Identity()` do? **Answer:** removes the ImageNet head, so the network outputs its 512 features.
- From the label curve, at about how many labels did frozen features pass the from-scratch CNN? **Answer:** by 100 tiles (0.755 vs 0.719 with 2,000).
- Why a smaller learning rate for layer4? **Answer:** its weights are already good; big steps could destroy what we borrowed.

Workbook self-check, items Q1 to Q6.

Answers: **Q1** A) Frozen weights never change, so each tile's features are the same in every epoch; **Q2** `1731`; **Q3** C) Probably not: the curve has flattened (1,000 to 2,000 added under a point); fine-tuning is the better next step; **Q4** B) Batch normalization uses each batch's statistics, so a tile's features depend on its batch, and the stored ImageNet statistics are overwritten; **Q5** D) `RuntimeError`: the first convolution expects 3 channels and the image has 1; **Q6** `Linear(in_features=512, out_features=1000, bias=True)`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: give the board's three-column box to anyone lost and have them fill it during the demo. In Lab A, let pairs copy the CV line from Module 1 notes: the point is that features are a table.

Stretch: Lab A's PCA stretch (10 / 50 / 200 components: 0.808 / 0.870 / 0.896); then the lesson's Exercise 2 (MobileNetV3-small: 0.913 with 576 features) and the Bonus Challenge (Fashion-MNIST through ResNet, predict first).

11.1 Text Classification

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Clean and tokenize text with Python's `re` module, and explain what scikit-learn's vectorizers do to a sentence
- Build bag-of-words and TF-IDF feature matrices, and read the sparse matrix and vocabulary they produce
- Train a leak-free sentiment classifier with a `Pipeline` of `TfidfVectorizer` and `LogisticRegression`, and compare it with a baseline
- Explain the stop-word trap, and use bigrams so that "not waterproof" and "not bad" become features of their own
- Describe what word embeddings add over TF-IDF, and when the simpler model is the right choice

TIMING IN THE ROOM

10m	Frame: the clerk with the tally sheet; a document becomes a fixed-length row
30m	Live demo: reviews, baseline, analyzer, toy bag of words, sparse matrix, TF-IDF, the classifier and its words, stop words, bigrams, mistakes
20m	Guided lab: `recommend_flag_starter.py`, then `category_router_starter.py`
15m	Start the project: review triage, Steps 1 to 3

MENTAL MODEL TO SAY

"Hand each review to a clerk with a long tally sheet listing every word customers have ever used. For every word in the review, one tick. No grammar, no word order: only which words, how often. That is a bag of words." Then: "It sounds too crude, yet a tally of love, leaked, returned and highly recommend says a lot. Where it fails is just as instructive: not bad and bad leave almost the same ticks. By the end of this block you will know how to keep the not."

Draw the pipeline: raw text → lowercase and tokenize → vocabulary and IDF (**fit on train**) → sparse TF-IDF matrix → logistic regression → P(positive). "After the second arrow it is the scikit-learn you already know."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Notebook `lesson_11_1`. Everything here runs in seconds: no CPU plan needed. Ask for a prediction before Steps 7, 10 and 12.

1. **Load and count (2m).** "4,000 reviews, a 1-to-5 star rating each."

```
import pandas as pd
reviews = pd.read_csv("data/reviews.csv")
print(reviews.shape)
print(reviews["rating"].value_counts().sort_index().to_dict())
```

```
(4000, 8)
{1: 230, 2: 558, 3: 905, 4: 1093, 5: 1214}
```

2 **Read before you model (2m).** "There is no better way to learn what the model is up against."

```
for _, row in reviews.sample(3, random_state=42).iterrows():
    print(row["rating"], "|", row["text"])
```

```
3 | really happy with this. it the zipper failed after a week. it arrived quic
5 | Love it. This is my second one. It arrived quickly. It still looks new aft
5 | Exactly what I needed. This trekking poles is easy to set up.
```

The first one: lowercase, a slip ("it the zipper"), opens happy and ends with rain getting in. A 3-star review in every sense.

3 **Frame it and measure doing nothing (3m).** "4 or 5 stars positive, 1 or 2 negative, 3-star reviews set aside (the project scores them). X is a Series of strings, not a table: vectorizers want an iterable of documents."

```
from sklearn.model_selection import train_test_split
from sklearn.dummy import DummyClassifier

labeled = reviews[reviews["rating"] != 3].copy()
labeled["positive"] = (labeled["rating"] >= 4).astype(int)
X_train, X_test, y_train, y_test = train_test_split(
    labeled["text"], labeled["positive"], test_size=0.25, stratify=labeled["po
)
print(X_train.shape, X_test.shape)
baseline = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)
print(f"baseline accuracy: {baseline.score(X_test, y_test):.3f}")
```

```
(2321,) (774,)
baseline accuracy: 0.745
```

Board: 0.745.

4 **What does the vectorizer do to a sentence? (2m).** "Ask it: build_analyzer."

```
from sklearn.feature_extraction.text import CountVectorizer
sample = "Really happy with this!! It is NOT waterproof at all... 2-person ten
analyzer = CountVectorizer().build_analyzer()
print(analyzer(sample))
print(analyzer("I think a tent is a tent"))
```

```
['really', 'happy', 'with', 'this', 'it', 'is', 'not', 'waterproof', 'at', 'a
['think', 'tent', 'is', 'tent']
```

Digits kept, one-letter words dropped, 2-person becomes person. If single letters matter (sizes M, L) you pass your own token_pattern.

5 **The tally sheet, on three toy reviews (3m).** "One column per word, one tick per occurrence. Look at rows 0 and 1."

```
toy = ["The tent is warm", "The tent is not warm", "Not bad, warm and light"]
counter = CountVectorizer()
counts = counter.fit_transform(toy)
print(pd.DataFrame(counts.toarray(), columns=counter.get_feature_names_out()))
```

```
   and  bad  is  light  not  tent  the  warm
0    0    0   1      0    0     1    1     1
1    0    0   1      0    1     1    1     1
2    1    1   0      1    1     0    0     1
```

The meaning flipped, the numbers moved by a single 1 in the not column. Keep that in mind.

6 **The real matrix is sparse (2m).** "Fit on the training text only: the vocabulary is learned from data, like a scaler's mean."

```
bow = CountVectorizer().fit(X_train)
X_train_counts = bow.transform(X_train)
n_rows, n_cols = X_train_counts.shape
print(type(X_train_counts).__name__, X_train_counts.shape)
print(f"stored values: {X_train_counts.nnz:,} of {n_rows * n_cols:,} cells ({X
```

```
csr_matrix (2321, 333)
stored values: 40,981 of 772,893 cells (5.3%)
```

333 words: these reviews are repetitive (real ones reach tens of thousands). 5.3% filled: never call .toarray() on a real one.

7 **TF-IDF by hand (3m).** "Words everywhere say little; rare words say a lot." Write idf = ln((1 + n) / (1 + df)) + 1 on the board.

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf_toy = TfidfVectorizer().fit(toy)
print(pd.Series(tfidf_toy.idf_, index=tfidf_toy.get_feature_names_out()).round
print("idf of 'tent' by hand:", round(np.log((1 + 3) / (1 + 2)) + 1, 3))
```

```
{'and': 1.693, 'bad': 1.693, 'is': 1.288, 'light': 1.693, 'not': 1.288, 'tent
idf of 'tent' by hand: 1.288
```

warm is in all three documents: exactly 1, the smallest value. tent is in two: 1.288, matching the hand calculation.

8 **The whole model (3m).** "Vectorizer then logistic regression, in one pipeline, so the vocabulary is learned inside fit." Read the negative row aloud.

```
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

sentiment = make_pipeline(TfidfVectorizer(), LogisticRegression())
sentiment.fit(X_train, y_train)
pred = sentiment.predict(X_test)
print(classification_report(y_test, pred, target_names=["negative", "positive"
```

```
              precision    recall  f1-score   support

    negative      0.823     0.660     0.732       197
    positive      0.891     0.951     0.920       577

    accuracy                          0.877       774
   macro avg      0.857     0.806     0.826       774
weighted avg      0.874     0.877     0.873       774
```

0.877 against 0.745. For the care team the negative row matters: recall 0.660 means a third of unhappy customers are missed at the 0.5 cut-off. The project fixes that with a threshold.

9 **What did it learn? (2m).** "Every coefficient belongs to a word: this model explains itself."

```
vectorizer = sentiment.named_steps["tfidfvectorizer"]
classifier = sentiment.named_steps["logisticregression"]
word_weights = pd.Series(classifier.coef_[0], index=vectorizer.get_feature_nam
print("most negative:", word_weights.head(5).round(2).to_dict())
print("most positive:", word_weights.tail(5).round(2).to_dict())
print("bad:", round(word_weights["bad"], 2), " problem:", round(word_weights["
```

```
most negative: {'not': -4.75, 'regret': -3.16, 'returned': -2.35, 'very': -2.1
most positive: {'great': 2.2, 'all': 2.27, 'highly': 2.29, 'really': 2.37, 'lo
bad: 1.97  problem: 1.66
```

Ask: "Why would bad and problem pull toward POSITIVE?" (Not bad at all; Never had a problem.) Single words lose their phrase.

10 **The stop-word trap (2m).** "Many tutorials remove stop words by default. Read the list first."

```
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS
print([w for w in ["not", "no", "never", "cannot", "very", "but"] if w in ENGL
stopped = make_pipeline(TfidfVectorizer(stop_words="english"), LogisticRegress
print(f"stop words removed {stopped.score(X_test, y_test):.3f}   all words kep
```

```
['not', 'no', 'never', 'cannot', 'very', 'but']
stop words removed 0.864   all words kept 0.877
```

11 **Probe with sentences (2m).** "Accuracy is a blunt summary. Ask the model about sentences you understand."

```
probes = ["It is not waterproof at all.", "Not bad at all.", "Would not buy ag
for text in probes:
    p_all = sentiment.predict_proba([text])[0, 1]
    p_stop = stopped.predict_proba([text])[0, 1]
    print(f"{text:30s} all words {p_all:.2f}   stop words removed {p_stop:.2f}
```

```
It is not waterproof at all.   all words 0.56   stop words removed 0.41
Not bad at all.                all words 0.85   stop words removed 0.94
Would not buy again.           all words 0.21   stop words removed 0.51
It is waterproof.              all words 0.59   stop words removed 0.41
```

Would not buy again. becomes the single word buy: 0.21 turns into a coin flip.

12 **Bigrams keep a little word order (3m).** "Pairs of words become columns: not waterproof, not bad."

```
bigram = make_pipeline(TfidfVectorizer(ngram_range=(1, 2), min_df=2), Logistic
bigram.fit(X_train, y_train)
print(f"unigrams + bigrams accuracy: {bigram.score(X_test, y_test):.3f}")
bi_weights = pd.Series(bigram.named_steps["logisticregression"].coef_[0],
                       index=bigram.named_steps["tfidfvectorizer"].get_feature
print(bi_weights[["waterproof", "not waterproof", "bad", "not bad"]].round(2).
for text in probes:
    print(f"{text:30s} P(positive) {bigram.predict_proba([text])[0, 1]:.2f}")
```

```
unigrams + bigrams accuracy: 0.877
{'waterproof': -0.46, 'not waterproof': -1.04, 'bad': 0.74, 'not bad': 1.46}
It is not waterproof at all.   P(positive) 0.47
Not bad at all.                P(positive) 0.94
Would not buy again.           P(positive) 0.11
It is waterproof.              P(positive) 0.73
```

Same accuracy, a much better model: not waterproof takes the blame, not bad gets its own positive weight, Would not buy again. is clearly negative. One probe stays near the middle (0.47): two-word windows cannot connect not waterproof with at all.

13 **Two loud mistakes (1m).** "A DataFrame instead of a column, and a second vectorizer for the test text."

```
frame_model = make_pipeline(TfidfVectorizer(), LogisticRegression())
frame_model.fit(labeled[["text"]], labeled["positive"])

train_vec = TfidfVectorizer().fit(X_train)
clf = LogisticRegression().fit(train_vec.transform(X_train), y_train)
clf.predict(TfidfVectorizer().fit_transform(X_test))
```

```
ValueError: Found input variables with inconsistent numbers of samples: [1, 3(
ValueError: X has 331 features, but LogisticRegression is expecting 333 featur
```

Single brackets for text (the opposite of numeric X). And transform the test text with the TRAINING vectorizer, or use a pipeline: if the sizes had matched you would get confident nonsense instead of an error.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **A DataFrame where a column of text belongs.** `ValueError: Found input variables with inconsistent numbers of samples: [1, 3095]`. Coach: "How many documents does the vectorizer think it got?" (one: the column name). Single brackets.
- **A second vectorizer for the test text, or one fitted before the split.** Loud only by luck: `ValueError: X has 331 features, but LogisticRegression is expecting 333 features as input.`.

Coach: "Which vocabulary does column 57 belong to?" Use the pipeline.

- **Trusting accuracy and a default stop-word list.** Silent: 0.864 instead of 0.877, and Would not buy again. becomes a coin flip. Coach: "Read the list. Probe with three sentences you understand." Missing text is the fourth one (`np.nan is an invalid document`): fill or drop.

GUIDED LAB

- **Lab A (everyone, 12m):** `labs/module-09/l11_1-text-classification/recommend_flag_starter.py`; reference `labs/module-09/l11_1-text-classification/recommend_flag_solution.py`. A new target: the review form's "would recommend" checkbox, predicted from text, on ALL 4,000 reviews including the mixed 3-star ones. Three vectorizer settings cross-validated, one test, the words, accuracy by star rating, probes.
- **Expected, Lab A:** baseline 0.655; CV accuracy 0.726 unigrams, 0.719 stop words removed, 0.732 bigrams; test 0.755, not-recommended recall 0.481 / precision 0.716; strongest terms not (-2.81) and highly recommend (1.58); accuracy by stars 1: 0.794, 2: 0.543, 3: 0.464, 4: 0.845, 5: 0.963; It leaked on the first night. gets P(recommended) 0.66.
- **Debrief (2m):** an honest negative result. The checkbox is far noisier than the stars (45% of 3-star reviewers still recommend), so the same pipeline reaches 0.755 instead of 0.877, and it is weakest exactly on the mixed reviews. The leaked probe: leaked appears in mixed reviews that are still recommended. "A harder target is not a worse model; it is a different question."
- **Lab B (fast pairs, 8m):** `labs/module-09/l11_1-text-classification/category_router_starter.py`; reference `labs/module-09/l11_1-text-classification/category_router_solution.py`. Ten classes: route a message to the right product team. Expected: baseline 0.237; accuracy 0.615, macro F1 0.585; lowest recall Tents 0.333, Camp Kitchen 0.377, Footwear 0.408; Paddling → Apparel 63 times; all three probe messages land in Apparel or Accessories (≤ 0.31); routing only above 0.5 confidence sends 42.7% automatically at 0.932 accuracy. Stretch: `class_weight="balanced"` lifts Tents recall to 0.452 but drops accuracy to 0.580.
- **Fast finishers:** `labs/module-09/l11_1-text-classification/debug_text_pipeline.py`: a DataFrame and missing text (loud), stop words and a threshold chosen on the test set (silent). Fixed: 0.877, threshold 0.2, P(negative) `[0.05 0.3 0.13]`, and a refund email the model barely notices.

START THE LESSON PROJECT

Review triage: a daily list of reviews to follow up, with a threshold chosen without the test set. In the room: Steps 1 to 3.

- Step 1: whole rows in the split. **Check:** `2321 train, 774 test, 905 three-star reviews held aside`.
- Step 2: the bigram pipeline, P(negative) = 1 - P(positive). **Check:** 0.877 at the 0.5 cut-off.
- Step 3: cross-validated probabilities on the training reviews, thresholds 0.5 to 0.2. **Check:** recall 0.570 /

0.756 / 0.851 / 0.926; 0.3 is the highest threshold reaching 85%.

- **Goes home:** Step 4 (grade once: recall 0.858, precision 0.698), Step 5 (read the worst mistakes: titles hold the complaint), Step 6 (`review_follow_up.csv` , 698 reviews; 456 of 905 three-star reviews flagged).

CHECKS FOR UNDERSTANDING

- Why is X a Series and not a one-column DataFrame here? **Answer:** a vectorizer iterates over documents; iterating a DataFrame gives column names.
- What does IDF do to a word that appears in every review? **Answer:** gives it the smallest weight, exactly 1.
- Why did bad get a positive coefficient? **Answer:** it appears mostly in Not bad at all; single words lose their phrase.
- How do you choose the triage threshold without leaking? **Answer:** cross_val_predict probabilities on the training reviews; then grade once on test.

Workbook self-check, items Q7 to Q12.

Answers: **Q7** `['am', 'person', 'tent', 'fan']` ; **Q8** C) The vocabulary and IDF values are learned from data, so they must come from the training text only (also inside every CV fold); **Q9** A) `['buy']` ; **Q10** D) Almost every cell is zero, so storing only the non-zero values saves memory, which matters with real vocabularies; **Q11** `1.511` ; **Q12** B) Catches more unhappy customers (recall up) at the cost of more false alarms (precision down). Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: pairs who struggle with the sparse-matrix idea print `counts.toarray()` for the toy only, and say aloud what each column is. Keep the pipeline picture on the board.

Stretch: Exercise 3 (character n-grams for typos: similarity 0.694 vs 0.200) and Exercise 4 (tuning the pipeline: best CV 0.892, test 0.881); the Bonus Challenge (title vs text) raises a fair-feature question worth a journal entry.

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

1. Name the backbone and the head of a pretrained ResNet-18, and say which one you replace and which one you freeze for feature extraction. **Answer:** backbone = the convolutional layers (frozen, eval mode); head = `fc` (replaced by nn.Identity for features or a new nn.Linear for your classes).
2. Give one sentence a unigram bag of words gets wrong and the change that fixes it. **Answer:** e.g. Would not buy again. or Not bad at all.; bigrams (`ngram_range=(1, 2)`) give not buy / not bad their own columns. (Character n-grams for typos, embeddings for synonyms also accepted with a matching example.)

3. Name one thing that must be learned from the training data only, in each lesson. **Answer:** 10.5: the head's weights and anything chosen on validation (stage 1 vs 2), never the test set (the normalization statistics are ImageNet's, not learned at all). 11.1: the vocabulary and IDF values (and the threshold, via cross-validated probabilities).

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both projects.** 10.5 Steps 4 to 6 (stage 2 fine-tuning, test once: 0.928, save `landcover_resnet18.pt`); 11.1 Steps 4 to 6 (recall 0.858, precision 0.698, `review_follow_up.csv` with 698 reviews).
- **Lesson exercises.** 10.5: Exercises 1 to 3 (ImageNet's guesses, MobileNetV3-small, 64 vs 128 pixels); Exercise 4 is a stretch. 11.1: Exercises 1 to 3 (busiest terms, Naive Bayes 0.857, character n-grams); Exercise 4 is a stretch.
- **Both Quick Quizzes** and **both journal entries** (10.5: the three accuracies side by side and the 5,000-labels answer; 11.1: a sentence the bag of words gets wrong).
- **Unfinished labs:** Lab A of each lesson to its Expected line.
- **Before the Next Lesson (from 11.1):** five sentences of your own through the bigram model; look at `data/products_ml.csv` and `data/interactions.csv` with `.head()` and `.shape`.

BRING LIST FOR SESSION 10

- A laptop on which `check_environment.py` still ends with READY: Session 10 uses SciPy sparse matrices, scikit-learn, **MLflow** and **skops** (both installed by the course `requirements.txt`; nothing new to download, no image data).
- The `pinecrest-ml` folder with `data/` (`interactions.csv`, `products_ml.csv`, `customers_churn.csv`) and `labs/` (Session 10 starters are in `labs/module-10/`).
- The finished 10.5 project file `landcover_resnet18.pt` and the 11.1 project notebook with `review_follow_up.csv` (Lesson 12.1 talks about saving and versioning exactly such files).
- The workbook and the journal. A terminal you can run `mlflow ui` in (Lesson 12.1, optional in the room).

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** 10.5 demo Steps 6 to 9; 11.1 demo Steps 8 to 12; Lab A of each lesson to TODO 3; the project check numbers.
- **Cut first (saves 4m):** 10.5 demo Step 13 and 11.1 demo Step 13: say the two fixes.
- **Cut second (saves 5m):** 11.1 demo Steps 4 and 6 (analyzer and TF-IDF by hand): point at the lesson page's tables.
- **Cut third (saves 15m):** the 11.1 project start. Read the brief, give the Step 1 to 3 check numbers, send it home.

RUNNING SHORT: WHAT TO ADD

- Run the lesson's 10.5 Exercise 1 on the projector: ImageNet calls a lake a jellyfish and a highway a corkscrew. The best argument for a new head.
- Lab B of each lesson for everyone, with a debrief on the human-review queue vs the auto-routing threshold: the same idea in images and text.
- 11.1 Exercise 2 live: Naive Bayes on counts (0.857) vs TF-IDF logistic regression (0.877).

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 10: Recommendations and Reproducibility

How to teach Session 10 of Advanced Machine Learning: turn 32,000 purchases into a personal "Picked for You" list and prove with an honest leave-last-out test that it beats the best sellers (0.308 vs 0.209), then make results reproducible and models safe to share: seeds, fingerprints, versions, skops instead of pickles, and an MLflow store that answers "which settings gave this number?".

`2 lessons` `3 hrs session` `Lessons 11.2–12.1` `Session 10 of 13`

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame → live demo → guided lab → start the project. Exercises, quizzes and journals go home.
- Every code block below was run to produce its output (scikit-learn 1.9.1, SciPy, MLflow 3.17.1, skops 0.16.0, 4 CPU threads). MLflow run IDs and timings differ on every machine; nothing else should.
- Keep `applications_recommendation_systems.html` and `mlops_intro.html` open for the figures (long tail, the leave-last-out diagram, hit rate against k, the lab-notebook diagram). Labs are in `labs/module-10/`.
- No deep learning and no downloads today: the whole session runs in seconds per cell. The slowest lab (Lab 12.1 A) takes about 30 seconds, mostly MLflow setting up its database.

> **TWO HABITS THIS SESSION IS REALLY ABOUT**
>
> Both lessons are about evaluation discipline more than algorithms. In 11.2, hide something real (the last purchase), ask the model to find it, compare with a baseline that knows nothing about the customer, and choose settings on a validation item. In 12.1, make that number provable later: seed, fingerprint, versions, settings, and a model file that cannot run code when someone opens it. Keep returning to "compared with what?" and "could you prove it in six months?"

BEFORE THE SESSION

- ☐ **Kit check:** `python3 labs/check_labs.py module-10`: 8 PASS lines, about 60 seconds. The 12.1 labs write their MLflow database and models to a temporary `OUTPUT` folder during the check; on a learner's machine they go to `output/` next to the lab.

- ☐ **Environment:** `check_environment.py` READY covers MLflow and skops. Run `python -c "import mlflow, skops"` on the projector machine once beforehand: the first MLflow import takes several seconds.
- ☐ **Projector:** notebooks `lesson_11_2` and `lesson_12_1` in a FRESH folder for 12.1 (no old `mlflow.db`, so the query shows only today's runs); a second terminal for `mlflow ui`.
- ☐ **Board:** "random 0.047 | best sellers 0.209 | item-kNN | SVD" for 11.2; the five ingredients for 12.1.
- ☐ **Hand out:** starters, `debug_recommender.py`, `debug_reproducibility.py`.

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
`MlflowException: The filesystem tracking backend (e.g., './mlruns') is in maintenance mode...`	Old tutorial code or the default store	`mlflow.set_tracking_uri("sqlite:///mlflow.db")` before anything else.
`search_runs` shows yesterday's or someone else's runs	The same `mlflow.db` and experiment name reused	Fine in real life (that is the point); for the demo, a fresh folder or a new experiment name.
`mlflow ui` says the port is in use	Another MLflow (or app) on 5000	`mlflow ui --backend-store-uri sqlite:///mlflow.db --port 5001`.
A line `INFO mlflow.agent.hint: Load the ... skill` appears	MLflow 3.17 prints a hint for coding assistants	Harmless. Set the environment variable `MLFLOW_DISABLE_AGENT_HINT=1` to silence it (the labs do).
`UntrustedTypesFoundException`	skops wants a reviewed list	`get_untrusted_types`, read it, pass the types you accept in `trusted=`.
`MemoryError` or a very slow cell in 11.2	`.toarray()` on a big matrix, or customer × customer similarity on a laptop with little RAM	At 3,000 customers both are fine (72 MB for 3,000 × 3,000); never on a real shop's data.

SESSION 10 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Lessons 10.5 and 11.1; homework check (`landcover_resnet18.pt`, `review_follow_up.csv`).

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:10	10	**11.2** Frame and mental model	The clerk who remembers; the customer-by-product grid; leave the last one out.
0:20	30	**11.2** Live demo	The grid and the long tail, the split, the sparse matrix and helpers, random and best-seller baselines, item-kNN, one customer, SVD and its factors, k, content and cold start, coverage, mistakes.
0:50	20	**11.2** Guided lab	`labs/module-10/l11_2-recommendations/segment_baselines_starter.py`; fast pairs take `labs/module-10/l11_2-recommendations/user_based_starter.py`.
1:10	15	**11.2** Start the lesson project	Picked for You, Steps 1 to 3 (choose on the validation item); Steps 4 to 6 go home.
1:25	10	Break	Board: the five ingredients (code, data, environment, randomness, settings).
1:35	10	**12.1** Frame and mental model	The lab notebook; five ingredients; why a model file is code.
1:45	30	**12.1** Live demo	Seeds, a data fingerprint, versions, joblib and a pickle that runs code, skops and its trusted types, a model card, MLflow with SQLite: runs, a query, three loud mistakes.
2:15	20	**12.1** Guided lab	`labs/module-10/l12_1-reproducibility/track_demand_starter.py`; fast pairs take `labs/module-10/l12_1-reproducibility/safe_handover_starter.py`.
2:35	15	**12.1** Start the lesson project	A tracked, reproducible churn model, Steps 1 to 3 (four candidates in MLflow); Steps 4 to 6 go home.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 11.
Total	180		Ends at 3:00.

In the room. The lessons estimate 110 + 105 = 215 minutes for a solo learner; the room has 150. So each lesson gets the mental model, the demo, one guided lab and the first steps of its project.

Goes home (2 to 3 hours). Project 11.2 Steps 4 to 6 (test once, `picked_for_you.csv`) and Project 12.1 Steps 4 to 6 (`churn_final.skops`, `churn_final.json`, the from-scratch check); Exercises 1 to 3 of each lesson; both quizzes and journals; the unfinished lab.

Protect, in this order: 11.2 demo Steps 4 to 9 (split, baselines, item-kNN: "0.209 vs 0.308"); 12.1 demo Steps 6 to 8 (the pickle that runs code, skops); the MLflow query (Step 12); Lab A of each lesson. **Cut first:** 11.2 demo Steps 11 and 14 (factors, coverage: point at the page), then 12.1 Step 4, then the 12.1 project start.

WARM-UP

0:00 Warm-up: Transfer Learning and Text

10 min

HOW TO RUN IT

- Five questions on the slide, two minutes alone, answers by show of hands. Homework check: who has `landcover_resnet18.pt` (12.1 will talk about saving files like it) and `review_follow_up.csv`.

QUESTIONS AND ANSWERS

W1

What does this print?

```
from torch import nn
from torchvision import models
m = models.resnet18(weights=None)
for p in m.parameters():
    p.requires_grad = False
m.fc = nn.Linear(512, 5)
print(sum(p.numel() for p in m.parameters() if p.requires_grad))
```

`2565`

> **Why:** Frozen first, then a new 5-class head: 512 × 5 + 5 = 2,565. New layers are trainable by default.

W2

What does this print?

```
from sklearn.feature_extraction.text import CountVectorizer
print(CountVectorizer().build_analyzer()("Not bad, NOT bad at all!"))
```

`['not', 'bad', 'not', 'bad', 'at', 'all']`

> **Why:** Lowercased, punctuation dropped, every occurrence kept in order by the analyzer. Counted as a bag: not 2, bad 2, at 1, all 1. A unigram model cannot tell Not bad from bad.

W3

Why must the backbone be in `eval()` mode while you extract features?

Batch normalization must use its stored ImageNet statistics; in training mode a tile's features depend on its batch and the stored statistics drift.

> **Why:** no_grad does not do this: it only switches off gradient tracking.

W4

Someone fits a `TfidfVectorizer` on all reviews, then splits. What is wrong, and how do you fix it?

Test reviews shape the vocabulary and the IDF values: a leak. Split first, fit inside a pipeline.

> **Why:** Today's recommender has the same rule in a new costume: the matrix is built from training purchases only.

W5

Frozen features: 0.885 with 500 labels, 0.902 with 2,000. What is the better next step than labeling more?

Fine-tune the top block (layer4) with a small learning rate: validation 0.876 → 0.924 in the project.

> **Why:** Measuring what more data buys, before paying for it, is the habit.

11.2 Recommendation Systems

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Build a sparse customer-by-product matrix from purchase records with SciPy
- Evaluate a recommender with a leave-last-out split and hit rate at 10, against random and best-seller baselines
- Implement item-based collaborative filtering with cosine similarity, and matrix factorization with `TruncatedSVD`
- Use product attributes for content-based recommendations, and explain how they rescue a brand-new product
- Judge a recommender on more than accuracy: catalog coverage and popularity bias

TIMING IN THE ROOM

10m	Frame: the clerk who remembers; the grid; leave the last one out
30m	Live demo: grid, long tail, split, helpers, baselines, item-kNN, one customer, SVD, k, cold start, coverage
20m	Guided lab: `segment_baselines_starter.py`, then `user_based_starter.py`
15m	Start the project: Picked for You, Steps 1 to 3

MENTAL MODEL TO SAY

"A long-serving clerk at the Bend store has watched thousands of customers. When a regular buys a climbing harness she does not reach for the best-selling socks: 'People who buy that harness usually come back for a chalk bag.' No rule book; she noticed which products end up with the same customers. That is collaborative filtering. A new clerk can only point at the best sellers (popularity) or read the labels (content-based). Today we build all three and measure which helps customers most."

Then the shape of the problem: no target column, a big mostly empty grid of customers × products, and the job is to RANK the empty cells. "The tools you know: cosine similarity from kNN, a cousin of PCA, and holding out data you never train on."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Notebook `lesson_11_2`. Everything runs in about a second per cell. Ask for predictions before Steps 7, 9 and 13.

1. **Two files (2m).** "One row per customer and product bought; a product table. No target column this time."

```
import numpy as np
import pandas as pd
interactions = pd.read_csv("data/interactions.csv")
products = pd.read_csv("data/products_ml.csv")
print(interactions.shape, products.shape)
print(interactions.head(3))
```

```
(32178, 5) (200, 8)
  customer_id product_id  purchases  rating last_purchase_date
0     C100001   APL-3021          2     5.0         2024-03-20
1     C100001   KIT-3003          2     5.0         2024-08-27
2     C100001   KIT-3013          1     NaN         2024-09-23
```

Implicit feedback (purchases) vs explicit (ratings, 38% of rows). We use purchases.

2. **How empty is the grid? (2m).** "A typical customer has bought 9 of 200 products."

```
n_customers = interactions["customer_id"].nunique()
n_products = interactions["product_id"].nunique()
print(f"{n_customers:,} customers x {n_products} products: {len(interactions)
print(interactions.groupby("customer_id").size().describe()[["mean", "50%", "m
```

```
3,000 customers x 200 products: 5.4% filled
{'mean': 10.726, '50%': 9.0, 'min': 3.0, 'max': 45.0}
```

An empty cell is not a dislike: they may have bought a tent elsewhere. That is why we RANK empty cells instead of predicting yes or no.

3. **The long tail (2m).** "Two sunscreens reach a third of all customers each; the typical product sells to about 120."

```
buyers = interactions.groupby("product_id").size().sort_values(ascending=False
names = products.set_index("product_id")["name"]
print(names[buyers.index[:2]].tolist(), buyers.head(2).tolist())
print(f"median buyers {buyers.median():.0f}; top 20 products = {buyers.head(20
```

```
['Larkspur Ridge Lookout Sunscreen SPF 50', 'Juniper Vane Tempo Sunscreen SPF
median buyers 120; top 20 products = 29.9% of purchases
```

Show the lesson's long-tail chart. A best-seller list shows its left end to everyone.

4 **Leave the last one out (3m).** "Hide each customer's most recent purchase, train on the rest, ask for ten suggestions. Hit = the hidden product is on the list." Draw the lesson's flow diagram.

```
interactions = interactions.sort_values(["customer_id", "last_purchase_date",
test = interactions.groupby("customer_id").tail(1)
train = interactions.drop(test.index)
print(f"{len(train):,} training rows, {len(test):,} test rows (one per custome
```

```
29,178 training rows, 3,000 test rows (one per customer)
```

Latest, not random: it mimics predicting the next purchase and keeps the future out of training (Lesson 1.2's time-split rule).

5 **A sparse matrix from training rows (2m).** "Rows are customers, columns products, a 1 where they bought."

```
from scipy.sparse import csr_matrix
customer_ids = np.sort(interactions["customer_id"].unique())
product_ids = products["product_id"].to_numpy()
row_of = pd.Series(np.arange(len(customer_ids)), index=customer_ids)
col_of = pd.Series(np.arange(len(product_ids)), index=product_ids)
R = csr_matrix(
    (np.ones(len(train)), (row_of[train["customer_id"]].to_numpy(), col_of[tra
    shape=(len(customer_ids), len(product_ids)),
)
print(R.shape, f"{R.nnz:,} stored ones")
```

```
(3000, 200) 29,178 stored ones
```

6 **Two helpers for every recommender (3m).** "Any recommender is a score matrix. Mask what they own, sort descending, take ten." Narrate each line of top_k.

```
owned = R.toarray() > 0
test_col = col_of[test.set_index("customer_id").loc[customer_ids, "product_id"

def top_k(scores, k=10):
    scores = np.array(scores, dtype=float)      # a copy, so the caller's matr
    scores[owned] = -np.inf                     # never recommend what they al
    return np.argsort(-scores, axis=1)[:, :k]   # column numbers of the k high

def hit_rate(scores, k=10):
    return (top_k(scores, k) == test_col[:, None]).any(axis=1).mean()

print(owned.shape, test_col[:5])
```

```
(3000, 200) [196  46 197  98 107]
```

7 **The floor and the baseline (2m).** Ask for predictions first: "Random? Best sellers?"

```
rng = np.random.default_rng(42)
random_scores = rng.random(R.shape)
popularity = np.asarray(R.sum(axis=0)).ravel()
popularity_scores = np.tile(popularity, (R.shape[0], 1))
print(f"random      HR@10 = {hit_rate(random_scores):.3f}")
print(f"popularity  HR@10 = {hit_rate(popularity_scores):.3f}")
```

```
random      HR@10 = 0.047
popularity  HR@10 = 0.209
```

Board: random 0.047, best sellers 0.209. "A personal recommender must beat 0.209 to earn its complexity."

8 **The clerk's trick: item similarity (3m).** "Two products are similar if mostly the same customers bought them."

```
from sklearn.metrics.pairwise import cosine_similarity
item_sim = cosine_similarity(R.T)        # products x products
np.fill_diagonal(item_sim, 0)
harness = col_of["CLM-3001"]
nearest = np.argsort(-item_sim[harness])[:4]
print(item_sim.shape, names.iloc[harness])
print(products.iloc[nearest][["name"]].assign(similarity=item_sim[harness, nea
```

```
(200, 200) Juniper Vane Headwater Climbing Harness
                                        name  similarity
121    Juniper Vane Summit Climbing Harness       0.282
113  Pinecrest Timberline Locking Carabiner       0.268
125         Cedar Notch Wayfinder Chalk Bag       0.267
124                Pinecrest Tempo 60m Rope       0.254
```

Nobody told the computer that harnesses, carabiners and ropes belong together.

9 **Score = similarity to what they own (2m).** "One multiplication: R @ item_sim."

```
knn_scores = R @ item_sim
print(type(knn_scores).__name__, knn_scores.shape)
print(f"item-kNN    HR@10 = {hit_rate(knn_scores):.3f}")
```

```
ndarray (3000, 200)
item-kNN    HR@10 = 0.308
```

0.308: about 50% more hits than the best sellers. The core result, in two lines of linear algebra.

10 **One customer (2m).** "Numbers convince; one example builds intuition."

```
r = row_of["C100027"]
print("bought:", names.iloc[np.flatnonzero(owned[r])].tolist())
print("recommended:", names.iloc[top_k(knn_scores, 3)[r]].tolist())
print("really bought next:", names.iloc[test_col[r]])
```

```
bought: ['Wrenfield Trailhead Approach Shoes', 'Pinecrest Wayfinder Approach S
recommended: ['Juniper Vane Summit Climbing Harness', 'Juniper Vane Tundra Cli
really bought next: Cedar Notch Cairn Climbing Harness
```

Three climbing purchases in, the model offers climbing gear; the harness bought next is on the full top-5 list (lesson page).

11 **Matrix factorization (3m).** "Maybe 200 preferences are really a handful of tastes." Write R ≈ customer factors @ product factors.

```
from sklearn.decomposition import TruncatedSVD
svd = TruncatedSVD(n_components=5, random_state=42)
customer_factors = svd.fit_transform(R)
product_factors = svd.components_
svd_scores = customer_factors @ product_factors
print(customer_factors.shape, product_factors.shape, f"variance captured {svd.
print(f"SVD k=5     HR@10 = {hit_rate(svd_scores):.3f}")
```

```
(3000, 5) (5, 200) variance captured 17.1%
SVD k=5     HR@10 = 0.311
```

12 **What did the factors learn? (2m).** "The activity column was never shown to the model: a fair check."

```
activity = products["primary_activity"]
for f in range(5):
    top_products = np.argsort(-product_factors[f])[:10]
    print(f"factor {f}:", activity.iloc[top_products].value_counts().to_dict()
```

```
factor 0: {'general': 8, 'hiking': 2}
factor 1: {'paddling': 4, 'general': 3, 'running': 3}
factor 2: {'hiking': 7, 'general': 3}
factor 3: {'climbing': 10}
factor 4: {'running': 8, 'general': 1, 'camping': 1}
```

Hiking, climbing, running come out cleanly; factor 0 is popularity. Naming a factor is your interpretation.

13 **How many factors? (2m).** "More factors fit the past better. Do they predict better?"

```
svd_hr = {}
for k in [2, 5, 10, 20, 40]:
    model = TruncatedSVD(n_components=k, random_state=42)
    svd_hr[k] = round(float(hit_rate(model.fit_transform(R) @ model.components
print(svd_hr)
```

```
{2: 0.22, 5: 0.311, 10: 0.258, 20: 0.227, 40: 0.19}
```

Peak at 5, worse than best sellers at 40: bias-variance in a new costume. Then confess: "We just picked k on the test item. The project does it properly."

14 **Content and the cold start (2m).** "A product launched yesterday has no buyers: collaborative filtering can never show it."

```
from sklearn.preprocessing import OneHotEncoder
attributes = OneHotEncoder(sparse_output=False).fit_transform(products[["prima
content_sim = cosine_similarity(attributes)
np.fill_diagonal(content_sim, 0)
content_scores = R @ content_sim
print(f"content     HR@10 = {hit_rate(content_scores):.3f}")
new_product = col_of["PDL-3018"]
cold = R.copy().tolil()
cold[:, new_product] = 0                        # wipe its history: a brand-new
print("largest item-kNN similarity after wiping:", cosine_similarity(cold.tocs
print("content neighbors:", names.iloc[np.argsort(-content_sim[new_product])[:
```

```
content     HR@10 = 0.192
largest item-kNN similarity after wiping: 0.0
content neighbors: ['Pinecrest Timberline Kayak Paddle', 'Larkspur Ridge Footl
```

15 **Beyond hit rate (2m).** "Two lists with the same hit rate can be very different for the shop."

```
best_sellers = np.argsort(-popularity)[:20]
methods = {"best sellers": popularity_scores, "item-kNN": knn_scores, "SVD k=5
for name, scores in methods.items():
    lists = top_k(scores)
    print(f"{name:12s} HR@10 {hit_rate(scores):.3f}  products shown {len(np.un
          f"top-20 slots {np.isin(lists, best_sellers).mean():.0%}")
```

```
best sellers HR@10 0.209  products shown  31  top-20 slots 100%
item-kNN     HR@10 0.308  products shown 161  top-20 slots 51%
SVD k=5      HR@10 0.311  products shown 153  top-20 slots 51%
content      HR@10 0.192  products shown 200  top-20 slots 20%
```

Then the responsibility box: recommendations create the purchases they are trained on (feedback loops), and purchase lists are personal data.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Similarity on the wrong axis.** `ValueError: matmul: dimension mismatch with signature (n,k=200),(k=3000,m)->(n,m)`. Coach: "Print the shape of your similarity matrix. Products × products?" Pass `R.T`.
- **Sorting ascending, or forgetting the mask.** Silent: HR@10: 0.001 (ascending) and owned slots: 14%, HR@10 without the mask: 0.270. Coach: "Compare with random (0.047). A model that loses to random is a bug."

- **Tuning on the test item.** Silent and tempting after the k chart. Coach: "Which purchase did you use to choose k?" The project's validation item is the answer.

GUIDED LAB

- **Lab A (everyone, 12m):** `labs/module-10/l11_2-recommendations/segment_baselines_starter.py`; reference `labs/module-10/l11_2-recommendations/segment_baselines_solution.py`. Is a smarter best-seller list enough? Best sellers of the customer's home store and of their favorite activity, against global best sellers and item-kNN, on the lesson's honest test.
- **Expected, Lab A:** global 0.209; home-store best sellers 0.209 (36 products shown, 98% of slots from the top 20: stores barely differ); favorite-activity best sellers 0.265 (183 products, 31%); item-kNN 0.308 (161, 51%). Stretch: customers whose favorite is "general" 0.229 vs 0.280 for item-kNN.
- **Debrief (2m):** a cheap, explainable segment baseline closes about half the gap between best sellers and item-kNN, and shows more of the catalog. That is the bar a vendor's "AI recommendations" should be measured against, not the global best-seller list.
- **Lab B (fast pairs, 8m):** `labs/module-10/l11_2-recommendations/user_based_starter.py`; reference `labs/module-10/l11_2-recommendations/user_based_solution.py`. User-based filtering with n neighbors, n chosen on a validation item, then one test score; and a "because you just bought" list. Expected: validation 0.202 / 0.282 / 0.292 / 0.266 for 10 / 50 / 200 / 1,000 neighbors (item-kNN 0.286); chosen 200; test user-kNN 0.321, item-kNN 0.308; last purchase only 0.212. The last-purchase result is an honest negative: in this data the purchase dates within the period carry little order, so the whole history beats the most recent item.
- **Fast finishers:** `labs/module-10/l11_2-recommendations/debug_recommender.py`: a broadcasting error and the wrong axis (loud); the FIRST purchase as the test item and an ascending sort (silent). Fixed: random 0.047, best sellers 0.209, item-kNN 0.308.

START THE LESSON PROJECT

Picked for You: five products per customer, the model chosen on a validation item and scored once. In the room: Steps 1 to 3.

- Step 1: three-way split by time. **Check:** `fit 26,178 validation 3,000 test 3,000`.
- Step 2: the helpers as functions of their inputs. **Check:** `(3000, 200) 26178`.
- Step 3: candidates on the validation item. **Check:** best sellers 0.209, item-kNN 0.286, SVD k=5 0.285: chosen item-kNN (a near tie; either is defensible).
- **Goes home:** Step 4 (test once: 0.308 vs 0.209, +47%), Step 5 (retrain on everything, `picked_for_you.csv`, 15,000 rows), Step 6 (sanity checks: 0 owned, 112 products, 63% of slots from the top 20).

CHECKS FOR UNDERSTANDING

- What does one row of R @ item_sim hold? **Answer:** each product's summed similarity to everything that customer owns.
- Why is a random list's hit rate about 0.05? **Answer:** 10 picks from about 190 unowned products.
- Why can item-kNN never recommend a brand-new product? **Answer:** its column is all zeros, so its similarity to everything is 0.
- Name one cost of the best-seller list besides hit rate. **Answer:** coverage (31 products); it buries the long tail and new products.

Workbook self-check, items Q1 to Q6.

Answers: **Q1** B) A bug, such as sorting the scores ascending or forgetting to mask owned products; **Q2** `[1. 0.41 0.41]`; **Q3** D) Owned products are similar to what the customer bought, so they score high and crowd the list; customers would see items they already have; **Q4** A) Catalog coverage: how many different products appear on anyone's list; **Q5** `(6, 3) (3, 8)`; **Q6** C) k was tuned on the test item, so 0.311 is optimistic; choose k on a validation item and score the test item once. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: the workbook's worked example (four customers, four products) by hand on paper before the demo's 3,000 × 200.

Stretch: Exercise 4 (who benefits: 11+ products 0.357 vs 0.265 for 2 to 5) and the Bonus Challenge (a blended hybrid chosen on validation).

12.1 Reproducibility and Experiment Tracking

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Name the five ingredients of a reproducible result (code, data, environment, randomness, settings) and capture each one
- Fingerprint a data file with SHA-256 and record library versions next to a model
- Save and load a fitted pipeline with `joblib`, and explain why loading a pickle from an untrusted source can run arbitrary code
- Save and load models with `skops.io`, reviewing the untrusted types before you trust them
- Track parameters, metrics and models with MLflow, query the runs, and load a logged model back

TIMING IN THE ROOM

10m	Frame: the lab notebook; five ingredients; a model file is code
30m	Live demo: seeds, fingerprint, versions, joblib and the pickle, skops, model card, MLflow runs and a query, three mistakes
20m	Guided lab: `track_demand_starter.py`, then `safe_handover_starter.py`
15m	Start the project: four tracked candidates, Steps 1 to 3

MENTAL MODEL TO SAY

"A good chemist writes every experiment in a bound lab notebook: which batch of each chemical, which instrument, the temperature, the result, even the failures. Anyone can repeat it, and nobody argues about what was done." Write the five ingredients: **code** (Git), **data** (the exact bytes), **environment** (versions), **randomness** (seeds), **settings** (hyperparameters, features, split). "Record all five plus the results and you have a lab notebook for models. MLflow is a lab notebook that fills itself in."

Then the uncomfortable half: "A saved model file is not data. A pickle is a small program, and opening it runs it. How you load a model decides whether a stranger's file can run commands on your machine."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Notebook `lesson_12_1` in a fresh folder (so the MLflow query shows only today's runs). The first MLflow call prints INFO lines while it creates the database: say so before it happens.

1. **The model we look after (2m).** "Module 2's churn pipeline: preprocessing inside, so it travels with the model."

```
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

customers = pd.read_csv("data/customers_churn.csv")
numeric = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value", "day
           "return_rate", "support_tickets_12m", "discount_share", "app_user",
categorical = ["home_store", "region", "age_group", "loyalty_tier", "preferred
X, y = customers[numeric + categorical], customers["churned"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, stra

def build_pipeline(model):
    prep = ColumnTransformer([
        ("num", make_pipeline(SimpleImputer(strategy="median"), StandardScaler
        ("cat", make_pipeline(SimpleImputer(strategy="most_frequent"), OneHotE
    ])
    return Pipeline([("prep", prep), ("model", model)])

print(X_train.shape, X_test.shape)
```

```
(4500, 16) (1500, 16)
```

2 **Randomness: seeds (3m).** "Fit the same forest twice without a seed, then with one."

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score

def forest_probabilities(seed):
    forest = build_pipeline(RandomForestClassifier(n_estimators=50, min_sample
    return forest.fit(X_train, y_train).predict_proba(X_test)[:, 1]

print("no seed:  two fits identical?", np.array_equal(forest_probabilities(Non
print("seed 42:  two fits identical?", np.array_equal(forest_probabilities(42)
for seed in [42, 7]:
    print(f"seed {seed:2d}: test AUC {roc_auc_score(y_test, forest_probabiliti
```

```
no seed:  two fits identical? False
seed 42:  two fits identical? True
seed 42: test AUC 0.8612
seed  7: test AUC 0.8560
```

Fix the seed so results repeat; and a third-decimal difference between seeds is noise, not a finding.

3. **Data: a fingerprint, not a file name (2m).** "Change one character and all 64 change."

```
import hashlib

def file_sha256(path):
    digest = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(1_000_000), b""):   # 1 MB at a time:
            digest.update(chunk)
    return digest.hexdigest()

data_hash = file_sha256("data/customers_churn.csv")
print(len(data_hash), "hex characters, starting", data_hash[:16])
```

```
64 hex characters, starting ac541c7e8691d33f
```

Same first 16 characters as the lesson page means the same file.

4. **Environment: write down the versions (1m).** Then show (do not run) `python -m pip freeze > requirements-lock.txt`.

```
import platform
from importlib.metadata import version
environment = {"python": platform.python_version()}
for package in ["scikit-learn", "pandas", "numpy", "skops", "mlflow"]:
    environment[package] = version(package)
print(environment)
```

```
{'python': '3.12.14', 'scikit-learn': '1.9.1', 'pandas': '3.0.6', 'numpy': '2.
```

Their versions may differ; what matters is that they record theirs.

5. **Saving with joblib (2m).** "Save the whole pipeline: imputer medians, scaler means and categories are part of the model."

```
import joblib
from sklearn.ensemble import HistGradientBoostingClassifier
churn_model = build_pipeline(HistGradientBoostingClassifier(random_state=42)).
test_auc = roc_auc_score(y_test, churn_model.predict_proba(X_test)[:, 1])
print(f"test AUC: {test_auc:.4f}")
joblib.dump(churn_model, "churn_model.joblib")
reloaded = joblib.load("churn_model.joblib")
print("same predictions after reload:", np.array_equal(reloaded.predict_proba(
```

```
test AUC: 0.8598
same predictions after reload: True
```

6 **A pickle runs code (3m).** "Now the catch. This file contains no model; watch what LOADING it does."

```
import pickle

class NotReallyAModel:
    def __reduce__(self):                        # tells pickle how to "rebuild
        return (print, ("This line ran while the file was being LOADED.",))

with open("innocent_looking_model.pkl", "wb") as f:
    pickle.dump(NotReallyAModel(), f)
loaded = joblib.load("innocent_looking_model.pkl")
print("what came back:", loaded)
```

```
This line ran while the file was being LOADED.
what came back: None
```

Pause here. "It was print. It could have been anything, with your permissions." Never load a pickle you did not make or fully trust.

7 **skops: review before you trust (2m).** "skops describes objects instead of storing instructions. Ask the file what it needs before loading it."

```
import skops.io as sio
sio.dump(churn_model, "churn_model.skops")
print(sio.get_untrusted_types(file="churn_model.skops"))
```

```
['numpy.dtype', 'sklearn.ensemble._hist_gradient_boosting.predictor.TreePredi
```

8 **Load with a list you wrote out (2m).** "Type the list; do not paste whatever the file reports. A future file that asks for something new should stop you."

```
trusted_types = ["numpy.dtype", "sklearn.ensemble._hist_gradient_boosting.pred
safe_model = sio.load("churn_model.skops", trusted=trusted_types)
print(type(safe_model).__name__, [name for name, _ in safe_model.steps])
print("same predictions:", np.array_equal(safe_model.predict_proba(X_test), ch
```

```
Pipeline ['prep', 'model']
same predictions: True
```

9 **A model card (2m).** "A model without its story is half a lab notebook."

```
import json
model_card = {
    "model_file": "churn_model.skops", "estimator": type(churn_model[-1]).__na
    "features": numeric + categorical, "data_file": "data/customers_churn.csv"
    "split": {"test_size": 0.25, "stratify": "churned", "random_state": 42},
    "train_rows": len(X_train), "test_auc": round(test_auc, 4),
    "environment": environment, "skops_trusted_types": trusted_types,
}
with open("churn_model.json", "w") as f:
    json.dump(model_card, f, indent=2)
print({key: model_card[key] for key in ["estimator", "train_rows", "test_auc"]
```

```
{'estimator': 'HistGradientBoostingClassifier', 'train_rows': 4500, 'test_auc
```

Add aloud what professional cards add: intended use ("rank customers for retention offers, not decisions about individual accounts"), limitations, fairness checks.

10 **MLflow with a SQLite store (2m).** "A lab notebook that fills itself in. Local file, no server, no account." Warn about the INFO lines on first use.

```
import mlflow
mlflow.set_tracking_uri("sqlite:///mlflow.db")     # a single local database f
mlflow.set_experiment("churn-lesson")
print(mlflow.get_tracking_uri())
```

```
sqlite:///mlflow.db
```

11 **One run per candidate (3m).** "Log every hyperparameter, including the defaults you never typed."

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
candidates = {"logreg": LogisticRegression(max_iter=1000), "hgb": HistGradient
for name, model in candidates.items():
    with mlflow.start_run(run_name=name):
        scores = cross_val_score(build_pipeline(model), X_train, y_train, cv=c
        mlflow.log_params(model.get_params())
        mlflow.set_tags({"estimator": type(model).__name__, "data_sha256": dat
        mlflow.log_metrics({"cv_auc": scores.mean(), "cv_auc_std": scores.std(
        print(f"{name:7s} CV AUC {scores.mean():.3f} ± {scores.std():.3f}")
```

```
logreg  CV AUC 0.832 ± 0.023
hgb     CV AUC 0.861 ± 0.013
```

12 **Query the store (2m).** "In a real project the runs come from different days and people: query, do not scroll."

```
runs = mlflow.search_runs(experiment_names=["churn-lesson"], order_by=["metric
print(runs[["tags.mlflow.runName", "metrics.cv_auc", "params.max_iter"]].round
```

```
tags.mlflow.runName  metrics.cv_auc params.max_iter
                hgb          0.8615             100
             logreg          0.8324            1000
```

Parameters are text, metrics numbers. If time allows, `mlflow ui --backend-store-uri sqlite:///mlflow.db` in a terminal and open http://127.0.0.1:5000 for a minute; Ctrl+C to stop.

13 **Three loud mistakes (3m).** "No reviewed list; an old tutorial's mlruns folder; changing a parameter inside a run." (First line of each message.)

```
try:
    sio.load("churn_model.skops")
except Exception as e:
    print(type(e).__name__ + ":", str(e).splitlines()[0])

try:
    mlflow.set_tracking_uri("./mlruns")
    mlflow.set_experiment("churn-lesson")
except Exception as e:
    print(type(e).__name__ + ":", str(e).split(". ")[0] + ".")
mlflow.set_tracking_uri("sqlite:///mlflow.db")

try:
    with mlflow.start_run(run_name="oops"):
        mlflow.log_param("learning_rate", 0.1)
        mlflow.log_param("learning_rate", 0.05)
except Exception as e:
    print(type(e).__name__ + ":", str(e).split(". ")[0] + ".")
```

```
UntrustedTypesFoundException: Untrusted types found in the file: ['numpy.dtype
MlflowException: The filesystem tracking backend (e.g., './mlruns') is in mair
MlflowException: Changing param values is not allowed.
```

Each refusal is a feature: skops makes you decide, MLflow 3.17 needs a database (sqlite:///mlflow.db), a logged parameter cannot be rewritten (one run per setting; the failed run stays in the store).

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Saving the classifier instead of the pipeline.** `ValueError: could not convert string to float: 'Asheville'` (raw columns reach a model that expects preprocessed numbers). Coach: "What does your file contain: `type(obj)`?" Save the whole Pipeline.
- **Trusting whatever the file asks for.** `trusted=sio.get_untrusted_types(file=path)` silences skops and defeats it. Coach: "Read me the list. Which of these did you expect for this model?" (Lab 12.1 B: the list depends on the model.)
- **The old `mlruns` store and parameters changed inside a run.** Loud, with good messages. Coach: one run per setting, `sqlite:///mlflow.db`, and read the message to the end: it names the fix.

GUIDED LAB

- **Lab A (everyone, 12m):** `labs/module-10/l12_1-reproducibility/track_demand_starter.py`; reference `labs/module-10/l12_1-reproducibility/track_demand_solution.py`. A lab notebook for the weekly demand forecast: CONFIG, fingerprint, three candidates tracked in MLflow (SQLite and artifacts inside `OUTPUT`), the

winner chosen by querying the store (lowest MAE: `ASC`), one test score, a filtered query and a seed check. About 30 seconds.

- **Expected, Lab A:** fingerprint `c48cd2582060519c`; fit 3,840 / validation 1,248 / test 4,896; validation MAE ridge 12.63, forest 11.31, hgb 10.74; hgb first; test MAE (2025) 8.21; below 12: hgb and forest; 4 runs in the store; hgb 10.7445 for every seed; forest 11.3146 twice and 11.2821 with seed 7.
- **Debrief (2m):** two traps learners hit: `DESC` for an error metric (it puts the worst first), and believing a seed matters for every model. The HGB regressor on 3,840 rows uses no randomness (early stopping only switches on above 10,000 rows), so its seed changes nothing; the forest's does. The validation weeks (October to December) are harder than 2025 as a whole, so validation MAE is higher than test MAE: a split by season, not a bug.
- **Lab B (fast pairs, 8m):** `labs/module-10/l12_1-reproducibility/safe_handover_starter.py`; reference `labs/module-10/l12_1-reproducibility/safe_handover_solution.py`. Hand a fraud model to another team: skops, a refused load, trust lists for three model kinds, a model card, and a verify function that rebuilds the score from the card alone. Expected: AP 0.4743; HGB needs only TreePredictor; logistic regression `[]`; forest `['sklearn.tree._tree.Tree']`; verify (True, True, 0.4743) for the original, a byte copy and a pandas re-save; (False, True, 0.4743) after a one-cent edit; skops 462 kB vs joblib 142 kB.
- **Fast finishers:** `labs/module-10/l12_1-reproducibility/debug_reproducibility.py`: `./mlruns` and a parameter logged twice (loud); an unseeded forest and an automatic trust list (silent). Fixed, two runs print the same lines ending `test AUC 0.8637`.

START THE LESSON PROJECT

A tracked, reproducible churn model. In the room: Steps 1 to 3.

- Step 1: one CONFIG, one fingerprint. **Check:** `data fingerprint: ac541c7e8691d33f`.
- Step 2: data, split, four candidates. **Check:** `(4500, 16) ['logreg', 'forest', 'hgb', 'hgb-slow']`.
- Step 3: one run per candidate. **Check:** CV AUC logreg 0.8324, forest 0.8678, hgb 0.8615, hgb-slow 0.8588.
- **Goes home:** Step 4 (pick from the store: forest; test once 0.8612), Step 5 (`churn_final.skops` with trusted `numpy.dtype` and `sklearn.tree._tree.Tree`, `churn_final.json`, the final run), Step 6 (`Reproduced : True`). **Keep both files and `mlflow.db`: Lesson 12.2 serves exactly this model.**

CHECKS FOR UNDERSTANDING

- Name the five ingredients. **Answer:** code, data, environment, randomness, settings.
- Same seed, same data, different laptop: could the AUC differ? **Answer:** in the last digits, yes (thread count, math libraries, versions): that is why the environment is recorded.

- What did the harmless pickle prove? **Answer:** loading a pickle runs code chosen by whoever made the file.
- Why write the trusted list by hand? **Answer:** so a file that asks for something new stops you instead of loading.

Workbook self-check, items Q7 to Q12.

Answers: **Q7** C) The random seed of the forest; **Q8** `64 True`; **Q9** A) `UntrustedTypesFoundException`: it names the HistGradientBoosting tree type, which you must review and pass in `trusted=`; **Q10** D) Loading a joblib file (a pickle) can run arbitrary code; skops only rebuilds known types or types you trust explicitly; **Q11** `True None`; **Q12** B) `mlflow.search_runs(experiment_names=["churn"], order_by=["metrics.cv_auc DESC"])`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: the five-ingredient checklist from the workbook next to the keyboard; tick each one as the demo records it.

Stretch: Exercise 4 (`train_churn.py`, run twice); the Bonus Challenge (autolog vs explicit logging).

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

1. Item-kNN scores HR@10 0.308 and the best sellers 0.209. Explain in one sentence what that comparison means for Pinecrest's email. **Answer:** the personal list contains the product a customer really bought next for about 31% of customers against 21% for the best-seller row: about 50% more hits, from the same purchase history.
2. Name the five ingredients of a reproducible result and one tool or habit for each. **Answer:** code (Git, a script), data (SHA-256 fingerprint), environment (versions, `pip freeze`), randomness (seeds), settings (a CONFIG dict, logged params).
3. A teammate sends `model.joblib`. What do you do before loading it? **Answer:** only load it if you fully trust the source; better, ask for a skops file, review `get_untrusted_types`, and load with an explicit `trusted=` list.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both projects.** 11.2 Steps 4 to 6 (test HR@10 0.308 vs 0.209, `picked_for_you.csv`, sanity checks); 12.1 Steps 4 to 6 (forest, test AUC 0.8612, `churn_final.skops` + `churn_final.json`, `Reproduced : True`).
- **Lesson exercises.** 11.2: Exercises 1 to 3 (list lengths, purchase counts 0.302, cold start: 100% paddlers); 12.1: Exercises 1 to 3 (fingerprint sensitivity, a logreg trust list, a filtered query). Exercise 4 of each is a stretch.
- **Both Quick Quizzes** and **both journal entries** (12.1: the five ingredients for an earlier project of yours).

- **Before the Next Lesson (from 12.1):** run `mlflow ui --backend-store-uri sqlite:///mlflow.db` and explore; add `mlflow.db`, `mlruns/` and `*.skops` to `.gitignore`.

BRING LIST FOR SESSION 11

- A laptop on which `check_environment.py` ends with READY: Session 11 uses **FastAPI**, **uvicorn** and **httpx** (all in the course `requirements.txt`; nothing new to install).
- The 12.1 project files `churn_final.skops` and `churn_final.json` (and `mlflow.db`): Lesson 12.2 serves exactly this model. (12.2 can recreate them, but bring yours.)
- The `pinecrest-ml` folder with `data/` (`customers_churn.csv`) and `labs/` (Session 11 starters are in `labs/module-11/`).
- The workbook and the journal; a terminal that can start `uvicorn`.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** 11.2 demo Steps 4 to 9; 12.1 demo Steps 6 to 8 and 12; Lab A of each lesson to TODO 3.
- **Cut first (saves 4m):** 11.2 demo Steps 11 and 14: show the lesson page's tables.
- **Cut second (saves 3m):** 12.1 demo Steps 4 and 9: one sentence each.
- **Cut third (saves 15m):** the 12.1 project start: read the brief, give the Step 3 check numbers, send it home.

RUNNING SHORT: WHAT TO ADD

- `mlflow ui` on the projector: compare the two runs side by side, open a run's parameters.
- Lab B of each lesson for everyone; debrief user-kNN beating item-kNN on test (0.321 vs 0.308) after being chosen honestly on validation.
- 11.2 Exercise 4 live: who benefits from personalization.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 11: Serving, Monitoring and the Churn Case Study

How to teach the last two technical lessons of Advanced Machine Learning and launch the capstone: serve the churn model behind a validated FastAPI service with a match test and a drift monitor, then run the whole churn case study from a manager's brief to a memo in dollars. Learners leave with a capstone problem chosen and a one-paragraph plan written.

2 lessons | 3 hrs session | Lessons 12.2–13.1 | Session 11 of 13

HOW TO USE THIS GUIDE

- Two lessons that close the technical course: 12.2 takes the churn model out of the notebook (a validated API and a drift monitor); 13.1 runs the whole churn project end to end, ending in a memo. The last ten minutes before the wrap-up start the capstone: each learner chooses one of Lesson 13.2's five problems and writes a one-paragraph plan.
- Same rhythm for both lessons: frame → live demo (you type, they follow) → guided kit lab (new practice: a FRAUD API and monitor; a leak audit and cost scenarios) → start the lesson project. Exercises, quizzes and journals are homework.
- Every code block below was run to produce the output under it (Python 3.12, pandas 3.0.6, scikit-learn 1.9.1, FastAPI 0.143, pydantic 2.14, skops 0.16, 4 CPU threads). Ask for a prediction before each run that prints a number.
- Keep `mlops_deployment.html` and `case_study_churn.html` open for the diagrams and charts (the order-window flow, the drift histograms, the profit curve, the partial-dependence plots). Labs are in `labs/module-11/`; the capstone reference notes for Sessions 12 and 13 are in `final-assessment.html` at the kit root: read the problem summaries before today's last segment.

> **THE #1 RISK OF SESSION 11: TWO BIG LESSONS AND A CAPSTONE LAUNCH IN THREE HOURS**
>
> 12.2 has two new libraries and two new files; 13.1 has twelve steps that each recap a module. Both demos are long, and the capstone choice at 2:40 must not be eaten. **Protect the clock:** start the 13.1 randomized search (demo Step 6) while you talk, never wait for it in silence; if 12.2 runs over, cut 12.2 Steps 11 to 13 to the PSI table alone (it is on the lesson page) rather than touching the capstone segment. A learner who leaves without a chosen problem loses the first hour of Session 12.

BEFORE THE SESSION

- ☐ **Run the kit check:** `python3 labs/check_labs.py module-11` from the kit folder: 8 passed (about 40 seconds). Run `fraud_api_solution.py` once on the projector machine so the imports are warm.
- ☐ **Learner files from Session 10:** the Lesson 12.1 project's `churn_final.skops` and `churn_final.json`. Anyone without them runs the recreate block at the top of Lesson 12.2's "Package the Model" (about 5 seconds): no blocker.
- ☐ **Packages:** `fastapi`, `pydantic`, `httpx` (TestClient needs it), `uvicorn` and `skops` are in the course `requirements.txt` and in the Session 1 environment check. No downloads today: no image data, no weights, no internet.
- ☐ **Projector machine:** a new folder `serving/` next to `data/`, an editor tab for `schemas.py` and one for `serve.py` (empty), a notebook `lesson_12_2`, a second notebook `lesson_13_1`, a terminal with `(.venv)` for the optional `uvicorn` demo. Font 18 pt or more.
- ☐ **Board, before learners arrive:** the four-row monitoring table (input drift · data quality · prediction drift: no labels; performance: labels, months later); the PSI rules of thumb (below 0.1 stable, 0.1 to 0.25 watch, above 0.25 shift); and the 13.1 outcome table (TP +$45, FP -$15, FN $0, TN $0).
- ☐ **Print or share:** the workbook (its last page has the capstone plan box), and the five problem summaries from Lesson 13.2 (one screen or a printed half-page per table).

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
`StarletteDeprecationWarning` printed when importing `fastapi.testclient`	A deprecation notice inside the installed Starlette	Harmless (stderr only). The labs filter it; say so once.
`ModuleNotFoundError: No module named 'schemas'` in the notebook	`schemas.py` was saved in another folder than the notebook's working folder	`print(Path.cwd())`; save both files next to the notebook (the `serving/` folder) and restart the kernel.
Changes to `serve.py` do not show up in the notebook	Python imported the module once and caches it	Restart the kernel (or `importlib.reload(serve)`); with uvicorn, `--reload` restarts on save.
`ERROR: [Errno 98] Address already in use` (or 10048 on Windows) from uvicorn	A server from earlier still runs on port 8000	Ctrl+C in its terminal, or `uvicorn serve:app --port 8001`.
`'uvicorn' is not recognized` / `command not found`	The terminal's environment is not active	Activate `.venv`; or `python -m uvicorn serve:app --reload`.

SYMPTOM	CAUSE	FIX
`UntrustedTypesFoundException` when loading the skops file	The file needs a type that is not in the trusted list	Print `sio.get_untrusted_types(file=...)`, review it, write the list out. Never pass the file's list blindly.
`InconsistentVersionWarning` while loading the model	The file was saved with another scikit-learn version	Treat as an error: reinstall the pinned versions or re-save the model in this environment.
`ValueError: Out of range float values are not JSON compliant: nan`	A DataFrame with NaN sent straight to the client	`to_json_records`: missing values travel as `None`.
The 13.1 randomized search runs a minute or more	Slow laptop, or another heavy program	Fine: start it and keep talking; or set `n_iter=5` (the best settings may differ slightly; say so).
Windows asks to allow Python through the firewall when uvicorn starts	A server opens a port	Allow on private networks only, or cancel: 127.0.0.1 works either way.

SESSION 11 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Lessons 11.2 and 12.1; homework check: who has `churn_final.skops` and `churn_final.json` from the 12.1 project.
0:10	10	**12.2** Frame and mental model	The kitchen and the order window; four things to watch with and without labels.
0:20	30	**12.2** Live demo	Load like a server, serving metadata, the schema, FastAPI and TestClient, the match test, a silent skew, PSI and prediction drift.
0:50	20	**12.2** Guided lab	`labs/module-11/l12_2-serving/fraud_api_starter.py`; fast finishers `fraud_monitor_starter.py` or `debug_serving.py`.
1:10	10	**12.2** Start the lesson project	The weekly drift report, Steps 1 to 3; Steps 4 to 6 go home.
1:20	10	Break	Leave the four-row monitoring table on the board.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
1:30	10	**13.1** Frame and mental model	The expedition plan: destination and budget first, boots last; the brief in dollars.
1:40	30	**13.1** Live demo	Break-even, explore, split and two policies, the leak check, compare, tune, threshold by cost, open the test set once, explain, the memo.
2:10	20	**13.1** Guided lab	`labs/module-11/l13_1-case-study/leak_audit_starter.py`, then `cost_scenarios_starter.py` in pairs.
2:30	10	**13.1** Start the lesson project	The retention campaign plan, Steps 1 and 2; Steps 3 to 6 go home.
2:40	10	Capstone choice and one-paragraph plan	Each learner picks one of the five problems in Lesson 13.2 and writes the plan paragraph in the workbook.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 12.
Total	180		Ends at 3:00.

In the room. The pages estimate 105 + 120 = 225 minutes for a solo learner; the room has 140 for the two lessons (70 each), plus ten minutes for the capstone choice. So the room protects the mental model, the demo, one kit lab and the first project steps of each lesson.

Goes home (2 to 3 hours). The weekly drift report Steps 4 to 6 and the retention campaign plan Steps 3 to 6; Exercises 1 to 3 of each lesson (Exercise 4 is a stretch); both quizzes and journal entries; reading the whole of Lesson 13.2 and finishing the capstone plan (Milestone 1).

Protect, in this order: the capstone choice segment; the 12.2 match test (Step 9) and the silent skew (Step 10); the 13.1 leak check, the threshold by cost and the one look at the test set (Steps 4, 7, 8). **Cut first:** 12.2 Steps 11 and 13 (say the three answers in one breath), then 13.1 Step 2 (the shapes are on the page), then the 13.1 project start (send all six steps home).

WARM-UP

0:00 Retrieval from Lessons 11.2 and 12.1 — 10 min

FIVE QUESTIONS (WITH ANSWERS)

1. Lesson 11.2 hid each customer's LATEST purchase for testing, not a random one. Why? **Answer:** It mimics the real job (predict the next purchase from earlier ones) and keeps future purchases out of the training matrix: the same rule as splitting by time in Lesson 1.2.
2. Best sellers scored a hit rate at 10 of 0.209 and item-kNN 0.308. Say what 0.308 means in one sentence. **Answer:** For about 31% of customers, the product they really bought next was in their personal top-10 list (already-owned products excluded): about 50% more often than the best-seller list.
3. Why did SVD with 40 factors recommend worse than with 5? **Answer:** With many factors it memorizes each customer's past basket instead of broad tastes, so it generalizes worse (0.19 against 0.311 at k = 5).
4. A colleague emails you `churn_model.joblib`. What is the risk of loading it, and what did Lesson 12.1 use instead? **Answer:** A joblib file is a pickle: loading it can run arbitrary code. Lesson 12.1 saved with skops, which loads only reviewed types passed to `trusted=`.
5. Your model card stores a SHA-256 of `customers_churn.csv`. Today the hash differs. What do you know, and what don't you? **Answer:** The file is not byte-for-byte the one the model was trained on; the hash cannot say what changed (one row, the order, a column).

HOW TO RUN IT

Questions on the slide, four minutes on paper (workbook Warm-up box), then answers called out. While they write, ask who has the 12.1 files (`churn_final.skops`, `churn_final.json`); the others will run the recreate block in demo Step 1.

12.2 Serving and Monitoring Models

70 min

OBJECTIVES (FROM THE LESSON PAGE)

- Package a fitted Pipeline with the metadata a server needs (features, dtypes, threshold, version)
- Describe valid input with a pydantic model and read its validation errors
- Serve predictions with FastAPI, test every endpoint with TestClient, and start the server with uvicorn
- Measure input drift with the Population Stability Index and prediction drift with simple batch statistics
- Tell a data-quality bug from a real change in customers, and say which one needs labels to confirm

TIMING IN THE ROOM

10m	Frame: the kitchen and the order window; the four things to watch
30m	Live demo: Steps 1 to 13
20m	Guided lab: `fraud_api_starter.py`
10m	Start the project: the weekly drift report, Steps 1 to 3

MENTAL MODEL TO SAY

"The fitted model is a kitchen that can cook one dish. Customers never walk into the kitchen: they hand an order through a window, and the person at the window checks it first. Is every field filled in? Is Denver one of our stores? Is a discount share of 1.4 even possible? Bad orders go back with a note; good ones go to the kitchen. FastAPI is the window, pydantic is the person checking orders, your Pipeline is the kitchen."

Then point at the monitoring table on the board: "A deployed model does not wear out. It fails quietly: the customers change, or a pipe breaks, and it keeps answering with status 200. Pinecrest only learns who churned six months later, so we watch what we can see now: inputs, data quality, the scores."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Work in the `serving/` folder: the two files in the editor, everything else in `lesson_12_2`. Optional, if time allows after Step 13: run `uvicorn serve:app --reload` in the terminal and show `http://127.0.0.1:8000/docs` (the lesson does not run it; port and log differ per machine).

1. **Load the model the way a server will (3m).** "A server cannot look back into your notebook. It reads the card first, checks the data fingerprint, and trusts only the types a person reviewed." If anyone lacks the 12.1 files, they run the lesson's recreate block first (it prints `test AUC 0.8612`).

```
from sklearn.dummy import DummyClassifier
with open("churn_final.json") as f:
    card = json.load(f)
churn_model = sio.load(card["model_file"], trusted=card["skops_trusted_types"]
print(card["estimator"], card["test_auc"], card["skops_trusted_types"])
print("data unchanged:", file_sha256(card["config"]["data_file"]) == card["dat
cfg = card["config"]
NUMERIC, CATEGORICAL = cfg["numeric"], cfg["categorical"]
FEATURES = NUMERIC + CATEGORICAL
customers = pd.read_csv(cfg["data_file"])
X_train, X_test, y_train, y_test = train_test_split(
    customers[FEATURES], customers[cfg["target"]], test_size=cfg["test_size"],
    stratify=customers[cfg["target"]], random_state=cfg["seed"])
test_auc = roc_auc_score(y_test, churn_model.predict_proba(X_test)[:, 1])
baseline = DummyClassifier(strategy="prior").fit(X_train, y_train)
print(f"baseline AUC {roc_auc_score(y_test, baseline.predict_proba(X_test)[:,
```

```
forest 0.8612 ['numpy.dtype', 'sklearn.tree._tree.Tree']
data unchanged: True
baseline AUC 0.500   model AUC 0.8612
```

Same AUC as the card: the model survived the trip. Ask: "What two things in the card make this load safe?" (the reviewed trusted types; the matching data hash).

2 **What a server needs beyond the card (4m).** "Four facts the notebook knows and the server does not: the features, their dtypes, the threshold from Lesson 1.4's costs, a version. And what normal looks like." Stress why `cross_val_predict`: predictions on training rows are over-confident.

```
from sklearn.model_selection import cross_val_predict
oof_proba = cross_val_predict(churn_model, X_train, y_train, cv=5, method="pre
THRESHOLD = 0.25
serving = {
    "model_card": "churn_final.json",
    "version": "churn-forest-2025-06-30",
    "features": FEATURES,
    "dtypes": {col: str(dtype) for col, dtype in X_train.dtypes.items()},
    "threshold": THRESHOLD,
    "reference_mean_probability": round(float(oof_proba.mean()), 4),
    "reference_offer_rate": round(float((oof_proba >= THRESHOLD).mean()), 4),
}
with open("churn_serving.json", "w") as f:
    json.dump(serving, f, indent=2)
reference = X_train.assign(churn_probability=oof_proba)
reference.to_csv("churn_reference.csv", index=False)
print(serving["dtypes"]["age_group"], serving["dtypes"]["email_opens_90d"], se
print(serving["reference_mean_probability"], serving["reference_offer_rate"])
```

```
str float64 int64
0.1662 0.2436
```

Write `0.1662` and `24.4%` on the board: the reference every later batch is compared with.

3 **The order window: schemas.py (3m).** Create `schemas.py` in the editor, not a notebook cell. "Every limit comes from the data dictionary, not from the model." Point at `extra="forbid"`, the two optional fields and the `Literal` stores.

```python
"""schemas.py: what a valid request to the churn API looks like."""
from typing import Literal

from pydantic import BaseModel, ConfigDict, Field

Store = Literal["Online", "Boise", "Bend", "Flagstaff", "Asheville"]
Region = Literal["Northwest", "Southwest", "Southeast", "Mountain", "Midwest",
AgeGroup = Literal["18-24", "25-34", "35-44", "45-54", "55-64", "65+"]
Tier = Literal["Basic", "Silver", "Gold"]
Category = Literal["Accessories", "Apparel", "Camp Kitchen", "Climbing", "Foot
                   "Packs", "Sleeping", "Snow", "Tents"]


class Customer(BaseModel):
    """One customer's 12-month summary, in the same units the model was traine
    model_config = ConfigDict(extra="forbid")      # unknown fields are an err
    tenure_months: int = Field(ge=1, le=600)
    orders_12m: int = Field(ge=0, le=500)
    spend_12m: float = Field(ge=0)
    avg_order_value: float = Field(ge=0)
    days_since_last_order: int = Field(ge=0, le=365)
    returns_12m: int = Field(ge=0)
    return_rate: float = Field(ge=0, le=1)
    support_tickets_12m: int = Field(ge=0)
    discount_share: float = Field(ge=0, le=1)
    app_user: int = Field(ge=0, le=1)
    email_opens_90d: float | None = Field(default=None, ge=0)   # may be unkno
    home_store: Store
    region: Region
    age_group: AgeGroup | None = None                           # may be unkno
    loyalty_tier: Tier
    preferred_category: Category


class Score(BaseModel):
    """What the API sends back for one customer."""
    churn_probability: float
    offer: bool
    model_version: str
```

Ask: "Which two fields may be missing, and why those two?" (they are missing in training too).

4 **pydantic checks and converts (2m).** "Predict first: what type comes back for 5, and does the text \"5\" fail?"

```
from pydantic import ValidationError
from schemas import Customer
example = {
    "tenure_months": 90, "orders_12m": 5, "spend_12m": 1378.70, "avg_order_val
    "days_since_last_order": 53, "returns_12m": 1, "return_rate": 0.306, "supp
    "discount_share": 0.14, "app_user": 0, "email_opens_90d": 5, "home_store":
    "region": "Southeast", "age_group": "25-34", "loyalty_tier": "Gold", "pref
}
customer = Customer(**example)
print(customer.email_opens_90d, type(customer.email_opens_90d).__name__)
print(Customer(**{**example, "orders_12m": "5"}).orders_12m + 1)
```

```
5.0 float
6
```

Lax mode converts: 5 becomes 5.0, "5" becomes 5. Friendly to callers that send numbers as text.

5 **A bad order, every problem at once (2m).** "This is what a caller needs: not 'something went wrong', but 'fix these four fields'."

```
bad = {**example, "discount_share": 1.4, "home_store": "Denver", "app_user": "
del bad["tenure_months"]
try:
    Customer(**bad)
except ValidationError as err:
    print(err.error_count(), "problems")
    for problem in err.errors():
        print(problem["loc"], "->", problem["msg"])
```

```
4 problems
('tenure_months',) -> Field required
('discount_share',) -> Input should be less than or equal to 1
('app_user',) -> Input should be a valid integer, unable to parse string as a
('home_store',) -> Input should be 'Online', 'Boise', 'Bend', 'Flagstaff' or
```

6 **serve.py: the whole service (4m).** Create `serve.py`. Read it top to bottom with them: the model loads ONCE at start-up; `to_frame` is the most important function (columns, order AND dtypes); three endpoints.

```
"""serve.py: the Pinecrest churn-scoring API."""
import json

import pandas as pd
import skops.io as sio
from fastapi import FastAPI

from schemas import Customer, Score

with open("churn_serving.json") as f:
    SERVING = json.load(f)
with open(SERVING["model_card"]) as f:
    CARD = json.load(f)
MODEL = sio.load(CARD["model_file"], trusted=CARD["skops_trusted_types"])   #
FEATURES = SERVING["features"]
DTYPES = SERVING["dtypes"]
THRESHOLD = SERVING["threshold"]
VERSION = SERVING["version"]

app = FastAPI(title="Pinecrest churn scoring", version=VERSION)


def to_frame(customers: list[Customer]) -> pd.DataFrame:
    """Validated customers to a DataFrame with the training columns, order and
    rows = [c.model_dump() for c in customers]
    return pd.DataFrame(rows, columns=FEATURES).astype(DTYPES)


def to_score(p: float) -> Score:
    return Score(churn_probability=round(float(p), 4), offer=bool(p >= THRESHO


@app.get("/health")
def health() -> dict:
    return {"status": "ok", "model_version": VERSION}


@app.post("/predict")
def predict(customer: Customer) -> Score:
    return to_score(MODEL.predict_proba(to_frame([customer]))[0, 1])


@app.post("/predict_batch")
def predict_batch(customers: list[Customer]) -> list[Score]:
    return [to_score(p) for p in MODEL.predict_proba(to_frame(customers))[:, 1
```

Say: "If this file has a bug, it is almost always in `to_frame`."

7 **Test it without a server (2m).** "TestClient sends requests straight to the app object: no network, milliseconds." Some machines print a `StarletteDeprecationWarning` on the import: harmless (trap table).

```
import warnings
warnings.filterwarnings("ignore", message=".*Starlette.*")
from fastapi.testclient import TestClient
import serve
client = TestClient(serve.app)
print(client.get("/health").json())
response = client.post("/predict", json=example)
print(response.status_code, response.json())
```

```
{'status': 'ok', 'model_version': 'churn-forest-2025-06-30'}
200 {'churn_probability': 0.0775, 'offer': False, 'model_version': 'churn-fore
```

8 **The bad order through the API (1m).** "422: the model never ran."

```
response = client.post("/predict", json=bad)
print(response.status_code)
for problem in response.json()["detail"]:
    print(problem["loc"], "->", problem["msg"])
```

```
422
['body', 'tenure_months'] -> Field required
['body', 'discount_share'] -> Input should be less than or equal to 1
['body', 'app_user'] -> Input should be a valid integer, unable to parse strin
['body', 'home_store'] -> Input should be 'Online', 'Boise', 'Bend', 'Flagsta
```

9 **A batch, and the most valuable test (3m).** "NaN is not JSON, so missing values travel as null." Then the match test: the API's answers against the notebook's for the same customers.

```
def to_json_records(frame):
    """DataFrame rows as dicts, with missing values as None (JSON null)."""
    return frame.astype(object).where(frame.notna(), None).to_dict(orient="rec
week = X_test.head(5)
scores = pd.DataFrame(client.post("/predict_batch", json=to_json_records(week)
print(scores)
direct = churn_model.predict_proba(week)[:, 1]
print("API matches the notebook:", np.allclose(scores["churn_probability"], di
```

```
   churn_probability  offer            model_version
0             0.0775  False  churn-forest-2025-06-30
1             0.3810   True  churn-forest-2025-06-30
2             0.1457  False  churn-forest-2025-06-30
3             0.1908  False  churn-forest-2025-06-30
4             0.1178  False  churn-forest-2025-06-30
API matches the notebook: True
```

Say: "Keep a test like the last line next to every model you deploy."

10 **A bug with no error (2m).** "Build the one-row frame WITHOUT the stored dtypes. Watch the probability."

```
row = {**example, "age_group": None}
no_dtypes = pd.DataFrame([row], columns=FEATURES)
with_dtypes = no_dtypes.astype(serving["dtypes"])
print(no_dtypes["age_group"].dtype, round(churn_model.predict_proba(no_dtypes)
print(with_dtypes["age_group"].dtype, round(churn_model.predict_proba(with_dty
```

```
object 0.0719
str 0.0708
```

Training-serving skew: a None makes the column `object`, the imputer passes it through, the encoder treats it as unknown. Small here; it can flip an offer.

11 **A rough season, simulated (2m).** "Half the customers hit by a 60-day delay, a long sale, a broken email tracker."

```
rng = np.random.default_rng(42)       # seeded, so the simulation is the same
stable = X_test.copy()
shifted = X_test.copy()
delayed = rng.random(len(shifted)) < 0.5
shifted.loc[delayed, "days_since_last_order"] = (shifted.loc[delayed, "days_si
shifted["discount_share"] = (shifted["discount_share"] * 1.5).clip(upper=0.833
lost_email = rng.random(len(shifted)) < 0.25
shifted.loc[lost_email, "email_opens_90d"] = np.nan
for name, frame in [("reference", X_train), ("stable", stable), ("shifted", sh
    print(f"{name:10s} days {frame['days_since_last_order'].mean():5.1f}   "
          f"discount {frame['discount_share'].mean():.3f}   email missing {fra
```

```
reference  days  83.5   discount 0.209   email missing 9.3%
stable     days  83.6   discount 0.204   email missing 7.9%
shifted    days 112.7   discount 0.299   email missing 31.8%
```

12 **PSI points at what moved (3m).** Write the PSI formula and the 0.1 / 0.25 conventions on the board. "Chosen before you look."

```
def psi(reference, current, bins=10):
    """Population Stability Index of one numeric column; missing values get th
    edges = np.unique(np.quantile(reference.dropna(), np.linspace(0, 1, bins +
    edges[0], edges[-1] = -np.inf, np.inf
    def shares(values):
        counts = np.append(np.histogram(values.dropna(), bins=edges)[0], value
        return np.clip(counts / len(values), 1e-4, None)      # avoid log(0) f
    ref, cur = shares(reference), shares(current)
    return float(np.sum((cur - ref) * np.log(cur / ref)))
drift = pd.DataFrame({
    "psi_stable": {col: psi(X_train[col], stable[col]) for col in NUMERIC},
    "psi_shifted": {col: psi(X_train[col], shifted[col]) for col in NUMERIC},
})
drift["status"] = pd.cut(drift["psi_shifted"], [-np.inf, 0.1, 0.25, np.inf], l
print(drift.round(3))
```

```
                      psi_stable  psi_shifted  status
tenure_months              0.008        0.008  stable
orders_12m                 0.001        0.001  stable
spend_12m                  0.005        0.005  stable
avg_order_value            0.012        0.012  stable
days_since_last_order      0.002        0.336   SHIFT
returns_12m                0.000        0.000  stable
return_rate                0.008        0.008  stable
support_tickets_12m        0.001        0.001  stable
discount_share             0.013        0.200   watch
app_user                   0.000        0.000  stable
email_opens_90d            0.009        0.350   SHIFT
```

Untouched features stay stable; recency and email opens SHIFT, discount share 'watch'. That is where the investigation starts.

13 **Prediction drift, against the reference (2m).** "More customers cross 0.25: the offer budget was planned for 24%."

```
batch_scores = {}
for name, frame in [("stable", stable), ("shifted", shifted)]:
    batch_scores[name] = pd.DataFrame(client.post("/predict_batch", json=to_js
    s = batch_scores[name]
    print(f"{name:9s} mean probability {s['churn_probability'].mean():.3f}   o
print(f"reference mean probability {serving['reference_mean_probability']:.3f}
      f"offers {serving['reference_offer_rate']:.1%}")
```

```
stable    mean probability 0.163   offers 24.1%
shifted   mean probability 0.209   offers 35.7%
reference mean probability 0.166   offers 24.4%
```

Close with the three answers: email = data bug (fix the feed), delay = real change (someone decides about budget), sale = possible concept drift (only labels confirm).

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **NaN sent as JSON.** `client.post(..., json=frame.to_dict("records"))` with gaps gives `ValueError: Out of range float values are not JSON compliant: nan`. Coach: "Which value is not JSON?" → `to_json_records`.
- **The frame built without the stored columns or dtypes.** No error, a slightly different number (Step 10). Coach with the match test: "Does the API agree with the notebook on rows WITH gaps?"
- **Drift read as a verdict.** "PSI is high, retrain!" Coach: "Data bug or real change? Which labels would you retrain on?" A broken feed needs a fix; retraining on it teaches the model the bug.

GUIDED LAB

- **File:** `labs/module-11/l12_2-serving/fraud_api_starter.py` (reference `labs/module-11/l12_2-serving/fraud_api_solution.py`). The same three habits on a NEW model: card fraud. A model trained on January to April, a threshold at the review team's capacity (the riskiest 1% of May), a `Payment` schema, `/score` and `/score_batch`, then June through the API.
- **Expected:** threshold 0.0775; `bad` has 4 problems and the API answers 422; the API matches the notebook (`True`); June: 83 of 8,262 held (1.00%), 35 of the 64 frauds among them.
- **A teaching moment in TODO 1:** skops asks to trust `functools.partial` and `check_array` besides the tree predictor (they come from `categorical_features="from_dtype"`). Ask: "Would you trust `functools.partial` from a stranger's file?" That is why the list is reviewed and written out, never copied from the file.
- **Circulate for:** a threshold set at 0.5 (nothing is held: fraud scores are tiny); categories rebuilt without the training list; `predict` used for holds.
- **Fast finishers:** `labs/module-11/l12_2-serving/fraud_monitor_starter.py`: REAL months, no simulation. Every PSI stays under 0.01 and average precision is steady as the fraud rate falls; then a broken `velocity_24h` feed drops the hold rate to 0.25% and the frauds held from 19 to 10. Or `labs/module-11/l12_2-serving/debug_serving.py`: four bugs between JSON and the model.

START THE LESSON PROJECT

The project is **a weekly drift report**: a `monitor.py` module, three simulated weeks scored through `/predict_batch`, and a note per week. In the room: Steps 1 to 3 (load, write `monitor.py`, simulate).

- **Done in the room:** `churn-forest-2025-06-30 (4500, 17) (1500, 16)`; `monitor.py` saved; tenure 44.0 / 25.0 / 42.7 and email missing 8% / 6% / 100% for weeks 1 to 3.
- **Goes home:** Steps 4 to 6: 500 scores per week with status 200; the summary table (week 2 tenure PSI 0.383, week 3 email PSI 8.368); `drift_report.csv` and three different actions.

CHECKS FOR UNDERSTANDING

- Why does the model load once, at start-up? **Answer:** loading takes far longer than predicting; per request it would make every call slow.
- Which monitoring check needs labels? **Answer:** performance (AUC, offer precision), months later. Input drift, data quality and prediction drift do not.
- Week 3's PSI is 8.4 but the scores barely moved. How? **Answer:** the imputer filled every missing email value with the median; only the missing-rate and PSI checks warned.

Workbook self-check, Q1 to Q6:

Answers: **Q1** B) pydantic rejects it before the model runs: status 422, and the body names `home_store` and the allowed stores; **Q2** `60 1.0`; **Q3** D) A one-row table with a `None` gets the `object` dtype, the imputer does not see it as missing, and the model scores a slightly different customer with no error; **Q4** A) A broken data feed: fix it and re-score the week; do not retrain on it; **Q5** C) `0.416`; **Q6** B) No: with enough rows any difference is detectable; PSI says the shift is tiny, and an alert should be about size. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: pairs share one keyboard for the lab; give strugglers the schema and `to_frame` from the lesson to adapt rather than write. **Stretch:** 12.2 Exercise 4 (KS vs PSI) and the Bonus Challenge (request logging, then the label check); `debug_serving.py`.

13.1 Case Study: Predicting Churn

70 min

OBJECTIVES (FROM THE LESSON PAGE)

- Translate a business goal into a target, a metric and a dollar value for each kind of right and wrong prediction
- Detect a target leak with cross-validation and a crosstab, and remove it before modeling
- Compare and tune models inside a Pipeline with cross-validation, keeping the test set locked until one final check
- Choose a decision threshold from business costs with TunedThresholdClassifierCV and report the result in dollars
- Explain the model honestly and write a short recommendation for a non-technical stakeholder

TIMING IN THE ROOM

10m	Frame: the expedition plan; the brief in dollars
30m	Live demo: Steps 1 to 12
20m	Guided lab: `leak_audit_starter.py`, then `cost_scenarios_starter.py` in pairs
10m	Start the project: the retention campaign plan, Steps 1 and 2

MENTAL MODEL TO SAY

"A mountain guide does not start by choosing boots. Destination and budget first, then hazards on the route, a fallback plan, and only then packing. At the end, a report to the people who paid, in their words. In a machine-learning project the algorithm is the boots: important, chosen late, and rarely what the client asks about."

Then the brief: "The manager asks: who should get the offer, and what will it earn? That is a decision, not a prediction. Every step today serves that decision."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook `lesson_13_1`. Start Step 6's search and keep talking; it takes about 20 seconds on a laptop, longer on slow ones. Ask for a prediction before Steps 4, 5, 7 and 8.

1. **The brief in dollars (2m).** "Start from the decision and its costs. They set the metric, the baselines and even the threshold."

```
OFFER_COST = 15     # dollars per customer contacted
CHURNER_VALUE = 60  # expected dollars per real churner contacted
print(f"net gain per churner offered     : ${CHURNER_VALUE - OFFER_COST}")
print(f"net cost per non-churner offered : ${OFFER_COST}")
print(f"break-even churn probability     : {OFFER_COST / CHURNER_VALUE:.2f}")
```

```
net gain per churner offered     : $45
net cost per non-churner offered : $15
break-even churn probability     : 0.25
```

Write the four-row outcome table: TP +$45, FP -$15, FN $0, TN $0. Accuracy is not on the list.

2 **Explore with a question in mind (3m).** "Is there signal, and what shape is it?"

```
import numpy as np
import pandas as pd
pd.set_option("display.width", 150)
customers = pd.read_csv("data/customers_churn.csv")
print(customers.shape, f"churn rate {customers['churned'].mean():.1%}")
recency = pd.cut(customers["days_since_last_order"], [-1, 30, 90, 150, 250, 36
print(customers.groupby(recency, observed=True)["churned"].mean().round(3))
```

```
(6000, 20) churn rate 16.5%
days_since_last_order
(-1, 30]      0.091
(30, 90]      0.095
(90, 150]     0.152
(150, 250]    0.415
(250, 364]    0.558
Name: churned, dtype: float64
```

A threshold near 150 days, not a slope. Mention the other two shapes in one breath (tickets × app interaction, tenure U): they argue for trees.

3 **Split once, and the two policies that need no model (3m).** "Lock the test set away now."

```
from sklearn.model_selection import train_test_split
NUMERIC = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value", "day
           "returns_12m", "return_rate", "support_tickets_12m", "discount_shar
CATEGORICAL = ["home_store", "region", "age_group", "loyalty_tier", "preferred
FEATURES = NUMERIC + CATEGORICAL
train, test = train_test_split(customers, test_size=0.25, stratify=customers["
as_float = {col: float for col in NUMERIC}
X_train, y_train = train[FEATURES].astype(as_float), train["churned"]
X_test, y_test = test[FEATURES].astype(as_float), test["churned"]
def campaign_profit(y_true, flagged):
    """Expected profit of contacting every flagged customer."""
    y_true, flagged = np.asarray(y_true), np.asarray(flagged)
    tp = ((flagged == 1) & (y_true == 1)).sum()
    return CHURNER_VALUE * tp - OFFER_COST * flagged.sum()
print(X_train.shape, X_test.shape)
print("offer to nobody  ", campaign_profit(y_train, np.zeros(len(y_train), dty
print("offer to everyone", campaign_profit(y_train, np.ones(len(y_train), dtyp
```

```
(4500, 16) (1500, 16)
offer to nobody   0
offer to everyone -22860
```

Offering to everyone loses about $23,000 on 4,500 customers: five offers in six go to stayers.

4 **The leak check (4m).** "Nobody tells you which column is the trap. Add the suspicious one and see."

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score
def make_prep(numeric, categorical):
    return ColumnTransformer([
        ("num", make_pipeline(SimpleImputer(strategy="median"), StandardScaler
        ("cat", make_pipeline(SimpleImputer(strategy="most_frequent"),
                              OneHotEncoder(handle_unknown="ignore", sparse_ou
    ])
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
with_reason = train.assign(cancellation_reason=train["cancellation_reason"].fi
leaky = make_pipeline(make_prep(NUMERIC, CATEGORICAL + ["cancellation_reason"]
honest = make_pipeline(make_prep(NUMERIC, CATEGORICAL), LogisticRegression(max
for name, pipe, data in [("with cancellation_reason", leaky, with_reason), ("w
    scores = cross_val_score(pipe, data, y_train, cv=cv, scoring="roc_auc")
    print(f"{name:25s} CV AUC {scores.mean():.3f}")
print(pd.crosstab(train["cancellation_reason"].notna(), train["churned"]))
```

```
with cancellation_reason  CV AUC 0.937
without                   CV AUC 0.832
churned                 0    1
cancellation_reason
False                3756  278
True                    0  466
```

0.937 against 0.832 and a zero in the stayers' column: recorded after the outcome. Today's lab turns this into a function for every column.

5 **Compare fairly (4m).** "Same preprocessing, same folds, same metric." Predict the ranking before running.

```
from sklearn.dummy import DummyClassifier
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClass
candidates = {
    "baseline (prior)": DummyClassifier(strategy="prior"),
    "logistic regression": LogisticRegression(max_iter=1000),
    "random forest": RandomForestClassifier(n_estimators=200, min_samples_leaf
    "hist gradient boosting": HistGradientBoostingClassifier(random_state=42),
}
cv_auc = {}
for name, model in candidates.items():
    scores = cross_val_score(make_pipeline(make_prep(NUMERIC, CATEGORICAL), mo
    cv_auc[name] = scores
    print(f"{name:23s} AUC {scores.mean():.3f} +/- {scores.std():.3f}")
```

```
baseline (prior)        AUC 0.500 +/- 0.000
logistic regression     AUC 0.832 +/- 0.023
random forest           AUC 0.868 +/- 0.020
hist gradient boosting  AUC 0.861 +/- 0.013
```

Forest and boosting are within one spread: honestly undecided.

6 **Tune the one with room to improve (4m).** "Note the `model__` prefix." The search takes about 20 seconds: talk through why boosting's defaults suit bigger data.

```
from sklearn.model_selection import RandomizedSearchCV
from sklearn.pipeline import Pipeline
hgb_pipe = Pipeline([("prep", make_prep(NUMERIC, CATEGORICAL)),
                     ("model", HistGradientBoostingClassifier(random_state=42)
param_distributions = {
    "model__learning_rate": [0.03, 0.05, 0.1, 0.2],
    "model__max_leaf_nodes": [7, 15, 31],
    "model__min_samples_leaf": [20, 40, 80],
    "model__l2_regularization": [0.0, 0.1, 1.0],
    "model__max_iter": [100, 200, 300],
}
search = RandomizedSearchCV(hgb_pipe, param_distributions, n_iter=10, cv=cv, s
search.fit(X_train, y_train)
print(search.best_params_)
print(f"best CV AUC {search.best_score_:.3f}  (defaults: {cv_auc['hist gradien
best_model = search.best_estimator_
```

```
{'model__min_samples_leaf': 80, 'model__max_leaf_nodes': 7, 'model__max_iter'
best CV AUC 0.879  (defaults: 0.861)
```

The gentlest settings won: small trees, slow learning, 80 customers per leaf.

7 **Threshold by cost, confirmed on data (3m).** "Theory says 0.25 if the probabilities are honest. Check it."

```
from sklearn.metrics import make_scorer
from sklearn.model_selection import TunedThresholdClassifierCV
profit_scorer = make_scorer(campaign_profit)
tuned = TunedThresholdClassifierCV(best_model, scoring=profit_scorer, cv=cv, s
tuned.fit(X_train, y_train)
print(f"best threshold {tuned.best_threshold_:.3f}   mean profit per validatio
curve = pd.Series(tuned.cv_results_["scores"], index=tuned.cv_results_["thresh
for t in [0.1, 0.25, 0.5]:
    print(f"threshold near {t:.2f}: ${curve.iloc[np.abs(curve.index - t).argmi
```

```
best threshold 0.233   mean profit per validation fold (900 customers) $3,373
threshold near 0.10: $2,284
threshold near 0.25: $3,302
threshold near 0.50: $2,358
```

Show the lesson's profit curve: flat from about 0.2 to 0.3; the default 0.5 gives up about a third.

8 **Open the test set, once (3m).** Make a small ceremony of it: "Every decision is made. No more changes after this."

```
from sklearn.metrics import roc_auc_score, precision_score, recall_score
test_proba = tuned.predict_proba(X_test)[:, 1]
offers = tuned.predict(X_test)
print(f"test AUC          {roc_auc_score(y_test, test_proba):.3f}")
print(f"offers sent       {offers.sum()} of {len(offers)}")
print(f"precision         {precision_score(y_test, offers):.3f}   (share of of
print(f"recall            {recall_score(y_test, offers):.3f}   (share of churn
print(f"model plan        ${campaign_profit(y_test, offers):7,}")
print(f"offer to everyone ${campaign_profit(y_test, np.ones(len(y_test), dtype
print(f"offer to nobody   ${campaign_profit(y_test, np.zeros(len(y_test), dtyp
```

```
test AUC          0.875
offers sent       384 of 1500
precision         0.477   (share of offers that reach a churner)
recall            0.738   (share of churners who get an offer)
model plan        $  5,220
offer to everyone $ -7,620
offer to nobody   $      0
```

0.875 against CV 0.879: the choices held up. About half the offers reach a churner, and at $15 against $60 that is plenty.

9 **Explain: what the model relies on (2m).** "The manager will ask 'why these customers?'"

```
from sklearn.inspection import permutation_importance
result = permutation_importance(tuned, X_test, y_test, scoring="roc_auc", n_re
importance = pd.Series(result.importances_mean, index=FEATURES).sort_values(as
print(importance.head(5).round(3))
```

```
days_since_last_order    0.155
discount_share           0.088
tenure_months            0.058
app_user                 0.016
support_tickets_12m      0.016
dtype: float64
```

Say the sentence that goes in every memo: these describe the model, not why customers leave.

10 **The memo, built from variables (2m).** "So it can never disagree with your analysis." Show the first lines; the full memo is on the lesson page.

```
plan_profit = campaign_profit(y_test, offers)
memo = f"""TO: Customer-care manager          RE: Who should get the retention
Recommendation: send the offer to customers whose predicted churn risk is abov
On 1,500 customers the model had never seen, that meant {offers.sum()} offers
{precision_score(y_test, offers):.0%} of the offers reached someone who was ab
{recall_score(y_test, offers):.0%} of all customers who left.
Expected value: about ${plan_profit:,} for every 1,500 customers, using your f
$60 per would-be leaver reached)."""
print(memo)
```

```
TO: Customer-care manager          RE: Who should get the retention offer?
Recommendation: send the offer to customers whose predicted churn risk is abov
On 1,500 customers the model had never seen, that meant 384 offers (26% of cus
48% of the offers reached someone who was about to leave, and the list include
74% of all customers who left.
Expected value: about $5,220 for every 1,500 customers, using your figures ($1
$60 per would-be leaver reached).
```

Point at the last line on the lesson page: the hold-out experiment.

11 **Mistake: deciding with predict() (1m).** "Same model, same ranking, a different cut-off."

```
default_offers = best_model.predict(X_test)        # predict uses 0.5
print(f"threshold 0.5   : {default_offers.sum()} offers, ${campaign_profit(y_t
print(f"tuned threshold : {offers.sum()} offers, ${campaign_profit(y_test, off
```

```
threshold 0.5   : 144 offers, $3,600
tuned threshold : 384 offers, $5,220
```

12 **Mistake: a threshold picked on the test set (1m).** "Grading your own homework with the answer key open."

```
grid = np.arange(0.05, 0.9, 0.01)
test_profits = [campaign_profit(y_test, (test_proba >= t).astype(int)) for t i
peek = grid[int(np.argmax(test_profits))]
print(f"threshold picked on the test set {peek:.2f}: ${max(test_profits):,} (l
print(f"threshold picked by CV           {tuned.best_threshold_:.2f}: ${campai
```

```
threshold picked on the test set 0.26: $5,280 (looks better)
threshold picked by CV           0.23: $5,220 (honest)
```

Only $60 here, because CV already chose well; it grows with smaller test sets and more knobs.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **The step name forgotten in a Pipeline search.** `ValueError: Invalid parameter 'learning_rate' for estimator Pipeline(steps=[('prep',...` Coach: "Which step owns `learning_rate`?" → `model__learning_rate`; `hgb_pipe.get_params().keys()` lists the names.
- **Rows with gaps dropped.** `dropna` keeps 5,007 of 6,000 customers, and the dropped ones churn more (0.216 vs 0.155). Coach: "Who did you throw away, and will they stop arriving in production?"
- **The default threshold, or a threshold tuned on the test set.** `predict` sends 144 offers for $3,600 instead of 384 for $5,220; the test-set threshold looks $60 better and is not. Coach: "Where did the number 0.5 come from? Which data chose your threshold?"

GUIDED LAB

- **Lab A (10m, everyone):** `labs/module-11/l13_1-case-study/leak_audit_starter.py` (reference `labs/module-11/l13_1-case-study/leak_audit_solution.py`). The case study checked one column by hand; this writes an `audit()` that screens EVERY column: cross-validated AUC of the column alone, "filled for only one class", and ID-like text.
- **Expected, Lab A:** churn flags `cancellation_reason` (0.815 alone; filled for 63% of churners and 0% of stayers) and `customer_id` (ID-LIKE, 0.500); the strongest honest column is `days_since_last_order` (0.715). Fraud: only `transaction_id` is flagged; `velocity_24h` is the strongest single column (0.782). **Debrief:** "A clean audit is a finding. Write it in your capstone's data section."

- **Lab B (10m, pairs):** `labs/module-11/l13_1-case-study/cost_scenarios_starter.py`. The case-study model without age, out-of-fold probabilities on the training customers. Four offer costs: does the best threshold follow cost / 60? Then the profit of each block of 100 offers, and one look at the test set for a $25 offer.
- **Expected, Lab B:** best thresholds 0.17 / 0.24 / 0.31 / 0.43 for $10 / $15 / $20 / $25 (break-even 0.167 / 0.250 / 0.333 / 0.417); the first losing block of 100 offers ends at offer 1,200 (lowest p 0.213); test, $25 offer: 0.25 gives 360 offers, profit $1,620, the new threshold 183 offers, profit $2,745.
- **Debrief:** the threshold belongs to the business numbers, so it must be re-derived when they change, and served with the model (Lesson 12.2's `churn_serving.json`).

START THE LESSON PROJECT

The project is **the retention campaign plan**: a $3,000 budget (200 offers), no `age_group`, a group check and an updated memo. In the room: Steps 1 and 2.

- **Done in the room:** `4500 1500 offers allowed: 200`; CV AUC with age 0.879, without 0.880.
- **Goes home:** Steps 3 to 6: 360 above break-even, lowest probability on the list 0.403; the group report; 131 churners reached and $4,860 against -$540 for 200 random offers; `campaign_list.csv` with IDs and priority only.

CHECKS FOR UNDERSTANDING

- Where does the 0.25 come from? **Answer:** $15 / $60: an offer pays when p × 60 > 15, if the probabilities are calibrated.
- Why not use accuracy here? **Answer:** with 83.5% stayers it rewards offering to nobody; AUC compares rankings, profit judges the decision.
- Name one thing you may NOT write in the memo. **Answer:** a cause ("discounts cause churn"); permutation importance describes the model.

Workbook self-check, Q7 to Q12:

Answers: **Q7** C) `0.417`; **Q8** `60`; **Q9** A) The data does not yet say which tree family is better: the gap is inside one fold-to-fold spread; **Q10** D) Split, leak check, compare with CV, tune with CV, threshold with CV, test once, explain and write up; **Q11** B) Hold back the offer from a random 10% of the list and compare who stays; **Q12** A) The 200 riskiest customers, and never anyone below the break-even probability. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: in Lab A, give the per-column loop and ask only for the flag rules. **Stretch:** 13.1 Exercise 4 (three hand-made features lift logistic regression from 0.832 to 0.877) and the Bonus Challenge (the save rate between 10% and 40%).

CAPSTONE CHOICE

2:40 Choose the Capstone Problem and Write the Plan

10 min

THE FIVE PROBLEMS (LESSON 13.2) AND THEIR TEMPLATE BASELINES

PROBLEM	DATA, SPLIT, METRIC	TEMPLATE BASELINE
A · Demand forecasting	`weekly_demand.csv`; train 2024, test 2025; MAE in units	13.419 (per-product mean)
B · Fraud review queue	`transactions.csv`; train Jan to May, test June; average precision + capacity	0.008 (no model)
C · Review triage	`reviews.csv` (3 stars dropped); stratified 75/25; accuracy + negative recall	0.745 (always positive)
D · Recommendations	`interactions.csv` + `products_ml.csv`; leave-last-out; hit rate at 10	0.209 (popular unowned)
E · Customer segments	`customers_churn.csv` behavior columns; no target; silhouette + stability + profiles	0.079 (loyalty tiers)

HOW TO RUN IT

- **2 minutes:** put the five problems on screen. Churn (13.1) and daily store sales (the lesson's worked example) are taken. Say: "Choose the one you are most curious about; curiosity carries you through the dull parts better than ease does. And 'do not deploy' can be a full answer."
- **6 minutes:** each learner writes the one-paragraph plan in the workbook's Capstone box: decision, user, target, prediction moment, split, metric and its business meaning, baseline, and what result would make them say "do not deploy". This is the start of Milestone 1.
- **2 minutes:** a show of hands per problem (A to E); write the counts on the board and photograph the plans (or collect them). Anyone undecided picks by the end of the wrap-up.
- **Coaching notes** for each problem (strong work, traps, honest negative results) are in `final-assessment.html`. Read them before Session 12; do not hand them out.
- **Own problem:** allowed if it passes the lesson's three checks (a decision, data and labels in `data/`, not an existing project). Approve it in writing before Session 12.

WHAT A GOOD PARAGRAPH SOUNDS LIKE (READ ONE ALOUD AS A MODEL)

"The two-person review team checks about 1% of online payments each day (83 in June). I will rank June payments by fraud risk with a model trained on January to May and report average precision against the no-model 0.008, plus how many frauds the top 1% holds. A payment is scored at checkout, so only fields known

then are features. I will not recommend the model if its top 1% holds no more frauds than a simple velocity rule."

CLOSING

EXIT TICKET (3 QUESTIONS, 4 MINUTES)

1. Name two places where a churn API can return a wrong number with status 200, and the check that catches each. **Answer:** a frame built without the training dtypes (the match test on real rows with gaps); a broken feed filled by the imputer (missing-rate and PSI checks); also an impossible value without schema limits (pydantic `Field` limits).
2. The offer cost rises to $20 (value $60). The new break-even probability? Would you trust it without a check? **Answer:** 0.333. Check it on out-of-fold predictions or with `TunedThresholdClassifierCV`, because it assumes calibrated probabilities (the lab's best was 0.31).
3. Your capstone: the problem letter, the decision it supports and the baseline number you must beat. **Answer:** for example "B: which 83 payments the team reviews each day; no-model average precision 0.008". Anyone without a letter gets a message from you tonight.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both projects.** 12.2 weekly drift report Steps 4 to 6 (`drift_report.csv`, three actions); 13.1 retention campaign plan Steps 3 to 6 (`campaign_list.csv`, the updated memo).
- **Exercises.** 12.2: 1 to 3 (rejections, a model-info endpoint, category PSI); 13.1: 1 to 3 (profit by store, the better model in dollars, calibration). Exercise 4 of each is a stretch.
- **Both Quick Quizzes** and both journal entries.
- **Before the Next Lesson:** 12.2: start the real server and send one request from `/docs`; change the threshold to 0.4 and watch the week-2 offer rate. 13.1: reread the memo as the manager; one sentence per other Pinecrest table.
- **Capstone (about 60 minutes):** read all of Lesson 13.2 (the brief, the five problems, the worked example, the model card skeleton, the checklist, the rubric). Finish Milestone 1: create `capstone.ipynb` with the plan cell, starting from today's paragraph.
- **Optional:** `fraud_monitor_starter.py`, `debug_serving.py`, and whichever lab you did not reach.

BRING LIST FOR SESSION 12

- A laptop with the course environment (nothing new to install) and the `pinecrest-ml` folder with `data/` and `labs/` (Session 12's files are in `labs/module-12/`).
- Your capstone folder with `capstone.ipynb` whose first cell holds the written plan (Milestone 1), and the problem letter you chose.

- Your Lesson 13.1 notebook (its structure is the capstone template) and the 12.2 files `serve.py`, `schemas.py`, `monitor.py` (for the optional serving milestone).
- The workbook and journal. No downloads.

IF YOU RUN LONG OR SHORT

RUNNING LONG

- **Never cut** the capstone choice segment: move the 13.1 project start home instead.
- **Cut first (saves 5m):** 12.2 Steps 11 and 13; show the PSI table from the page and say the three answers.
- **Cut second (saves 6m):** 13.1 Steps 2 and 9 (the shapes and the importance are on the page).
- **Cut third (saves 10m):** the 12.2 project start; give its check numbers instead.

RUNNING SHORT

- Run uvicorn and the `/docs` page live; send one request from the browser.
- `fraud_monitor_starter.py` for everyone: a model on real months that does not drift, and a feed that breaks.
- Let learners compare capstone plans in pairs with the five-question test: decision? prediction moment? split? metric? 'do not deploy' bar?

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 12: Capstone Kickoff and Studio I

How to run the capstone kickoff: frame Lesson 13.2's brief and rubric, test every plan in a clinic, and run a timed studio for Milestones 2 to 4 (data checks and the feature-timing table, two honest baselines and an evaluation harness, a first pipeline). The capstone reference numbers and coaching notes are in final-assessment.html.

1 lesson | 3 hrs session | Lesson 13.2 | Session 12 of 13 | Studio

HOW TO USE THIS GUIDE

- One lesson, no new tools. Sessions 12 and 13 are studios for Lesson 13.2 (480 minutes for a solo learner). Today: the kickoff and Milestones 1 to 4 (plan, data checks and split, baselines and harness, a first pipeline). Milestone 5 (improve) goes home; Session 13 does Milestones 6 to 8 and the showcase.
- Your answer notes are in `final-assessment.html` at the kit root: for each problem A to E the steps of a sound project, the reference numbers (baselines, good models, traps, honest negative results), follow-up questions and the rubric with the scoring rules. Read the notes for the problems your room chose (the Session 11 plan photos tell you the mix). Nothing from it is said aloud as an answer: the lesson has no answer key.
- Coach by asking. Every move in the studio is a question ("which columns do you know on the day you predict?"). The template already fixes the split, the metric and the baseline: today makes sure nobody quietly breaks them.
- The labs in `labs/module-12/l13_2-capstone/` are capstone tools, not exercises: a second honest baseline per problem and an evaluation harness with a first pipeline. Each learner runs only the part for their own problem; the solutions hold reference numbers for all five.

> **THE ONE RISK OF SESSION 12: AN EVALUATION SETUP THAT IS QUIETLY WRONG**
>
> The most expensive capstone mistake is made in the first hour and discovered in the last: a random split on time-ordered data, a feature known only after the prediction moment, a test item the customer already owned, a k chosen on the test set. Nothing errors, and every later number is wrong. **So the room protects Milestones 2 and 3:** the feature-timing table and the harness are checked by you or a partner before anyone builds a model. A learner who leaves with a correct harness and a modest first pipeline is in better shape than one with a tuned model on a broken split.

BEFORE THE SESSION

- ☐ Read `final-assessment.html`, at least the Kickoff traps and the reference numbers for the problems your learners chose. Group tables by problem if the room allows (two to four learners per table).
- ☐ Run `python3 labs/check_labs.py module-12`: 4 passed (about 15 seconds). The folder also holds `capstone_start.py`, the lesson's starter template as a module, which the two labs import.
- ☐ Traffic-light cards or three colors of sticky notes per table; a visible countdown timer; the rubric (lesson page, The Rubric) on the second screen.
- ☐ The board, before learners arrive: the five plan-clinic questions; a log grid (learner, problem, baseline, harness score, first pipeline) for the 2:40 milestone check.
- ☐ No downloads, no servers, no internet needed. Learners need their capstone folder with `data/` beside the notebook and `labs/module-12/`.
- ☐ Own problems approved in writing since Session 11 (the lesson's three checks: a decision, data and labels in `data/`, not an existing project).

SESSION 12 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Four retrieval questions on 12.2 and 13.1, then each learner's capstone sentence.
0:10	15	**13.2** Frame: the brief, the rubric and the plan	The trail guide's report; eight rubric rows; what the template fixes; today's clock.
0:25	20	**13.2** Plan clinic (Milestone 1)	Pairs swap plans and test them with five questions; fixes written into the plan cell.
0:45	40	**13.2** Studio I · Milestone 2: load, check, split	Check-ins at 0, 15 and 30 minutes; the feature-timing table.
1:25	10	Break	Restart the kernel and Run All once before leaving the room.
1:35	35	**13.2** Studio I · Milestone 3: baselines and harness	`labs/module-12/l13_2-capstone/baselines_starter.py` and `harness_starter.py`: the baseline measured in class.
2:10	30	**13.2** Studio I · Milestone 4: a first pipeline	The simplest sensible pipeline through the harness; check-ins at 0 and 15 minutes.
2:40	10	**13.2** Milestone check	Stand-up: each learner reads baseline, harness score and first-pipeline score; the board log.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
2:50	10	Wrap-up	Exit ticket (3 questions), Milestone 5 at home, bring list for Session 13.
Total	180		Ends at 3:00.

THE 480 MINUTES OF LESSON 13.2, SPLIT ACROSS THE KIT

MILESTONE (LESSON ESTIMATE)	MINUTES	WHERE IT HAPPENS IN THIS KIT
1 · Choose and plan	30	Session 11 (choice + paragraph, 10) and home; checked in the Session 12 plan clinic
2 · Load, check and split	40	Session 12 studio
3 · Baseline and evaluation harness	30	Session 12 studio
4 · A first pipeline	45	Session 12 studio (started), home (finished)
5 · Improve	90	Home, between Sessions 12 and 13
6 · Test once, then analyze the errors	45	Session 13 studio
7 · Explain and review	40	Session 13 studio
8 · Figures and model card	45	Session 13 (the figure and the first two card sections, 20), home (25)
9 · Reproduce, save and review	30	Home, within a week of Session 13
Optional · Serve it	60	Home, only after Milestone 9

In the room today: about 125 minutes of milestone work (plan clinic, Milestones 2 and 3, most of 4). At home before Session 13: finish Milestone 4 and do Milestone 5 (about 110 minutes, with a log of attempts). Learners must NOT open the test set at home: Milestone 6 happens in Session 13.

Protect, in this order: the feature-timing table (Milestone 2), the harness with the baseline in it (Milestone 3), the milestone check. **Cut first:** shorten the frame to 10 minutes and the clinic to 15; never cut the 0:30 check-in of Milestone 2.

WARM-UP

0:00 Retrieval from Session 11 and a Capstone Check — 10 min

FIVE QUESTIONS (WITH ANSWERS)

1. Why does `serve.py` rebuild the request frame with the stored training dtypes? **Answer:** Otherwise a None in a one-row table makes an object column the imputer does not treat as missing: a silently different score (training-serving skew).
2. Of input drift, data quality, prediction drift and performance, which needs labels? **Answer:** Only performance (AUC, offer precision), and the labels arrive months later.
3. Where does the churn threshold 0.25 come from, and what does it assume? **Answer:** $15 / $60: an offer pays when $p \times 60 > 15$. It assumes calibrated probabilities, which the case study confirmed with TunedThresholdClassifierCV.
4. In the case study, when is the test set used? **Answer:** Once, after features, model, hyperparameters and threshold are all chosen.
5. Your capstone in one sentence: the decision, and the baseline number you must beat. **Answer:** No single answer. Listen for a decision (not a topic) and a number; anyone without one gets the first visit in the plan clinic.

HOW TO RUN IT

W1 to W4 on paper for four minutes, answers called out in two. Then W5 around the room, ten seconds each; write problem letters on the board. Anyone with no decision or no number is your first visit in the plan clinic.

LESSON PLAN

13.2 Capstone Project

160 min

OBJECTIVES (FROM THE LESSON PAGE)

- Frame a business request as a machine-learning task with a decision, a target, a leakage-safe split, a metric and a baseline, written down before any modeling
- Build, compare and improve models in leakage-proof pipelines, touching the test set once
- Analyze errors by subgroup and explain the model honestly, without claiming causes
- Review your model against a responsible-ML checklist and document it in a model card
- Deliver a project that reruns from a fresh start and score it yourself against a rubric

FRAME · 15 MIN: THE TRAIL GUIDE'S REPORT

Say: "A guide who scouts a new route hands the club a short report: where it goes, how it compares with the old route, where it is dangerous, who should not attempt it. Your notebook is the scouting trip; the model card is the report. Pinecrest's teams will read the card, not the code, and their data team will rerun the notebook."

Put the rubric on screen and read the Proficient column of each row aloud. Then three sentences from the lesson, on the board: **"Do not deploy" can be a full answer. The one loop that is not allowed is from the test set back to tuning. The template is the agreed rules of the competition** (split, metric, baseline): change one only with a written reason.

Today's clock, said once: plan clinic, Milestone 2 with check-ins every 15 minutes, a break with Run All, Milestone 3 with the baselines measured in class, Milestone 4, and a stand-up where everyone reads three numbers.

PLAN CLINIC · 20 MIN (MILESTONE 1)

- **Pairs, ideally across problems (8 min):** each reads the other's plan cell and asks the five questions on the board: (1) Who makes which decision with the predictions, and how often?; (2) What is the prediction moment, and which columns are known then?; (3) Which split does the template use, and does it match the data's structure (time, groups)?; (4) Which metric, in the business's units, and what does the baseline score on it?; (5) What result would make you write "do not deploy"? Every "I don't know" becomes a line in the plan.
- **Rewrite (7 min):** each learner fixes their plan cell. Circulate to the learners who had no decision in the warm-up.
- **Two plans read aloud (5 min),** with permission: one strong, one that improved. Name the part that made it strong (usually the prediction moment or the "do not deploy" bar).

- **Done when** someone else could read the plan and predict roughly what the notebook will contain (the lesson's own done-when).

STUDIO I · 40 MIN · MILESTONE 2, LOAD, CHECK AND SPLIT

- **0:00 check-in (2 min).** Cards: green (data loaded with the template, split printed), amber (loaded, checks unclear), red (not loaded). Reds first; ambers pair with a green.
- **Work (13 min).** Shapes, dtypes, date ranges, missing values by group, duplicate keys, the target's distribution, odd values; one line of conclusion under each. The relevant part of `data/README.md`.
- **0:15 check-in (2 min).** One word: "timing", "quirks", "stuck". Then everyone starts the **feature-timing table**: every column, "known at prediction time? yes / no / only as a plan".
- **0:30 check-in (3 min).** Partners swap feature-timing tables for 60 seconds each: "Which yes would you challenge?" Coaching round on the challenged rows.
- **Done when** each learner can say what one row is, how many rows train and test hold, and which columns they will never use.

STUDIO I · 35 MIN · MILESTONE 3, BASELINES AND THE HARNESS

- **Baselines first (12 min):** `labs/module-12/l13_2-capstone/baselines_starter.py` with `PROBLEM` set to their letter. One more honest baseline on the same split and metric, and the "do not deploy" sentence written into the plan.
- **Reference (from the solution):** demand per product × store 12.953, same week last year 13.810 (template 13.419); fraud velocity rule average precision 0.250, 19 of 64 frauds in its top 1%; reviews keyword rule accuracy 0.788, negative recall 0.477; recs random unowned 0.053; segments five random groups -0.023.
- **Then the harness (23 min):** `labs/module-12/l13_2-capstone/harness_starter.py`: ONE function that scores any candidate on the training data the way the split works (rolling folds for demand, May for fraud, stratified folds with negative recall for reviews, a second hidden purchase for recs, silhouette plus seed stability for segments). The baseline goes through it first.
- **Coach:** "Does any row of your validation data come from AFTER its training rows?" "Does the baseline go through the same function?" Learners stuck for more than five minutes look at the solution's structure for their problem only.

STUDIO I · 30 MIN · MILESTONE 4, A FIRST PIPELINE

- **0:00 (2 min):** "The simplest sensible model, as a Pipeline, through your harness." One-hot + ridge; scaled numbers + one-hot + logistic regression; TF-IDF + logistic regression; item-kNN; K-Means on scaled features.
- **Work (13 min), 0:15 check-in (2 min):** one word: "beats", "ties", "loses". Every "loses" gets a visit: look for a bug before a better model.

- **Reference (harness solution, training data only):** demand ridge 13.128 vs baseline 13.924 in rolling folds (pessimistic: unseen months, see Q12); fraud logistic regression average precision 0.539 on May vs 0.008; reviews TF-IDF + LR accuracy 0.857 with negative recall 0.577; recs item-kNN 0.286 vs popularity 0.209 on the validation item; segments k = 5 silhouette 0.470, seed ARI 1.000.
- **Last 13 minutes:** start the attempts log (a small DataFrame or `results.json`): candidate, CV score, spread, one line of why. Milestone 5 at home fills it.

KICKOFF COACHING NOTES BY PROBLEM (NO ANSWERS ALOUD)

A · Demand. Strong early work: Plans the order decision three weeks ahead, so only planned fields are features (store, product, calendar, price and promo as planned); adds the per product × store mean as a second baseline (12.95) and a harness that respects time.
Traps to listen for: Actual `avg_high_temp_f` as a feature (known only after the week); a random split of weekly rows; the 14 planted stockout zeros read as zero demand; reading the 40%-off weeks (9.7 units against 20.1 at full price) as "discounts lower sales" (they are off-season clearance).
Ask: "Which columns do you know on the day the order is placed?" · "Is a zero always zero demand?" · "Inside 2024, which weeks validate which?"

B · Fraud. Strong early work: Keeps the January-to-May / June split, validates inside it on May, names the team's capacity (1% of June = 83 payments) and measures a one-line rule (velocity) as the second baseline (average precision 0.250, 19 of 64 frauds in its top 1%).
Traps to listen for: Accuracy (0.9923 for "nothing is fraud" in June); a stratified random split that mixes months; resampling before the split; ignoring the drift in the fraud rate (1.00% in January to 0.77% in June).
Ask: "How many payments can the team look at, and how many frauds does that hold today?" · "Which month validates your choices?" · "Who is kept waiting by a hold?"

C · Reviews. Strong early work: Uses the text (and says whether titles are allowed), keeps the stratified split, reports accuracy AND the recall of negative reviews from the first harness, and decides up front what the live system does with 3-star reviews.
Traps to listen for: Accuracy alone ("always positive" is 0.745 and catches no complaint); the English stop-word list, which removes "not" (accuracy drops to 0.864); fitting the vectorizer on all reviews before the split.
Ask: "Which error is the costly one here?" · "What happens to a 3-star review in production?" · "Does your vocabulary know the word 'not'?"

D · Recs. Strong early work: Keeps leave-last-out for the test, carves a SECOND hidden purchase out of the training rows for validation, masks owned products everywhere, and measures random (about 0.05) and popularity (0.209) first.
Traps to listen for: Choosing k on the test hit rate (Lesson 11.2 shows the k curve on the test set for teaching; a capstone must use the validation item); forgetting the mask (an unmasked item-kNN scores 0.113 because 69.1% of its slots are already owned); evaluating on training purchases.
Ask: "Which purchase validates your k?" · "Can your list contain something the customer owns?" · "What does a brand-new customer see?"

E · Segments. Strong early work: Chooses behavior columns only, scales (and considers logs), compares k with several signals (silhouette, inertia, BIC, stability) and plans the stability check (seeds, halves) before naming anything.
Traps to listen for: Unscaled features (silhouette 0.545, HIGHER than the scaled 0.470, because average order value dominates: a better number from a worse clustering); all 11 numeric columns (silhouette 0.189 at k = 5); `age_group` in the clustering; choosing k from one number.
Ask: "Which column dominates the distances, and why?" · "Do your segments survive another seed and half the data?" · "What will you do with age_group?"

MILESTONE CHECK · 10 MIN

Stand-up in problem order. Each learner reads three numbers: the baseline, the baseline through their harness, their first pipeline through the harness. Write them in the board grid and photograph it: it is your list for the first coaching round of Session 13 (anyone without a harness, and anyone whose pipeline loses, gets an email tonight).

CHECKS FOR UNDERSTANDING

Workbook self-check, all twelve items (project skills), as homework or a five-minute pair activity:

Answers: **Q1** A) Each morning the two-person review team decides which of the day's payments to hold; the model ranks them; **Q2** B) `avg_high_temp_f`, the actual temperature of the forecast week; **Q3** `[3, 5]`; **Q4** C) The model will score future months, and the fraud rate drifts; validating on a later month mimics that; **Q5** D) The best k on a validation purchase carved from the training rows (each customer's second-to-last); **Q6** A) The scaled ones: unscaled distances are dominated by average order value, so the higher number comes from a worse clustering; **Q7** `2.0`; **Q8** C) Look for a bug (a split, a feature, a leak in reverse) before looking for a better model; **Q9** D) Say what the live system does with them and measure it (how many get flagged as negative); **Q10** B) Before any modeling, in the plan; **Q11** A) Name who bears the errors, keep a human review with a fast release path, and report results by account age; **Q12** C) Each validation block contains months the folds have never seen, so a month effect learned from one year cannot be checked forward inside that year. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: learners who arrive without a plan write it in the clinic with you and narrow the scope (one metric, one baseline, one pipeline); they still meet every rubric row. **Stretch:** the Exemplary column early: a leak demonstrated with before-and-after scores, a second baseline that is genuinely hard to beat, a written "do not deploy" bar in the business's units.

CLOSING

EXIT TICKET (3 QUESTIONS, 4 MINUTES)

1. Write your harness in one sentence: which rows train, which rows validate, which metric. **Answer:** for example "January to April train, May validates, average precision". Check that validation never precedes its training

rows on time-ordered data.

2. Your two baselines and their numbers. **Answer:** the template's and the one from today's lab, on the same split and metric.

3. Name one column you will never use, and why. **Answer:** for example the actual week's temperature (unknown at order time), `transaction_id` (a label), `age_group` in the segments (a sensitive attribute; check it afterwards instead).

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish Milestone 4** if it is not done: the first pipeline through the harness, beating the baseline (or a bug found).
- **Milestone 5, improve (about 90 minutes):** better features, then a stronger model, then modest tuning; keep a change only if it beats the previous best by more than the fold spread. Log every attempt with its CV score.
- **Do NOT score the test set.** Choose a final candidate on the training data only; Session 13 opens the test set once, in the room.
- **The lesson's warm-ups:** Exercise 1 (an interval for an improvement) and Exercise 2 (who gets held) are good practice for Milestone 6; the Quick Quiz (five questions).
- **Journal:** one entry on the hardest decision so far (the lesson's prompt is for the end).

BRING LIST FOR SESSION 13

- A laptop with the course environment and `pinecrest-ml` with `data/` and `labs/` (Session 13's files are in `labs/module-13/`).
- Your capstone notebook through Milestone 5: plan, checks, harness, the attempts log, and ONE chosen final candidate, with the test set still unopened.
- The rubric (lesson page) and the model card skeleton, open in a tab; your journal.

IF YOU RUN LONG OR SHORT

RUNNING LONG

- Shorten the frame to 10 minutes and the plan clinic to 15; keep all Milestone 2 check-ins.
- If Milestone 3 runs over, Milestone 4 starts at home: the harness with the baseline in it is the protected outcome.

RUNNING SHORT

- The twelve self-check items in pairs.
- Pairs read each other's harness code and try to find a row that validates before it trains.
- Start Milestone 5's first idea in the room (one feature, through the harness).

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 13: Capstone Studio II and Showcase

How to run the last session: a timed studio for Lesson 13.2's Milestones 6 to 8 (one look at the test set with an interval, errors by subgroup, the responsible-ML review, a finding-first figure and the first sections of the model card), a showcase, and the close of the course. Reference numbers and the scoring rules are in final-assessment.html.

`1 lesson` `3 hrs session` `Lesson 13.2` `Session 13 of 13` `Studio + showcase`

HOW TO USE THIS GUIDE

- The last session of Advanced Machine Learning and of the course. A studio for Lesson 13.2's Milestones 6 to 8 (test once, explain and review, the figure and the first sections of the card), a showcase, and the close. Milestones 8 (rest) and 9 go home, with the submission due one week from today.
- Your answer notes and the rubric with the scoring rules are in `final-assessment.html`: for each problem the reference test numbers, the honest negative results, the traps and the follow-up questions. The short versions below are for coaching only; nothing is said aloud as an answer.
- The labs in `labs/module-13/l13_2-capstone/` are capstone tools: intervals (paired and by week), and the hand-in steps (results.json, skops round trip, a card written from the results, a number check). `debug_capstone.py` is a classmate's notebook to review: it breaks four checklist rules and still prints a rosy number.

> **THE ONE RISK OF SESSION 13: THE TEST SET OPENED AT HOME, THEN "IMPROVED"**
> Some learners will have peeked. Do not punish it; make it visible. In the warm-up status ask "has anyone already scored the test set?" and say: "Then your card says so, and the score you report is the first one you saw." The Evaluation row rewards honesty about this far more than a polished number. And in the room, nobody tunes after Milestone 6: post the rule on the board.

BEFORE THE SESSION

- ☐ Read the results notes in `final-assessment.html` for the problems in your room; the Session 12 board photo tells you who has no harness yet (first visits today).
- ☐ Run `python3 labs/check_labs.py module-13`: 4 passed (about 25 seconds).
- ☐ Decide the showcase format from the room size (table below); print rotation cards for a gallery walk; a visible timer.

- ☐ The board: "Milestone 6 happens once. No tuning after it." and the responsible-ML checklist (ten items from the lesson page).
- ☐ The course-close content: next steps (the four "going further" readings, a second capstone on open data), your feedback form, the submission deadline (one week from today).

SESSION 13 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Four retrieval questions and a one-line status from everyone.
0:10	10	**13.2** Frame: test once, then read the errors	The test-once ceremony, the Evaluation row, the responsible-ML checklist.
0:20	40	**13.2** Studio II · Milestone 6: test once, interval, errors	`labs/module-13/l13_2-capstone/bootstrap_ci_starter.py`; check-ins at 0, 15 and 30 minutes.
1:00	30	**13.2** Studio II · Milestone 7: explain and review	Permutation importance or SHAP in two sentences; every checklist item answered.
1:30	10	Break	Restart the kernel and Run All.
1:40	20	**13.2** Studio II · Milestone 8: the figure and the card's first sections	One finding-first figure; Recommendation and Performance written from results (`model_card_starter.py`).
2:00	10	**13.2** Showcase prep	One figure and three sentences.
2:10	30	**13.2** Showcase	Three-minute talks or a gallery walk, timed for the room.
2:40	10	Course close and what next	What the course built, the last journal entry, next steps, feedback.
2:50	10	Wrap-up	Exit ticket, submission rules and deadline.
Total	**180**		Ends at 3:00.

THE 480 MINUTES OF LESSON 13.2, SPLIT

MILESTONE (LESSON ESTIMATE)	MINUTES	WHERE IT HAPPENS IN THIS KIT
1 · Choose and plan	30	Session 11 (choice + paragraph, 10) and home; checked in the Session 12 plan clinic

MILESTONE (LESSON ESTIMATE)	MINUTES	WHERE IT HAPPENS IN THIS KIT
2 · Load, check and split	40	Session 12 studio
3 · Baseline and evaluation harness	30	Session 12 studio
4 · A first pipeline	45	Session 12 studio (started), home (finished)
5 · Improve	90	Home, between Sessions 12 and 13
6 · Test once, then analyze the errors	45	Session 13 studio
7 · Explain and review	40	Session 13 studio
8 · Figures and model card	45	Session 13 (the figure and the first two card sections, 20), home (25)
9 · Reproduce, save and review	30	Home, within a week of Session 13
Optional · Serve it	60	Home, only after Milestone 9

In the room today: Milestone 6 (40), Milestone 7 (30) and the first 20 minutes of Milestone 8. After the session: the rest of Milestone 8 (25), Milestone 9 (30) and the optional serving milestone (60).

Protect, in this order: Milestone 6 (one test score, with an interval, next to the baseline); the checklist answers of Milestone 7; the showcase. **Cut first:** showcase prep to 5 minutes; the course close to the journal prompt and the deadline.

WARM-UP

0:00 Retrieval and Status — 10 min

FIVE QUESTIONS (WITH ANSWERS)

1. Why does the fraud harness validate on May, not on shuffled folds? **Answer:** The model scores future months and the fraud rate drifts; validating on a later month mimics that.
2. What is the one loop the lesson does not allow? **Answer:** From the test set back to tuning: once you have looked at the test score, the model is finished.
3. Name two parts of a model card that a weak card usually lacks. **Answer:** The baseline on the same test set (with an interval for the model), and limitations specific to the data; also often the out-of-scope uses.
4. What does "explanations are not causes" mean for your card? **Answer:** Importance and SHAP describe what the model relies on; words like "causes" or "drives" need a design that supports them (none in this course).
5. Status: your final candidate, its CV score and the baseline's, in one line. **Answer:** No single answer. Listen for a candidate chosen on training data, and for anyone who has already looked at the test set: they write that in their card.

HOW TO RUN IT

W1 to W4 on paper, four minutes; answers in two. W5 around the room: write each status on the board grid next to the Session 12 numbers; anyone without a final candidate is your first visit.

LESSON PLAN

13.2 Capstone Project

130 min

OBJECTIVES (FROM THE LESSON PAGE)

- Frame a business request as a machine-learning task with a decision, a target, a leakage-safe split, a metric and a baseline, written down before any modeling
- Build, compare and improve models in leakage-proof pipelines, touching the test set once
- Analyze errors by subgroup and explain the model honestly, without claiming causes
- Review your model against a responsible-ML checklist and document it in a model card
- Deliver a project that reruns from a fresh start and score it yourself against a rubric

FRAME · 10 MIN: TEST ONCE, THEN READ THE ERRORS

Say: "Today you open the test set, once. Refit the final pipeline on all the training data, score the test set, write the number down next to the baseline, and never change the model again. Then the interesting part starts: where is it wrong, and who pays for those errors?" Read the Evaluation and Error analysis rows of the rubric. Point at the checklist on the board: "Every item gets an answer specific to your problem. 'Not applicable, because...' is fine; skipping is not."

STUDIO II · 40 MIN · MILESTONE 6, TEST ONCE, INTERVAL, ERRORS

- **0:00 (3 min):** each learner says their final candidate aloud, then runs the test once. Write the number in the notebook and on the board grid.
- **Work (12 min):** a bootstrap interval for the test metric, paired with the baseline (`labs/module-13/l13_2-capstone/bootstrap_ci_starter.py` shows both kinds: paired resamples on fraud, whole weeks on demand). Choose the operating point if the problem has one (the review team's 1%, top 10 for recommendations).
- **Reference (lab solution):** fraud June average precision logistic regression 0.539 (95% CI 0.408 to 0.658), tuned boosting 0.556, difference +0.017 (-0.046 to +0.077): not distinguishable; top 1% holds 35 of 64 frauds (recall 0.55, 0.43 to 0.68). Demand: units saved 5.23, row interval 4.90 to 5.53, week interval 4.68 to 5.81.
- **0:15 check-in (2 min):** one word: "interval", "operating point", "stuck".
- **Work (13 min):** errors by at least one subgroup (store, category, account age, history length, segment) with the baseline's errors beside them; read the twenty worst cases.
- **0:30 check-in (3 min):** each learner writes ONE finding about where the model fails. Read four aloud. **Done when** the test metric has an interval next to the baseline, and one written failure finding exists.

STUDIO II · 30 MIN · MILESTONE 7, EXPLAIN AND REVIEW

- **Explain (10 min):** permutation importance or SHAP on the test set; two sentences that describe the MODEL ("relies on"), never causes.
- **Review (15 min):** every responsible-ML checklist item answered for the problem: leakage audit, one look, baseline, subgroups, who bears the errors, sensitive attributes, humans in the loop, explanations vs causes, privacy, monitoring and fallback. If an answer changes the system (a review step, a removed feature, a threshold), make the change and note its effect.
- **Pairs (5 min):** swap checklists; each challenges one answer that is too generic.

RESULTS COACHING NOTES BY PROBLEM (NO ANSWERS ALOUD)

A · Demand. A sound project finds: Gradient boosting on store, product, category, month, prices, promo and holiday: 2025 MAE 8.19 against 13.42 (95% CI of the units saved 4.91 to 5.53 resampling rows; a week bootstrap gives 4.68 to 5.81); ridge with one-hot 11.29; Poisson loss 8.12.
Honest negatives and traps: Temperature adds nothing measurable once the month is in: actual 8.16, last year's for that week 8.18, none 8.19. "Same week last year" (13.81) is worse than the plain per-product mean. Errors are largest for Footwear and Camp Kitchen in units; Snow in summer is near zero both ways.

B · Fraud. A sound project finds: June average precision: logistic regression 0.539, boosting with defaults 0.477, tuned boosting 0.556 (95% CI 0.423 to 0.668); the top 1% (83 held) catches 33 / 32 / 35 of 64 frauds.
Honest negatives and traps: Tuned boosting minus logistic regression: 0.017 (95% CI -0.046 to 0.077): the two cannot be told apart, and on May validation logistic regression is as good (0.539 vs 0.532). New accounts are 8.4% of legitimate June payments but 29.2% of the false alarms: a human review step and a fast release path belong in the card.

C · Reviews. A sound project finds: TF-IDF + logistic regression: test accuracy 0.877 (95% CI 0.855 to 0.901) but negative recall only 0.660; with `class_weight="balanced"` accuracy 0.873 and negative recall 0.873; a threshold set on CV for 90% negative recall gives 0.909 on the test set while flagging 36.8% of reviews.
Honest negatives and traps: Bigrams barely help on this vocabulary (CV 0.860 vs 0.857). Titles alone score 0.969: legitimate if titles arrive with the review, but they restate the rating, so the card must say which was used. On 3-star reviews the model flags 20.8% at 0.5 and 64.8% at the 90%-recall threshold: unvalidated, and a caveat.

D · Recs. A sound project finds: Item-kNN 0.308 and SVD with 5 factors 0.311 against popularity 0.209 (item-kNN minus popularity 0.099, 95% CI 0.082 to 0.116); validation picks k = 5 (0.285).
Honest negatives and traps: SVD minus item-kNN: 0.003 (95% CI -0.009 to 0.016): a tie, so the simpler, explainable model is a fine choice. Content-based on activity alone scores 0.176, BELOW popularity (with a popularity tie-break 0.272): an honest negative, but it is the only answer for a new product. Popularity recommends 31 distinct products, item-kNN 161.

E · Segments. A sound project finds: K-Means on the four scaled behavior columns: silhouette by k peaks at 5 (0.47; log + scaled 0.478), seed stability ARI 1.000, half-sample ARI 0.986 to 0.996; five nameable profiles whose churn runs from 0.03 to 0.32 (never trained on).

Honest negatives and traps: Gaussian-mixture BIC keeps falling past 5 (43175 at 5, 42990 at 6, 42803 at 7): BIC alone would not pick 5, which is why several signals are needed. k = 4 is unstable across seeds (ARI down to 0.512).

STUDIO II · 20 MIN · MILESTONE 8, THE FIGURE AND THE CARD

- **The figure (10 min):** one chart that carries the argument (model against baseline by subgroup, a capacity or threshold chart, segment profiles), with a title that states the finding, saved from code.
- **The card (10 min):** Recommendation and Performance, written FROM the results dictionary (`labs/module-13/l13_2-capstone/model_card_starter.py` shows the pattern and the number check). The rest of the card goes home.
- **Reference (model-card lab, review triage):** CV picks the balanced model (negative recall 0.849 vs 0.57); test accuracy 0.873 (95% CI 0.85 to 0.895) vs 0.745, negative recall 0.873; skops needs no extra trusted types; a hand-typed 0.877 is caught.

SHOWCASE PREP · 10 MIN

One figure and three sentences: the decision and the recommendation; the main number against the baseline with its interval; one specific limitation. A learner whose answer is "do not deploy" says so first, with the evidence: model that sentence once for the room.

SHOWCASE · 30 MIN

LEARNERS	LIGHTNING TALKS (30 MIN)	GALLERY WALK (30 MIN)
8	One circle: 8 × (3 min + 30 s for one question) = 28 min. You keep time.	Two halves of 4: 4 visits of 3 min (13 min), swap, 13 min, 4 min debrief.
16	Two circles of 8 in parallel, a learner timekeeper in the second; or 2-minute talks in one circle.	Two halves of 8: each visitor makes 4 visits of 3 min to assigned stations; swap; 4 min debrief.
24	Three circles of 8 with timekeepers.	Two halves of 12 with printed rotation cards. Recommended at this size.

Audience job: one question per presenter on a sticky note; the presenter keeps them for the card's Limitations. Your job: keep time, say nothing evaluative in public, note one strength per learner for the written feedback.

COURSE CLOSE · 10 MIN

- **What the course built (2 min):** from `fit` and `predict` in Lesson 1.1 to a model chosen, checked for leaks, explained honestly, served, monitored and documented.
- **The last journal entry (4 min):** the lesson's prompt: the one decision they are proudest of, in three sentences as to an interviewer; then reread the Lesson 1.1 entry and write what they would tell that person about baselines.

- **What next (2 min):** the four "going further" readings (AutoML, federated learning, quantum machine learning, genetic algorithms), a second capstone on open public data, depth in one area. Repeat the rule: never practice on private or work data without written permission; publish `.skops`, never pickles.
- **Feedback (2 min):** your form, and one spoken question: "Which session would you change first, and how?"

CHECKS FOR UNDERSTANDING

Workbook self-check, all twelve items, as homework with the submission or as a pair activity if the showcase runs short:

Answers: **Q1** A) After every choice (features, model, hyperparameters, threshold or list size) is made on the training data, once; **Q2** `[ 3.475 97.525]`; **Q3** C) The two cannot be told apart on 64 June frauds; choose on other grounds (simplicity, explainability, cost) and say so; **Q4** D) The week interval: rows in the same week share the same conditions, so they are not independent; **Q5** B) On 774 held-out reviews the model finds 87% of the negative reviews (the baseline finds none), at accuracy 0.873 (95% CI 0.85 to 0.895) against 0.745; **Q6** D) Customers in this segment churned more often (about 32%); the segments describe behavior, not why customers leave; **Q7** `{'Bend': 3.0, 'Boise': 5.0}`; **Q8** B) Compare each group's error with its baseline and its typical volume: Snow sells about 12 units a week, Footwear about 46; **Q9** A) Hit rate at 10 by customer history length, for the model and the baseline; **Q10** C) Keep it out of the clustering, then check whether the segments differ by age; **Q11** D) Write the card from the results (results.json and an f-string), and check every card number against it; **Q12** A) The decision and the recommendation, the main number against the baseline with its interval, one specific limitation, pointing at one figure. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: learners without a final candidate take their first pipeline as final today, run Milestone 6 on it, and say so in the card: a modest, honest model meets every rubric row. **Stretch:** the Exemplary column: an error finding turned into a fix and measured (the worked example's closed-day rule), a system change because of a checklist answer, the optional FastAPI endpoint with a match test and a drift check.

CLOSING

EXIT TICKET (3 QUESTIONS, 4 MINUTES)

1. Your main result in one sentence: the metric with its interval, against the baseline. **Answer:** for example "June average precision 0.556 (95% CI 0.42 to 0.67) against 0.008 with no model; the top 1% holds 35 of 64 frauds."
2. One limitation specific to your data (not "more data"). **Answer:** for example "only 64 June frauds, so two good models cannot be ranked" or "3-star reviews are not validated".
3. What is left, in order, before you submit? **Answer:** the rest of the card, then Milestone 9: save with skops, results.json and requirements.txt, Restart and Run All, check every card number against the output, self-score on the rubric.

SUBMISSION (WITHIN ONE WEEK) AND WHAT GOES HOME

- **After class (about 55 minutes required):** the rest of Milestone 8 (the card's remaining sections), Milestone 9 (save, pin, Restart and Run All, check numbers, self-score). Optional: Serve it (60).
- **Hand in** the project folder: `capstone.ipynb` (reruns from a fresh kernel), `model_card.md`, one to three figures, `model.skops`, `results.json`, `requirements.txt`, a README (what it is, how to run it, the fictional-data line, and what help was used), and the self-scored rubric. Deadline: one week from today. Rules, late options and the scoring sheet are in `final-assessment.html`.
- **Bring list:** there is no Session 14. Do not hand in the course `data/` folder: link to `pinecrest-ml-data.zip`, as the lesson's Sharing Your Work section says.

IF YOU RUN LONG OR SHORT

RUNNING LONG

- Showcase prep to 5 minutes; the course close to the journal prompt and the deadline.
- Never cut the one test-set run or the interval in Milestone 6.

RUNNING SHORT

- Everyone reviews `labs/module-13/l13_2-capstone/debug_capstone.py` in pairs: four broken rules, which rubric row each costs, and the honest numbers.
- The twelve self-check items in pairs.
- Start Milestone 9 in the room: Restart and Run All, then check three card numbers against the output.
